Web Services Applications

4. Design a URL Shortening Service

0) Requirements & Sizing

Functional

- Shorten long URLs, support custom aliases, link expiration, and disabling
- **302 redirects** (analytics-friendly) with very low latency
- Click analytics: total clicks, uniques, referrer/UA/geo, time series
- Admin: update/transfer ownership, bulk create, abuse reporting, preview/expand

Non-functional

Metric	Target
Scale	~100M creates/day → ~1.16k writes/sec avg
Read:Write ratio	100:1 → ~116k reads/sec avg
Peak design	10× → **~1M RPS reads**
p99 redirect latency	< 20–30 ms from edge
Availability	Four 9s (99.99%) on read path
Global	Multi-region active-active, no SPOF

Capacity

```
7-char Base62 → 62⁷ ≈ 3.52 trillion codes
At 100M/day → ~96 years of headroom (future-proof)
```

Design Reasoning

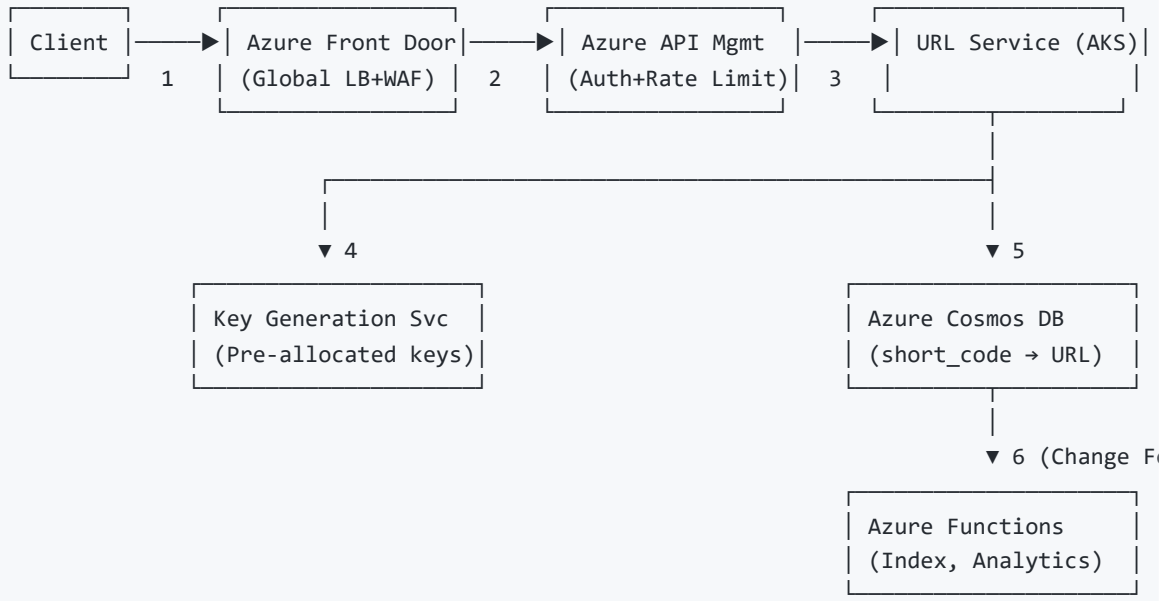
> **Core philosophy**: Optimize the read path for extreme low latency (sub-20ms) since reads outnumber writes 100:1, using a pre-generated key pool to eliminate write-path coordination and a multi-tier cache to serve the vast majority of redirects without touching the database.

Decision	Choice	Why	Alternative Considered
ID generation	Pre-generated Base62 key batches	Zero coordination on the hot write path; pods draw from local pools with no collisions	Snowflake IDs — require Base62 encoding and scatter across partitions unpredictably
Database	Azure Cosmos DB	Single-digit-ms point reads by short_code; global replication for active-active multi-region	Azure SQL — higher latency for simple KV lookups, harder to scale globally
Hot cache	Azure Cache for Redis	95-99% cache hit rate keeps p99 redirect latency under 20ms for popular URLs	Cosmos DB only — 5-10ms reads still too slow at 1M RPS peak without caching
Redirect type	HTTP 302 (temporary)	Enables per-click analytics since browsers re-request each time instead of caching	HTTP 301 (permanent) — browsers cache and bypass server, losing analytics data
Analytics pipeline	Event Hubs + Azure Data Explorer	Fire-and-forget async click events keep redirect latency off the analytics critical path	Synchronous DB writes — would add 10-20ms to every redirect request
Global entry point	Azure Front Door	Anycast routing to nearest region, built-in WAF and DDoS protection	Regional load balancer — higher latency for users far from single region

---

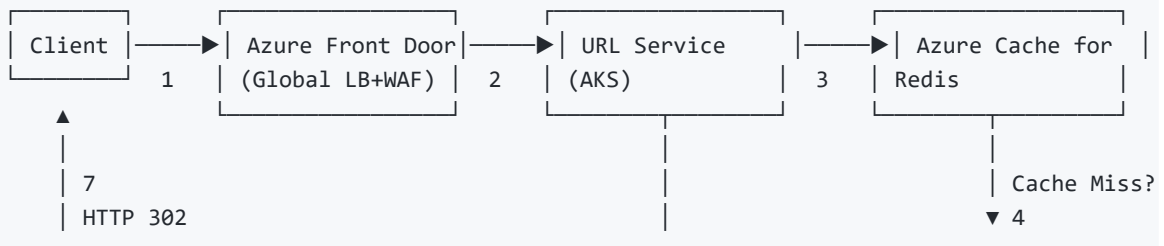
1) High-Level Architecture

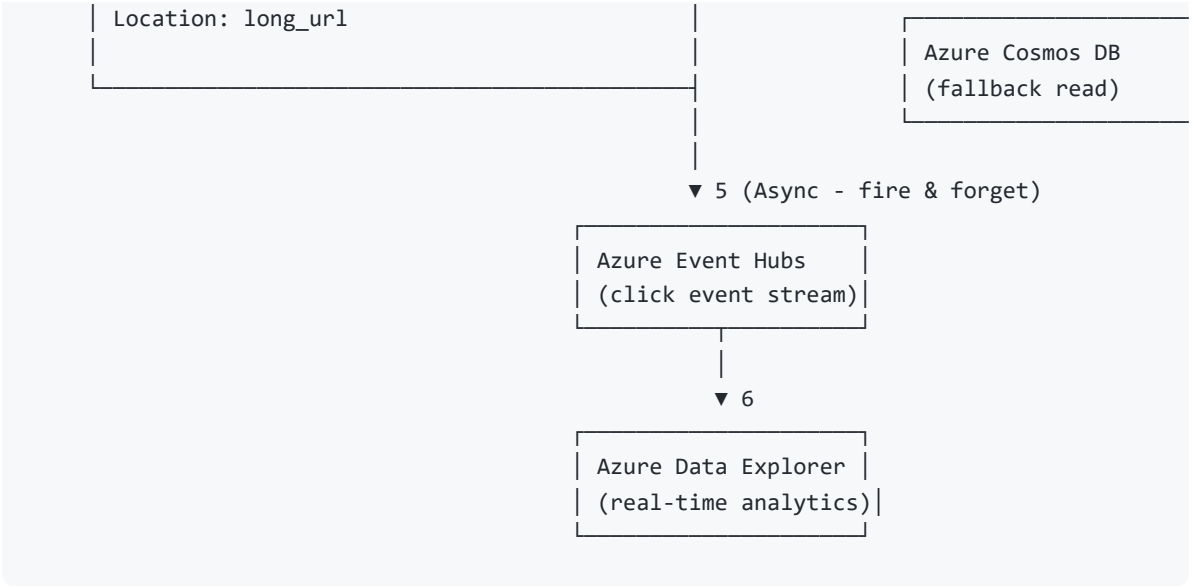
Write Path (Create Short URL)



Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	HTTPS request with long URL	SYNC
2	Front Door	API Management	Route to regional backend, WAF check	SYNC
3	API Management	URL Service (AKS)	Validate auth, apply rate limits	SYNC
4	URL Service	Key Generation Svc	Get next available short code from batch	SYNC
5	URL Service	Cosmos DB	Write mapping `{short_code → long_url}`	SYNC
6	Cosmos DB	Azure Functions	Change feed triggers indexing & analytics	ASYNC

Read Path (Redirect with Analytics)





Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	GET `short.ly/abc123`	SYNC
2	Front Door	URL Service (AKS)	Route to nearest region	SYNC
3	URL Service	Azure Cache for Redis	Lookup `short_code` in cache	SYNC
4	Redis (miss)	Cosmos DB	Point read by `short_code`	SYNC
5	URL Service	Event Hubs	Emit click event `{code, ip, ua, ts}`	**ASYNC**
6	Event Hubs	Azure Data Explorer	Stream processing → dashboards	ASYNC
7	URL Service	Client	Return `HTTP 302 Location: long_url`	SYNC

> **Key insight**: Step 5 is ASYNC (fire-and-forget). Redirect returns immediately; analytics processed in background.

Components Summary

Component	Azure Service	Purpose

Global Entry	**Azure Front Door**	Anycast DNS, global load balancing, WAF
API Gateway	**Azure API Management**	Authentication, rate limiting, quotas
Compute	**Azure Kubernetes Service (AKS)**	Run URL service pods
Hot Cache	**Azure Cache for Redis**	~95-99% cache hit rate
Database	**Azure Cosmos DB**	Persistent KV store, global replication
Event Stream	**Azure Event Hubs**	Click events at high throughput
Analytics	**Azure Data Explorer (ADX)**	Sub-second aggregation queries
Serverless	**Azure Functions**	Change feed processing

API Design

Core APIs

Method	Endpoint	Description
`POST`	`/api/v1/shorten`	Create a short URL
`GET`	`/{shortCode}`	Redirect to original URL (302)
`GET`	`/api/v1/urls/{shortCode}`	Get URL metadata
`DELETE`	`/api/v1/urls/{shortCode}`	Delete a short URL

Request/Response Examples

Create Short URL:

```
POST /api/v1/shorten
{
  "long_url": "https://example.com/very/long/path",
  "custom_alias": "my-link",    // optional
  "expiry_days": 30             // optional
}

Response: 201 Created
{
```

```
  "short_url": "https://short.ly/abc123",
  "short_code": "abc123",
  "expires_at": "2025-04-15T00:00:00Z"
}
```

Redirect:

```
GET /abc123
Response: 302 Found
Location: https://example.com/very/long/path
```

Analytics APIs

Method	Endpoint	Description
`GET`	`/api/v1/urls/{shortCode}/stats`	Get click analytics
`GET`	`/api/v1/urls/{shortCode}/stats?range=7d`	Get time-range analytics

2) ID Generation

✅ Recommended: Pre-generated Batches

```
Key Service pre-generates Base62 keys by ranges
  ├── Batch [aaa...azz] → API Pod 1
  ├── Batch [baa...bzz] → API Pod 2
  └── Batch [caa...czz] → API Pod 3

On create: pop from local pool
If empty: atomically fetch next batch

✅ No coordination on hot path
✅ No collisions
✅ No SPOF
```

Alternative: Snowflake ID

```
┌──────────────┬──────────────┬──────────────┬──────────┐
│  Timestamp   │ Datacenter   │  Machine     │ Sequence │
│  (41 bits)   │  (5 bits)    │  (5 bits)    │ (12 bits)│
└──────────────┴──────────────┴──────────────┴──────────┘
→ Base62 encode → short code
```

```
✅ Distributed, no coordination
⚠️ Set region bits to scatter across partitions
```

❌ Counter: Reject

Single bottleneck + SPOF. Not scalable.

Custom Aliases

- Conditional write: "create if not exists" (Cosmos `If-None-Match`)
- Reserved terms list + profanity filter (enterprise requirement)

---

3) Data Model (Cosmos DB)

```
{
  "id": "7HgT2mA",              // short_code (partition key)
  "long_url": "https://...",
  "owner_id": "aad:...",
  "created_at": "2026-01-27T00:00:00Z",
  "expires_at": "2027-01-27T00:00:00Z",
  "status": "active|disabled|deleted",
  "flags": {
    "allow_params": true,
    "safe": true
  }
}
```

- **TTL** on `expires_at` for auto-delete
- **Change feed** → secondary container indexed by `(owner_id, created_at)` for "my links" view

---

3.1) Storage Options: Table Storage vs Cosmos DB vs Blob

Azure Storage Account as KV Store?

Service	Pattern	Best For
Table Storage	PartitionKey + RowKey → value	Cheap, simple KV, moderate throughput
Blob Storage	blob name → blob content	Large objects, not high-frequency KV

Cosmos DB	id → document	Low latency, global, high throughput

Comparison

Aspect	Table Storage	Cosmos DB
Latency	~10-50ms	**<10ms** (single-digit)
Throughput	≤20K ops/sec per table	**10M+ ops/sec**
Global distribution	❌	✅
Automatic indexing	❌ (PK+RK only)	✅ (all properties)
Cost	**Cheapest**	Higher
Query patterns	PK+RK point queries only	Rich queries

When to Use What

Use Table Storage if:

- Budget is tight
- Queries are only `PartitionKey + RowKey → value`
- No global distribution needed
- Moderate throughput (≤20K ops/sec)

Use Cosmos DB if:

- Need predictable low latency (<10ms)
- Large scale or global distribution
- High throughput requirements
- Need advanced queries or change feed

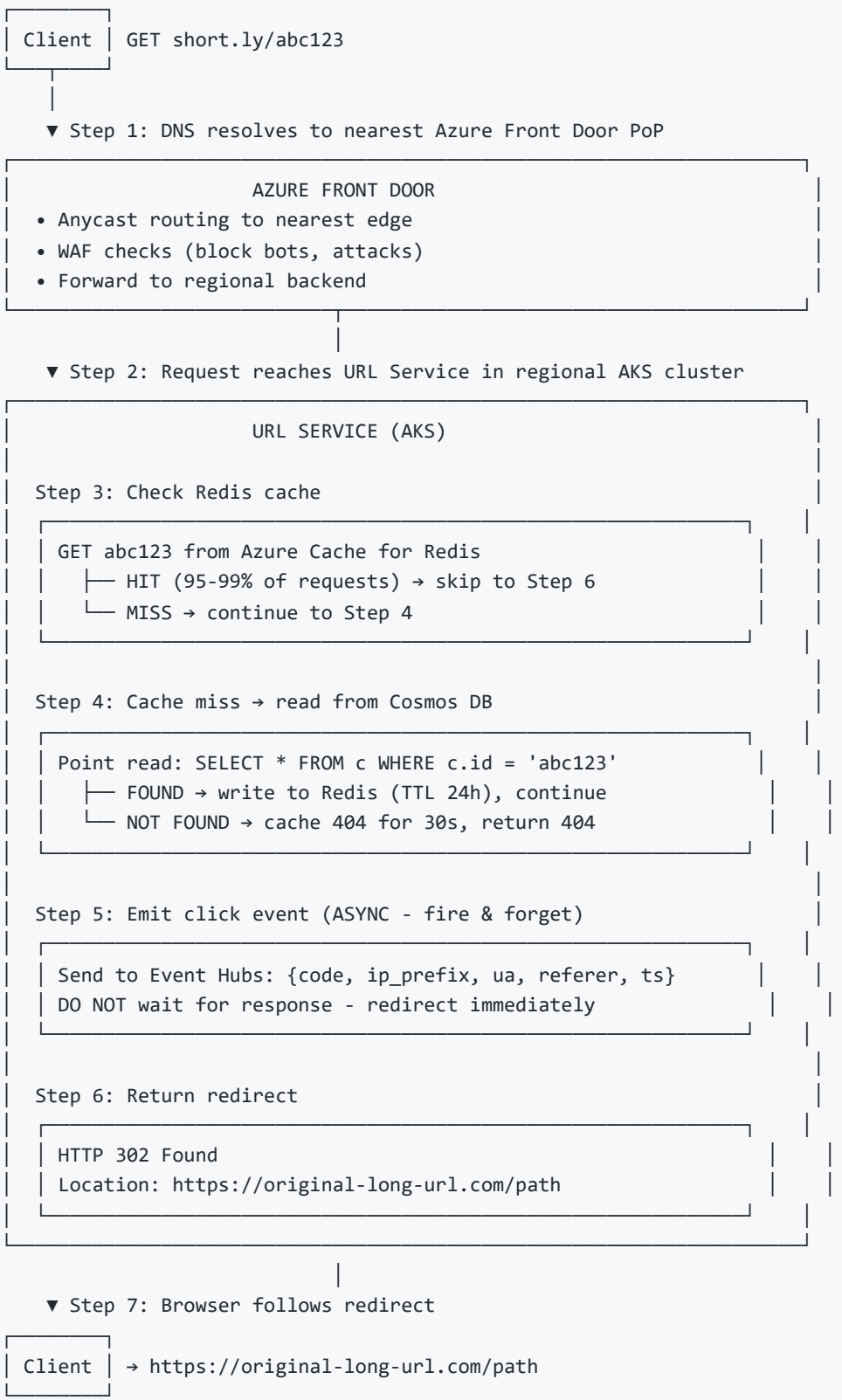
For URL Shortener: Cosmos DB

Why?

- Point reads by `short_code` = perfect KV pattern
- Need low latency for redirects (<30ms p99)
- Change feed powers analytics pipeline
- Multi-region active-active required

---

4) Read Path Detail

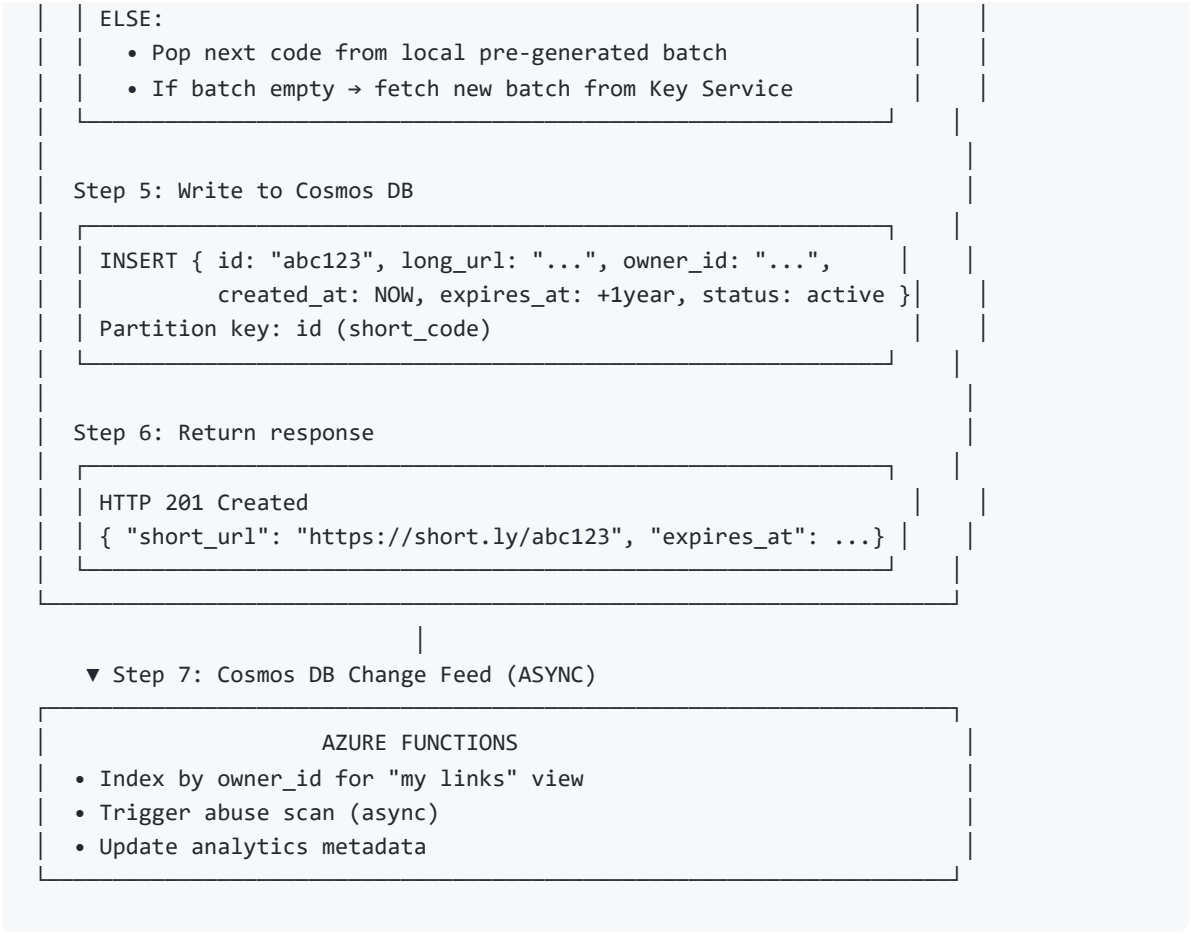


Cache Strategy:

Setting	Value	Why
TTL	24h-7d	URLs rarely change
Early refresh	Probabilistic	Prevent thundering herd on expiry
Eviction	LRU	Keep hot URLs, evict cold
Negative cache	30s for 404s	Block brute-force scanning
Dogpile protection	Single-flight per key	One DB call per miss

5) Write Path Detail





---

6) CDN vs Analytics (Critical Trade-off)

Choice	Performance	Analytics
CDN caches redirects	✅ Fastest	❌ Can't count every click
No CDN, use LB+Redis	✅ Fast	✅ Count every click

> ⚠️ **If every click must be counted, do NOT cache redirects at CDN.** Use Front Door as L7 LB + WAF only, serve from app/Redis.

Options:

- Per-link toggle: cache 301 for very hot links that don't need analytics
- Edge workers: increment counter + issue 302 (but beware log loss under cache hits)

---

7) Analytics (Clicks, Uniques)

On each redirect, emit to Event Hubs:

```
{ "ts": "...", "code": "7HgT2mA", "ip_prefix": "1.2.3.0/24",
  "ua": "...", "referer": "...", "region": "westus" }
```

Streaming jobs:

- Real-time counters in Redis or ADX materialized views
- **HyperLogLog** per (code, day) for memory-efficient unique counts
- Persist detailed events to ADX; roll older data to cold tier

---

8) Multi-Region & DR

```
Active-active: East US, West Europe, Central India
  │
  ├── Front Door anycast → nearest healthy region
  ├── Cosmos DB multi-write (last-writer-wins, safe for rare writes per key)
  ├── Redis: independent per region (no cross-region replication)
  └── Event Hubs: per-region with geo-replication to centralized ADX
```

Key allocation: Add region prefix in Snowflake or disjoint key ranges to avoid collisions.

---

9) Security & Governance

Concern	Solution
AuthN/Z	AAD for UI, API keys with scopes
WAF	Front Door WAF for bot signatures
Ownership	Require two owners or group; profanity/reserved checks
PII	Truncate IPs (/24), hash UA for uniques
GDPR	Deletion pipeline for click rows
SSRF	Block private IPs, enforce public host allow-list

---

10) Failure Modes

Failure	Mitigation
Redis down	Fall back to DB; shed load with token buckets; 503 for tail
DB region loss	Multi-region Cosmos continues; Front Door routes to healthy region
Key service outage	Writes degrade; reads unaffected; keep local key buffer for hours
Analytics lag	Decouple via Event Hubs; redirects continue; backpressure to consumers

---

11) Cost Estimates

Component	Sizing
Redis	Hot set ~20M keys × 300B = ~6GB + overhead per shard
Cosmos DB	Point reads ~1 RU; size for peak writes; autoscale
Event Hubs	Throughput units for peak click RPS; retention ≤7 days

---

12) 60-Second Explanation

> "It's a **read-heavy key-value system**. I generate 7-char Base62 codes using **pre-allocated batches** to avoid coordination. The mapping lives in **Cosmos DB** (globally distributed) fronted by a large **Redis cache** (~99% hit rate).
>
> Redirects return **302** so I can count clicks. Every hit emits a tiny event to **Event Hubs**; stream jobs aggregate into **ADX** for real-time dashboards.
>
> The whole thing is **active-active** across regions behind **Front Door**. We prevent SPOFs, use conditional writes for vanity conflicts, TTL/negative caches to stop brute forcing, and change feed for side effects.
>
> This keeps **p99 < 30ms** and scales to **~1M RPS peaks** while preserving accurate analytics."

Technologies Deep Dive

Technology	Role	Reference

Cosmos DB	Distributed KV store	Tech Guide
Redis	Hot cache, counters	Tech Guide
Kafka/Event Hubs	Click event streaming	Tech Guide
HyperLogLog	Unique click counts	Tech Guide
Consistent Hashing	Shard distribution	Tech Guide

Appendix: Base62 Encoding

```
String alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";

String encode(long n) {
    StringBuilder s = new StringBuilder();
    do {
        s.append(alphabet.charAt((int)(n % 62)));
        n /= 62;
    } while (n > 0);
    return s.reverse().toString();
}

long decode(String s) {
    long n = 0;
    for (char c : s.toCharArray()) {
        n = n * 62 + alphabet.indexOf(c);
    }
    return n;
}
```

---

5. Design Search Autocomplete/Typeahead

---

0) Requirements & Sizing

Functional

- Return **top 5-10 suggestions** as users type prefix queries
- Rank by popularity, recency, and optionally personalization
- Support **multi-language** queries (Unicode-aware)

- **Filter** offensive/inappropriate suggestions
- Handle **trending** queries (boost recent spikes)
- Optional: personalized suggestions based on user history

Non-functional

Metric	Target
Scale	~100K QPS (peak) for suggestions
p99 latency	< **50-100ms** (critical for UX)
Trie size	~~10M unique queries~~ (1-2GB in memory)
Update frequency	Hourly batch + real-time trending
Availability	99.9% (suggestions are non-critical graceful degradation)
Global	Multi-region for low latency

Capacity

```
Assumptions:
- 10M unique searchable phrases
- Average phrase length: 20 characters
- Top-10 suggestions per node: 10 × 8 bytes = 80 bytes
- Trie overhead: ~50 bytes per node

Memory estimate:
- 10M phrases × 20 chars = 200M nodes (worst case)
- Optimized trie with sharing: ~50M nodes
- 50M × 50 bytes = ~2.5GB per replica

With compression + only storing top suggestions:
- Practical size: **1-2GB per server**
```

Design Reasoning

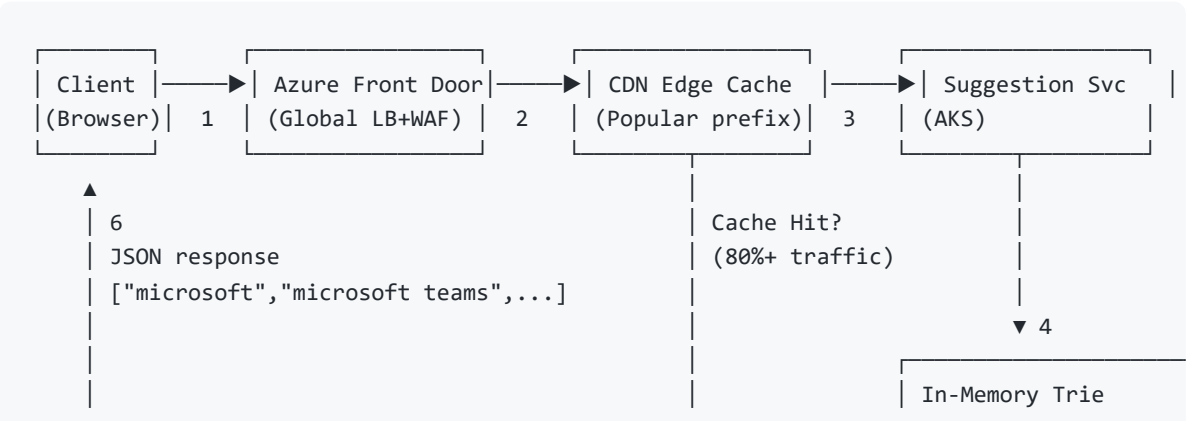
> Core philosophy: Serve suggestions in under 50ms by keeping the entire trie in memory per pod and caching popular prefixes at the CDN edge, while using a separate offline pipeline to rebuild rankings without impacting read latency.

Decision	Choice	Why	Alternative Considered

Data structure	Trie with pre-computed top-K per node	O(1) prefix lookup returns pre-ranked suggestions instantly without runtime sorting	Inverted index (Elasticsearch) — higher query latency for prefix matching at 100K QPS
Serving model	In-memory trie per pod (~1-2GB)	Eliminates network hops; entire lookup is a pointer traversal in local memory	Centralized Redis trie — adds 1-2ms network latency per keystroke at scale
Edge caching	Azure CDN for popular prefixes	80%+ of queries hit common prefixes; CDN serves them in under 10ms globally	No CDN — all 100K QPS hit backend pods, requiring far more compute capacity
Trie updates	Hourly batch rebuild with hot-swap	Predictable, consistent rankings; no read-path contention during updates	Real-time trie mutation — complex locking, risk of inconsistent rankings mid-update
Trending queries	Redis sliding window (real-time)	Captures breaking news/events within minutes and merges with static trie at query time	Batch-only — trending queries would be delayed by up to an hour
Query logging	Azure Event Hubs (async, fire-and-forget)	Decouples analytics from the read path; no latency impact on suggestion responses	Synchronous logging — adds latency to every keystroke request

1) High-Level Architecture

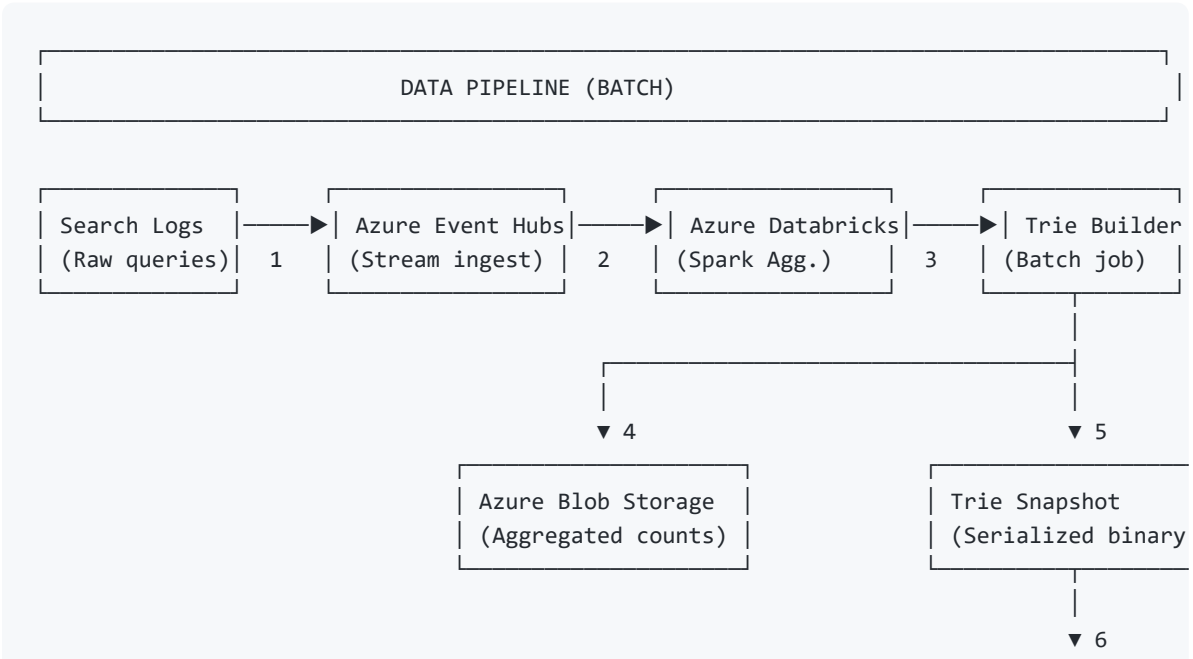
Read Path (Get Suggestions)

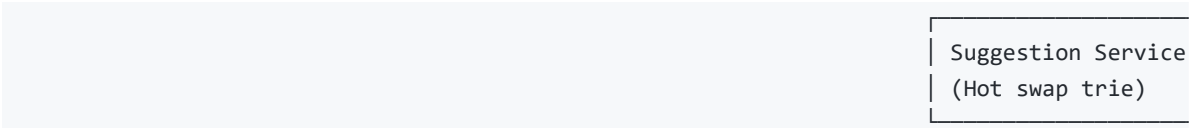




Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	GET `/suggest?q=micros` (debounced 100-200ms)	SYNC
2	Front Door	CDN Edge	Check edge cache for popular prefix	SYNC
3	CDN (miss)	Suggestion Service	Route to nearest AKS cluster	SYNC
4	Suggestion Service	In-Memory Trie	Lookup prefix → return pre-computed top-K	SYNC
5	Suggestion Service	Event Hubs	Log query for analytics (fire & forget)	**ASYNC**
6	Suggestion Service	Client	Return JSON array of suggestions	SYNC

Write Path (Update Trie)

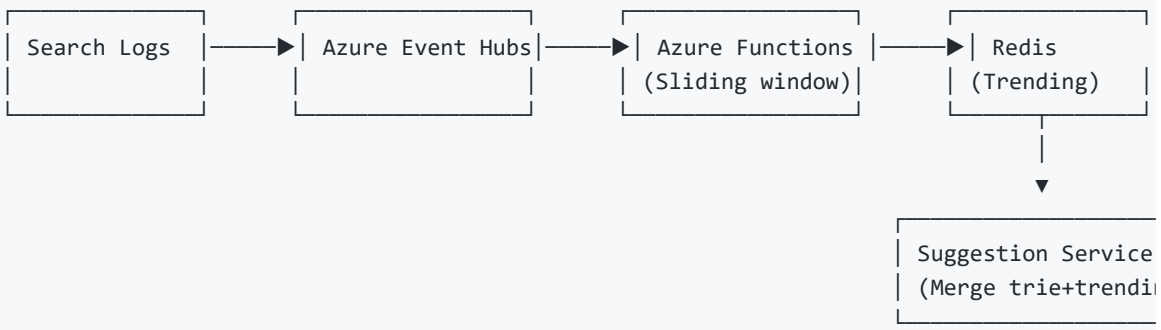




Step	From	To	Action	Frequency
1	Search Service	Event Hubs	Stream completed search queries	Real-time
2	Event Hubs	Databricks	Consume and aggregate by time window	Continuous
3	Databricks	Trie Builder	Compute frequency scores, filter, rank	Hourly
4	Trie Builder	Blob Storage	Store aggregated phrase counts	Hourly
5	Trie Builder	Blob Storage	Serialize trie to binary snapshot	Hourly
6	Suggestion Service	Memory	Load new trie snapshot (hot swap)	Hourly

---

Real-Time Trending Path



> **Key insight**: Trending queries (last 1-4 hours) are computed in near real-time and merged with the static trie at query time.

Components Summary

Component	Azure Service	Purpose

Global Entry	**Azure Front Door**	Anycast DNS, WAF, route to nearest region
Edge Cache	**Azure CDN**	Cache popular prefixes (80%+ hit rate)
Compute	**Azure Kubernetes Service (AKS)**	Run suggestion service pods
In-Memory Store	**Application Memory**	Hold trie (~1-2GB per pod)
Trending Cache	**Azure Cache for Redis**	Real-time trending queries
Event Stream	**Azure Event Hubs**	Ingest search query logs
Batch Processing	**Azure Databricks**	Aggregate query frequencies
Storage	**Azure Blob Storage**	Trie snapshots, aggregated data
Filter Service	**Azure ML**	Offensive content detection

API Design

Core APIs

Method	Endpoint	Description
`GET`	`/api/v1/suggestions?q={prefix}&limit=10`	Get autocomplete suggestions
`POST`	`/api/v1/searches`	Log a completed search (for ranking updates)

Request/Response Examples

Get Suggestions:

```
GET /api/v1/suggestions?q=micro&limit=5

Response: 200 OK
{
  "suggestions": [
    { "text": "microsoft teams", "score": 9500 },
```

```
    { "text": "microservices", "score": 8200 },
    { "text": "microsoft azure", "score": 7800 },
    { "text": "microcontroller", "score": 3100 },
    { "text": "micro frontend", "score": 2900 }
  ],
  "query_time_ms": 12
}
```

Design Considerations

- Response time must be < **100ms** for good UX
- Use **HTTP GET** (cacheable) rather than POST
- Support **debouncing** on client side (200-300ms delay)
- Return results sorted by **popularity score**

2) Data Structure Options

✅ Recommended: Trie with Pre-computed Top-K




```
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    List<Suggestion> topK = new ArrayList<>(10);  // Pre-computed!
    boolean isEndOfWord;
}

class Suggestion {
    String phrase;
```

```
    long frequency;
    double score;       // α*frequency + β*recency + γ*personalization
    long lastUpdated;
}
```

Why pre-compute top-K at each node?

- Without: O(prefix) + O(traverse entire subtree) = slow
- With: O(prefix) + O(1) = **fast** (<1ms lookup)

Alternative: Prefix Hash Map

```
Map<String, List<Suggestion>> prefixToSuggestions;
// "mic" → ["microsoft", "microwave", ...]
// "micr" → ["microsoft", "microwave", ...]
// "micro" → ["microsoft", "microwave", ...]
```

Aspect	Trie	Prefix Hash Map
Memory	More efficient (shared prefixes)	Redundant storage
Lookup	O(prefix length)	O(1) hash lookup
Updates	Complex (bubble up changes)	Simple (regenerate)
Memory for 10M phrases	~1-2GB	~5-10GB

Recommendation: **Trie** for memory efficiency, **Hash Map** if simplicity > memory.

❌ Rejected: Database Lookup

- Too slow for <50ms p99
- Even with Redis, serialization overhead adds latency

---

3) Data Model

Query Aggregation (Blob Storage / Data Lake)

```
{
  "phrase": "microsoft teams",
  "frequency_1h": 15420,
  "frequency_24h": 245000,
```

```
    "frequency_7d": 1850000,
    "last_searched": "2026-01-27T08:00:00Z",
    "language": "en",
    "category": "technology",
    "is_safe": true
  }
```

Trie Snapshot Format (Binary)

```
Header:
  - Version: uint32
  - Node count: uint32
  - Phrase count: uint32

Nodes (BFS order):
  - Character: uint16 (Unicode)
  - Children offset: uint32
  - Top-K offset: uint32
  - IsEndOfWord: bool

Suggestions:
  - Phrase offset: uint32
  - Score: float32

Strings (compressed):
  - Null-terminated UTF-8 strings
```

Redis Trending Cache

```
Key: trending:{region}:{hour}
Type: Sorted Set
Score: frequency in last hour
Value: query phrase

ZADD trending:westus:2026012708 15420 "microsoft teams"
ZREVRANGE trending:westus:2026012708 0 99  // Get top 100
```

---

4) Read Path Detail

```
┌──────────┐
│ Client │ User types "micros" (debounced)
└──────────┘
    │
    ▼ Step 1: Debounce on client (100-200ms)
┌──────────────────────────────────────────────────────────────────────┐
│                          CLIENT                                      │
```


• Wait for user to stop typing (debounce)
• Check local cache (last 5 min of suggestions)
• If cache miss → send request
GET /suggest?q=micros
▼ Step 2: Edge routing via Front Door
AZURE FRONT DOOR
• Route to nearest PoP
• WAF check (block malicious requests)
• Forward to CDN
▼ Step 3: CDN cache check
AZURE CDN (EDGE CACHE)
Cache Key: suggest:{prefix}:{lang}:{region}
TTL: 5-15 minutes (balance freshness vs hit rate)
Cache HIT? (80%+ of requests for popular prefixes)
YES → Return cached response immediately
NO → Forward to origin
Cache miss
▼ Step 4: Suggestion Service processes request
SUGGESTION SERVICE (AKS)
Step 4a: Normalize prefix
• Lowercase
• Remove special characters (keep Unicode letters)
• Trim whitespace
• Detect language (for multi-lingual support)
Step 4b: Trie lookup
TrieNode node = root;
for (char c : "micros".toCharArray()) {
node = node.children.get(c);
if (node == null) return empty;
}
return node.topK; // Pre-computed, O(1)!
Step 4c: Merge with trending (Redis)
• Fetch trending queries starting with "micros"
• Merge-sort with trie results
• Boost recently trending items



---

5) Ranking Algorithm

Score Calculation

```
Score = α × frequency + β × recency + γ × personalization + δ × trending

Where:
  α = 0.5  (base popularity weight)
  β = 0.2  (recency decay factor)
  γ = 0.2  (user personalization, if logged in)
  δ = 0.1  (trending boost)
```

Frequency Weighting

```
  double frequencyScore(long freq7d) {
      // Log scale to prevent popular queries from dominating
      return Math.log10(freq7d + 1);
  }
```

Recency Decay

```
  double recencyScore(long lastSearchedTs, long nowTs) {
      long hoursAgo = (nowTs - lastSearchedTs) / 3600000;
      // Exponential decay: half-life of 24 hours
```

```
        return Math.exp(-0.029 * hoursAgo);
    }
```

Trending Boost

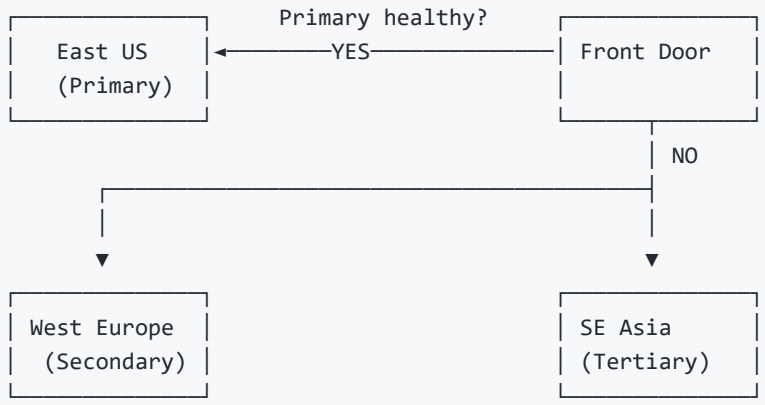
```
    double trendingScore(long freqLastHour, long avgHourlyFreq) {
        if (avgHourlyFreq == 0) return 0;
        double ratio = (double) freqLastHour / avgHourlyFreq;
        // Boost if 2x+ normal traffic
        return ratio > 2.0 ? Math.log(ratio) : 0;
    }
```

---

6) Multi-Region & DR

```
Active-Active: East US, West Europe, Southeast Asia
  │
  ├── Front Door anycast → nearest healthy region
  ├── Each region has:
  │     ├── Full trie replica (loaded from same Blob snapshot)
  │     ├── Regional Redis (trending cache)
  │     └── CDN edge nodes
  │
  ├── Trie updates: Push snapshot to all regions simultaneously
  └── Trending: Regional (no cross-region sync needed)
```

Failover Strategy



---

7) Security & Governance

Concern	Solution
Offensive content	ML-based filter + blocklist; review flagged suggestions
Query injection	Sanitize input, limit length (100 chars max)
PII in suggestions	Never include email, phone, SSN patterns
Abuse (scraping)	Rate limit by IP (100 req/min), CAPTCHA for suspicious
Logging PII	Hash/truncate user IDs, don't log full queries with PII
GDPR	Allow users to opt-out of personalization

Input Validation

```
String sanitizePrefix(String input) {
    if (input == null || input.length() > 100) {
        throw new BadRequestException("Invalid prefix");
    }
    // Remove control characters, normalize Unicode
    return Normalizer.normalize(input, Normalizer.Form.NFC)
                     .replaceAll("[\\p{Cntrl}]", "")
                     .toLowerCase()
                     .trim();
}
```

---

8) Failure Modes

Failure	Impact	Mitigation
Trie service down	No suggestions	CDN serves stale; client shows "no suggestions" gracefully
Redis (trending) down	No trending boost	Fall back to static trie only (still functional)
Trie build fails	Stale suggestions	Keep serving previous snapshot; alert on-call
CDN outage	Higher origin load	Front Door routes directly; auto-scale AKS

Event Hubs down	Missing analytics	Suggestions still work; backfill from logs

Graceful Degradation

```
List<Suggestion> getSuggestions(String prefix) {
    try {
        List<Suggestion> results = trie.lookup(prefix);
        try {
            results = mergeWithTrending(results, redis.getTrending(prefix));
        } catch (RedisException e) {
            log.warn("Trending unavailable, using static trie");
        }
        return results;
    } catch (Exception e) {
        log.error("Trie lookup failed", e);
        return Collections.emptyList();  // Graceful degradation
    }
}
```

---

9) Cost Estimates

Component	Sizing	Monthly Cost (Est.)
AKS	6 nodes × Standard_D4s_v3 (3 regions × 2 for HA)	~$1,500
Azure CDN	100TB egress, 1B requests	~$3,000
Redis	P1 (6GB) × 3 regions	~$750
Event Hubs	10 TUs, 7-day retention	~$700
Databricks	4 DBU-hours/day for batch	~$400
Blob Storage	100GB snapshots + logs	~$50
Front Door	1B requests	~$400
Total		**~$6,800/month**

---

10) Trade-offs Discussion

Decision	Option A	Option B	Choice	Reasoning
Data structure	Trie	Hash Map	**Trie**	5x memory savings for large vocabularies
Top-K storage	Pre-computed	On-demand	**Pre-computed**	O(1) vs O(subtree traversal) at query time
Update model	Real-time	Batch	**Batch + trending**	Simpler, lower cost; trending handles spikes
Personalization	Built-in	Separate	**Separate service**	Keeps suggestion service stateless
CDN caching	Aggressive	Conservative	**Aggressive (5-15min)**	80%+ hit rate worth slight staleness

---

11) 60-Second Explanation

> "Typeahead is a **read-heavy, latency-critical** system. I use an **in-memory Trie** with **pre-computed top-K suggestions at each node** for O(prefix-length) + O(1) lookups.
>
> The trie is built **hourly** from search logs aggregated via **Event Hubs → Databricks → Blob Storage**. Each suggestion service pod loads the same snapshot (~1-2GB).
>
> For **real-time trending**, I use a **Redis sorted set** with sliding window counts, merged at query time.
>
> **CDN caches 80%+ of requests** (popular prefixes like 'how to'). Client-side **debouncing** (100-200ms) reduces load by 5-10x.
>
> The system is **stateless** and **horizontally scalable**. With multi-region deployment behind **Front Door**, we achieve **<50ms p99** and handle **100K+ QPS** at peak."

Technologies Deep Dive

Technology	Role	Reference
Trie	Prefix lookup data structure	Tech Guide

Redis	Trending cache, sorted sets	Tech Guide
Kafka/Event Hubs	Query log streaming	Tech Guide
CDN	Edge caching for popular prefixes	Tech Guide
Bloom Filter	Quick prefix existence check	Tech Guide

Appendix: Trie Implementation

```
public class TypeaheadTrie {
    private final TrieNode root = new TrieNode();
    private static final int TOP_K = 10;

    static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        List<Suggestion> topSuggestions = new ArrayList<>();
        boolean isEndOfWord;
    }

    static class Suggestion implements Comparable<Suggestion> {
        String phrase;
        double score;

        @Override
        public int compareTo(Suggestion o) {
            return Double.compare(o.score, this.score); // Descending
        }
    }

    // Build trie with pre-computed top-K at each node
    public void build(List<Suggestion> allSuggestions) {
        // Sort all suggestions by score descending
        allSuggestions.sort(Comparator.comparingDouble(s -> -s.score));

        for (Suggestion suggestion : allSuggestions) {
            TrieNode node = root;
            for (char c : suggestion.phrase.toCharArray()) {
                node = node.children.computeIfAbsent(c, k -> new TrieNode());
                // Add to top-K if not full or better than worst
                if (node.topSuggestions.size() < TOP_K) {
                    node.topSuggestions.add(suggestion);
                }
            }
            node.isEndOfWord = true;
        }
    }

    // O(prefix length) lookup
    public List<Suggestion> getSuggestions(String prefix) {
```

```
        TrieNode node = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            node = node.children.get(c);
            if (node == null) {
                return Collections.emptyList();
            }
        }
        return node.topSuggestions;
    }
}
```

Appendix: Client-Side Debouncing

```
// Debounce to avoid flooding server on every keystroke
function debounce(func, wait) {
    let timeout;
    return function executedFunction(...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() => func.apply(this, args), wait);
    };
}

const fetchSuggestions = debounce(async (prefix) => {
    if (prefix.length < 2) return; // Min 2 chars

    // Check local cache first
    const cached = localStorage.getItem(`suggest:${prefix}`);
    if (cached) {
        const { suggestions, ts } = JSON.parse(cached);
        if (Date.now() - ts < 300000) { // 5 min cache
            renderSuggestions(suggestions);
            return;
        }
    }

    const response = await fetch(`/suggest?q=${encodeURIComponent(prefix)}`);
    const suggestions = await response.json();

    // Cache locally
    localStorage.setItem(`suggest:${prefix}`,
        JSON.stringify({ suggestions, ts: Date.now() }));

    renderSuggestions(suggestions);
}, 150); // 150ms debounce

searchInput.addEventListener('input', (e) => fetchSuggestions(e.target.value));
```

6. Design Uber/Lyft Ride-Sharing Platform

0) Requirements & Sizing

Functional

- **Real-time location tracking**: Drivers update location every 2-5 seconds
- **Ride matching**: Match riders with nearest available drivers
- **ETA estimation**: Accurate pickup and dropoff time predictions
- **Surge pricing**: Dynamic pricing based on supply/demand
- **Payments**: Pre-authorization, capture after ride, driver payouts
- **Ride lifecycle**: Request → Match → Pickup → In-progress → Complete
- **Notifications**: Push notifications, SMS fallback
- **Fraud detection**: GPS spoofing, fake rides, collusion detection

Non-functional

Metric	Target
Scale	**10M rides/day**, 500+ cities
Online drivers	**1-2M concurrent** drivers sending location
Location write rate	**400K-1M writes/sec** (2-5 sec intervals)
Matching latency	< **5 seconds** to find a driver
ETA accuracy	**±2-3 minutes** in normal traffic
Payment consistency	**Strongly consistent** (CP)
Availability	**99.9%** for ride services
Global	Multi-region, per-city isolation

Capacity

```
Location Updates:
- 2M drivers × 1 update/3 sec = 667K writes/sec
- Each update: ~200 bytes (driver_id, lat, lng, timestamp, heading, speed)
- Hot data: 2M × 200B = 400MB in Redis GEO

Rides:
```

```
- 10M rides/day = ~116 rides/sec average
- Peak: 10x = 1,160 rides/sec
- Each ride record: ~2KB

Storage:
- Redis (hot): ~500MB per city cluster
- PostgreSQL (rides): 10M × 2KB × 365 = ~7TB/year
- Analytics (cold): Petabytes over time
```

Design Reasoning

> **Core philosophy**: Prioritize real-time driver-rider matching with sub-5-second latency by keeping live driver locations in an in-memory geospatial index, while using strongly consistent storage for financial transactions and ride state.

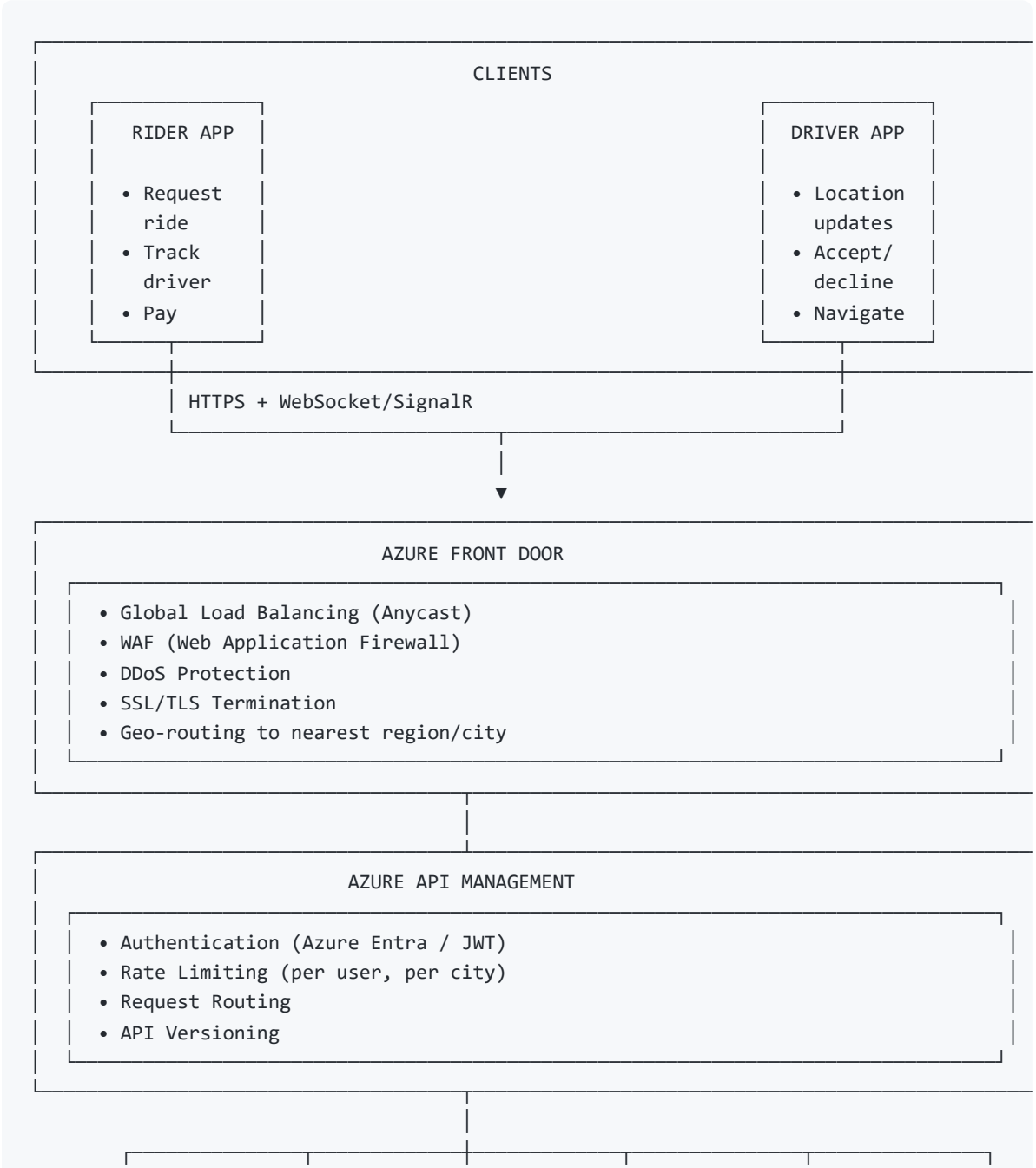
Decision	Choice	Why	Alternative Considered
Location index	Redis GEO (in-memory)	Sub-ms geospatial queries for 2M concurrent drivers; GEORADIUS finds nearest drivers instantly	PostGIS — disk-based queries too slow for 667K location writes/sec
Ride database	Azure PostgreSQL	ACID transactions ensure ride state and payment records are never inconsistent or lost	Cosmos DB — eventual consistency risks double-charges or lost ride records
Driver dispatch	Azure Service Bus (scheduled messages)	Built-in scheduled delivery for offer expiry timeouts; dead-letter for failed dispatches	Direct HTTP to drivers — no retry/timeout guarantees, cascade failures
Real-time push	Azure SignalR Service	Managed WebSocket connections push driver location and ride updates to riders in real-time	HTTP polling — too slow for live map tracking, wastes bandwidth
Matching strategy	Wave-based offering (nearest first)	Offers ride to closest drivers sequentially, reducing pickup time and driver idle distance	Broadcast to all nearby — multiple drivers accept simultaneously, causing conflicts
Surge pricing	Azure Functions (serverless)	Spiky compute demand during surge events; pay-	Always-on AKS pods — wasteful since surge

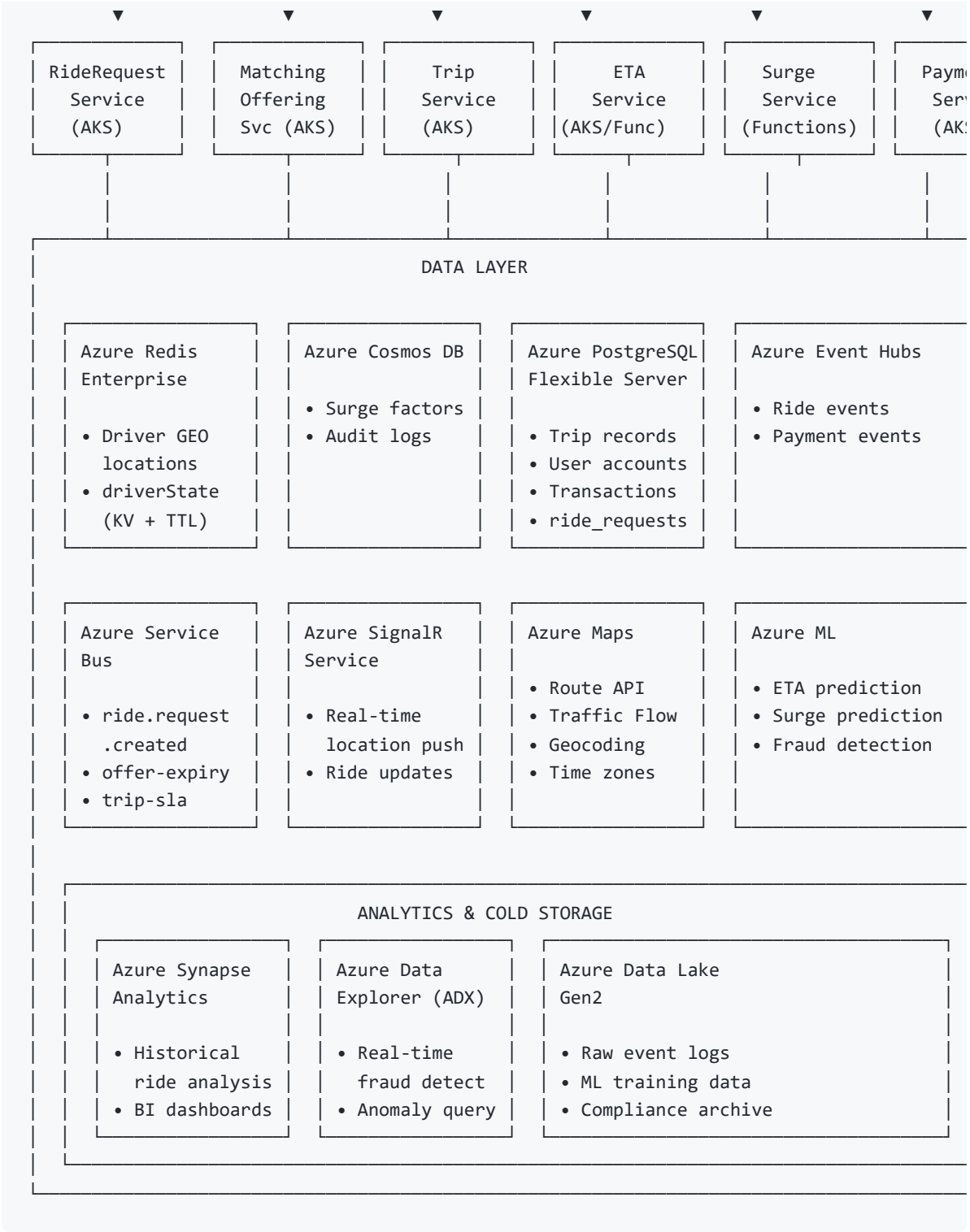
		per-invocation avoids idle capacity costs	calculations are infrequent and bursty
Maps and routing	Azure Maps API	Native Azure integration for ETA, traffic-aware routing, and geocoding	Google Maps API — external dependency, egress costs, vendor lock-in

---

1) High-Level Architecture

Complete Azure Architecture





Components Summary

Component	Azure Service	Purpose
Global Entry	**Azure Front Door**	Anycast DNS, WAF, geo-routing
API Gateway	**Azure API Management**	Auth, rate limiting, routing

Compute	**Azure Kubernetes Service (AKS)**	Microservices per city
Ride Ingestion	**RideRequestService (AKS)**	Persist request, produce to Service Bus
Matching	**MatchingOfferingService (AKS)**	Wave-based driver offering, CAS state
Trip Management	**TripService (AKS)**	Trip persistence, state machine
Pricing	**PricingService (AKS)**	Fare estimate & finalization
Real-time Location	**Azure Redis Enterprise**	GEO index, driver state (KV+Geo, TTL)
Event Streaming	**Azure Event Hubs**	Ride, payment events
Driver Dispatch	**Azure Service Bus**	ride.request.created, offer-expiry, trip-sla-timeout (scheduled)
Real-time Push	**Azure SignalR Service**	WebSocket to riders/drivers
Routing & Maps	**Azure Maps**	Route API, traffic, ETA
ML Models	**Azure ML Managed Endpoints**	ETA adjustment, fraud detection
Surge Compute	**Azure Functions**	Serverless surge calculation
Ride Database	**Azure PostgreSQL**	ACID transactions for rides
Document Store	**Azure Cosmos DB**	Driver status, surge factors
Notifications	**Azure Notification Hubs**	APNs + FCM push
Analytics	**Azure Synapse + ADX**	Real-time and batch analytics

---

API Design

WebSocket Events

Driver → Server

Event	Payload	Description
`driver.location.update`	`{ driverId, lat, lon, heading, speed, status }`	Location heartbeat (every 3-5s). `status` : `available` / `assigned` / `in_trip`
`driver.ride.accept`	`{ driverId, tripRequestId }`	Driver accepts an offered ride
`driver.ride.decline`	`{ driverId, tripRequestId }`	Driver declines an offered ride
`driver.ride.arrived`	`{ tripId }`	Driver arrived at pickup location
`driver.ride.pickup`	`{ tripId }`	Rider picked up, trip starts
`driver.ride.dropoff`	`{ tripId }`	Rider dropped off, trip ends

User → Server

Event	Payload	Description
`user.ride.request`	`{ userId, sourceLocation, targetLocation }`	Request a new ride

Server → User

Event	Payload	Description
`user.ride.ack`	`{ tripRequestId }`	Immediate ack after ride request persisted
`user.ride.assigned`	`{ tripRequestId, tripId, driver, pickupETA, dropoffETA }`	Driver matched and assigned
`user.ride.driverLocation`	`{ tripId, location }`	Real-time driver location during trip
`user.ride.update`	`{ tripId, status }`	Status change: `driverArrived` / `start` / `completed`
`user.ride.noDriverFound`	`{ tripRequestId }`	No driver available (SLA exceeded)

Server → Driver

Event	Payload	Description
`driver.ride.request`	`{ driverId, tripRequestId, sourceLocation, targetLocation, fare, expiresAt }`	Ride offer (wave-based, expires in 2-5s)
`driver.ride.assigned`	`{ driverId, tripRequestId, tripId }`	Confirmed assignment after accept
`driver.ride.offerClosed`	`{ driverId, tripRequestId }`	Offer closed (another driver accepted)

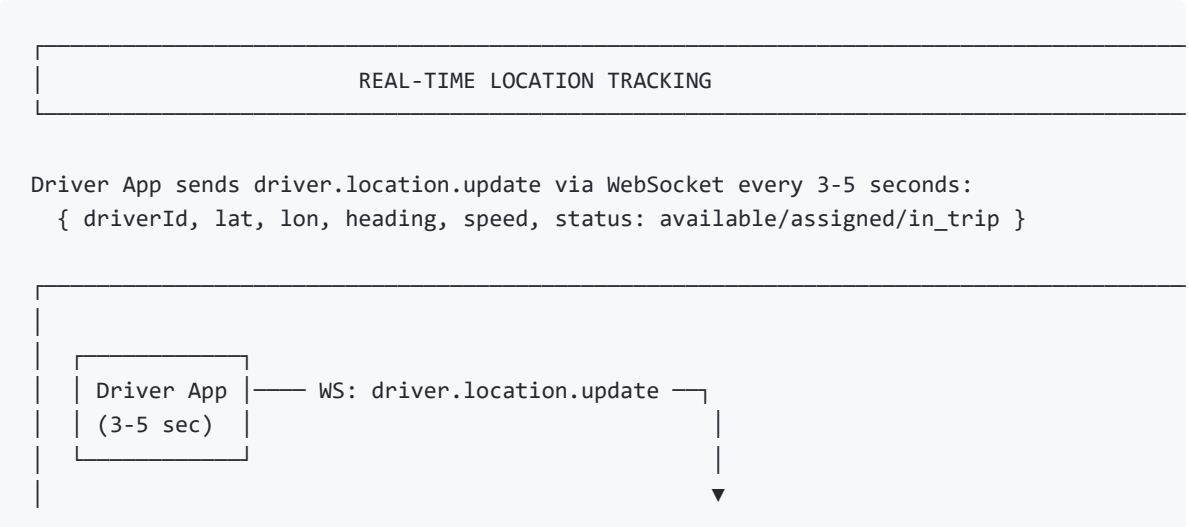
REST APIs

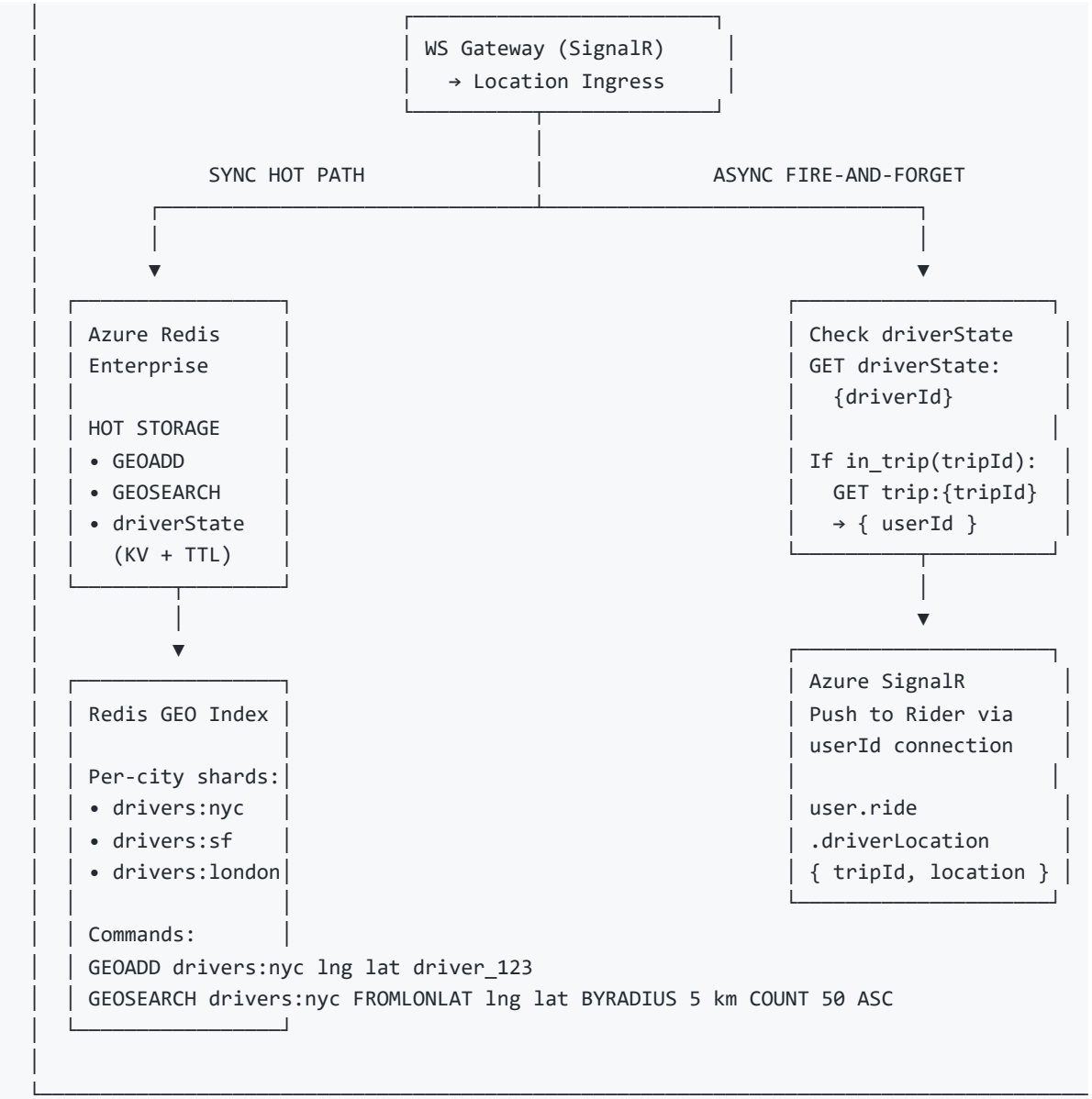
Payment

Method	Endpoint	Request	Response
`POST`	`/api/v1/user/payment/authorize`	`{ tripId, paymentMethodToken }`	`{ authorizationId, status }`
`POST`	`/api/v1/user/payment/capture`	`{ tripId, authorizationId }`	`{ receiptId, status }`

2) Real-Time Location Tracking (HOT PATH)

Architecture





Scale:

- 2M drivers × 1 update/3 sec = 667K writes/sec
- Redis Enterprise: 1M+ ops/sec per cluster
- Shard by city/geohash prefix

> 💡 **Why WebSocket, not HTTP polling?** At 667K updates/sec, HTTP polling would mean 667K TCP connections/sec (handshake overhead). WebSocket keeps one persistent connection per driver — orders of magnitude more efficient.

> 💡 **Why fire-and-forget for rider push?** The hot path (Redis write) must return in <5ms to keep up with 667K/sec. If SignalR push is slow or the rider's connection is flaky, it must not block the location write. The side path is async — dropping one push out of hundreds is invisible to the rider.

Location Update Flow

Step	From	To	Action	Sync/Async
1	Driver App	WS Gateway (SignalR)	`driver.location.update` via WebSocket	SYNC
2	WS Gateway	Location Ingress	Validate, transform	SYNC
3	Location Ingress	Redis GEO + KV	GEOADD to city shard + SET driverState (TTL)	SYNC
4	Location Ingress	Redis KV	**Fire-and-forget:** GET `driverState:{driverId}` — check if `in_trip(tripId)`	ASYNC
5	Location Ingress	Redis KV	If in_trip: GET `trip:{tripId}` → resolve `userId`	ASYNC
6	Location Ingress	SignalR → User App	Push `user.ride.driverLocation { tripId, location }` to rider's connection by `userId`	ASYNC

Redis GEO Commands

```
# Add/Update driver location
GEOADD "drivers:nyc" -73.985428 40.748817 "driver_123"

# Find nearest drivers within 5km radius
GEOSEARCH "drivers:nyc" FROMLONLAT -73.985428 40.748817 BYRADIUS 5 km COUNT 50 ASC W

# Result:
# 1) "driver_456" "0.3"   (300 meters)
# 2) "driver_789" "0.8"   (800 meters)
# 3) "driver_321" "1.2"   (1.2 km)

# Remove driver (went offline)
ZREM "drivers:nyc" "driver_123"

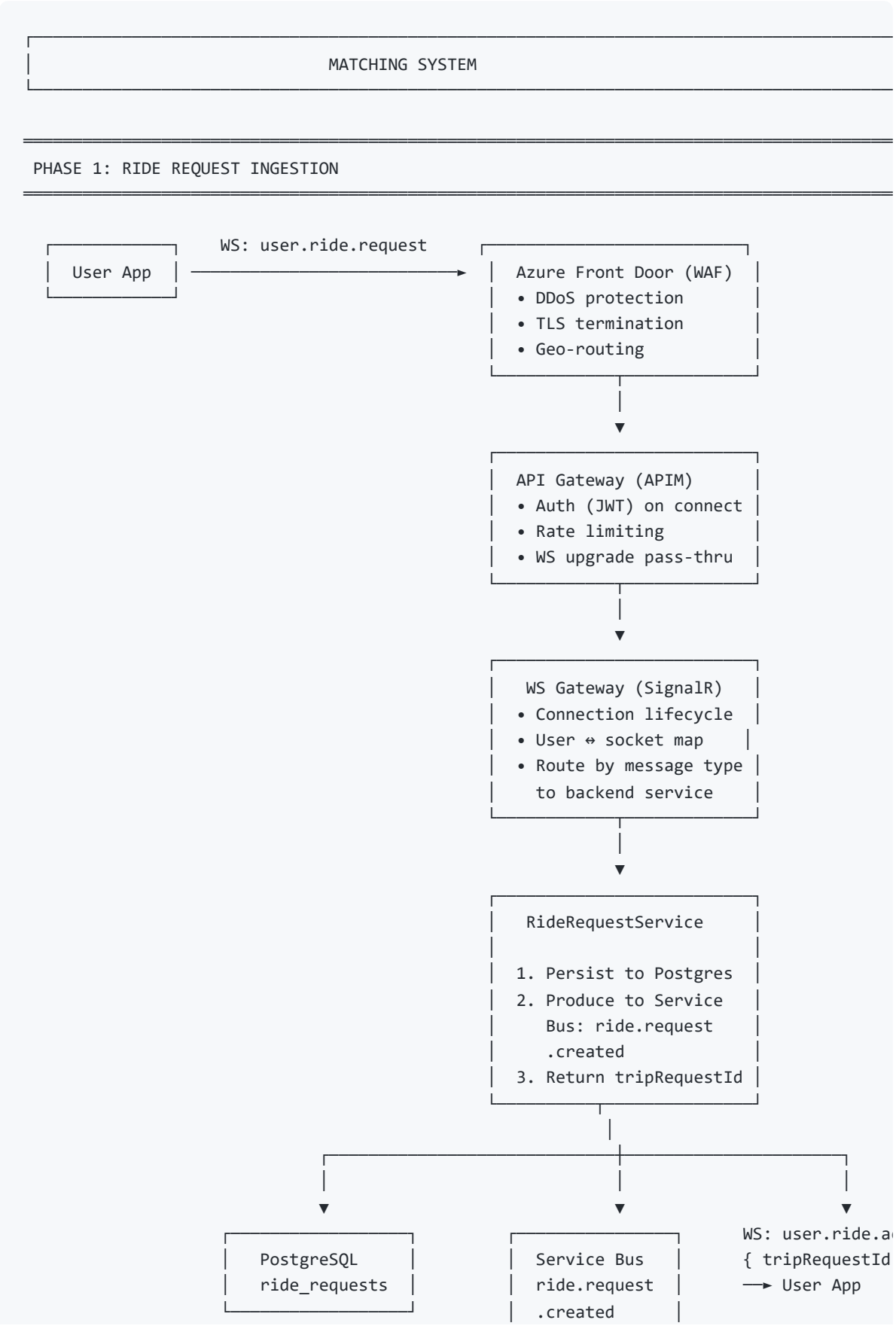
# Trip-to-rider mapping (set when trip is created, TTL = max trip duration)
SET "trip:trip_abc" '{"userId":"user_456","driverId":"driver_123"}' EX 7200

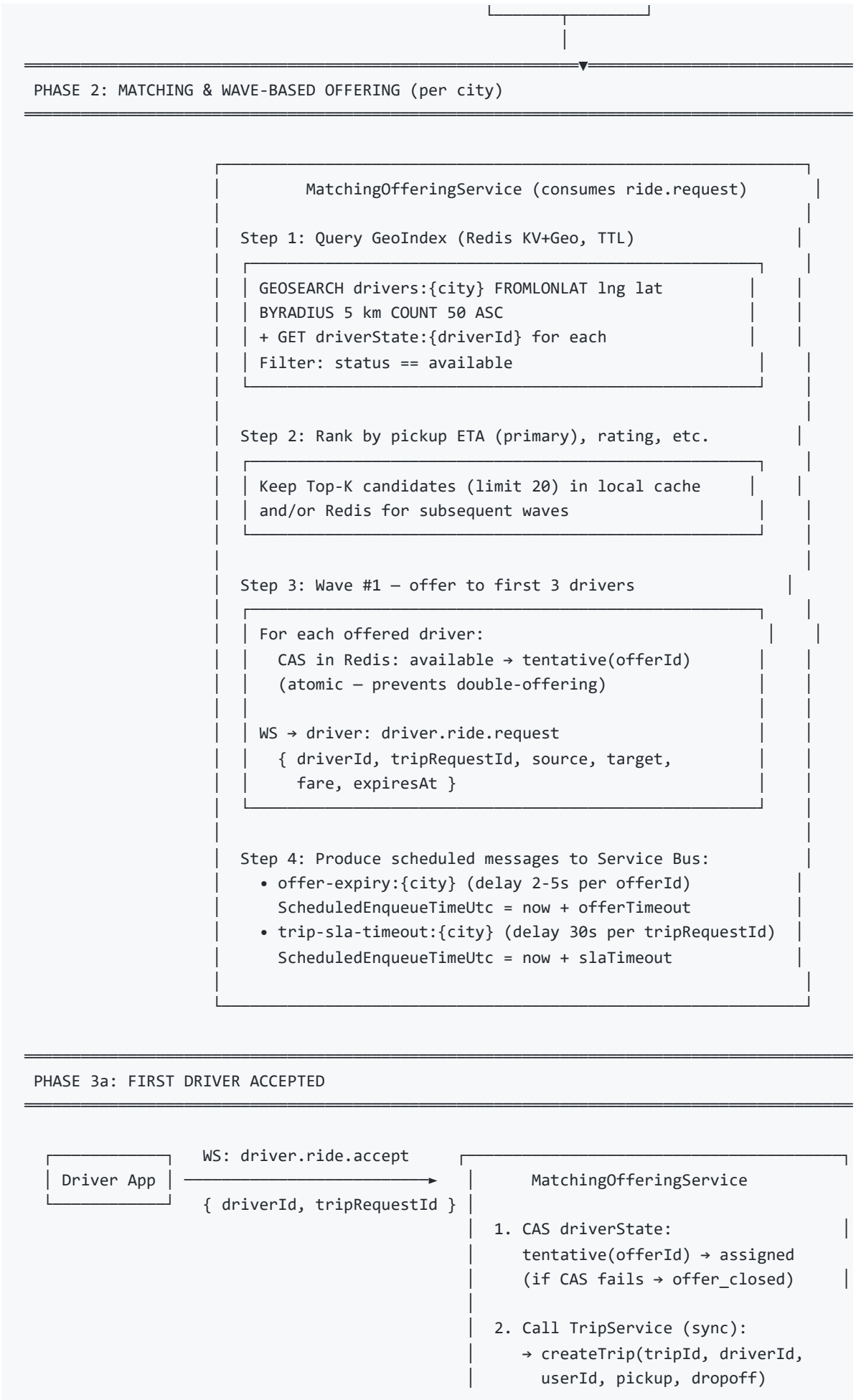
# Location push routing: check if driver is in active trip
GET "driverState:driver_123"          # → "in_trip:trip_abc"
GET "trip:trip_abc"                    # → {"userId":"user_456"} → push to user_456
```

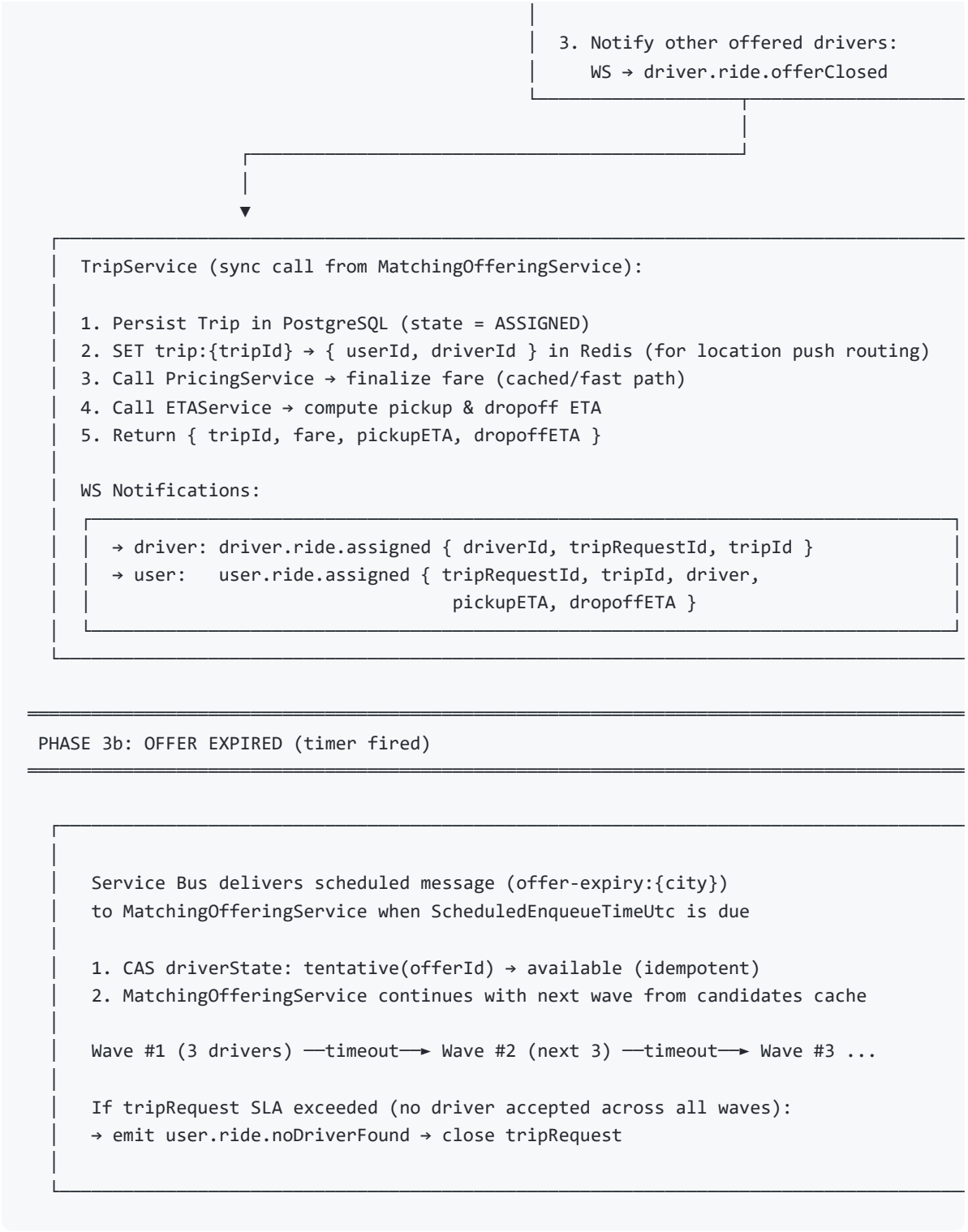
---

3) Matching System (City-Local, Wave-Based Offering)

Architecture




PHASE 2: MATCHING & WAVE-BASED OFFERING (per city)
MatchingOfferingService (consumes ride.request)
Step 1: Query GeoIndex (Redis KV+Geo, TTL)
GEOSEARCH drivers:{city} FROMLONLAT lng lat
BYRADIUS 5 km COUNT 50 ASC
+ GET driverState:{driverId} for each
Filter: status == available
Step 2: Rank by pickup ETA (primary), rating, etc.
Keep Top-K candidates (limit 20) in local cache
and/or Redis for subsequent waves
Step 3: Wave #1 — offer to first 3 drivers
For each offered driver:
CAS in Redis: available → tentative(offerId)
(atomic — prevents double-offering)
WS → driver: driver.ride.request
{ driverId, tripRequestId, source, target,
fare, expiresAt }
Step 4: Produce scheduled messages to Service Bus:
• offer-expiry:{city} (delay 2-5s per offerId)
ScheduledEnqueueTimeUtc = now + offerTimeout
• trip-sla-timeout:{city} (delay 30s per tripRequestId)
ScheduledEnqueueTimeUtc = now + slaTimeout
PHASE 3a: FIRST DRIVER ACCEPTED
Driver App
WS: driver.ride.accept
{ driverId, tripRequestId }
MatchingOfferingService
1. CAS driverState:
tentative(offerId) → assigned
(if CAS fails → offer_closed)
2. Call TripService (sync):
→ createTrip(tripId, driverId,
userId, pickup, dropoff)



Step-by-Step Flow: Ride Request Ingestion (Phase 1)

Step	From	To	Action	Data
1	User App	Azure Front Door	WS connection via Anycast	`user.ride.r`
2	Front Door	API Gateway (APIM)	WAF + DDoS filtering, TLS	Same payloa

			termination, geo-route	
3	API Gateway	WS Gateway (SignalR)	Auth (JWT) on connect, rate limit, WS upgrade pass-thru	Same payloa context
4	WS Gateway	RideRequestService	Route by message type (`user.ride.request`)	`{ userId, sourceLocat targetLocat`
5	RideRequestService	PostgreSQL	Persist ride request (within tx)	INSERT `ride_reques`
6	RideRequestService	Service Bus	Produce event (after tx commit)	`ride.reques { tripReque userId, pic dropoff, ci`
7	WS Gateway	User App	Ack immediately after tx commit	`user.ride.a tripRequest`

Step-by-Step Flow: Matching & Wave Offering (Phase 2)

Step	From	To	Action
1	Service Bus	MatchingOfferingService	Consume `ride.request.created` (per city)
2	MatchingOfferingService	Redis GEO + KV	GEOSEARCH nearby + GET driverState for each
3	MatchingOfferingService	—	Rank by pickup ETA (primary), rating, acceptance rate; keep Top-20 in cache
4	MatchingOfferingService	Redis	CAS per wave driver: `available → tentative(offerId)`

			(atomic, prevents double-offering)
5	MatchingOfferingService	WS Gateway → Driver Apps	Send offers to wave (3 drivers)
6	MatchingOfferingService	Service Bus (scheduled)	Produce scheduled messages: offer-expiry (delay 2-5s) + trip-sla-timeout (delay 30s) via `ScheduledEnqueueTimeUtc`

Step-by-Step Flow: First Driver Accepted (Phase 3a)

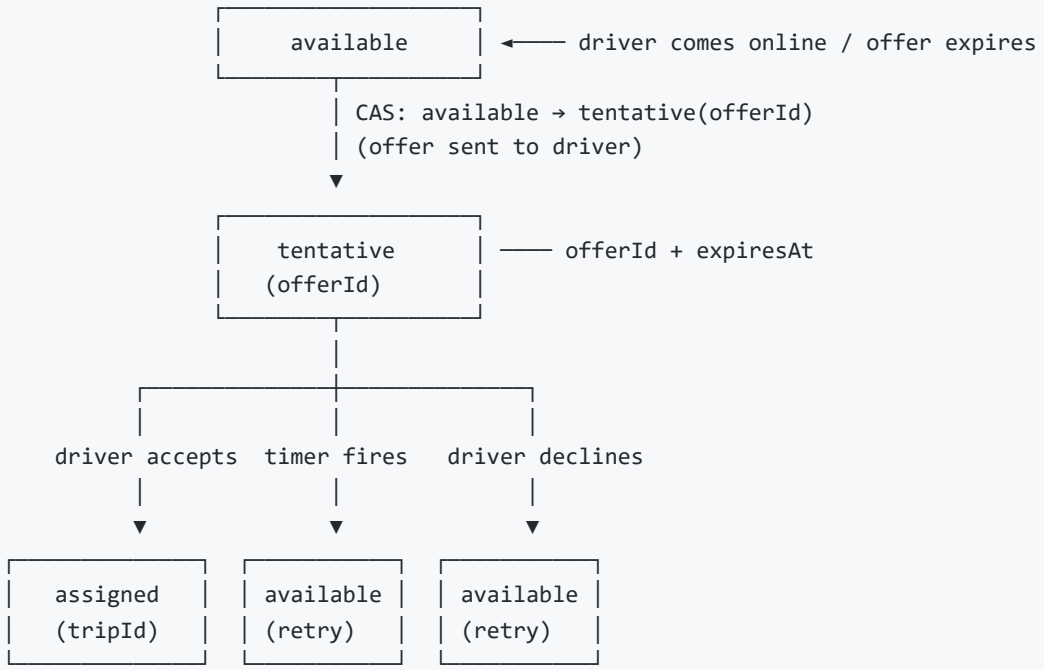
Step	From	To	Action	D
1	Driver App	WS Gateway	Accept ride offer	`{ d }`
2	WS Gateway	MatchingOfferingService	Route by message type	S
3	MatchingOfferingService	Redis	CAS: `tentative(offerId) → assigned(tripId)` (if fail → offer_closed)	`{`
4	MatchingOfferingService	TripService	Call `createTrip` (sync)	`{ u }`
4a	TripService	PostgreSQL	Persist Trip (state = ASSIGNED)	IN
4b	TripService	Redis	SET `trip:{tripId}` → `{ userId, driverId }` (for location push routing)	T

4c	TripService	PricingService	Finalize fare (cached/fast path)	C e
4d	TripService	ETAService	Compute pickup + dropoff ETA	{ }
5	MatchingOfferingService	WS Gateway → other drivers	Notify: offer closed / assigned elsewhere	{ t
5b	MatchingOfferingService	Redis	CAS other offered drivers: `tentative(offerId) → available`	P
6	WS Gateway	Driver App	Assignment confirmation	d t
7	WS Gateway	User App	Full match notification	t d d

Step-by-Step Flow: Offer Expired (Phase 3b)

Step	From	To	Action
1	Service Bus (scheduled)	MatchingOfferingService	Deliver offer-expiry message when `ScheduledEnqueueTimeUtc` is due
2	MatchingOfferingService	Redis	CAS: `tentative(offerId) → available` (idempotent)
3	MatchingOfferingService	—	Continue with next wave from candidates cache (Top-20)
4	MatchingOfferingService	WS Gateway → User App	**If SLA exceeded:** no driver found, close tripRequest

Driver State Machine (Redis CAS)



Key Design Decisions

> 💡 **Why Redis GEO, not PostGIS?** PostGIS gives richer spatial queries but runs on disk. Redis GEO (`GEOADD` / `GEOSEARCH`) runs entirely in-memory at **1M+ ops/sec** — critical for the 667K writes/sec location hot path. The trade-off: Redis is ephemeral (no durability), but location data is transient by nature — a 3-second-old location is already stale.

> 💡 **Why CAS (Compare-And-Swap), not distributed locks?** A distributed lock (e.g., Redlock) requires acquiring and releasing — if the process crashes while holding the lock, the driver is stuck. CAS is atomic and stateless: `available → tentative(offerId)` succeeds or fails in one call. No lock cleanup needed. If the process crashes, the offer-expiry timer (Service Bus scheduled message) resets the driver to `available` automatically.

> 💡 **Why wave-based offering (3 drivers), not broadcast to all?** Broadcasting to 20 drivers creates a thundering herd — 19 drivers get "offer closed" immediately after 1 accepts. Waves of 3 give each driver a fair window (2-5s), reduce wasted WS messages, and the Top-K cache means we don't re-query Redis between waves.

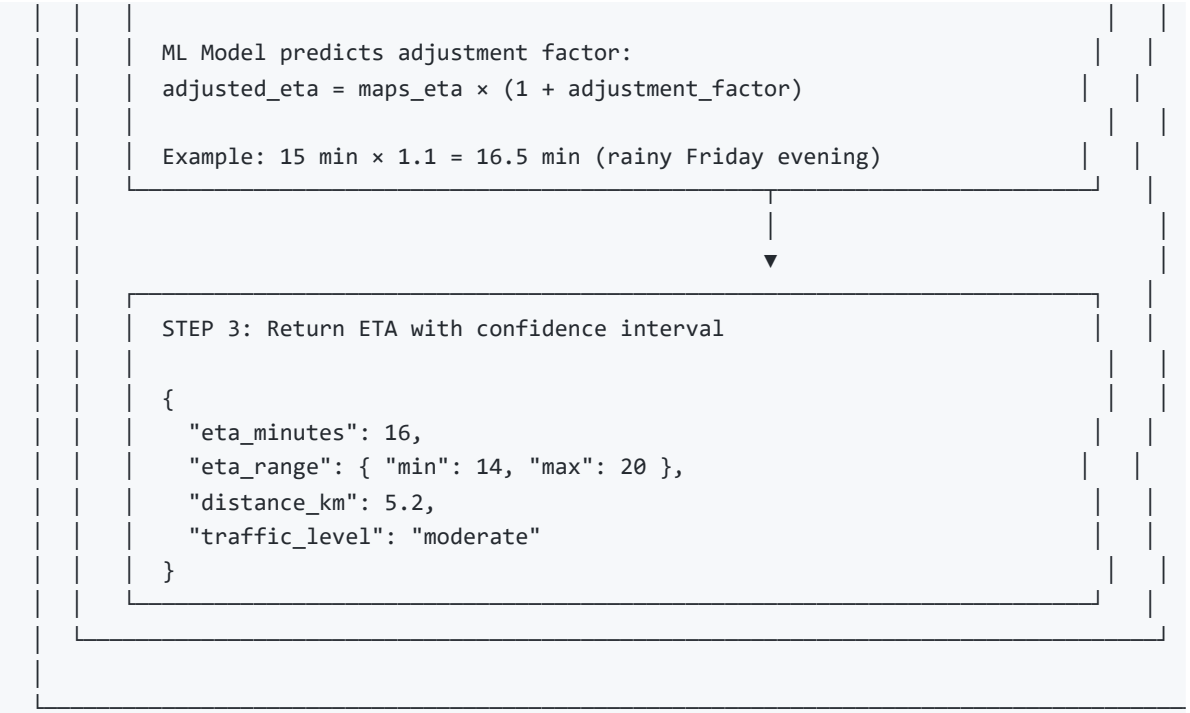
> 💡 **Why Service Bus (not Event Hubs) for ride events?** Ride requests are ~116/sec (not 667K/sec like locations). Service Bus handles this easily per city queue. The killer feature: **ScheduledEnqueueTimeUtc** — native delayed delivery for offer expiry and SLA timers. Event Hubs has no scheduling mechanism, so you'd need a separate timer infrastructure.

> 💡 **Why sync TripService call (not Service Bus event)?** The rider is waiting. An async `trip.created` event means eventual consistency — the rider might wait extra seconds for TripService to consume, persist, and compute ETA. At ~116 trips/sec, a sync call adds <50ms and gives the rider instant confirmation with fare and ETA.

4) ETA Service (Azure Maps + ML)

Architecture





Azure Maps APIs Used:

- Route API: Directions with real-time traffic
- Traffic Flow API: Current traffic conditions on road segments
- Weather API: Rain, snow, fog affecting travel
- Timezone API: Handle cross-timezone rides
- Geocoding API: Address to lat/lng conversion

Step-by-Step Flow

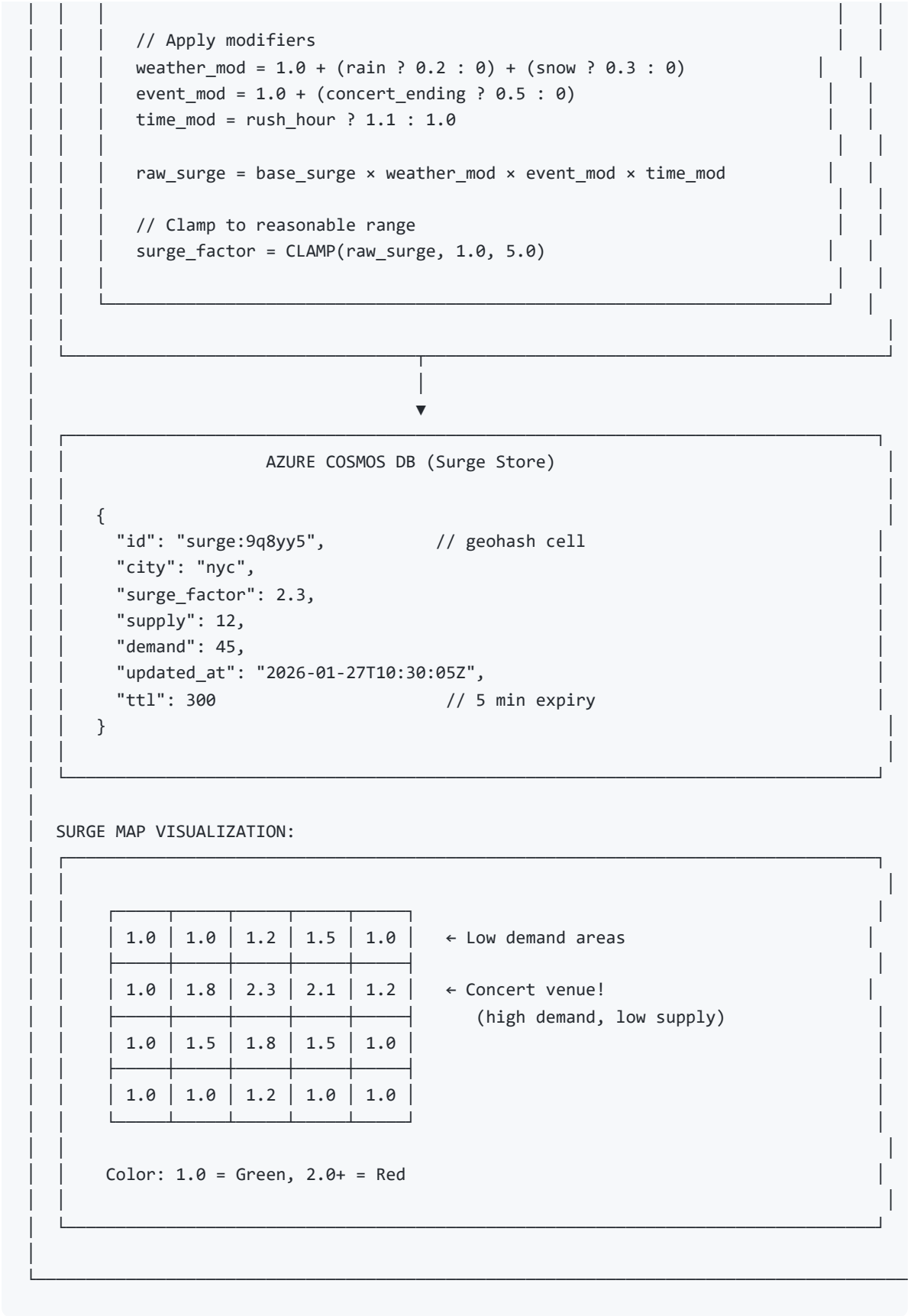
Step	From	To	Action	Data
1	Caller (MatchingOfferingService / PricingService)	ETA Service	Request ETA	`{ origin: {lat, lng}, destination: {lat, lng} }`
2	ETA Service	Azure Maps Route API	Get base travel time + traffic delay	`travelTimeInSeconds: 720, trafficDelayInSeconds: 180`
3	ETA Service	Azure ML Endpoint	Get ML adjustment factor	Features: `{ time_of_day, weather, historical_accuracy, events, surge }` → `adjustment_factor: 0.1`

4	ETA Service	—	Compute adjusted ETA	`adjusted_eta = maps_eta × (1 + adjustment_factor)` = 16.5 min
5	ETA Service	Caller	Return ETA with confidence	`{ eta_minutes: 16, eta_range: {min:14, max:20}, distance_km: 5.2, traffic_level: "moderate" }`

5) Surge Pricing System

Architecture





Step-by-Step Flow

Step	From	To	Action	Data

1	Azure Functions (timer: 5-10s)	Redis GEO + PostgreSQL	Query supply (GEOSEARCH per geohash cell) + demand (ride_requests count)	`{ cell, supply, demand }`
2	Stream Analytics	—	Tumbling window (5-10s) per geohash cell (precision 6 ≈ 600m)	Aggregate `supply` and `demand` per cell
3	Stream Analytics	—	Calculate surge: `(demand / supply) × weather_mod × event_mod × time_mod`, clamp to [1.0, 5.0]	`{ surge_factor, supply, demand }`
4	Stream Analytics	Cosmos DB	Write surge factor for cell (TTL: 5 min)	`{ cell: "surge:9q8yy5", surge_factor: 2.3, ttl: 300 }`
5	RideRequestService / PricingService	Cosmos DB	Read surge factor for fare estimate/finalization	`{ cell } → { surge_factor }`

---

6) Payment System (Strongly Consistent)

> 💡 **Why PostgreSQL (CP), not Cosmos DB (AP)?** Money requires strong consistency. If a payment capture is acknowledged, it must be durable — no "eventual consistency" for financial transactions. PostgreSQL gives ACID guarantees. Cosmos DB's eventual consistency could result in double-charges or lost payouts.

> 💡 **Why authorize/capture (two-phase), not charge-on-complete?** Authorize on assignment validates the rider can pay *before* the driver drives. If the card is declined, we cancel the match immediately — not after a 30-minute trip.

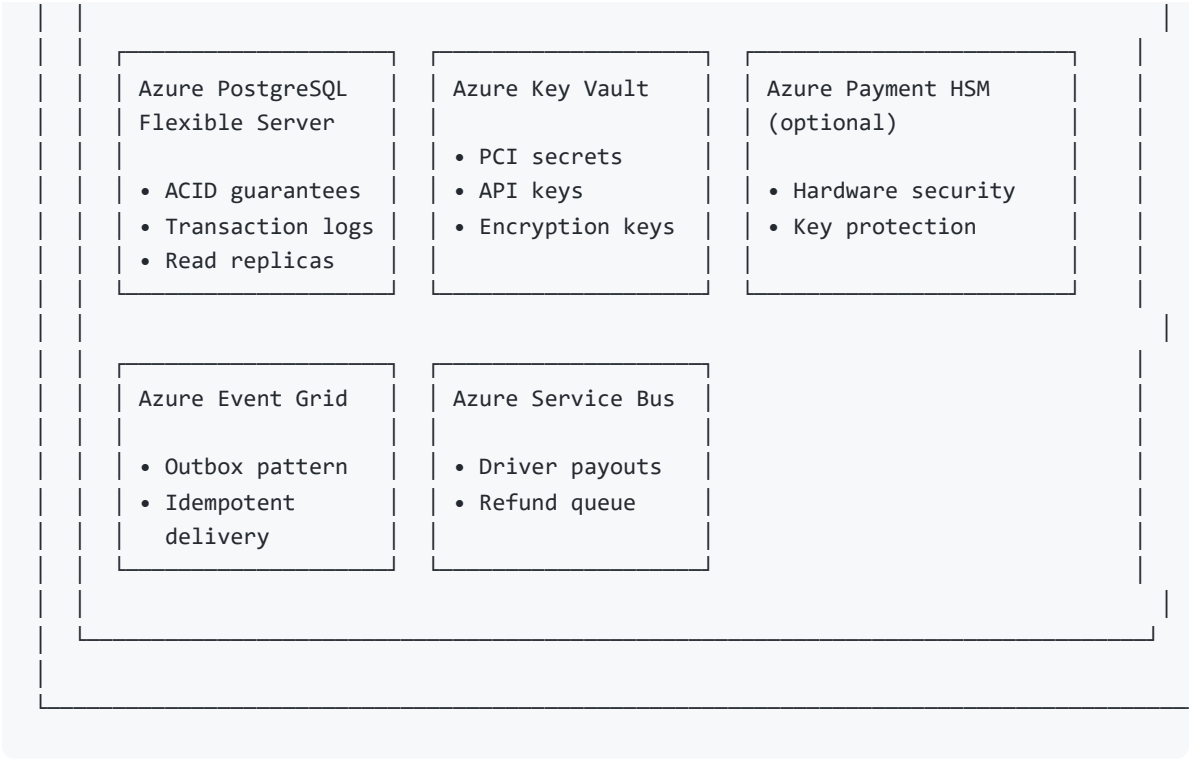
Architecture

```
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│                                PAYMENT SYSTEM (CP - Strongly Consistent)
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────┘

This MUST be CP (consistent + partition-tolerant). Use Azure PostgreSQL.

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                              │
│  PAYMENT FLOW:                                                                               │
│                                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────────────────┐    │
│  │                                                                                      │    │
│  │  1. TRIP ASSIGNED → Authorize payment (hold estimated amount)                        │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  POST /api/v1/user/payment/authorize                                         │    │    │
│  │  │  Request: { tripId, paymentMethodToken }                                     │    │    │
│  │  │  Response: { authorizationId, status }                                       │    │    │
│  │  │                                                                              │    │    │
│  │  │  INSERT INTO transactions (trip_id, amount, status, type)                    │    │    │
│  │  │  VALUES ('trip_123', 25.00, 'authorized', 'preauth')                         │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                  ↓                                                   │    │
│  │  2. TRIP COMPLETED → Capture actual amount                                           │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  POST /api/v1/user/payment/capture                                           │    │    │
│  │  │  Request: { tripId, authorizationId }                                        │    │    │
│  │  │  Response: { receiptId, status }                                             │    │    │
│  │  │                                                                              │    │    │
│  │  │  Final fare calculated:                                                      │    │    │
│  │  │    base_fare + (distance × rate) + (time × rate) × surge_factor              │    │    │
│  │  │    = $5 + (8km × $1.50) + (20min × $0.25) × 1.5 = $27.75                     │    │    │
│  │  │                                                                              │    │    │
│  │  │  BEGIN TRANSACTION;                                                          │    │    │
│  │  │    UPDATE transactions SET amount = 27.75, status = 'captured'               │    │    │
│  │  │    WHERE trip_id = 'trip_123';                                               │    │    │
│  │  │                                                                              │    │    │
│  │  │    INSERT INTO driver_payouts (driver_id, ride_id, amount)                   │    │    │
│  │  │    VALUES (456, 123, 22.20);  // 80% to driver                               │    │    │
│  │  │                                                                              │    │    │
│  │  │    INSERT INTO outbox (event_type, payload)                                  │    │    │
│  │  │    VALUES ('payment_captured', '{ride_id: 123, amount: 27.75}');             │    │    │
│  │  │  COMMIT;                                                                     │    │    │
│  │  │                                                                              │    │    │
│  │  │  Call Payment Gateway: capture(auth_code, $27.75)                            │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                  ↓                                                   │    │
│  │  3. ASYNC: Process outbox events                                                     │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  Azure Event Grid polls outbox table                                         │    │    │
│  │  │  → Emit 'payment_captured' event                                             │    │    │
│  │  │  → Service Bus: Queue payout to driver (weekly batch)                        │    │    │
│  │  │  → Notification: Send receipt to rider                                       │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                                                                      │    │
│  └──────────────────────────────────────────────────────────────────────────────────────┘    │
│                                                                                              │
│  AZURE SERVICES:                                                                             │
│  ┌──────────────────────────────────────────────────────────────────────────────────────┐    │
```



Step-by-Step Flow

Authorization (on trip assignment):

Step	From	To	Action	Data	S
1	User App	Payment Service	`POST /user/payment/authorize`	`{ tripId, paymentMethodToken }`	S
2	Payment Service	PostgreSQL	INSERT transaction (preauth, pending)	`{ trip_id, amount, status: 'authorized' }`	S
3	Payment Service	Payment Gateway	Authorize hold on card	Estimated fare amount	S
4	Payment Service	User App	Return authorization	`{ authorizationId, status }`	S

Capture (on trip completion):

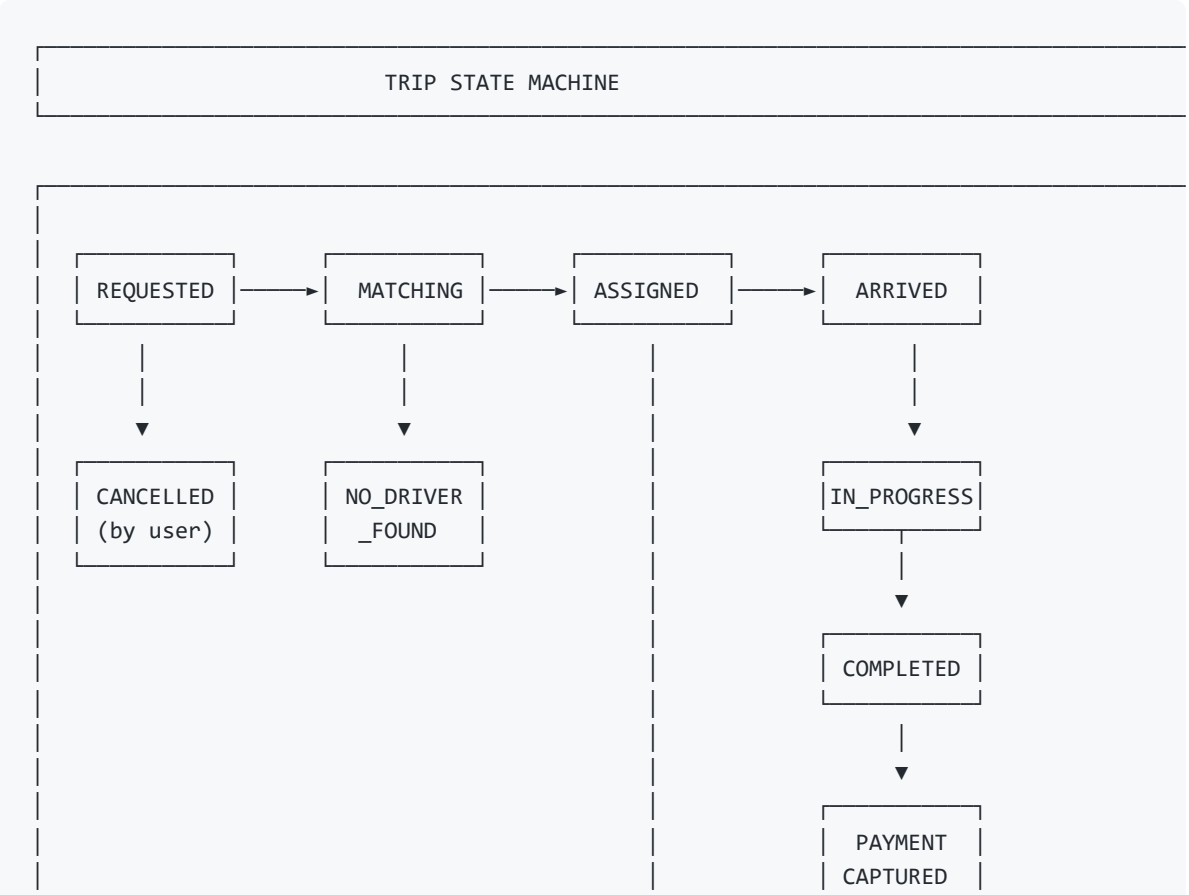
Step	From	To	Action	Data	Sync
5	User App	Payment Service	`POST /user/payment/capture`	`{ tripId, authorizationId }`	SYN

6	Payment Service	PricingService	Calculate final fare	`base + (dist × rate) + (time × rate) × surge`	SYN
7	Payment Service	PostgreSQL	BEGIN TX: update transaction + insert driver_payout + insert outbox	`{ amount: final, status: 'captured' }`	SYN
8	Payment Service	Payment Gateway	Capture actual amount	Final fare amount	SYN
9	Payment Service	User App	Return receipt	`{ receiptId, status }`	SYN
10	Outbox poller	Event Grid → Service Bus	Process outbox events	Driver payout queue + receipt notification	**ASY**

---

7) Ride Lifecycle Service (TripService)

State Machine



```
                                                  │                 └──────────┘
  │
  │  Triggers:
  │  ──────────────────────────────────────────────────────────────────────────────────────────
  │  REQUESTED → MATCHING    : RideRequestService persists + produces ride.request.created
  │  MATCHING  → ASSIGNED    : MatchingOfferingService: driver accepted, calls TripService
  │  MATCHING  → NO_DRIVER   : SLA timer exceeded, no driver accepted across all waves
  │  ASSIGNED  → ARRIVED     : driver.ride.arrived via WS
  │  ARRIVED   → IN_PROGRESS : driver.ride.pickup via WS
  │  IN_PROGRESS → COMPLETED : driver.ride.dropoff via WS
  │  COMPLETED → PAYMENT_CAPTURED : POST /user/payment/capture succeeds
  │
  │  WS notifications on each transition:
  │    → user:   user.ride.update { tripId, status }
  │    → driver: driver.ride.assigned (on ASSIGNED only)
  │
  └──────────────────────────────────────────────────────────────────────────────────────────────

TRIP RECORD (PostgreSQL — persisted by TripService on sync call from MatchingOfferingServ
  ┌──────────────────────────────────────────────────────────────────────────────────────────────
  │  {
  │    "trip_id": "trip_abc123",
  │    "trip_request_id": "req_xyz789",
  │    "user_id": "user_456",
  │    "driver_id": "driver_789",
  │    "status": "ASSIGNED",
  │    "pickup": { "lat": 40.7128, "lng": -74.0060 },
  │    "dropoff": { "lat": 40.7580, "lng": -73.9855 },
  │    "surge_factor": 1.5,
  │    "estimated_fare": 25.00,
  │    "final_fare": null,
  │    "pickup_eta": "2026-01-27T10:08:00Z",
  │    "dropoff_eta": "2026-01-27T10:30:00Z",
  │    "timestamps": {
  │      "requested_at": "2026-01-27T10:00:00Z",
  │      "assigned_at": "2026-01-27T10:00:05Z",
  │      "arrived_at": null,
  │      "started_at": null,
  │      "completed_at": null
  │    }
  │  }
  └──────────────────────────────────────────────────────────────────────────────────────────────
```

Step-by-Step Flow

Step	Transition	Trigger (inbound)	Service	Ac
1	REQUESTED → MATCHING	`user.ride.request` via WS	RideRequestService	Pe pr `ri` to

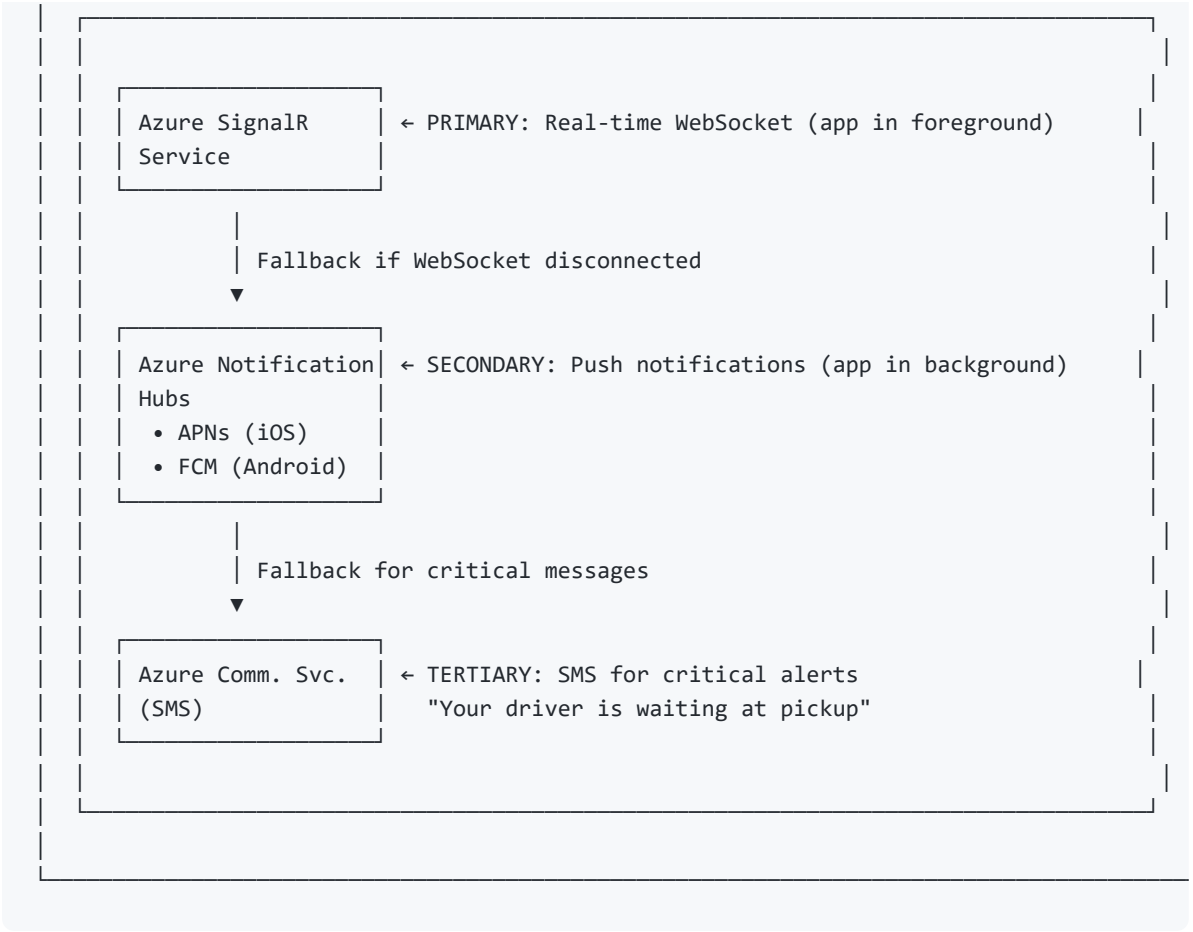
2	MATCHING → ASSIGNED	`driver.ride.accept` via WS	MatchingOfferingService → TripService	CA Tri pe far
3	MATCHING → NO_DRIVER_FOUND	SLA timer exceeded	MatchingOfferingService	All clo
4	ASSIGNED → ARRIVED	`driver.ride.arrived { tripId }` via WS	TripService	Up
5	ARRIVED → IN_PROGRESS	`driver.ride.pickup { tripId }` via WS	TripService	Up sta
6	IN_PROGRESS → COMPLETED	`driver.ride.dropoff { tripId }` via WS	TripService	Up ca
7	COMPLETED → PAYMENT_CAPTURED	`POST /user/payment/capture`	PaymentService	Ca en

8) Notifications

Architecture

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          NOTIFICATION SYSTEM                                │
└─────────────────────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│  NOTIFICATION TYPES:                                                        │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │  • Ride matched: "Your driver is on the way!"                        │   │
│  │  • Driver arrived: "Your driver has arrived at pickup"               │   │
│  │  • Ride completed: "Your trip is complete. Rate your driver"         │   │
│  │  • Driver: "New ride request!" (with accept/decline buttons)         │   │
│  │  • Promo: "50% off your next ride!" (marketing)                      │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                             │
│  DELIVERY CHANNELS:
```



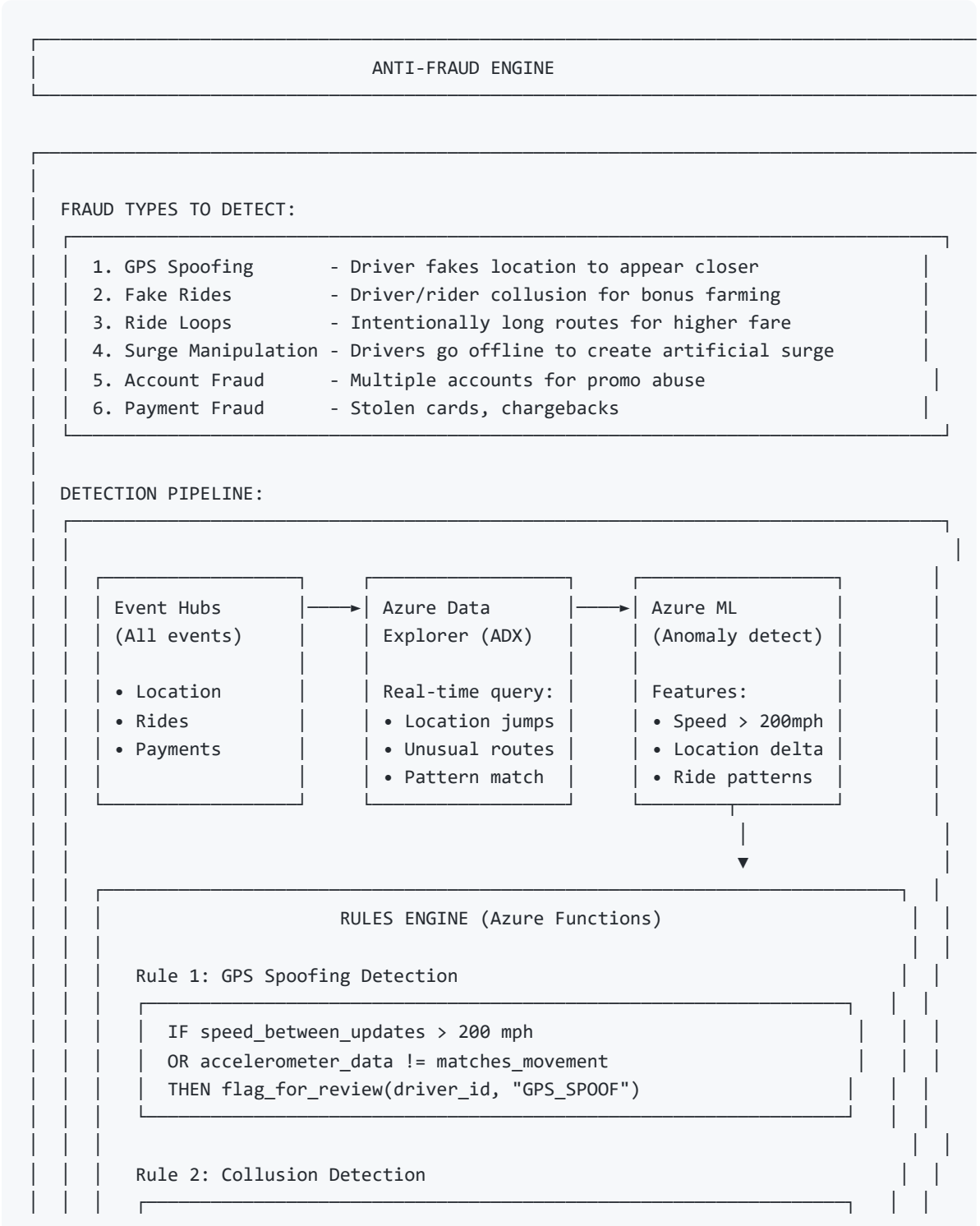
Step-by-Step Flow

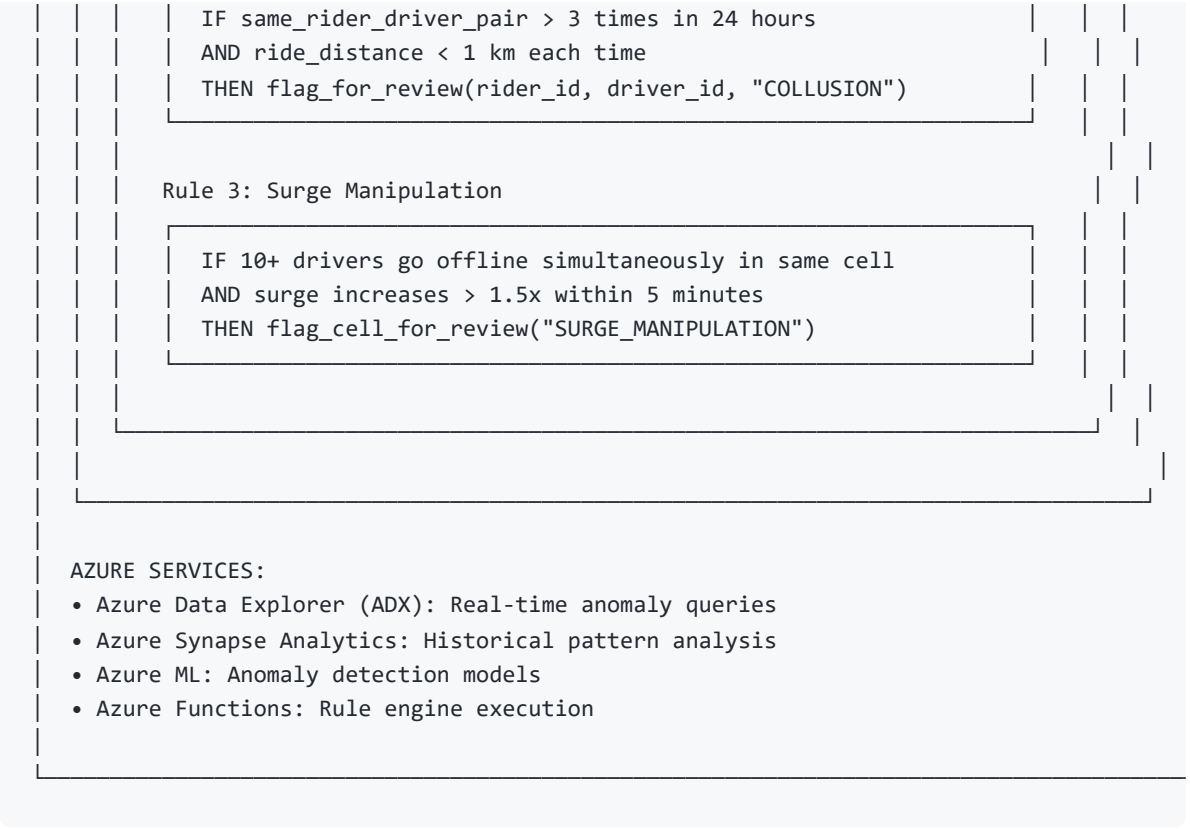
Step	From	To	Action	Data	Sync
1	Any Service	Notification Service	Emit notification event	`{ userId, type, channel[], template, data }`	ASYI
2	Notification Service	—	Check user WS connection state (SignalR connection registry)	`{ userId, isConnected }`	SYN
3a	Notification Service	Azure SignalR → User/Driver App	**If connected:** push via WebSocket	WS event (e.g., `user.ride.assigned`)	ASYI
3b	Notification Service	Azure Notification Hubs → APNs/FCM	**If disconnected:** push notification	`{ title, body, data }` → mobile push	ASYI

3c	Notification Service	Azure Communication Services → SMS	**If critical + no response:** send SMS	`"Your driver is waiting at pickup"`	ASYI

9) Anti-Fraud Engine

Architecture





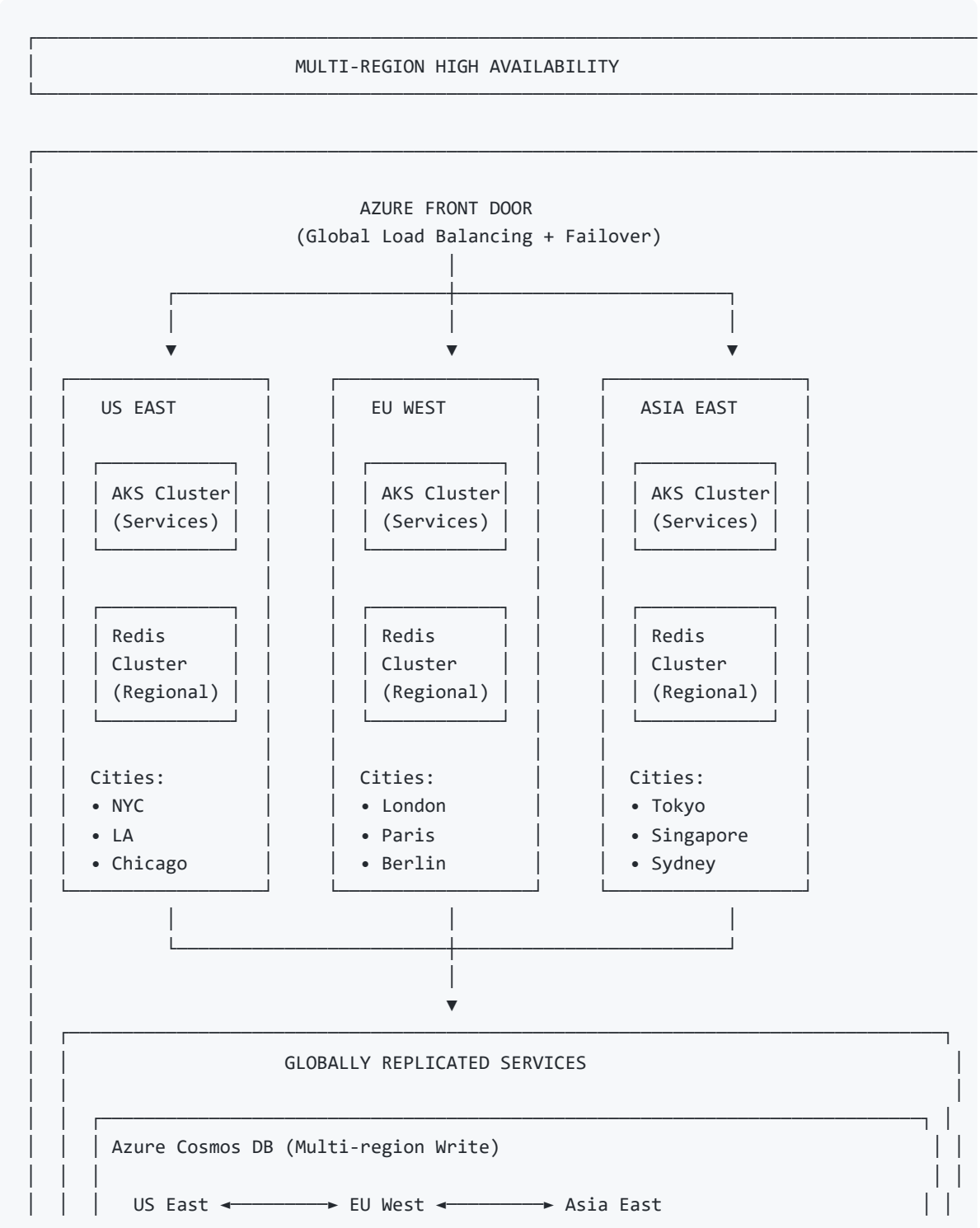
Step-by-Step Flow

Step	From	To	Action	Data	Sync/Async
1	All Services	Event Hubs	Emit events (location, rides, payments)	Raw event stream	ASYNC
2	Event Hubs	Azure Data Explorer (ADX)	Ingest events for real-time querying	Location jumps, unusual routes, patterns	ASYNC
3	ADX	Azure ML Endpoint	Extract features + anomaly scoring	`{ speed, location_delta, ride_patterns }`	SYNC
4	Azure ML	Rules Engine (Functions)	Evaluate fraud rules (GPS spoof, collusion, surge manipulation)	`{ anomaly_score, rule_matches[] }`	SYNC

5	Rules Engine	Action	Flag for review / block user / adjust payout	`{ driver_id, fraud_type, action }`	ASYNC

10) Multi-Region High Availability

Architecture





11) Trade-offs Discussion

Decision	Option A	Option B	Choice	Reasoning
Location store	PostgreSQL PostGIS	**Redis GEO**	**Redis**	In-memory, 1M+ ops/sec, ephemeral data fits (see §2 💡)
Matching isolation	Global service	**Per-city cluster**	**Per-city**	Blast radius isolation, lower latency
Spatial index	QuadTree	**Geohash**	**Geohash**	Redis native, simpler, good enough for 5km radius
Driver offering	Broadcast to all	**Wave-based (3/wave)**	**Waves**	Less wasted WS messages, fair driver windows (see §3 💡)
Trip creation	Async via Service Bus	**Sync call to TripService**	**Sync**	Rider wants instant confirmation, ~116/sec fits (see §3 💡)
Offer expiry timers	Redis ZSET + timer worker	**Service Bus scheduled msgs**	**Service Bus**	ScheduledEnqueueTimeUtc — no extra infra (see §3 💡)
Payment consistency	Eventual (Cosmos DB)	**Strong (PostgreSQL ACID)**	**Strong**	Money requires strong consistency (see §6 💡)
ETA accuracy	Maps only	**Maps + ML**	**Maps + ML**	Azure Maps + ML gives ±2min accuracy
Surge compute	On-demand	**Pre-computed (poll Redis)**	**Pre-computed**	5-10 sec refresh is fast enough
Event streaming	Kafka	**Event Hubs**	**Event Hubs**	Native Azure, managed, Kafka API compatible

---

12) 60-Second Explanation

> "A ride-sharing platform is fundamentally a **real-time location-based matching system**. I'll design for 10M rides/day across 500 cities.
>
> **Location tracking** is the hot path: 2M drivers updating every 3 seconds = 700K writes/sec. I use **Azure Redis Enterprise with GEO commands** — `GEOADD` to store locations, `GEOSEARCH`

> to find nearby drivers in O(log N). During active trips, a fire-and-forget side path looks up the rider via `trip:{tripId}` in Redis and pushes location via **SignalR**.
>
> **Matching is city-isolated and wave-based**. When a rider requests, **RideRequestService** persists to Postgres and produces `ride.request.created` to **Service Bus** (partitioned by city). **MatchingOfferingService** consumes the event, queries Redis GEO for the nearest 50 drivers, ranks by pickup ETA, and offers to 3 drivers at a time (waves). Each offered driver's state is CAS'd in Redis: `available → tentative(offerId)`. Offer expiry uses **Service Bus scheduled messages** (`ScheduledEnqueueTimeUtc`, 2-5s delay) — no separate timer infrastructure needed.
>
> **When a driver accepts**, MatchingOfferingService CAS's the state to `assigned(tripId)` and calls **TripService synchronously** — TripService persists the trip, calls **PricingService** and **ETAService** internally, and returns `{ tripId, fare, pickupETA, dropoffETA }`. Rider gets instant confirmation via WebSocket.
>
> **Surge pricing** polls Redis GEO every 5-10s per geohash cell, calculates supply/demand ratio with weather and event modifiers, and stores in **Cosmos DB** with 5-min TTL.
>
> **Payments are strongly consistent** in **PostgreSQL** — authorize on assignment, capture on completion, outbox pattern for payout events.
>
> Multi-region with **Front Door** routing, city-level blast radius isolation, and **99.9% availability**."

Technologies Deep Dive

Technology	Role	Reference
Redis GEO	Real-time driver location with GEOADD/GEOSEARCH	Tech Guide
Geohash	Spatial indexing for surge pricing cells	Tech Guide
Kafka/Event Hubs	Event streaming for location and ride events	Tech Guide
PostgreSQL	ACID transactions for rides and payments	Tech Guide
Cosmos DB	Globally replicated driver status	Tech Guide
SignalR	Real-time WebSocket push to riders/drivers	Tech Guide

---

Azure Service Mapping Table

Problem	Azure Solution
Real-time locations	**Azure Redis Enterprise GEO**
Massive ingestion	**Azure Event Hubs**
Matching engine	**Azure Kubernetes Service (AKS)**
ETA calculation	**Azure Maps + Azure ML**
Surge pricing	**Stream Analytics + Azure Functions**
Payments	**Azure PostgreSQL + Key Vault**
Real-time updates	**Azure SignalR Service**
Push notifications	**Azure Notification Hubs**
Fraud detection	**Azure Synapse + ADX + ML**
Long-term history	**Azure Data Lake + Synapse**
Multi-region routing	**Azure Front Door**
Driver dispatch	**Azure Service Bus**

Appendix: Geohash Precision Reference

```
┌──────────────────────────────────────────────────────────┐
│                  GEOHASH PRECISION LEVELS                │
├─────────────┬──────────────────┬─────────────────────────┤
│ Precision   │ Cell Size        │ Use Case                │
├─────────────┼──────────────────┼─────────────────────────┤
│     4       │ ~20 km           │ Regional aggregation    │
│     5       │ ~5 km            │ City-level surge zones  │
│     6       │ ~600 m           │ ✅ Surge pricing cells (default) │
│     7       │ ~150 m           │ Block-level precision   │
│     8       │ ~20 m            │ Street-level precision  │
└─────────────┴──────────────────┴─────────────────────────┘

Example: Times Square, NYC
  Precision 4: dr5r
  Precision 6: dr5ru6
  Precision 8: dr5ru6hm
```

Appendix: Wave-Based Matching Pseudocode

```
WAVE_SIZE = 3
TOP_K = 20
OFFER_TTL_SEC = 5

def on_ride_request_created(ride_request):
    """Consumed from Service Bus: ride.request.created (per city)."""

    # Step 1: Query nearby candidates from GeoIndex (Redis KV+Geo, TTL)
    nearby = redis.geosearch(
        key=f"drivers:{ride_request.city}",
        longitude=ride_request.pickup_lng,
        latitude=ride_request.pickup_lat,
        radius=5, unit="km", count=50, sort="ASC"
    )

    # Step 2: Filter by driverState == available
    candidates = []
    for driver_id, distance in nearby:
        state = redis.get(f"driverState:{driver_id}")
        if state == "available":
            candidates.append((driver_id, distance))

    # Step 3: Rank by pickup ETA (primary), rating, acceptance_rate
    scored = []
    for driver_id, distance in candidates:
        eta = eta_service.get_pickup_eta(driver_id, ride_request.pickup)
        rating = redis.hget(f"driver:{driver_id}", "rating")
        scored.append(ScoredDriver(driver_id, eta, rating, distance))

    scored.sort(key=lambda d: d.eta)  # Primary: pickup ETA
    top_k = scored[:TOP_K]

    # Cache Top-K for subsequent waves
    candidate_cache.put(ride_request.trip_request_id, top_k)

    # Step 4: Start wave #1
    send_wave(ride_request, top_k, offset=0)

    # Step 5: Produce scheduled messages to Service Bus
    service_bus.send_scheduled(
        queue=f"trip-sla-timeout:{ride_request.city}",
        body={"tripRequestId": ride_request.trip_request_id},
        scheduled_enqueue_time=now() + ride_request.sla_timeout
    )

def send_wave(ride_request, candidates, offset):
    """Send offers to next WAVE_SIZE drivers."""
    wave = candidates[offset : offset + WAVE_SIZE]

    for driver in wave:
        offer_id = generate_uuid()

        # CAS in Redis: available → tentative(offerId) — atomic
        locked = redis.cas(
```

```
            key=f"driverState:{driver.driver_id}",
            expected="available",
            new_value=f"tentative:{offer_id}"
        )

        if locked:
            # Send offer via WS Gateway
            ws_gateway.send(driver.driver_id, {
                "event": "driver.ride.request",
                "driverId": driver.driver_id,
                "tripRequestId": ride_request.trip_request_id,
                "sourceLocation": ride_request.pickup,
                "targetLocation": ride_request.dropoff,
                "fare": ride_request.fare_estimate,
                "expiresAt": now() + OFFER_TTL_SEC
            })

            # Produce scheduled offer-expiry message to Service Bus
            service_bus.send_scheduled(
                queue=f"offer-expiry:{ride_request.city}",
                body={"offerId": offer_id, "driverId": driver.driver_id},
                scheduled_enqueue_time=now() + OFFER_TTL_SEC
            )

def on_driver_accepted(driver_id, trip_request_id):
    """driver.ride.accept arrives via WS Gateway."""
    offer_id = extract_offer_id(driver_id)

    # CAS: tentative(offerId) → assigned(tripId); if fail → offer_closed
    trip_id = generate_trip_id()
    success = redis.cas(
        key=f"driverState:{driver_id}",
        expected=f"tentative:{offer_id}",
        new_value=f"assigned:{trip_id}"
    )

    if not success:
        ws_gateway.send(driver_id, {"event": "driver.ride.offerClosed"})
        return

    # Call TripService (sync) — orchestrates Pricing + ETA internally
    trip_result = trip_service.create_trip(
        trip_id=trip_id,
        driver_id=driver_id,
        user_id=ride_request.user_id,
        pickup=ride_request.pickup,
        dropoff=ride_request.dropoff
    )
    # trip_result = { tripId, fare, pickupETA, dropoffETA }

    # Notify other offered drivers: offer_closed
    for other_driver in get_offered_drivers(trip_request_id):
        if other_driver != driver_id:
            redis.cas(f"driverState:{other_driver}", f"tentative:*", "available")
            ws_gateway.send(other_driver, {"event": "driver.ride.offerClosed"})
```

```
    def on_offer_expired(offer_id, driver_id):
        """Service Bus delivers scheduled offer-expiry message when ScheduledEnqueueTime
        # CAS: tentative(offerId) → available (idempotent)
        redis.cas(
            key=f"driverState:{driver_id}",
            expected=f"tentative:{offer_id}",
            new_value="available"
        )

        # Continue with next wave from candidates cache
        trip_request_id = get_trip_request(offer_id)
        candidates = candidate_cache.get(trip_request_id)
        next_offset = get_next_offset(trip_request_id)

        if next_offset < len(candidates):
            send_wave(get_ride_request(trip_request_id), candidates, next_offset)

        # If tripRequest SLA exceeded → no driver found
        if sla_exceeded(trip_request_id):
            ws_gateway.send(get_user_id(trip_request_id), {
                "event": "user.ride.noDriverFound",
                "tripRequestId": trip_request_id
            })
            close_trip_request(trip_request_id)
```

---

8. Design WhatsApp/Slack Chat Messaging

Key Terms:

Term	Full Name	What it does
WebSocket	Full-duplex protocol	Persistent connection for real-time bidirectional messaging
ACK	Acknowledgment	Confirmation that message was received
Fanout	Message distribution	Sending one message to many recipients
TIMEUUID	Time-based UUID	UUID that's sortable by creation time
APNs	Apple Push Notification service	Delivers notifications to iOS devices
FCM	Firebase Cloud Messaging	Delivers notifications to Android/iOS
E2EE	End-to-End Encryption	Only sender and recipient can read messages

Problem Statement

Design a real-time messaging system supporting billions of messages daily with delivery guarantees, read receipts, and offline support.

Clarifying Questions to Ask

1. **Scale**: How many users? Messages per day? (2B users, 100B messages?)
2. **Message types**: Text only or media too?
3. **Group size**: Max participants per group? (256? 1000?)
4. **Delivery guarantees**: At-least-once or exactly-once?
5. **Message retention**: How long to store messages?
6. **End-to-end encryption**: Required?
7. **Features**: Read receipts, typing indicators, presence?

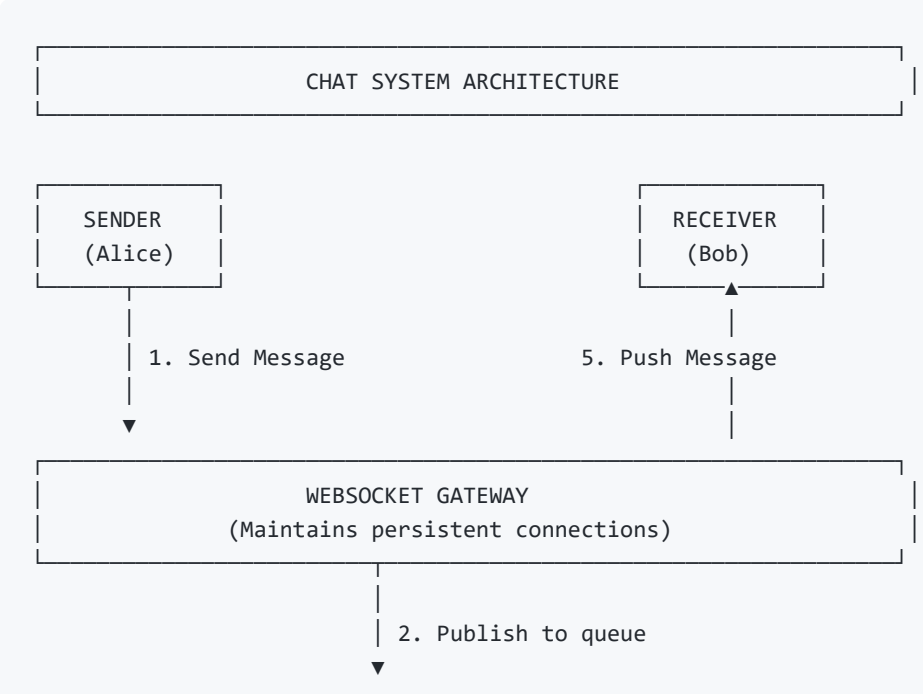
Design Reasoning

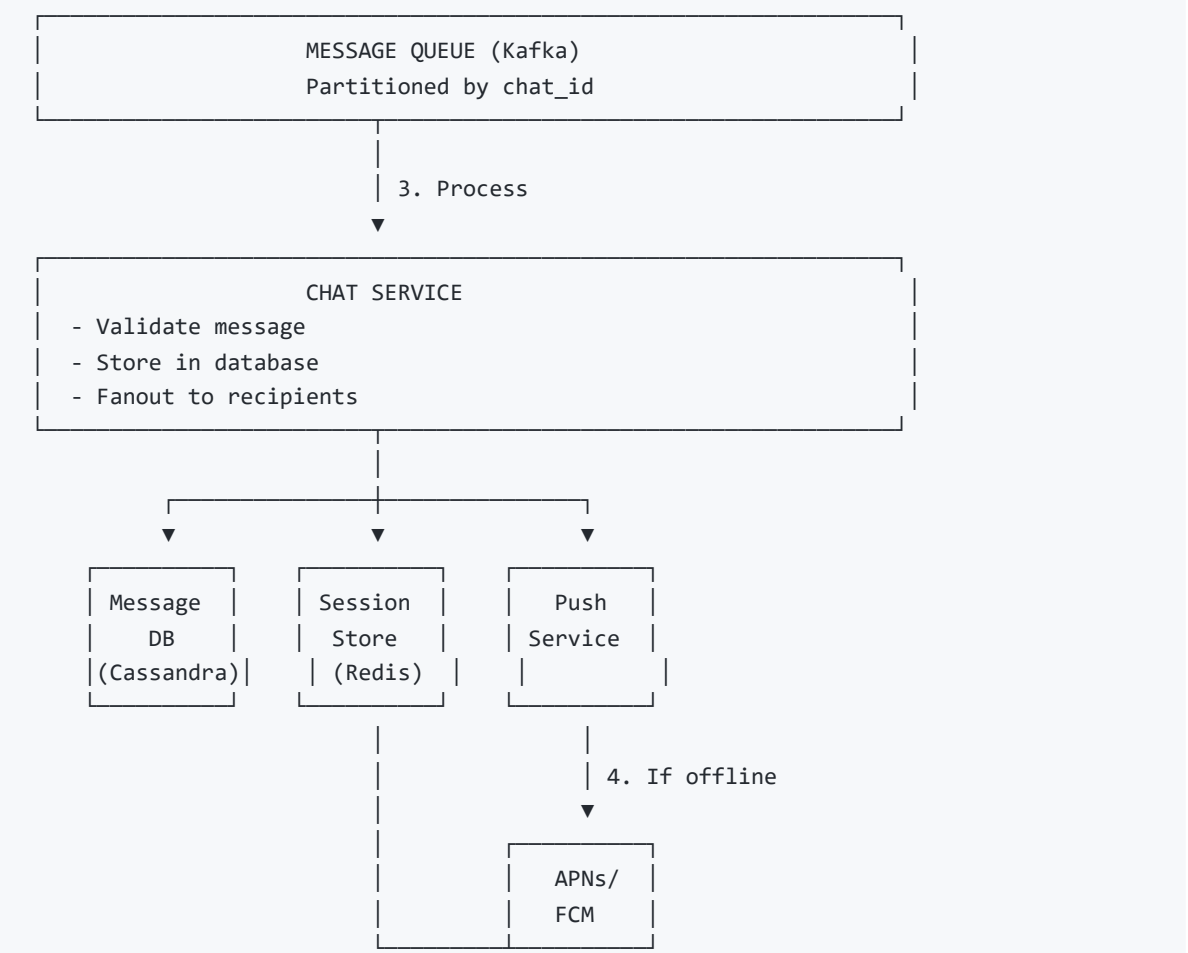
> **Core philosophy**: Deliver messages in real-time with at-least-once guarantees using persistent WebSocket connections, while optimizing for the write-heavy nature of chat (100B messages/day) with a storage engine designed for sequential writes and time-ordered reads.

Decision	Choice	Why	Alternative Considered
Client protocol	WebSocket (persistent)	Full-duplex connection enables instant push delivery and typing indicators without polling overhead	HTTP long-polling — higher latency, more server connections, wastes bandwidth at 2B users
Message queue	Kafka (partitioned by chat_id)	Guarantees message ordering within each chat; durable replay for crash recovery	Redis Pub/Sub — no persistence, messages lost if subscriber is offline
Message store	Cassandra	Optimized for write-heavy workloads; time-series clustering by TIMEUUID gives ordered reads for free	PostgreSQL — write throughput bottleneck at 100B messages/day, sharding is complex

Session/presence store	Redis	Sub-ms lookups for online status and WebSocket connection routing across gateway servers	Database polling — too slow for real-time presence with millions of concurrent users
Group delivery	Fanout-on-write (small groups)	Pre-writes to each member's inbox for fast reads; acceptable write amplification for groups under 256	Fanout-on-read — slower reads requiring joins across group membership, poor UX for active chats
Offline delivery	APNs/FCM push notifications	Platform-native push reaches offline devices without maintaining idle WebSocket connections	SMS fallback — expensive at scale, not suitable for rich message content
Message ordering	TIMEUUID	Globally unique and time-sortable; no coordination needed across distributed writers	Sequence counters — require centralized counter service, single point of failure

High-Level Design





API Design

Message APIs

Method	Endpoint	Description
`POST`	`/api/v1/messages`	Send a message
`GET`	`/api/v1/messages?chatId={id}&limit=50&before={cursor}`	Fetch message history
`PUT`	`/api/v1/messages/{messageId}`	Edit a message
`DELETE`	`/api/v1/messages/{messageId}`	Delete a message
`POST`	`/api/v1/messages/{messageId}/read`	Mark message as read

Chat/Group APIs

Method	Endpoint	Description
`POST`	`/api/v1/chats`	Create a 1:1 or group chat
`GET`	`/api/v1/chats`	List user's chats
`POST`	`/api/v1/chats/{chatId}/members`	Add member to group
`DELETE`	`/api/v1/chats/{chatId}/members/{userId}`	Remove member

WebSocket Events

Event	Direction	Payload
`message.new`	Server → Client	`{ chatId, messageId, content, sender, timestamp }`
`message.delivered`	Server → Client	`{ messageId, userId, timestamp }`
`message.read`	Server → Client	`{ messageId, userId, timestamp }`
`user.typing`	Client → Server	`{ chatId, isTyping }`
`user.online`	Server → Client	`{ userId, lastSeen }`

Send Message Request

```
POST /api/v1/messages
{
  "chat_id": "chat_123",
  "content": "Hello!",
  "type": "text",
  "reply_to": "msg_456",       // optional
  "client_msg_id": "uuid-789"  // for idempotency
}
```

Key Components

Component	Purpose	Technology
WebSocket Gateway	Persistent connections	Custom / SignalR

Session Store	Track user connections	Redis
Message Queue	Decouple and order	Kafka
Chat Service	Business logic	Microservice
Message Store	Persist messages	Cassandra
Push Service	Offline notifications	APNs, FCM
Media Service	Handle attachments	Blob Storage + CDN

Message Flow

```
1:1 CHAT:
  Alice → Server → Bob (direct delivery if online)
                 → Store for later (if offline)

GROUP CHAT (Fanout-on-write):
  Alice sends to Group(Alice, Bob, Charlie)
  Server writes to:
    - Bob's inbox
    - Charlie's inbox
  Pros: Fast reads
  Cons: Storage cost, write amplification

GROUP CHAT (Fanout-on-read):
  Alice sends to Group
  Server stores once in Group's messages
  Readers pull from Group
  Pros: Less storage
  Cons: Slower reads, complex queries
```

Message Delivery States

```
┌─────────────────────────────────────────────────────┐
│  SENT (✓)      → Server received the message        │
│  DELIVERED (✓✓) → Recipient's device received       │
│  READ (✓✓ blue) → Recipient opened the chat         │
└─────────────────────────────────────────────────────┘

Implementation:
  1. Client sends message → Server ACKs → Show single tick
  2. Server delivers to recipient → Recipient ACKs → Double tick
  3. Recipient opens chat → Send read receipt → Blue ticks
```

Database Schema

```
-- Messages (Cassandra - partitioned by chat_id)
CREATE TABLE messages (
    chat_id UUID,
    message_id TIMEUUID,  -- Sortable by time
    sender_id UUID,
    content TEXT,
    content_type VARCHAR,  -- text, image, video
    created_at TIMESTAMP,
    PRIMARY KEY (chat_id, message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);

-- User Inbox (for quick access to recent chats)
CREATE TABLE user_inbox (
    user_id UUID,
    chat_id UUID,
    last_message_id TIMEUUID,
    last_message_preview TEXT,
    unread_count INT,
    updated_at TIMESTAMP,
    PRIMARY KEY (user_id, updated_at)
) WITH CLUSTERING ORDER BY (updated_at DESC);
```

Scalability Considerations

- **Connection management**: Each gateway server handles ~500K connections
- **Partitioning**: Kafka partitioned by chat_id for ordering
- **Presence**: Use heartbeats + Redis pub/sub for online status
- **Group fanout**: Limit group size, use async fanout for large groups
- **Message ordering**: TIMEUUID ensures global ordering within a chat

Trade-offs

Decision	Option A	Option B	Recommendation
Protocol	HTTP polling	WebSocket	**WebSocket** - real-time, efficient
Group fanout	On write	On read	**On write** - for small groups
Message store	SQL	Cassandra	**Cassandra** - write-heavy scale
Ordering	Timestamps	Sequence IDs	**TIMEUUID** - sortable + unique

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Message ordering and delivery guarantee	Tech Guide
Redis	Online presence and connection state	Tech Guide
Cassandra	Write-optimized message storage	Tech Guide

How to Explain It Well

> **Opening**: "A chat system needs real-time delivery with offline support. I'll design for 2 billion users and 100 billion messages daily."

> **Key talking points**:
>
> - "WebSocket gateways maintain persistent connections - one per user"
> - "Kafka ensures message ordering per chat via partitioning"
> - "Fanout-on-write for groups under 256 users, on-read for larger"
> - "Cassandra for messages - optimized for write-heavy, time-series data"
> - "Three delivery states: sent, delivered, read - each requires an ACK"

---

9. Design a Scalable Web Crawler

Key Terms:

Term	Full Name	What it does
URL Frontier	URL Queue	Prioritized queue of URLs waiting to be crawled
robots.txt	Robots Exclusion	File telling crawlers which pages not to crawl
SimHash	Similarity Hash	Fingerprint for detecting near-duplicate content
MinHash	Min-wise Hashing	Algorithm for estimating similarity between sets
PageRank	Page Ranking	Algorithm scoring pages by incoming link importance

TF-IDF	Term Frequency-Inverse Document Frequency	Weights words by importance in document
BFS	Breadth-First Search	Crawl strategy visiting all pages at depth N before N+1
DNS	Domain Name System	Resolves domain names to IP addresses

Problem Statement

Design a web crawler like Bing's crawler that can index billions of web pages efficiently while being polite to websites.

Clarifying Questions to Ask

1. **Scale**: How many pages to crawl? (10B pages?)
2. **Freshness**: How often to recrawl? (Daily? Weekly?)
3. **Depth**: How deep to follow links? (5 levels?)
4. **Politeness**: Rate limits per domain? (1 req/sec?)
5. **Content types**: HTML only or PDFs, images too?
6. **Deduplication**: Handle duplicate/near-duplicate pages?
7. **Robots.txt**: Respect exclusions?

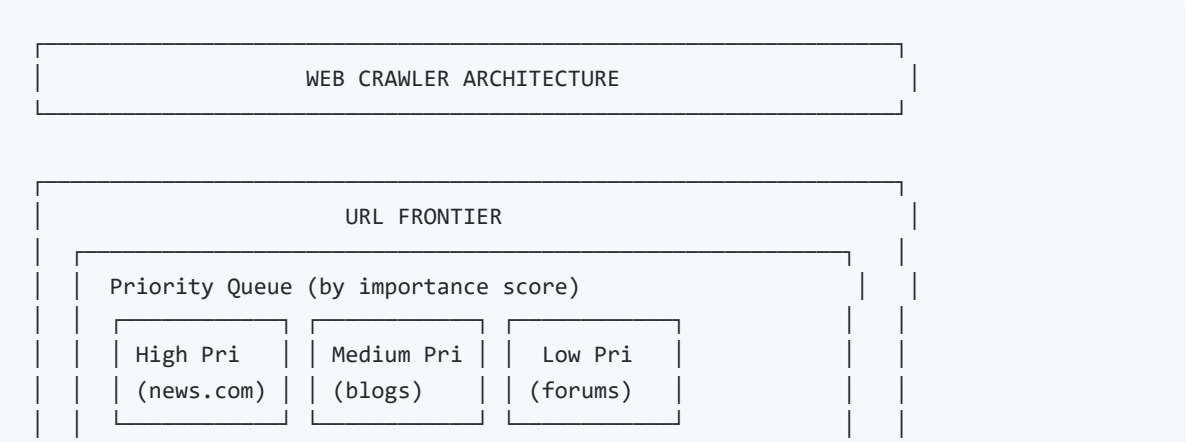
Design Reasoning

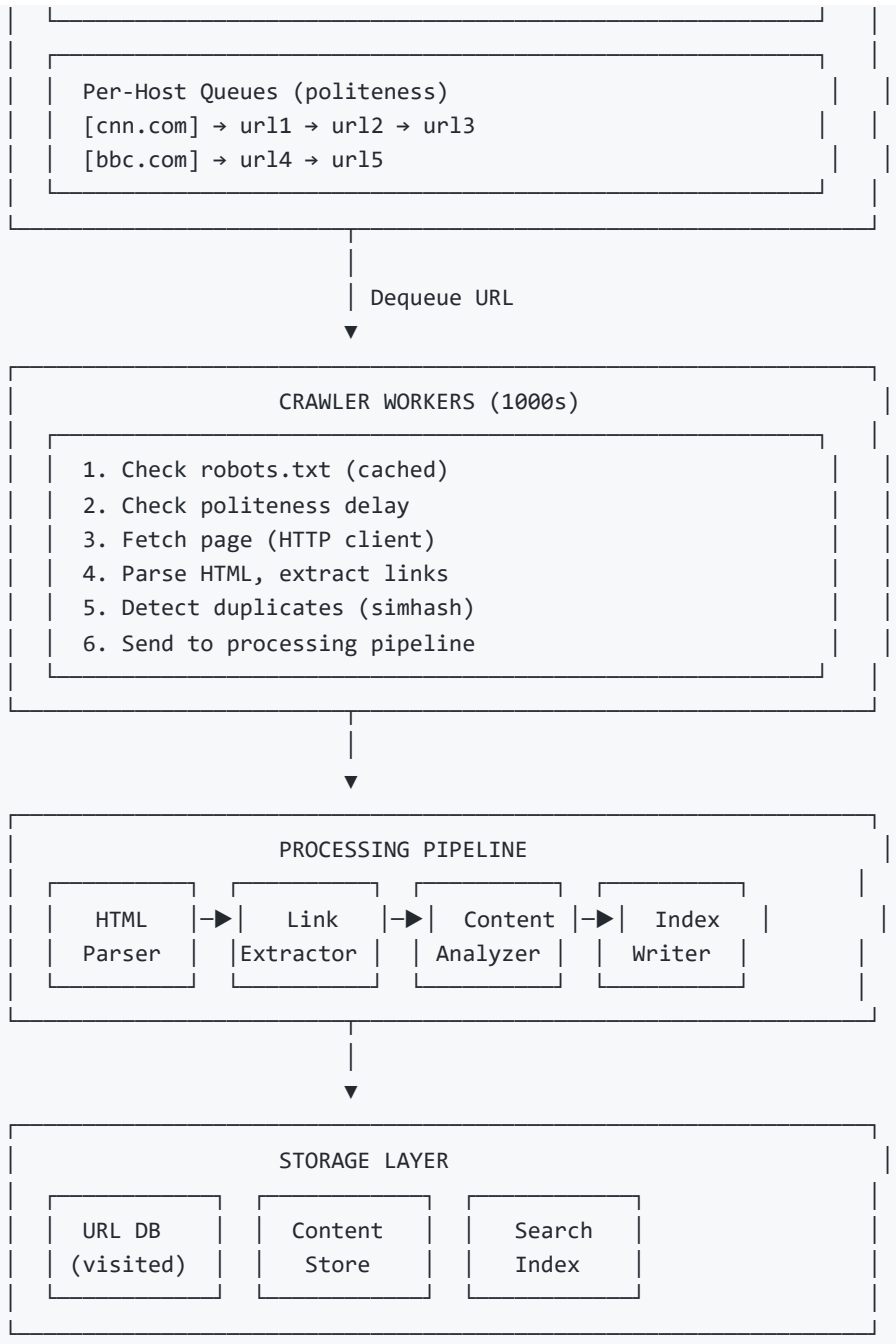
> **Core philosophy**: Crawl billions of pages efficiently while being a good citizen of the web — a two-level frontier separates what-to-crawl (priority) from how-fast-to-crawl (politeness), ensuring important pages are indexed first without overwhelming any single host.

Decision	Choice	Why	Alternative Considered
Crawl strategy	BFS (Breadth-First Search)	Better page coverage at each depth level; discovers important pages (linked from many sites) faster	DFS — gets trapped in deep site hierarchies, poor coverage breadth
URL frontier	Two-level queue (priority + per-	Priority queues rank by importance (PageRank);	Single FIFO queue — no way to prioritize

	host)	per-host queues enforce politeness delays per domain	important pages or rate-limit per host
Duplicate detection	SimHash fingerprinting	Detects near-duplicate content (Hamming distance under 3) with a compact 64-bit fingerprint per page	Exact URL matching — misses duplicate content served at different URLs
URL deduplication	Bloom filter	Memory-efficient probabilistic check (1% false positive) for billions of seen URLs using only ~1GB RAM	HashSet — requires storing all 10B URLs in memory, hundreds of GB
Worker partitioning	Hash(domain) to workers	Each worker owns a set of domains, enabling local robots.txt caching and connection reuse	Random assignment — same domain hit by multiple workers, violates politeness contracts
Content storage	Blob storage (raw) + search index	Raw HTML preserved for reprocessing; separate inverted index optimized for search queries	Database rows — too expensive for billions of full HTML pages
Recrawl scheduling	Adaptive frequency	News sites crawled hourly, static sites weekly; update frequency detected from content change history	Fixed schedule — wastes resources on unchanged pages, misses fast-changing content

High-Level Design





API Design

Crawl Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/crawls`	Start a new crawl job
`GET`	`/api/v1/crawls/{crawlId}`	Get crawl job status
`POST`	`/api/v1/crawls/{crawlId}/stop`	Stop a running crawl

`GET`	`/api/v1/crawls/{crawlId}/results?page={n}`	Get crawled pages

Seed URL Management

Method	Endpoint	Description
`POST`	`/api/v1/seeds`	Add seed URLs
`GET`	`/api/v1/seeds`	List seed URLs
`DELETE`	`/api/v1/seeds/{seedId}`	Remove a seed URL

Configuration APIs

Method	Endpoint	Description
`PUT`	`/api/v1/config/politeness`	Set crawl rate limits per domain
`PUT`	`/api/v1/config/filters`	Set URL inclusion/exclusion rules
`GET`	`/api/v1/stats`	Get crawler statistics (pages/sec, queue depth)

Key Components

Component	Purpose	Technology
URL Frontier	Prioritized URL queue	Redis + Custom
Crawler Workers	Fetch web pages	Distributed workers
DNS Resolver	Resolve hostnames	Local cache + DNS
Robots Parser	Parse robots.txt	Custom parser
Duplicate Detector	Identify duplicates	SimHash / MinHash
Content Store	Store raw HTML	Blob storage
Link Extractor	Extract outbound links	HTML parser

URL Frontier Design

```
TWO-LEVEL QUEUE SYSTEM:

Level 1: Priority Queues (what to crawl)
┌──────────────────────────────────────────────────────────────────────┐
```

```
│  Priority based on:                                                │
│  - PageRank score                                                  │
│  - Freshness (news sites crawled more often)                       │
│  - Update frequency (changed pages prioritized)                    │
└────────────────────────────────────────────────────────────────────┘

Level 2: Per-Host Queues (politeness)
┌────────────────────────────────────────────────────────────────────┐
│  Each host has its own queue                                       │
│  Enforces delay between requests to same host                      │
│  Respects robots.txt crawl-delay directive                         │
└────────────────────────────────────────────────────────────────────┘

Scheduling:
  1. Select from highest priority queue
  2. Find a host that's not rate-limited
  3. Dequeue URL from that host's queue
  4. Update host's last-access timestamp
```

Duplicate Detection

```
SIMHASH (Near-duplicate detection):
┌────────────────────────────────────────────────────────────────────┐
│  1. Extract features (word n-grams)                                │
│  2. Hash each feature                                              │
│  3. Weight by TF-IDF                                               │
│  4. Combine into 64-bit fingerprint                                │
│  5. Compare Hamming distance (< 3 = duplicate)                     │
└────────────────────────────────────────────────────────────────────┘

URL NORMALIZATION:
  http://Example.com:80/path/../page?b=2&a=1
  → https://example.com/page?a=1&b=2

  - Lowercase hostname
  - Remove default port
  - Resolve path (../)
  - Sort query parameters
  - Remove tracking params (utm_*)
```

Scalability Considerations

- **Distributed crawling**: Partition URLs by hash(domain) to workers
- **DNS caching**: Cache DNS lookups (most lookups are repeats)
- **Connection reuse**: HTTP keep-alive to same hosts
- **Compression**: Request gzip, store compressed
- **Checkpointing**: Save frontier state for crash recovery
- **Adaptive crawling**: Crawl frequently-updated sites more often

Trade-offs

Decision	Option A	Option B	Recommendation
BFS vs DFS	BFS	DFS	**BFS** - better coverage
Politeness	Per-IP	Per-domain	**Per-domain** - safer
Storage	Store all	Store hashes	**Both** - full for index, hash for dedup
Scheduling	FIFO	Priority	**Priority** - important pages first

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	URL frontier queue and crawl task distribution	Tech Guide
Redis	URL deduplication and crawl state caching	Tech Guide
Bloom Filter	Efficient URL seen-before check	Tech Guide
PostgreSQL	Crawl metadata and robots.txt storage	Tech Guide

How to Explain It Well

> **Opening**: "A web crawler needs to be fast yet polite. I'll design for 10 billion pages with politeness guarantees and duplicate detection."

> **Key talking points**:
>
> - "Two-level frontier: priority queues for importance, per-host queues for politeness"
> - "SimHash fingerprints detect near-duplicates with Hamming distance < 3"
> - "Partition by domain hash so each worker owns a set of domains"
> - "Cache robots.txt and DNS to avoid repeated lookups"
> - "Adaptive recrawl: news sites hourly, static sites weekly"

---

15. Design Meeting Scheduling System

Key Terms:

Term	Full Name	What it does
UTC	Coordinated Universal Time	Global time standard for storage
RRULE	Recurrence Rule (iCalendar)	Standard format for recurring events
DST	Daylight Saving Time	Seasonal time shift (complicates scheduling)
Free/Busy	Availability data	Shows when users are available/busy
Bitmap	Binary array	Efficient storage for availability slots
TIMESTAMPTZ	Timestamp with timezone	PostgreSQL type storing time + zone

Problem Statement

Design a calendar and meeting scheduling system like Outlook Calendar that handles time zones, recurring meetings, and room bookings.

Clarifying Questions to Ask

1. **Scale**: How many users? Meetings per day? (100M users, 50M meetings?)
2. **Features**: Free/busy, room booking, recurring meetings?
3. **Time zones**: Global users across all time zones?
4. **Integration**: Email invites, video conferencing links?
5. **Conflicts**: Auto-detect or allow double-booking?
6. **Availability**: Find available slots across multiple attendees?
7. **Reminders**: Email, push, SMS notifications?

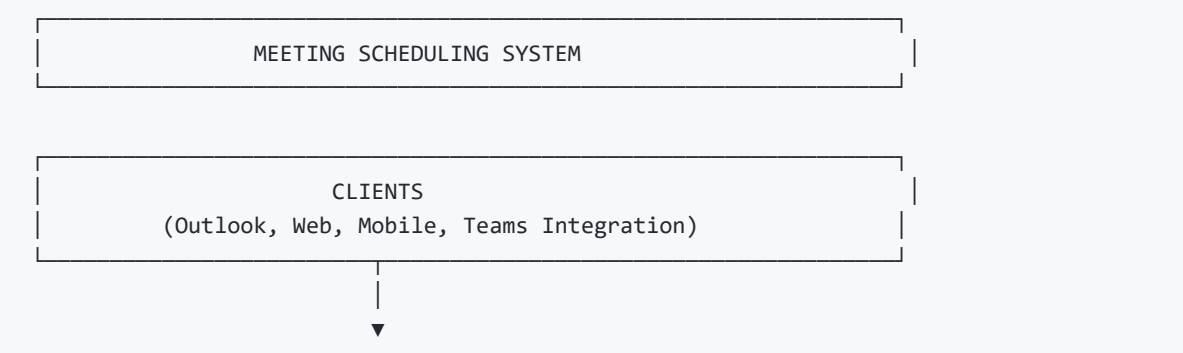
Design Reasoning

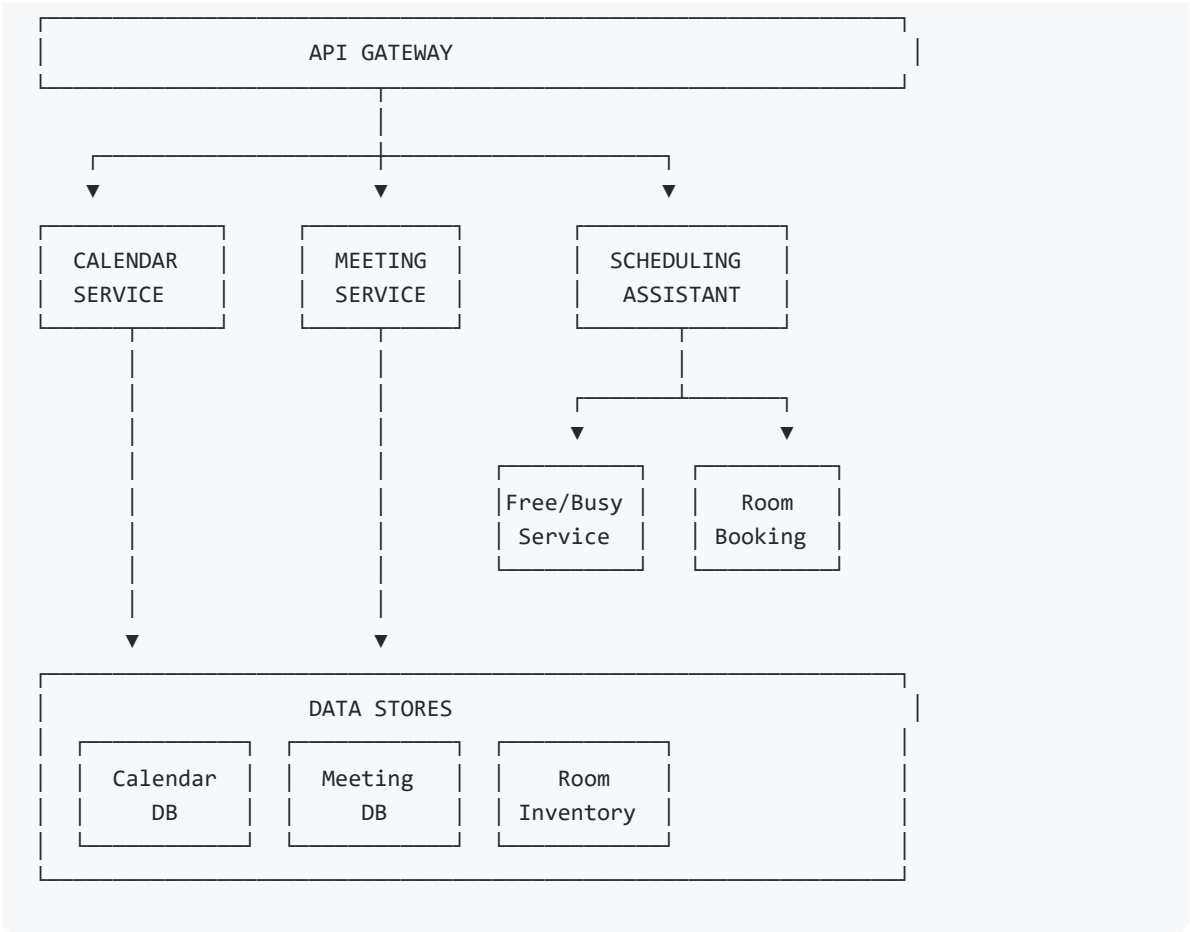
> **Core philosophy**: Store everything in UTC with timezone metadata, and use bitmap-based free/busy indexes to make multi-attendee availability queries fast — solving the two hardest calendar problems: time zones and scheduling at scale.

Decision	Choice	Why	Alternative Considered

Time Storage	UTC with timezone name stored separately	Correct DST handling; timezone name allows recalculation when rules change	Local time storage — breaks when DST rules change or users travel across zones
Free/Busy Index	Redis bitmaps (288 bits/user/day)	O(1) bitwise OR across attendees for availability; only 36 bytes per user per day	Query events table — requires scanning all events for each availability check
Primary Database	PostgreSQL (TIMESTAMPTZ)	Strong consistency for meeting bookings; native timezone support; relational joins for attendees	NoSQL — meetings need ACID transactions to prevent room double-booking
Recurring Events	Hybrid: RRULE + pre-expand 6 months	Fast reads for near-term calendar views; flexible expansion for far-future queries	Full pre-expansion (excessive storage) or rule-only (slow reads for common views)
Room Booking	Pessimistic locking (SELECT FOR UPDATE)	Prevents double-booking of conference rooms under concurrent requests	Optimistic locking — higher conflict rate during popular meeting time slots
Notifications	Async via message queue	Decouples scheduling from reminder/invite delivery; handles retries independently	Synchronous — blocks meeting creation on email/push delivery success

High-Level Design





API Design

Meeting APIs

Method	Endpoint	Description
`POST`	`/api/v1/meetings`	Create/schedule a meeting
`GET`	`/api/v1/meetings/{meetingId}`	Get meeting details
`PUT`	`/api/v1/meetings/{meetingId}`	Update meeting
`DELETE`	`/api/v1/meetings/{meetingId}`	Cancel meeting
`POST`	`/api/v1/meetings/{meetingId}/rsvp`	RSVP to a meeting

Availability APIs

Method	Endpoint	Description
`GET`	`/api/v1/availability?userIds={ids}&date={date}`	Get free/busy slots

`POST`	`/api/v1/availability/suggest`	Suggest meeting times for participants

Schedule Meeting Request

```
POST /api/v1/meetings
{
  "title": "Design Review",
  "start_time": "2025-03-15T10:00:00Z",
  "end_time": "2025-03-15T11:00:00Z",
  "timezone": "America/Los_Angeles",
  "attendees": ["user_1", "user_2", "user_3"],
  "recurrence": {
    "pattern": "weekly",
    "days": ["monday", "wednesday"],
    "end_date": "2025-06-15"
  },
  "room_id": "room_42"
}
```

Calendar APIs

Method	Endpoint	Description
`GET`	`/api/v1/calendars/{userId}?start={date}&end={date}`	Get calendar events
`POST`	`/api/v1/calendars/{userId}/block`	Block time on calendar

Key Components

Component	Purpose	Technology
Calendar Service	CRUD for calendar events	PostgreSQL
Meeting Service	Meeting lifecycle	PostgreSQL
Free/Busy Service	Availability queries	Redis + Bitmap
Room Booking	Conference room management	PostgreSQL
Scheduling Assistant	Find optimal times	Algorithm service
Notification Service	Reminders, invites	Email, Push

Time Zone Handling

```
STORAGE: Always store in UTC
┌──────────────────────────────────────────────────────────────┐
│  Meeting: "Team standup"                                     │
│  Start: 2024-01-15T14:00:00Z  (UTC)                          │
│  End:   2024-01-15T14:30:00Z  (UTC)                          │
│  Timezone: America/New_York   (for display)                  │
│                                                              │
│  Display for NYC user: 9:00 AM - 9:30 AM EST                 │
│  Display for London user: 2:00 PM - 2:30 PM GMT              │
│  Display for Tokyo user: 11:00 PM - 11:30 PM JST             │
└──────────────────────────────────────────────────────────────┘

RECURRING MEETING EDGE CASES:
  "Every day at 9 AM" in NYC
  - During DST transition, 9 AM moves in UTC
  - Store: recurring rule + timezone
  - Expand: compute UTC times at query time
```

Recurring Meeting Rules (iCalendar RRULE)

```
EXAMPLES:
  FREQ=DAILY;COUNT=10
    → Every day for 10 occurrences

  FREQ=WEEKLY;BYDAY=MO,WE,FR;UNTIL=20241231
    → Every Mon, Wed, Fri until end of 2024

  FREQ=MONTHLY;BYMONTHDAY=15
    → 15th of every month

  FREQ=YEARLY;BYMONTH=1;BYMONTHDAY=1
    → Every January 1st

STORAGE OPTIONS:
  1. Store rule, expand on query (less storage, slower reads)
  2. Pre-expand instances (more storage, faster reads)

  Recommendation: Hybrid
    - Store rule for future expansion
    - Pre-expand next 6 months for fast queries
```

Free/Busy Algorithm

```
FINDING AVAILABLE SLOTS:
┌──────────────────────────────────────────────────────────────┐
│  Input: [Alice, Bob, Charlie], duration: 30min, range: this week
```

```
│                                                                  │
│  1. Fetch busy times for each attendee                           │
│     Alice:  [9-10, 14-15]                                        │
│     Bob:    [10-11, 14-16]                                       │
│     Charlie:[9-11]                                               │
│                                                                  │
│  2. Merge into combined busy bitmap                              │
│     Combined: [9-11, 14-16]                                      │
│                                                                  │
│  3. Find gaps ≥ 30 minutes                                       │
│     Available: [11-14, 16-EOD]                                   │
│                                                                  │
│  4. Rank by preference (working hours, fewer conflicts)          │
│     Suggestion: 11:00 AM - 11:30 AM                              │
└──────────────────────────────────────────────────────────────────┘

BITMAP OPTIMIZATION:
  - Each day = 288 slots (5-minute granularity)
  - Store as bitmap: 288 bits = 36 bytes per user per day
  - Fast OR operation to combine multiple calendars
```

Database Schema

```sql
-- Events (includes meetings)
CREATE TABLE events (
    id BIGINT PRIMARY KEY,
    calendar_id BIGINT NOT NULL,
    title VARCHAR(255),
    description TEXT,
    start_time TIMESTAMPTZ NOT NULL,  -- UTC
    end_time TIMESTAMPTZ NOT NULL,    -- UTC
    timezone VARCHAR(50),              -- for display
    recurrence_rule TEXT,              -- RRULE format
    recurrence_id BIGINT,              -- parent for instances
    location VARCHAR(255),
    meeting_link TEXT,
    status VARCHAR(20),                -- confirmed, tentative, cancelled
    created_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ
);

-- Attendees
CREATE TABLE attendees (
    event_id BIGINT,
    user_id BIGINT,
    email VARCHAR(255),
    response_status VARCHAR(20),  -- accepted, declined, tentative
    is_organizer BOOLEAN,
    PRIMARY KEY (event_id, user_id)
);

-- Rooms
```

```
CREATE TABLE rooms (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    capacity INT,
    building VARCHAR(100),
    floor INT,
    amenities JSONB  -- {projector, whiteboard, video_conf}
);
```

Scalability Considerations

- **Read-heavy**: Cache user calendars in Redis
- **Partitioning**: Shard by user_id or calendar_id
- **Recurring expansion**: Pre-expand 6 months, lazy-expand beyond
- **Free/busy**: Bitmap indexes for fast availability queries
- **Notifications**: Async via message queue

Trade-offs

Decision	Option A	Option B	Recommendation
Recurrence	Store rule	Pre-expand	**Hybrid** - rule + 6mo expand
Time storage	Local time	UTC	**UTC** - consistent
Free/busy	Query events	Bitmap cache	**Bitmap** - faster
Conflicts	Prevent	Allow (warn)	**Allow + warn** - flexibility

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Bitmap-based free/busy caching for fast availability queries	Tech Guide
PostgreSQL	Primary storage for events, attendees, and room data	Tech Guide

How to Explain It Well

> **Opening**: "Calendar scheduling handles time zones, recurring events, and availability. I'll design for 100M users with fast availability queries."

> **Key talking points**:
>
> - "Store all times in UTC, convert to user timezone for display"
> - "RRULE format for recurring meetings, pre-expand 6 months"
> - "Bitmap indexes for O(1) free/busy lookups across multiple users"
> - "Room booking uses pessimistic locking to prevent double-booking"
> - "Handle DST transitions by storing timezone name, not offset"

---

18. Design Video Streaming Platform

Key Terms:

Term	Full Name	What it does
HLS	HTTP Live Streaming	Apple's adaptive streaming protocol using .m3u8 playlists
DASH	Dynamic Adaptive Streaming over HTTP	Open standard for adaptive streaming
Transcoding	Format conversion	Converting video to multiple resolutions/codecs
CDN	Content Delivery Network	Edge servers for global video delivery
Bitrate	Data rate	Video quality measure (e.g., 5Mbps for 1080p)
Segment	Video chunk	2-10 second piece of video for streaming
ABR	Adaptive Bitrate	Player switches quality based on bandwidth

Problem Statement

Design a video streaming platform like YouTube or Netflix that handles video upload, processing, storage, and playback at scale.

Clarifying Questions to Ask

1. **Scale**: How many videos? Daily views? (1B videos, 5B views/day?)

2. **Video length**: Short clips or long movies?
3. **Quality**: Multiple resolutions? Adaptive bitrate?
4. **Live streaming**: Pre-recorded only or live too?
5. **Latency**: How fast should videos be available after upload?
6. **Features**: Comments, likes, recommendations?
7. **Monetization**: Ads, subscriptions, or both?

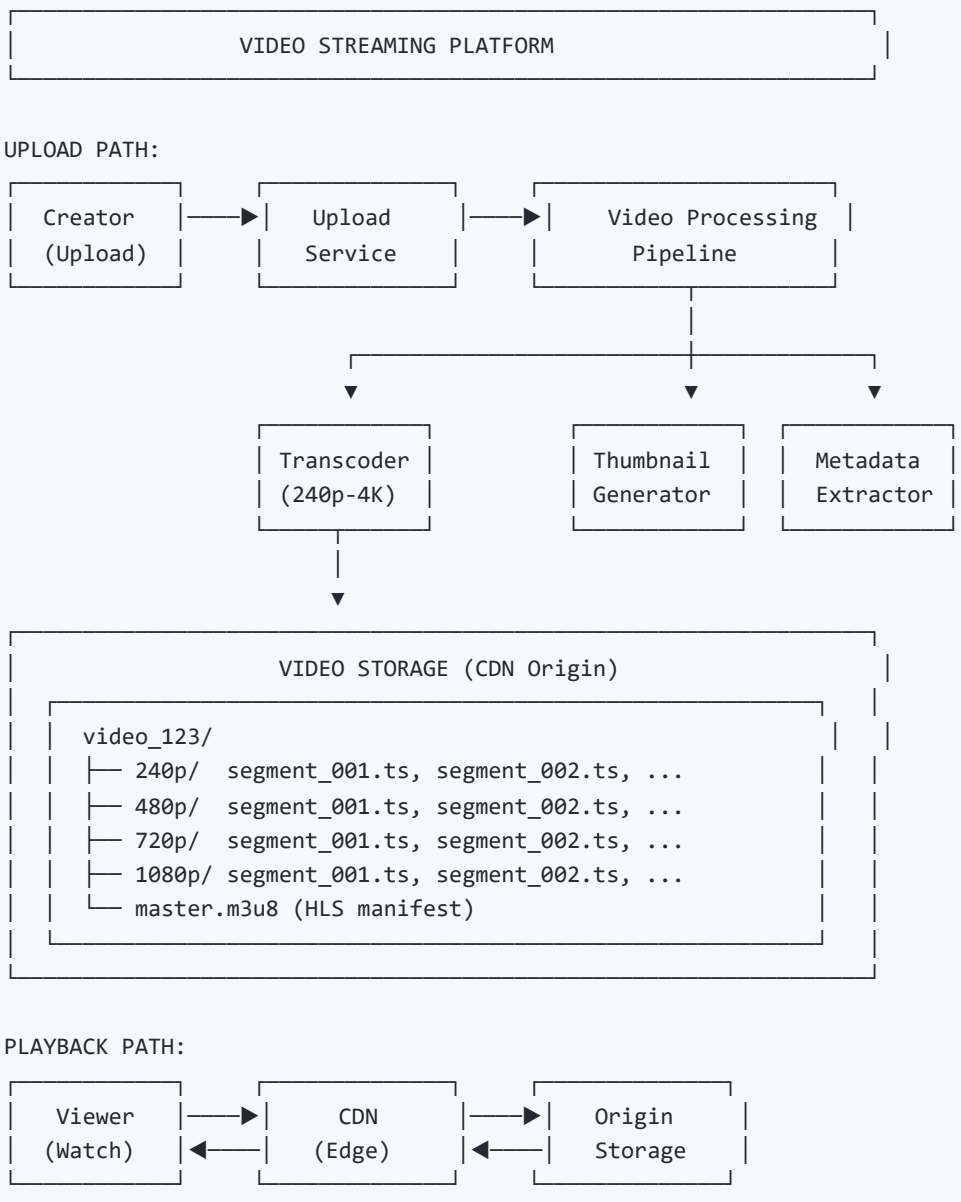
Design Reasoning

> **Core philosophy**: Separate the upload/processing pipeline from the playback path — transcode eagerly into multiple resolutions and push segments to a CDN, so playback is a simple edge-cache hit with adaptive bitrate switching for every viewer's bandwidth.

Decision	Choice	Why	Alternative Considered
Streaming Protocol	HLS (HTTP Live Streaming)	Universal device support, HTTP-based (CDN-friendly), adaptive bitrate built-in	DASH (less iOS support) / Progressive download (no adaptive quality switching)
Transcoding Strategy	Eager on-upload (240p–4K)	Playback is instant from pre-rendered segments; no processing delay at view time	On-demand transcoding — first viewer waits; popular videos re-transcoded repeatedly
Video Delivery	CDN with multi-tier edge caching	95%+ cache hit ratio; edge-served segments minimize origin load and viewer latency	Direct origin serving — cannot scale to billions of daily views globally
Video Storage	Azure Blob Storage with lifecycle tiering	Cost-effective at exabyte scale; hot tier for new videos, archive for old content	Self-hosted storage — high operational burden, no built-in geo-redundancy
Processing Queue	Kafka	Durable job queue for transcoding pipeline; supports replay if workers fail	Direct synchronous processing — no backpressure, workers overwhelmed during upload spikes

View Count Updates	Async batched aggregation	Avoids write contention on hot video rows; eventual consistency acceptable for counts	Real-time increment — database bottleneck for viral videos with millions of concurrent views
Video Segmentation	2-10 second HLS/DASH segments	Small enough for fast quality switching, large enough to reduce HTTP request overhead	Full-file streaming — no adaptive bitrate capability, long initial buffering

High-Level Design



API Design

Video APIs

Method	Endpoint	Description
`POST`	`/api/v1/videos/upload`	Initialize video upload
`PUT`	`/api/v1/videos/upload/{uploadId}/chunk`	Upload video chunk
`POST`	`/api/v1/videos/upload/{uploadId}/complete`	Complete upload & start processing
`GET`	`/api/v1/videos/{videoId}`	Get video metadata
`GET`	`/api/v1/videos/{videoId}/stream`	Get streaming manifest (HLS/DASH)
`DELETE`	`/api/v1/videos/{videoId}`	Delete a video

Playback APIs

Method	Endpoint	Description
`POST`	`/api/v1/videos/{videoId}/watch`	Record watch event & get playback token
`PUT`	`/api/v1/videos/{videoId}/progress`	Update watch progress
`GET`	`/api/v1/videos/{videoId}/manifest.m3u8`	HLS manifest

Feed & Discovery APIs

Method	Endpoint	Description
`GET`	`/api/v1/feed?page={n}`	Get personalized video feed
`GET`	`/api/v1/search?q={query}`	Search videos
`GET`	`/api/v1/videos/{videoId}/related`	Get related videos

Interaction APIs

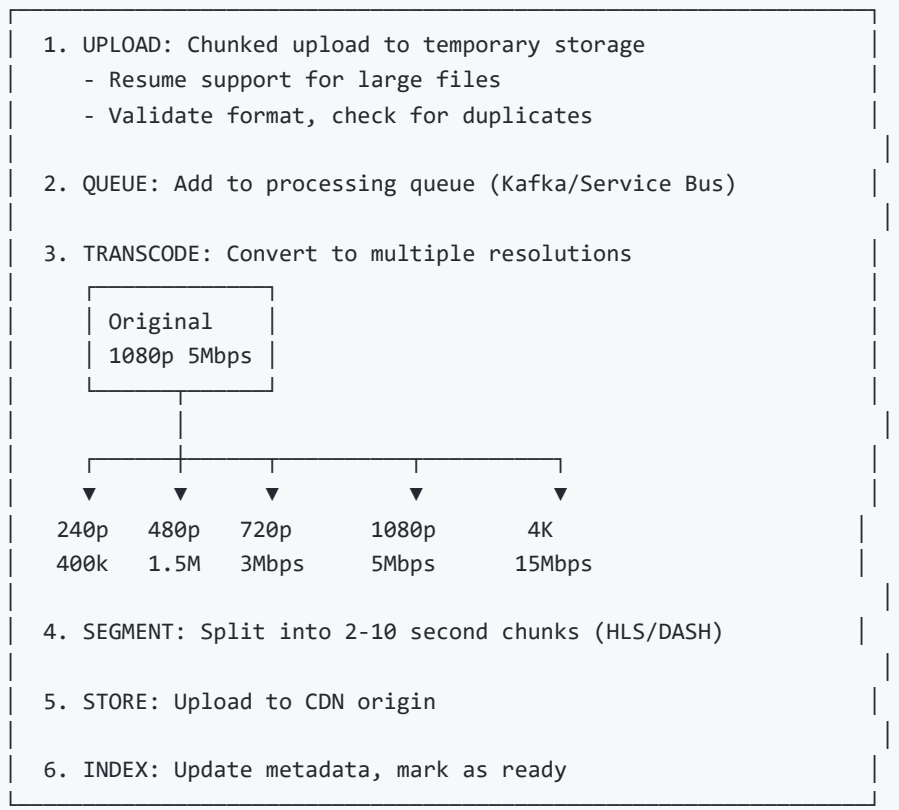
Method	Endpoint	Description
`POST`	`/api/v1/videos/{videoId}/like`	Like a video

`POST`	`/api/v1/videos/{videoId}/comments`	Post a comment
`GET`	`/api/v1/videos/{videoId}/comments?limit=20`	Get comments

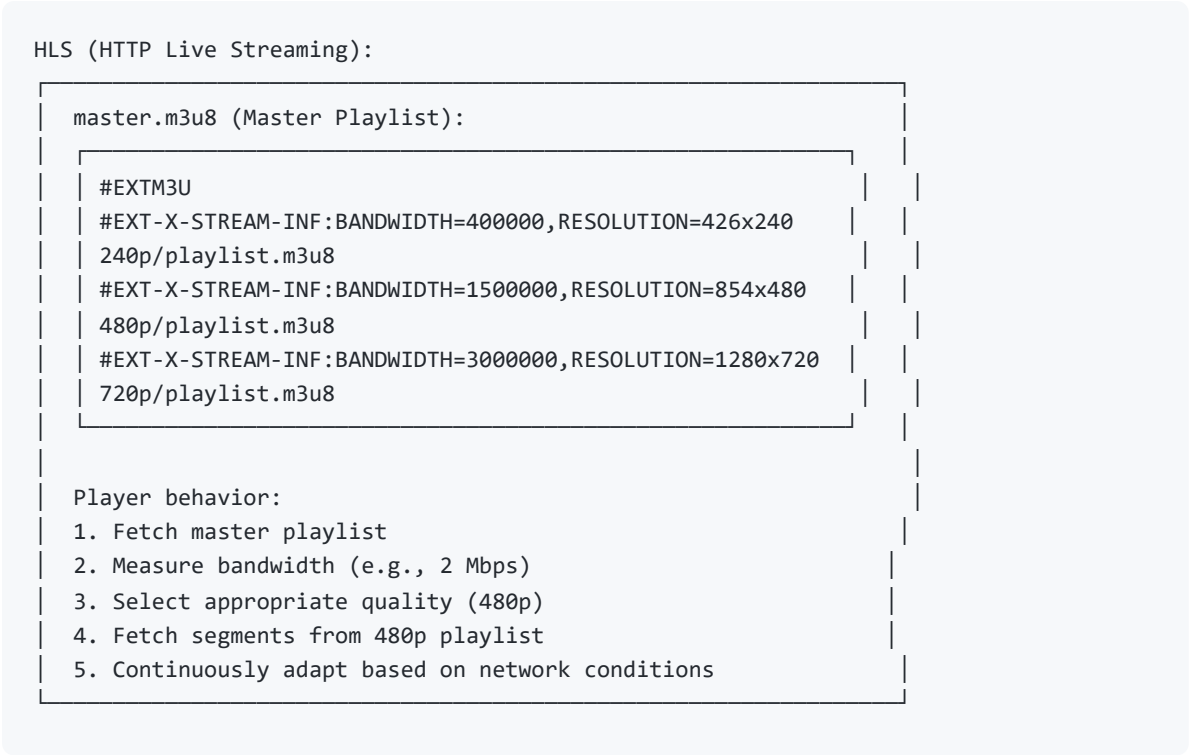
Key Components

Component	Purpose	Technology
Upload Service	Handle file uploads	Chunked upload API
Transcoding	Convert to multiple formats	FFmpeg, Media Services
Storage	Store video segments	Blob Storage
CDN	Global video delivery	Azure CDN, Akamai
Metadata DB	Video info, stats	PostgreSQL
Search	Video discovery	Elasticsearch
Recommendation	Personalized suggestions	ML service

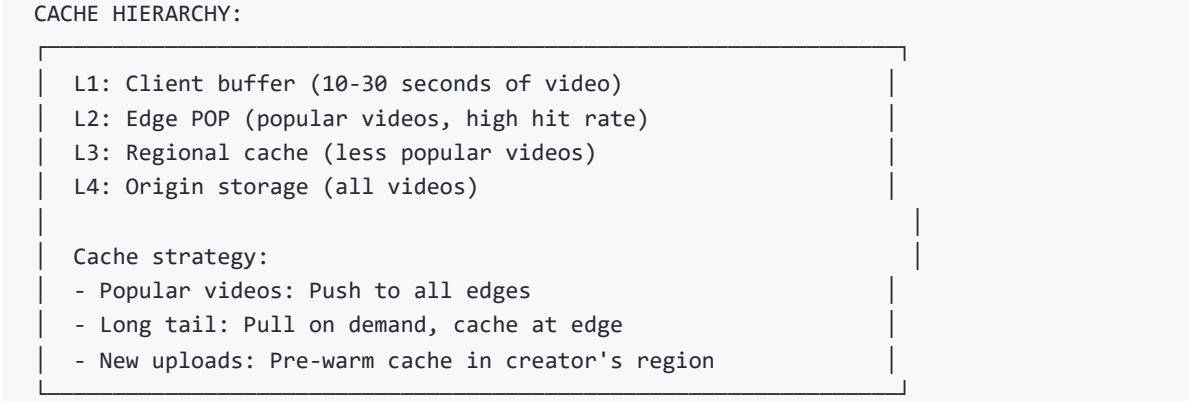
Video Processing Pipeline



Adaptive Bitrate Streaming



CDN and Caching



Database Schema

```sql
-- Videos
CREATE TABLE videos (
    id BIGINT PRIMARY KEY,
    creator_id BIGINT,
    title VARCHAR(255),
    description TEXT,
    duration_seconds INT,
```

```
    upload_status VARCHAR(20),  -- processing, ready, failed
    visibility VARCHAR(20),     -- public, private, unlisted
    view_count BIGINT DEFAULT 0,
    like_count BIGINT DEFAULT 0,
    created_at TIMESTAMP,
    published_at TIMESTAMP
);

-- Video Files (different qualities)
CREATE TABLE video_files (
    video_id BIGINT,
    resolution VARCHAR(10),     -- 240p, 480p, 720p, 1080p
    bitrate INT,
    codec VARCHAR(20),
    storage_path TEXT,
    file_size BIGINT,
    PRIMARY KEY (video_id, resolution)
);

-- View counts (separate for write scalability)
CREATE TABLE view_counts (
    video_id BIGINT PRIMARY KEY,
    view_count BIGINT,
    last_updated TIMESTAMP
);
```

Scalability Considerations

- **Upload**: Chunked uploads with resume capability
- **Processing**: Parallel transcoding jobs, auto-scale workers
- **Storage**: Tiered storage (hot for new, cold for old)
- **CDN**: 95%+ cache hit ratio target
- **View counts**: Async aggregation to avoid write contention

Trade-offs

Decision	Option A	Option B	Recommendation
Streaming	Progressive	Adaptive (HLS)	**HLS** - better UX
Transcoding	On-upload	On-demand	**On-upload** - faster playback
Storage	Single quality	All qualities	**All** - adaptive streaming
View counts	Real-time	Batched	**Batched** - scale

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
CDN	Global video segment delivery with edge caching	Tech Guide
Kafka	Video processing job queue and event distribution	Tech Guide
Redis	View count aggregation and metadata caching	Tech Guide

How to Explain It Well

> **Opening**: "Video streaming needs efficient processing and global delivery. I'll design for 5 billion daily views with adaptive bitrate streaming."

> **Key talking points**:
>
> - "Transcode to multiple resolutions: 240p to 4K for all devices"
> - "HLS segments allow adaptive bitrate - switch quality mid-stream"
> - "CDN serves 95%+ of requests from edge cache"
> - "Async view count aggregation to avoid database bottleneck"
> - "Chunked upload with resume for large file reliability"

---

19. Ensure Message Ordering in Chat

Key Terms:

Term	Full Name	What it does
Lamport Clock	Logical timestamp	Counter-based ordering without synchronized clocks
Vector Clock	Distributed timestamp	Tracks causality across multiple nodes
NTP	Network Time Protocol	Synchronizes clocks across machines
INCR	Redis increment	Atomic counter increment operation
Partition	Kafka shard	Ordered queue for messages with same key

Causal Ordering	Cause-before-effect	If A caused B, A appears before B for all viewers

Problem Statement

Design a chat system that guarantees messages appear in the correct order for all participants, even with network delays and distributed servers.

Clarifying Questions to Ask

1. **Scope**: Per-conversation or global ordering?
2. **Participants**: 1:1 chat or group chat? Max group size?
3. **Consistency**: Strict ordering or eventual ordering OK?
4. **Latency**: Real-time requirement? (<100ms?)
5. **Offline**: Handle offline users rejoining?
6. **Edits/Deletes**: Support editing/deleting messages?
7. **History**: How much history to maintain order for?

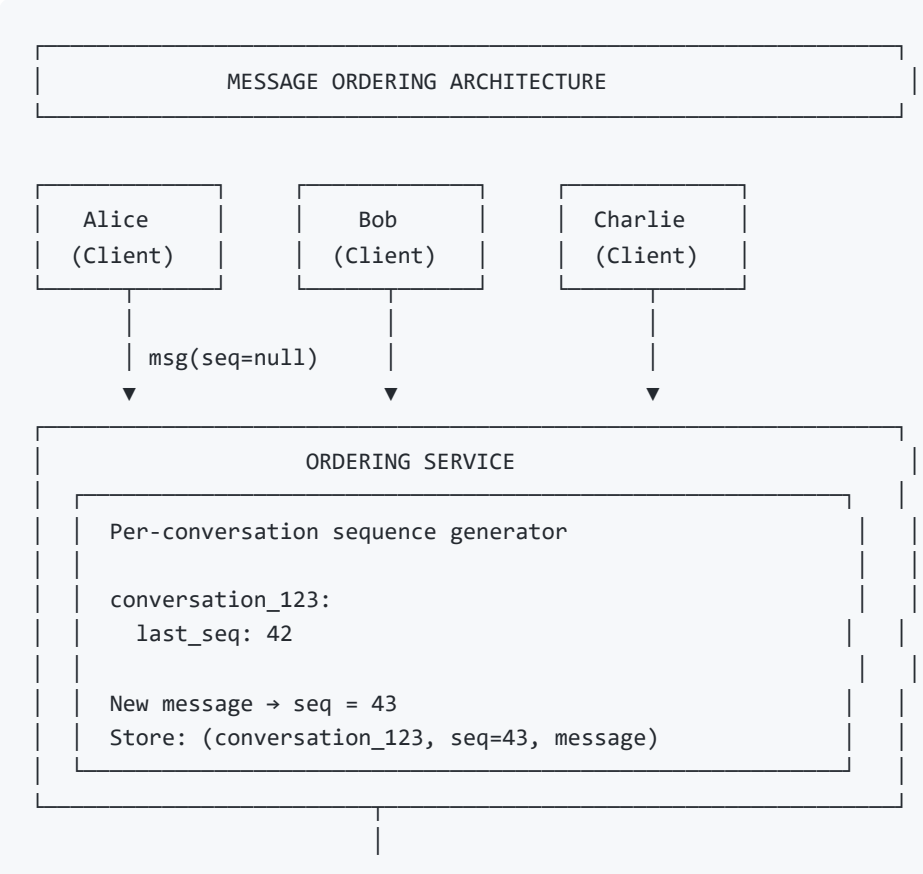
Design Reasoning

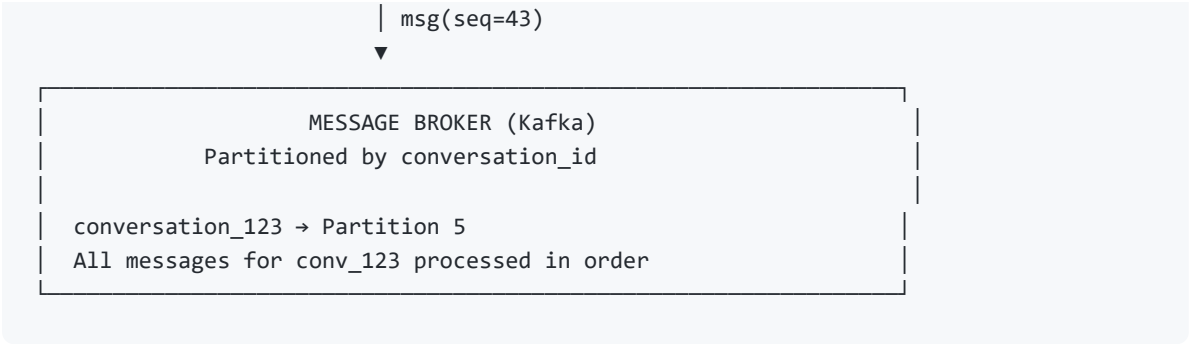
> Core philosophy: Use a centralized sequence generator per conversation to guarantee total ordering, with Kafka partitions ensuring ordered delivery at scale.

Decision	Choice	Why	Alternative Considered
Ordering scope	Per-conversation	Global ordering doesn't scale — conversations are the natural ordering boundary	Global ordering (bottleneck at high volume)
Sequence generation	Redis INCR	Atomic counter provides strict sequential IDs with sub-millisecond latency	Database auto-increment (higher latency per message)
Message broker	Kafka (partitioned by conversation_id)	Single partition = single consumer = guaranteed order within conversation	RabbitMQ (no partition-level ordering guarantee)

Timestamp strategy	Hybrid (wall clock + logical counter + client_id)	Combines real-time correlation with causal ordering and deterministic tie-breaking	Pure Lamport clocks (no real-time correlation)
Out-of-order handling	Client-side buffering with gap detection	Keeps server simple; clients reorder using sequence numbers and request missing messages	Server-side hold-and-release (adds latency for all messages)
Concurrent sends	Server-arrival order with deterministic tie-breaking	Predictable and consistent across all clients; avoids relying on unreliable client clocks	Client timestamp ordering (clock skew across devices)
Idempotency	Client-generated idempotency key per message	Prevents duplicate delivery on retries without server-side dedup logic	Server-side dedup window (memory overhead, time-limited)

High-Level Design





API Design

Message APIs (with ordering guarantees)

Method	Endpoint	Description
`POST`	`/api/v1/messages`	Send a message with sequence tracking
`GET`	`/api/v1/messages?chatId={id}&after={seqId}`	Fetch messages after a sequence ID
`GET`	`/api/v1/messages/sync?lastSeqId={id}`	Sync missed messages

Send Message Request (with ordering metadata)

```
POST /api/v1/messages
{
  "chat_id": "chat_123",
  "content": "Hello!",
  "client_timestamp": 1672531200000,
  "client_seq_id": "uuid-456",
  "idempotency_key": "idem-789"
}

Response: 201 Created
{
  "message_id": "msg_001",
  "server_seq_id": 42,
  "server_timestamp": 1672531200050,
  "chat_id": "chat_123"
}
```

WebSocket Events (ordered delivery)

Event	Direction	Payload

`message.ordered`	Server → Client	`{ chatId, message, seqId, prevSeqId }`
`message.ack`	Server → Client	`{ clientSeqId, serverSeqId }`
`sync.required`	Server → Client	`{ chatId, fromSeqId, toSeqId }`

The `seqId` field enables clients to detect gaps and request re-delivery.

Ordering Strategies

```
1. CENTRALIZED SEQUENCE GENERATOR
┌─────────────────────────────────────────────────────────────┐
│  Single source assigns sequence numbers per conversation    │
│                                                             │
│  Redis: INCR conversation:123:seq → 43                      │
│                                                             │
│  Pros: Simple, strict ordering                              │
│  Cons: Single point of failure, latency                     │
│                                                             │
│  Mitigation: Shard by conversation_id across Redis nodes    │
└─────────────────────────────────────────────────────────────┘

2. LAMPORT TIMESTAMPS
┌─────────────────────────────────────────────────────────────┐
│  Each client maintains local counter                        │
│  On send: timestamp = max(local, received) + 1              │
│                                                             │
│  Alice: (1, A) → (2, A) → (3, A)                            │
│  Bob:   (1, B) → (2, B)                                     │
│                                                             │
│  Merge: (1,A), (1,B), (2,A), (2,B), (3,A)                   │
│  Tie-breaker: client_id                                     │
│                                                             │
│  Pros: Decentralized, works offline                         │
│  Cons: Doesn't reflect real-time order                      │
└─────────────────────────────────────────────────────────────┘

3. HYBRID TIMESTAMP (Recommended)
┌─────────────────────────────────────────────────────────────┐
│  Combine wall clock + logical counter                       │
│                                                             │
│  timestamp = (physical_time, logical_counter, client_id)    │
│                                                             │
│  Example:                                                   │
│  (1705312345000, 0, alice) < (1705312345000, 1, bob)        │
│                                                             │
│  Pros: Causal ordering + real-time correlation              │
│  Cons: Requires synchronized clocks (NTP)                   │
└─────────────────────────────────────────────────────────────┘
```

Handling Out-of-Order Delivery

```
CLIENT-SIDE REORDERING:
┌─────────────────────────────────────────────────────────────────┐
│  Client maintains expected next sequence number                 │
│                                                                 │
│  Received: [seq=43]                                             │
│  Expected: seq=42                                               │
│  Action: Buffer seq=43, wait for seq=42                         │
│                                                                 │
│  Timeout: After 5s, display seq=43 with gap indicator           │
│  Late arrival: Insert seq=42 in correct position                │
│                                                                 │
│  UI: "Message may be out of order" or reorder animation         │
└─────────────────────────────────────────────────────────────────┘

SERVER-SIDE ORDERING:
┌─────────────────────────────────────────────────────────────────┐
│  Kafka partition per conversation                               │
│  Single consumer per partition = ordered processing             │
│                                                                 │
│  Partition 5 (conversation_123):                                │
│  [msg_seq_41] → [msg_seq_42] → [msg_seq_43]                     │
│                                                                 │
│  Consumer processes in order, delivers to clients               │
└─────────────────────────────────────────────────────────────────┘
```

Concurrent Send Handling

```
SCENARIO: Alice and Bob send at same millisecond
┌─────────────────────────────────────────────────────────────────┐
│  Alice sends "Hi" at T=1000ms                                   │
│  Bob sends "Hello" at T=1000ms                                  │
│                                                                 │
│  Solution: Server assigns order based on arrival                │
│                                                                 │
│  Server receives Alice first:                                   │
│    Alice "Hi" → seq=42                                          │
│    Bob "Hello" → seq=43                                         │
│                                                                 │
│  All clients see: [42: "Hi"], [43: "Hello"]                     │
│                                                                 │
│  Note: This is "server order" not "send order"                  │
│  For true causality, use vector clocks                          │
└─────────────────────────────────────────────────────────────────┘
```

Database Schema

```
-- Messages with ordering
CREATE TABLE messages (
    conversation_id BIGINT,
    sequence_num BIGINT,          -- Ordering within conversation
    message_id UUID,              -- Globally unique
    sender_id BIGINT,
    content TEXT,
    created_at TIMESTAMP,
    server_timestamp TIMESTAMP,   -- For debugging
    PRIMARY KEY (conversation_id, sequence_num)
);

-- Sequence tracking
CREATE TABLE conversation_sequences (
    conversation_id BIGINT PRIMARY KEY,
    last_sequence BIGINT,
    updated_at TIMESTAMP
);
```

Scalability Considerations

- **Partition by conversation**: Each conversation on one Kafka partition
- **Sequence generator sharding**: Shard Redis by conversation_id
- **Client buffering**: Handle temporary out-of-order on client
- **Idempotency**: Message ID prevents duplicate delivery

Trade-offs

Decision	Option A	Option B	Recommendation
Ordering scope	Global	Per-conversation	**Per-conversation** - scalable
Sequencing	Centralized	Distributed	**Centralized** - simpler
Ties	First-wins	Deterministic	**Deterministic** - consistent
Gaps	Block	Show with indicator	**Show + fetch** - better UX

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

Kafka	Partitioned message broker ensuring per-conversation ordering	Tech Guide
Redis	Centralized sequence number generator with atomic INCR	Tech Guide

How to Explain It Well

> **Opening**: "Message ordering requires careful coordination. I'll use per-conversation sequence numbers with Kafka for ordered delivery."

> **Key talking points**:
>
> - "Kafka partition per conversation guarantees processing order"
> - "Redis INCR assigns sequence numbers atomically"
> - "Client buffers out-of-order messages, reorders on display"
> - "Hybrid timestamps combine wall clock with logical counter"
> - "Tie-breaker: deterministic ordering by client_id for concurrent sends"

---

20. Secure Sensitive Data (GDPR/HIPAA)

Key Terms:

Term	Full Name	What it does
GDPR	General Data Protection Regulation	EU privacy law for personal data
HIPAA	Health Insurance Portability and Accountability Act	US healthcare data privacy law
PII	Personally Identifiable Information	Data that identifies individuals (name, SSN, email)
PHI	Protected Health Information	Medical records and health data
PCI-DSS	Payment Card Industry Data Security Standard	Credit card data protection
HSM	Hardware Security Module	Tamper-resistant device for cryptographic keys

AES-256	Advanced Encryption Standard 256-bit	Strong symmetric encryption algorithm
RBAC	Role-Based Access Control	Permissions assigned by role (admin, user)
ABAC	Attribute-Based Access Control	Permissions based on attributes (department, time)
DEK	Data Encryption Key	Key that encrypts actual data (encrypted by master key)
TLS	Transport Layer Security	Encryption for data in transit
OIDC	OpenID Connect	Authentication layer on top of OAuth 2.0

Problem Statement

Design a system that securely handles sensitive data while complying with GDPR, HIPAA, and other privacy regulations.

Clarifying Questions to Ask

1. **Data types**: PII, PHI, financial, or all?
2. **Regulations**: Which compliance frameworks? (GDPR, HIPAA, PCI-DSS?)
3. **Geography**: Where are users and data located?
4. **Retention**: How long to keep data? Right to deletion?
5. **Access**: Who needs access? Audit requirements?
6. **Encryption**: At rest, in transit, or end-to-end?
7. **Breach**: Notification requirements?

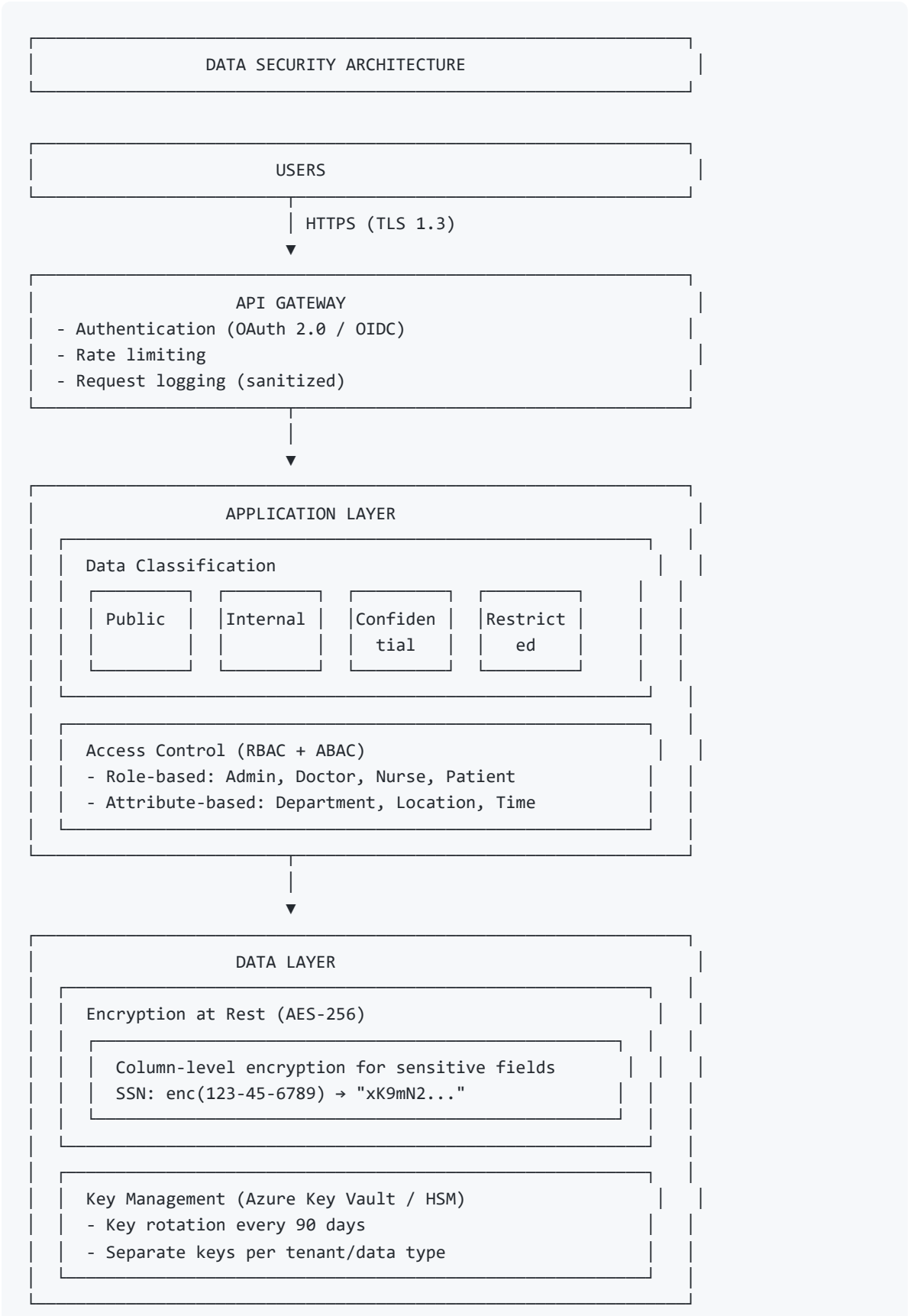
Design Reasoning

> **Core philosophy**: Defense in depth — layer encryption, access control, and audit logging so that no single failure exposes sensitive data.

Decision	Choice	Why	Alternative Considered
Encryption scope	Application-level (column-level)	Encrypt only sensitive fields (SSN, PHI) for surgical control; non-	Full database encryption (cannot query

		sensitive data stays queryable	encrypted fields, coarser control)
Key management	HSM-backed (Azure Key Vault)	Regulatory requirement — HIPAA/PCI-DSS mandate tamper-resistant key storage; master key never leaves hardware	Software key store (acceptable for non-regulated data, not compliant)
Key hierarchy	Master key → per-tenant DEKs	Envelope encryption limits blast radius — compromising one DEK only exposes one tenant's data	Single key for all data (one compromise exposes everything)
Access control	RBAC + ABAC combined	RBAC provides role-based baseline; ABAC adds context (department, time-of-day) required for HIPAA minimum necessary access	RBAC only (insufficient for context-sensitive healthcare access)
Payment data	Tokenization	PCI-DSS compliance — actual card numbers never stored; tokens are useless if breached	Encryption only (still stores card data, higher PCI scope)
Data deletion	Soft delete + scheduled purge	Supports GDPR right-to-erasure with audit trail; legal holds can pause purge	Hard delete (no recovery, harder to audit for compliance)
Audit logging	Immutable, cryptographically chained	HIPAA requires 7-year tamper-evident audit trail; chaining detects log manipulation	Append-only files (no tamper evidence, harder to verify integrity)
Data residency	Region-isolated storage with geo-routing	GDPR requires EU data stays in EU; routing by user country ensures compliance	Single-region (violates data residency laws for global users)

High-Level Design


DATA SECURITY ARCHITECTURE
USERS
HTTPS (TLS 1.3)
API GATEWAY
- Authentication (OAuth 2.0 / OIDC)
- Rate limiting
- Request logging (sanitized)
APPLICATION LAYER
Data Classification
Public
Internal
Confiden tial
Restrict ed
Access Control (RBAC + ABAC)
- Role-based: Admin, Doctor, Nurse, Patient
- Attribute-based: Department, Location, Time
DATA LAYER
Encryption at Rest (AES-256)
Column-level encryption for sensitive fields
SSN: enc(123-45-6789) → "xK9mN2..."
Key Management (Azure Key Vault / HSM)
- Key rotation every 90 days
- Separate keys per tenant/data type

API Design

Data Subject Rights APIs (GDPR)

Method	Endpoint	Description
`GET`	`/api/v1/data-subjects/{userId}/export`	Export all user data (Right of Access)
`DELETE`	`/api/v1/data-subjects/{userId}`	Delete all user data (Right to Erasure)
`PUT`	`/api/v1/data-subjects/{userId}/consent`	Update consent preferences
`GET`	`/api/v1/data-subjects/{userId}/consent`	Get current consent status
`POST`	`/api/v1/data-subjects/{userId}/portability`	Request data portability export

Encryption & Key Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/keys/rotate`	Trigger key rotation
`GET`	`/api/v1/keys/status`	Get encryption key status
`POST`	`/api/v1/encrypt`	Encrypt sensitive field
`POST`	`/api/v1/decrypt`	Decrypt sensitive field (audit-logged)

Audit APIs

Method	Endpoint	Description
`GET`	`/api/v1/audit/logs?resource={type}&from={date}`	Query audit logs
`GET`	`/api/v1/audit/access/{userId}`	Get access history for a user
`POST`	`/api/v1/audit/report`	Generate compliance report

Key Security Controls

```
1. ENCRYPTION
┌─────────────────────────────────────────────────────────────────┐
│  In Transit: TLS 1.3 for all connections                        │
```

```
│  At Rest: AES-256 for storage                                    │
│  Application-level: Encrypt sensitive fields before storage     │
│                                                                  │
│  Key hierarchy:                                                  │
│  ┌──────────────────────────────────────────────────────┐        │
│  │  Master Key (HSM) → never leaves hardware            │        │
│  │       │                                              │        │
│  │       ▼                                              │        │
│  │  Data Encryption Key (DEK) → encrypts actual data    │        │
│  │  (DEK encrypted by Master Key, stored alongside data)│        │
│  └──────────────────────────────────────────────────────┘        │
└──────────────────────────────────────────────────────────────────┘

2. DATA MASKING & TOKENIZATION

┌──────────────────────────────────────────────────────────────────┐
│  Tokenization: Replace sensitive data with non-sensitive token   │
│                                                                  │
│  Original: 4532-1234-5678-9012 (credit card)                     │
│  Token:    tok_abc123xyz                                         │
│  Mapping stored in secure token vault                            │
│                                                                  │
│  Masking: Show partial data                                      │
│  SSN: ***-**-6789                                                │
│  Email: j***@example.com                                         │
└──────────────────────────────────────────────────────────────────┘

3. ACCESS CONTROLS

┌──────────────────────────────────────────────────────────────────┐
│  RBAC (Role-Based Access Control):                               │
│    Doctor → read patient records in their department             │
│    Nurse → read vitals, update medications                       │
│    Admin → manage users, no patient data access                  │
│                                                                  │
│  ABAC (Attribute-Based Access Control):                          │
│    Allow IF:                                                     │
│      user.department == patient.department                       │
│      AND user.role IN ['doctor', 'nurse']                        │
│      AND time.current BETWEEN shift.start AND shift.end          │
└──────────────────────────────────────────────────────────────────┘
```

GDPR Compliance Features

```
1. RIGHT TO ACCESS (Data Portability)

┌──────────────────────────────────────────────────────────────────┐
│  User requests: "Give me all my data"                            │
│  System: Export all PII in machine-readable format (JSON)        │
│  Timeline: Within 30 days                                        │
└──────────────────────────────────────────────────────────────────┘

2. RIGHT TO BE FORGOTTEN (Data Deletion)

┌──────────────────────────────────────────────────────────────────┐
│  User requests: "Delete all my data"                             │
│  System:                                                         │
```

```
│    1. Delete from primary database                                  │
│    2. Delete from backups (within retention period)                 │
│    3. Delete from analytics/logs (or anonymize)                     │
│    4. Notify third parties to delete                                │
│    5. Generate deletion certificate                                 │
│                                                                     │
│  Exceptions: Legal holds, regulatory requirements                   │
└─────────────────────────────────────────────────────────────────────┘

3. CONSENT MANAGEMENT

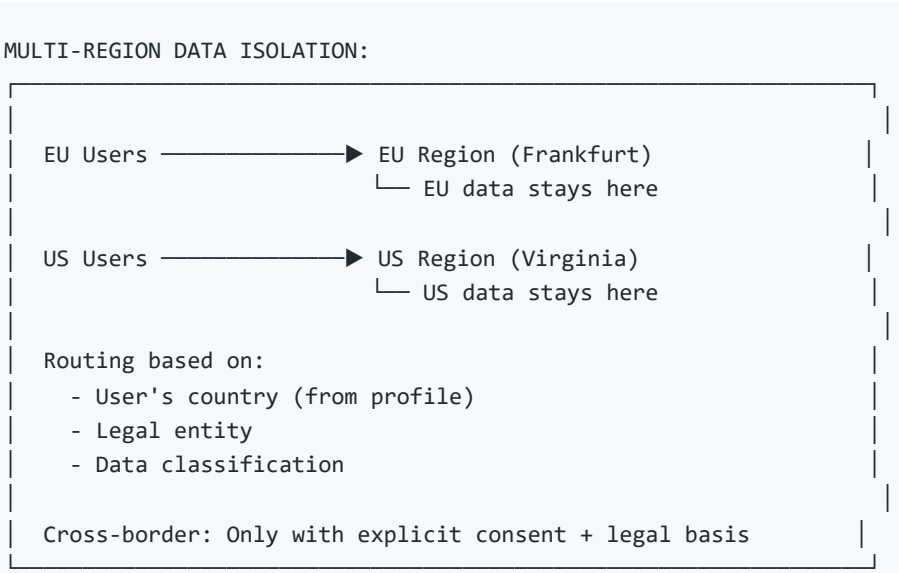
┌─────────────────────────────────────────────────────────────────────┐
│  Track consent per purpose:                                         │
│    - Marketing emails: Yes (2024-01-15)                             │
│    - Analytics: No                                                  │
│    - Third-party sharing: Yes (2024-01-15)                          │
│                                                                     │
│  Respect consent in all data processing                             │
└─────────────────────────────────────────────────────────────────────┘
```

Audit Logging

```
AUDIT LOG ENTRY:
{
  "timestamp": "2024-01-15T14:30:00Z",
  "actor": {
    "user_id": "user_123",
    "role": "doctor",
    "ip_address": "10.0.1.50",
    "session_id": "sess_abc"
  },
  "action": "READ",
  "resource": {
    "type": "patient_record",
    "id": "patient_456",
    "fields_accessed": ["name", "diagnosis", "medications"]
  },
  "context": {
    "reason": "treatment",
    "department": "cardiology"
  },
  "result": "SUCCESS"
}

REQUIREMENTS:
  - Immutable (write-once storage)
  - Tamper-evident (cryptographic chaining)
  - Retained for 7 years (HIPAA)
  - Searchable for investigations
```

Data Residency



Trade-offs

Decision	Option A	Option B	Recommendation
Encryption	DB-level	Application-level	**Application** - more control
Key storage	Software	HSM	**HSM** - for regulated data
Deletion	Hard delete	Soft delete + purge	**Soft + scheduled purge**
Logging	Minimal	Comprehensive	**Comprehensive** - compliance

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
HSM	Hardware-based master key storage for encryption key hierarchy	Tech Guide

How to Explain It Well

> **Opening**: "Data security requires defense in depth: encryption, access control, and audit logging. I'll design for GDPR and HIPAA compliance."

> **Key talking points**:
>
> - "Encrypt sensitive fields at application level, keys in HSM"

> - "Tokenization for payment data - actual numbers never stored"
> - "RBAC + ABAC: role defines base access, attributes refine it"
> - "Right to be forgotten: cascading delete with certificate"
> - "Immutable audit logs with 7-year retention for HIPAA"

---

21. Design Real-Time Location Tracking

Key Terms:

Term	Full Name	What it does
GPS	Global Positioning System	Satellite-based location (10m accuracy)
Geohash	Geographic Hash	Encodes lat/lng into hierarchical grid cells
GEORADIUS	Redis Geo Command	Finds all points within radius of location
Geofence	Geographic boundary	Virtual perimeter triggering enter/exit events
R-tree	Rectangle tree	Spatial index for efficient polygon queries
SSE	Server-Sent Events	One-way server-to-client streaming over HTTP
TimescaleDB	Time-series database	PostgreSQL extension optimized for time-series

Problem Statement

Design a system that tracks and displays real-time locations of moving objects (vehicles, delivery drivers, assets) on a map.

Clarifying Questions to Ask

1. **Scale**: How many tracked objects? (1M vehicles?)
2. **Update frequency**: How often to update? (Every 5 seconds?)
3. **Precision**: GPS accuracy needed? (10 meters?)
4. **Features**: Historical tracking, geofencing, ETA?
5. **Viewers**: How many simultaneous map viewers?
6. **Battery**: Mobile devices need power optimization?

7. **Offline**: Handle intermittent connectivity?

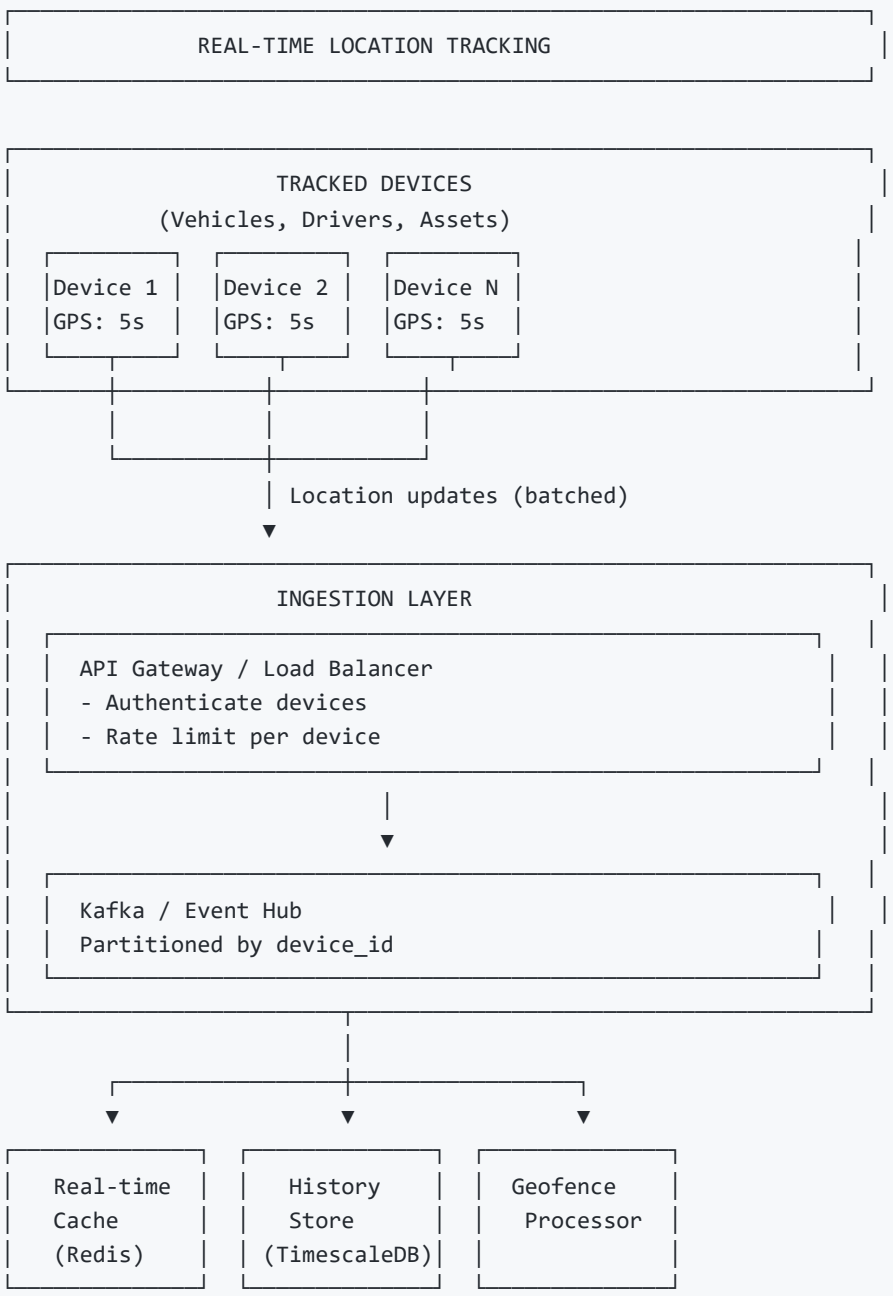
Design Reasoning

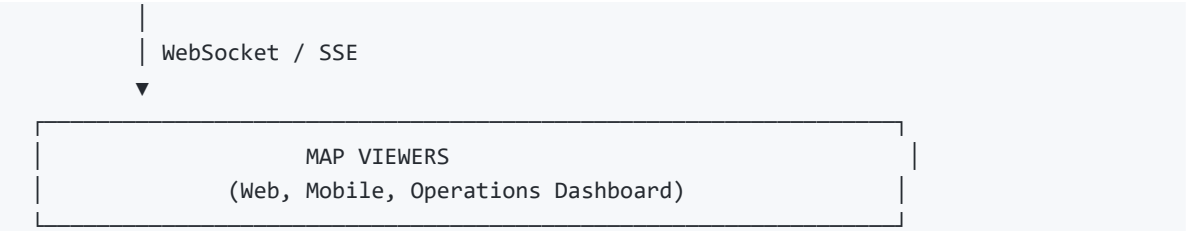
> **Core philosophy**: Separate hot-path (real-time current positions) from cold-path (historical tracking) to optimize each independently for latency and storage.

Decision	Choice	Why	Alternative Considered
Ingestion buffer	Kafka (partitioned by device_id)	Handles burst traffic from 1M+ devices; partition-per-device ensures ordered updates per entity	Direct-to-database writes (overwhelms DB during traffic spikes)
Real-time cache	Redis Geospatial (GEOADD/GEORADIUS)	O(log N) spatial queries for "nearby" lookups; in-memory for sub-millisecond current-position reads	PostgreSQL PostGIS (too slow for real-time at this write volume)
History store	TimescaleDB	Time-series optimized with automatic time-based partitioning; built on PostgreSQL for familiar query syntax	Raw PostgreSQL (lacks time-partitioning optimizations, slower range queries)
Client push	WebSocket with geohash subscriptions	Viewers only receive updates for their visible map region, reducing bandwidth by ~90%	Polling (high latency, wasteful for real-time maps)
Update batching	Adaptive frequency (3s moving, 60s stationary)	Reduces battery drain by 70% on mobile devices while maintaining accuracy during movement	Fixed interval (wastes battery when stationary, or misses movement)
Spatial partitioning	Geohash	Hierarchical grid enables efficient	Quadtree (more complex to

		region-based subscriptions and proximity queries	distribute across nodes)
Geofence evaluation	Stream processing with R-tree index	Spatial index avoids checking every geofence for every update; skip check if movement below threshold	Brute-force polygon checks (O(N) per update, doesn't scale)

High-Level Design





API Design

Location APIs

Method	Endpoint	Description
`PUT`	`/api/v1/locations/{entityId}`	Update entity location
`GET`	`/api/v1/locations/{entityId}`	Get current location
`GET`	`/api/v1/locations/{entityId}/history?from={t1}&to={t2}`	Get location history
`GET`	`/api/v1/locations/nearby?lat={lat}&lng={lng}&radius={m}`	Find nearby entities

Geofence APIs

Method	Endpoint	Description
`POST`	`/api/v1/geofences`	Create a geofence
`GET`	`/api/v1/geofences/{fenceId}`	Get geofence details
`DELETE`	`/api/v1/geofences/{fenceId}`	Delete a geofence

Location Update (High Frequency - via WebSocket/UDP)

```
{
  "entity_id": "driver_123",
  "lat": 47.6062,
  "lng": -122.3321,
  "accuracy": 10,
  "speed": 45.2,
  "heading": 270,
```

```
    "timestamp": 1672531200000
  }
```

WebSocket Events

Event	Direction	Payload
`location.updated`	Server → Client	`{ entityId, lat, lng, speed }`
`geofence.enter`	Server → Client	`{ entityId, fenceId, timestamp }`
`geofence.exit`	Server → Client	`{ entityId, fenceId, timestamp }`

Key Components

Component	Purpose	Technology
Device SDK	Collect & send GPS	Mobile SDK
Ingestion API	Receive location updates	REST / gRPC
Message Queue	Buffer and distribute	Kafka
Real-time Cache	Current locations	Redis (Geospatial)
History Store	Historical tracking	TimescaleDB
Geofence Engine	Enter/exit alerts	Stream processor
Map Service	Render locations	WebSocket + Mapbox

Location Update Protocol

```
DEVICE TO SERVER:
┌──────────────────────────────────────────────────────────────────┐
│  Batch updates to reduce battery and bandwidth:                  │
│                                                                   │
│  POST /locations                                                 │
│  {                                                               │
│    "device_id": "vehicle_123",                                   │
│    "locations": [                                                │
│      {"lat": 47.6062, "lng": -122.3321, "ts": 1705312340},      │
│      {"lat": 47.6065, "lng": -122.3318, "ts": 1705312345},      │
│      {"lat": 47.6068, "lng": -122.3315, "ts": 1705312350}       │
│    ],                                                            │
│    "battery": 85,                                                │
│    "accuracy": 10                                                │
│  }                                                               │
```

```
│                                                                 │
│  Adaptive frequency:                                            │
│    - Moving fast: every 3 seconds                               │
│    - Moving slow: every 10 seconds                              │
│    - Stationary: every 60 seconds                               │
└─────────────────────────────────────────────────────────────────┘
```

Geospatial Indexing

```
REDIS GEOSPATIAL:
┌─────────────────────────────────────────────────────────────────┐
│  GEOADD vehicles <lng> <lat> vehicle_123                        │
│  GEOADD vehicles <lng> <lat> vehicle_456                        │
│                                                                 │
│  Find vehicles within 5km of point:                             │
│  GEORADIUS vehicles -122.33 47.60 5 km WITHCOORD WITHDIST       │
│                                                                 │
│  Result:                                                        │
│  1) vehicle_123, 0.5km, (47.6062, -122.3321)                    │
│  2) vehicle_456, 2.1km, (47.5890, -122.3100)                    │
└─────────────────────────────────────────────────────────────────┘

GEOHASH FOR PARTITIONING:
┌─────────────────────────────────────────────────────────────────┐
│  Geohash divides world into grid cells                          │
│                                                                 │
│  47.6062, -122.3321 → geohash "c23nb"                           │
│                                                                 │
│  Subscribe to updates in region:                                │
│  SUBSCRIBE location:c23nb                                       │
│                                                                 │
│  Publish when vehicle moves:                                    │
│  PUBLISH location:c23nb {vehicle_123, lat, lng}                 │
└─────────────────────────────────────────────────────────────────┘
```

Geofencing

```
GEOFENCE DEFINITION:
┌─────────────────────────────────────────────────────────────────┐
│  {                                                              │
│    "fence_id": "warehouse_zone",                                │
│    "type": "polygon",                                           │
│    "coordinates": [[lat1,lng1], [lat2,lng2], ...],              │
│    "triggers": ["enter", "exit"],                               │
│    "actions": ["notify", "log"]                                 │
│  }                                                              │
└─────────────────────────────────────────────────────────────────┘

PROCESSING:
┌─────────────────────────────────────────────────────────────────┐
```

```
│  For each location update:                                    │
│    1. Find geofences near current location                    │
│    2. Check point-in-polygon                                  │
│    3. Compare with previous state                             │
│    4. If state changed → trigger event                        │
│                                                               │
│  Optimization:                                                │
│    - Spatial index for geofences (R-tree)                     │
│    - Cache previous state per device                          │
│    - Skip check if location change < threshold                │
└──────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Ingestion**: Kafka partitioned by device_id for parallel processing
- **Real-time cache**: Redis Cluster with geospatial indexing
- **Map viewers**: WebSocket with geohash-based subscriptions
- **History**: TimescaleDB with time-based partitioning
- **Battery**: Adaptive update frequency based on movement

Trade-offs

Decision	Option A	Option B	Recommendation
Protocol	REST	gRPC	**REST** - simpler, HTTP/2 for batching
Updates	Real-time	Batched	**Batched** - battery efficient
Storage	SQL	Time-series DB	**TimescaleDB** - optimized for time-series
Push	Polling	WebSocket	**WebSocket** - real-time updates

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	High-throughput ingestion of location updates	Tech Guide
Redis	Geospatial indexing with GEOADD/GEORADIUS commands	Tech Guide
Geohash	Spatial partitioning for region-based subscriptions	Tech Guide

How to Explain It Well

> **Opening**: "Location tracking needs efficient ingestion and real-time delivery. I'll design for 1M devices updating every 5 seconds."

> **Key talking points**:
>
> - "Batch location updates from devices - reduces battery by 70%"
> - "Redis GEOADD/GEORADIUS for O(log N) spatial queries"
> - "Geohash-based subscriptions: viewers only get updates in their viewport"
> - "Adaptive frequency: fast updates when moving, slow when stationary"
> - "Geofence events processed in stream with R-tree spatial index"

---

29. Design a Recommendation Engine

Key Terms:

Term	Full Name	What it does
Collaborative Filtering	User-based recommendations	"Users who liked X also liked Y"
Content-Based Filtering	Feature-based recommendations	Match item features to user preferences
Matrix Factorization	ML technique	Discovers latent factors from sparse user-item matrix
Cold Start	New user/item problem	No history to base recommendations on
ANN	Approximate Nearest Neighbors	Fast similarity search in embedding space
Two-Tower Model	Deep learning architecture	Separate user and item embedding networks
SVD	Singular Value Decomposition	Matrix factorization algorithm
ALS	Alternating Least Squares	Scalable matrix factorization for Spark

Problem Statement

Design a system that provides personalized recommendations for products, content, or connections based on user behavior and preferences.

Clarifying Questions to Ask

1. **Domain**: Products, movies, music, friends, or content?
2. **Scale**: How many users? Items? (1M users, 10M items?)
3. **Latency**: Real-time (<100ms) or precomputed?
4. **Cold start**: How to handle new users/items?
5. **Freshness**: How often should recommendations update?
6. **Explainability**: Need to explain why recommended?
7. **Diversity**: Balance relevance vs. discovery?

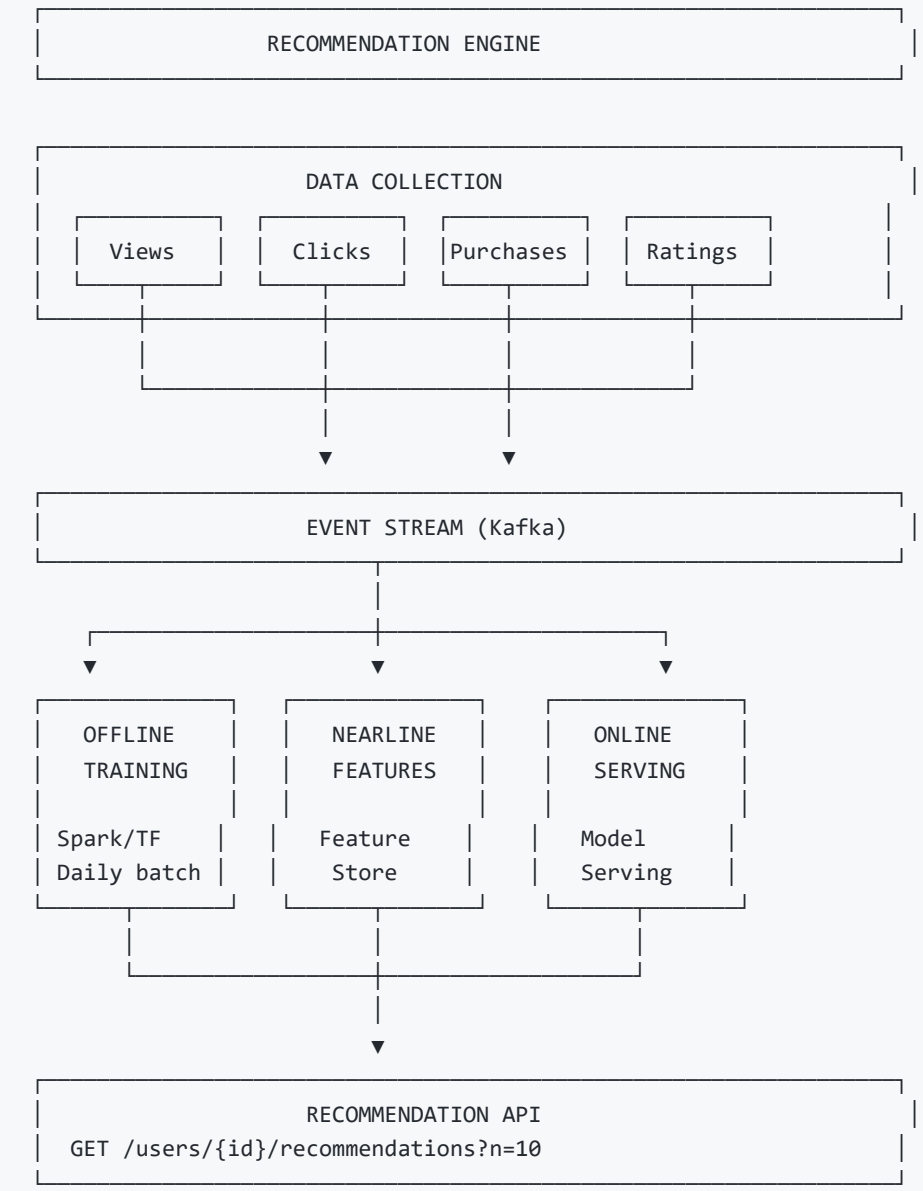
Design Reasoning

> **Core philosophy**: Combine offline model training with online serving to deliver personalized, low-latency recommendations while continuously learning from user behavior.

Decision	Choice	Why	Alternative Considered
Event streaming	Kafka	Handles bursty user-interaction events with durable, partitioned ingestion	Direct DB writes — can't buffer traffic spikes after viral content
ML approach	Hybrid (Collaborative + Content-Based)	Overcomes cold-start limitations of either approach alone	Pure collaborative filtering — fails completely for new items
Model architecture	Two-Tower model	Precompute item embeddings offline, compute user embedding online for <100ms serving	Single deep network — can't precompute, too slow at serving time
Serving strategy	Batch precompute + real-time re-rank	Fast reads from cache with fresh personalization on top	Pure real-time — too expensive at 1M+ users, latency spikes

Similarity search	ANN (Approximate Nearest Neighbors)	Sub-millisecond lookup in high-dimensional embedding space	Exact k-NN — O(N) per query, impractical at 10M items
Feature storage	Redis-backed feature store	Low-latency feature retrieval for online model inference	Database lookups — adds 10-50ms per request, breaks latency budget

High-Level Design



API Design

Recommendation APIs

Method	Endpoint	Description
`GET`	`/api/v1/recommendations?userId={id}&type={type}&limit=20`	Get personalized recommendations
`GET`	`/api/v1/recommendations/similar/{itemId}`	Get similar items
`GET`	`/api/v1/recommendations/trending?category={cat}`	Get trending items

User Interaction APIs (for training)

Method	Endpoint	Description
`POST`	`/api/v1/events`	Log user interaction event
`POST`	`/api/v1/events/batch`	Log batch of interaction events

Event Payload

```
POST /api/v1/events
{
  "user_id": "user_123",
  "item_id": "item_456",
  "event_type": "click",      // click, view, purchase, rate
  "rating": 4.5,              // optional
  "context": {
    "source": "homepage",
    "position": 3,
    "session_id": "sess_789"
  },
  "timestamp": 1672531200000
}
```

Model Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/models/retrain`	Trigger model retraining
`GET`	`/api/v1/models/active`	Get active model info
`GET`	`/api/v1/models/{modelId}/metrics`	Get model accuracy metrics

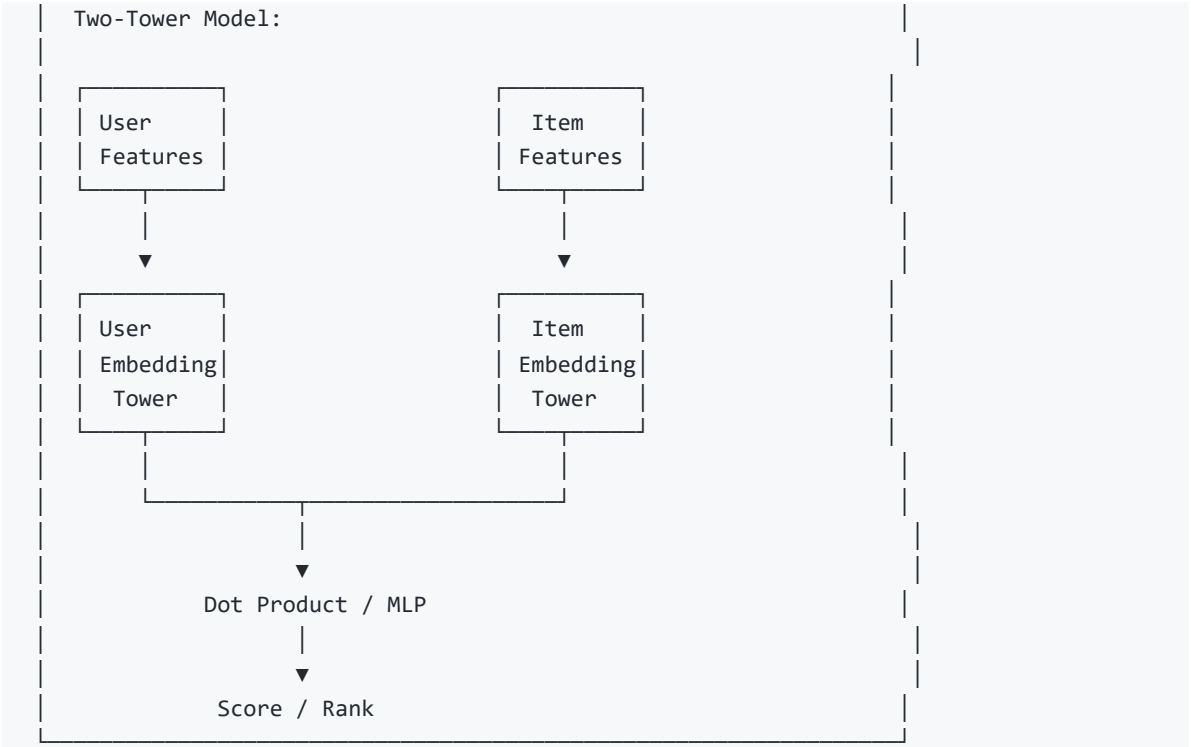
Recommendation Approaches

```
1. COLLABORATIVE FILTERING (Users who liked X also liked Y)
┌─────────────────────────────────────────────────────────────┐
│  USER-BASED:                                                │
│    Find users similar to target user                        │
│    Recommend items those similar users liked                │
│                                                             │
│    User A likes: [1, 2, 3]                                  │
│    User B likes: [1, 2, 4]     ← Similar to A               │
│    User C likes: [5, 6, 7]     ← Different from A           │
│                                                             │
│    Recommend to A: item 4 (from similar user B)             │
│                                                             │
│  ITEM-BASED:                                                │
│    Find items similar to what user liked                    │
│    More stable (items don't change as fast as user behavior)│
│                                                             │
│    User likes item 1                                        │
│    Item 1 similar to: [2, 3, 5]                             │
│    Recommend: [2, 3, 5]                                     │
└─────────────────────────────────────────────────────────────┘

2. CONTENT-BASED FILTERING (Items similar in features)
┌─────────────────────────────────────────────────────────────┐
│  Extract item features:                                     │
│    Movie: {genre: "action", actors: ["Tom"], director: "James"} │
│                                                             │
│  User profile: Aggregate features of liked items            │
│    User likes action movies with Tom                        │
│                                                             │
│  Recommend: Items matching user profile                     │
│    Other action movies with Tom                             │
│                                                             │
│  Pros: Works for new items (no cold start for items)        │
│  Cons: Limited discovery, needs good features               │
└─────────────────────────────────────────────────────────────┘

3. MATRIX FACTORIZATION (Discover latent factors)
┌─────────────────────────────────────────────────────────────┐
│  User-Item matrix (sparse):                                 │
│         Item1  Item2  Item3  Item4                          │
│  User1    5      ?      3      ?                           │
│  User2    ?      4      ?      2                           │
│  User3    3      ?      ?      5                           │
│                                                             │
│  Factorize into: Users × Latent × Items                     │
│  Predict missing ratings: User1 × Item2                     │
│                                                             │
│  Algorithms: SVD, ALS, NMF                                  │
└─────────────────────────────────────────────────────────────┘

4. DEEP LEARNING (Modern approach)
┌─────────────────────────────────────────────────────────────┐
```



Cold Start Solutions

```
NEW USER (No history):
┌──────────────────────────────────────────────────────────────┐
│  1. Popular items: Recommend trending/popular items          │
│  2. Onboarding: Ask preferences during signup                │
│  3. Demographics: Use age, location as initial signal        │
│  4. Content-based: If user searches "sci-fi", use that       │
└──────────────────────────────────────────────────────────────┘

NEW ITEM (No ratings):
┌──────────────────────────────────────────────────────────────┐
│  1. Content features: Use item metadata to find similar items │
│  2. Exploration: Show to some users to gather signals        │
│  3. Expert curation: Manual categorization/tagging           │
└──────────────────────────────────────────────────────────────┘
```

Real-Time vs. Batch

```
BATCH PIPELINE (Daily/Hourly):
┌──────────────────────────────────────────────────────────────┐
│  1. Train model on historical data                           │
│  2. Generate recommendations for all users                   │
│  3. Store in Redis/Database                                  │
│  4. API serves precomputed recommendations                   │
│                                                              │
│  Pros: Simple, handles large-scale computation               │
```

```
│  Cons: Not real-time, stale recommendations                    │
└─────────────────────────────────────────────────────────────────┘

REAL-TIME (Online):
┌─────────────────────────────────────────────────────────────────┐
│  1. User opens app                                              │
│  2. Fetch user features from feature store                      │
│  3. Candidate generation: ANN search for similar items          │
│  4. Ranking: ML model scores candidates                         │
│  5. Return top-K                                                │
│                                                                 │
│  Pros: Fresh recommendations, personalized                      │
│  Cons: Complex, latency challenges                              │
└─────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Approach	Collaborative	Content	**Hybrid** - best of both
Serving	Batch	Real-time	**Batch + real-time rerank**
Diversity	Pure relevance	Exploration	**Balance** - avoid filter bubble
Cold start	Popular	Onboarding	**Both** - graceful degradation

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event streaming for user behavior data	Tech Guide
Redis	Feature store and recommendation cache	Tech Guide
Flink/Spark	Offline model training and batch processing	Tech Guide

---

How to Explain It Well

> **Opening**: "Recommendations combine collaborative filtering (user behavior) and content-based (item features). I'll design a two-tower model with real-time serving."

> **Key talking points**:

> - "Collaborative filtering: users who liked X also liked Y"
> - "Matrix factorization discovers latent factors without explicit features"
> - "Two-tower model: precompute item embeddings, compute user online"
> - "Cold start: popular items for new users, content features for new items"
> - "Balance relevance and diversity to avoid filter bubbles"

---

30. Trade-off Discussions

Key Terms:

Term	Full Name	What it does
ADR	Architecture Decision Record	Document capturing design decision rationale
SLA	Service Level Agreement	Contractual uptime/performance guarantees
FTE	Full-Time Equivalent	One person working full-time (cost measure)
MVP	Minimum Viable Product	Smallest version to test market fit
A/B Testing	Split testing	Compare two options with real users
P95/P99	Percentile latency	Latency that 95%/99% of requests are faster than

Problem Statement

Walk through a tough trade-off you would make in a system design, explaining your reasoning and how you'd validate the decision.

Design Reasoning

> **Core philosophy**: Every architectural decision is a trade-off — use structured reasoning to quantify costs and benefits, then align choices with business requirements rather than technical preferences.

Decision	Choice	Why	Alternative Considered

Consistency model	Hybrid (reserve + confirm)	Balances inventory accuracy with checkout availability during flash sales	Strong consistency only — degrades availability under load
ML ranking	Two-stage (fast retrieval + complex re-rank)	Low latency with high accuracy by narrowing 1000 candidates to 10	Single complex model — 200ms latency unacceptable for search
Compute pricing	70% reserved / 30% on-demand	Predictable cost savings with flexibility for traffic spikes and experiments	All on-demand — 35% more expensive at steady-state baseline
Initial architecture	Monolith first	Faster MVP (2 months vs 6), easier iteration while product-market fit is uncertain	Microservices from day 1 — premature optimization, 3x slower to ship
Authentication	Buy (Auth0/Okta)	Security is hard to get right; 1-week integration vs 3-month build + ongoing maintenance	Build in-house — diverts engineering from core product, higher security risk
Session storage	Redis	Sub-millisecond reads critical when sessions are checked on every request	Database sessions — adds 5-10ms latency compounding across page loads

API Design

Trade-off Analysis APIs

Method	Endpoint	Description
`POST`	`/api/v1/analysis`	Submit a system for trade-off analysis
`GET`	`/api/v1/analysis/{id}/report`	Get analysis report
`GET`	`/api/v1/patterns?category={cat}`	List common trade-off patterns

Example: Consistency vs Availability Trade-off in APIs

```
// Option A: Strong consistency (higher latency)
GET /api/v1/inventory/{id}
Headers:
  X-Consistency: strong
  X-Expected-Latency: 200ms

// Option B: Eventual consistency (lower latency)
GET /api/v1/inventory/{id}
Headers:
  X-Consistency: eventual
  X-Expected-Latency: 20ms
  X-Max-Staleness: 5s
```

Key API Trade-offs to Discuss

Decision	Option A	Option B
Protocol	REST (simple, cacheable)	gRPC (fast, typed)
Pagination	Offset-based (simple)	Cursor-based (consistent)
Versioning	URL path (`/v1/`)	Header-based (`Accept:`)
Auth	API keys (simple)	OAuth2/JWT (secure)

Framework for Trade-off Discussions

```
STRUCTURED APPROACH:
┌─────────────────────────────────────────────────────────────┐
│  1. IDENTIFY: What are the competing concerns?              │
│  2. QUANTIFY: What are the costs/benefits of each option?   │
│  3. DECIDE: Which trade-off aligns with business requirements? │
│  4. VALIDATE: How would you verify the decision was correct? │
│  5. REVISIT: When would you reconsider this decision?       │
└─────────────────────────────────────────────────────────────┘
```

Common Trade-off Scenarios

```
1. CONSISTENCY vs. AVAILABILITY
┌─────────────────────────────────────────────────────────────┐
│  Scenario: E-commerce inventory during flash sale           │
│                                                             │
│  Option A: Strong consistency                               │
│    - Check inventory before each order                      │
│    - Reject if unavailable                                  │
│    - Risk: System slows down, orders fail                   │
```

```
│                                                                 │
│  Option B: Eventual consistency                                 │
│    - Accept order optimistically                                │
│    - Reconcile later                                            │
│    - Risk: Overselling, need to refund                          │
│                                                                 │
│  My recommendation: Hybrid                                      │
│    - Reserve inventory (soft lock)                              │
│    - Confirm within timeout                                     │
│    - Release if payment fails                                   │
│    Why: Balances accuracy with availability                     │
└─────────────────────────────────────────────────────────────────┘
```

2. LATENCY vs. ACCURACY

```
┌─────────────────────────────────────────────────────────────────┐
│  Scenario: Search ranking with ML model                         │
│                                                                 │
│  Option A: Complex model (better accuracy)                      │
│    - BERT-based ranking                                         │
│    - 200ms latency                                              │
│    - 5% higher click-through rate                               │
│                                                                 │
│  Option B: Simple model (lower latency)                         │
│    - Gradient boosted trees                                     │
│    - 20ms latency                                               │
│    - Baseline click-through rate                                │
│                                                                 │
│  My recommendation: Two-stage ranking                           │
│    - Fast model for candidate generation (1000 → 100)           │
│    - Complex model for final ranking (100 → 10)                 │
│    Why: Best of both - low latency with good accuracy           │
└─────────────────────────────────────────────────────────────────┘
```

3. COST vs. PERFORMANCE

```
┌─────────────────────────────────────────────────────────────────┐
│  Scenario: Choosing between on-demand and reserved instances    │
│                                                                 │
│  Option A: All on-demand                                        │
│    - $10,000/month                                              │
│    - Full flexibility                                           │
│    - No commitment                                              │
│                                                                 │
│  Option B: 1-year reserved instances                            │
│    - $6,500/month (35% savings)                                 │
│    - Committed capacity                                         │
│    - Less flexibility                                           │
│                                                                 │
│  My recommendation: 70% reserved, 30% on-demand                 │
│    - Reserve baseline load                                      │
│    - On-demand for spikes and experimentation                   │
│    Why: Predictable savings with flexibility for growth         │
└─────────────────────────────────────────────────────────────────┘
```

4. BUILD vs. BUY

```
┌─────────────────────────────────────────────────────────────────┐
│  Scenario: Authentication system                                │
```

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│  Option A: Build in-house                                    │
│    - Full control                                            │
│    - Customizable                                            │
│    - 3 months dev time                                       │
│    - Ongoing maintenance                                     │
│                                                              │
│  Option B: Use Auth0/Okta                                    │
│    - Quick integration (1 week)                              │
│    - Enterprise features included                            │
│    - $0.05/user/month                                        │
│    - Vendor dependency                                       │
│                                                              │
│  My recommendation: Buy (Auth0)                              │
│    - Security is hard to get right                           │
│    - Cost at 100K users = $5K/month vs. 3 FTEs to maintain   │
│    - Focus engineering on core product                       │
│    Why: Auth isn't our differentiator, shouldn't build       │
└──────────────────────────────────────────────────────────────┘

5. SIMPLICITY vs. SCALABILITY
┌──────────────────────────────────────────────────────────────┐
│  Scenario: Starting a new project                            │
│                                                              │
│  Option A: Microservices from day 1                          │
│    - Scalable architecture                                   │
│    - Team autonomy                                           │
│    - 6 months to MVP                                         │
│    - Complex operations                                      │
│                                                              │
│  Option B: Monolith                                          │
│    - Simple development                                      │
│    - Fast iteration                                          │
│    - 2 months to MVP                                         │
│    - May need to refactor later                              │
│                                                              │
│  My recommendation: Monolith first                           │
│    - Validate product-market fit quickly                     │
│    - Easier to refactor when domain is understood            │
│    - Extract services when hitting scale limits              │
│    Why: Premature optimization is the root of all evil       │
└──────────────────────────────────────────────────────────────┘
```

How to Present Trade-offs in Interviews

```
TEMPLATE:
┌──────────────────────────────────────────────────────────────┐
│  "We have two options here: [A] and [B].                     │
│                                                              │
│   Option A gives us [benefit], but costs us [drawback].      │
│   Option B gives us [benefit], but costs us [drawback].      │
│                                                              │
│   Given our requirement for [specific need], I'd recommend   │
```

```
│   Option [X] because [reasoning].                                  │
│                                                                     │
│   We can validate this by [measurement] and revisit if            │
│   [condition changes]."                                            │
└─────────────────────────────────────────────────────────────────────┘

EXAMPLE:
"We have two options for session storage: Redis and database.

Redis gives us sub-millisecond reads but costs more and adds
operational complexity.

Database is simpler and cheaper but adds 5-10ms latency per
request.

Given our requirement for <100ms page loads, I'd recommend
Redis because sessions are read on every request, and 10ms
adds up.

We can validate by A/B testing both approaches and monitoring
P95 latency. We'd revisit if Redis costs exceed 10% of
infrastructure budget."
```

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Session storage and caching trade-offs	Tech Guide
PostgreSQL	ACID transactions vs. eventual consistency	Tech Guide

---

How to Explain It Well

> **Opening**: "Every system design involves trade-offs. I use a structured approach: identify options, quantify costs/benefits, decide based on requirements, and plan to validate."

> **Key talking points**:
>
> - "No perfect solution - every choice has costs"
> - "Quantify trade-offs: 'X improves latency by 50% but costs 2x'"
> - "Align decisions with business goals, not technical preferences"
> - "Always have a validation plan and conditions to revisit"
> - "Document decisions (ADRs) for future reference"

Summary: Interview Success Tips

```
BEFORE THE INTERVIEW:
┌─────────────────────────────────────────────────────────────────┐
│  1. Practice drawing systems on whiteboard/paper                │
│  2. Know numbers: latency, throughput, storage estimates        │
│  3. Understand trade-offs for common patterns                   │
│  4. Prepare 2-3 deep dives you know well                        │
└─────────────────────────────────────────────────────────────────┘

DURING THE INTERVIEW:
┌─────────────────────────────────────────────────────────────────┐
│  1. CLARIFY: Ask questions, understand requirements             │
│  2. ESTIMATE: Back-of-envelope calculations                     │
│  3. DESIGN: High-level first, then dive deep                    │
│  4. TRADE-OFFS: Discuss alternatives and justify choices        │
│  5. EVOLVE: How would the system scale 10x?                     │
└─────────────────────────────────────────────────────────────────┘

COMMUNICATION:
┌─────────────────────────────────────────────────────────────────┐
│  ✓ Think out loud                                               │
│  ✓ Structure your answer (don't jump around)                    │
│  ✓ Use specific technologies (Redis, Kafka, not "cache")        │
│  ✓ Acknowledge trade-offs proactively                           │
│  ✓ Be honest about what you don't know                          │
└─────────────────────────────────────────────────────────────────┘
```

---

42. Design Ticket Booking System

Key Terms:

Term	Full Name	What it does
Virtual Queue	Waiting room	Controlled admission to prevent overload
Hold	Temporary reservation	Tickets reserved during checkout (10 min)
FCFS	First-Come-First-Served	Queue-based fairness
Overselling	Selling more than available	Must be prevented with atomic operations
CAPTCHA	Bot challenge	Distinguishes humans from automated bots

Saga Pattern	Distributed transaction	Sequence of steps with compensations
Scalping	Ticket resale for profit	Prevented with purchase limits

Problem Statement

Design a ticket booking system like Ticketmaster or Eventbrite that handles high-concurrency purchases for events with limited inventory.

Clarifying Questions to Ask

1. **Scale**: Concurrent users during hot sales? (100K-1M simultaneous?)
2. **Inventory**: Tickets per event? Seat selection required?
3. **Fairness**: First-come-first-served (FCFS) or lottery?
4. **Hold time**: How long to hold tickets during checkout? (5-15 min?)
5. **Pricing**: Fixed, tiered, or dynamic pricing?
6. **Payment**: In-system or external payment provider?
7. **Fraud**: Bot detection and prevention requirements?
8. **Resale**: Support ticket transfers or resale marketplace?

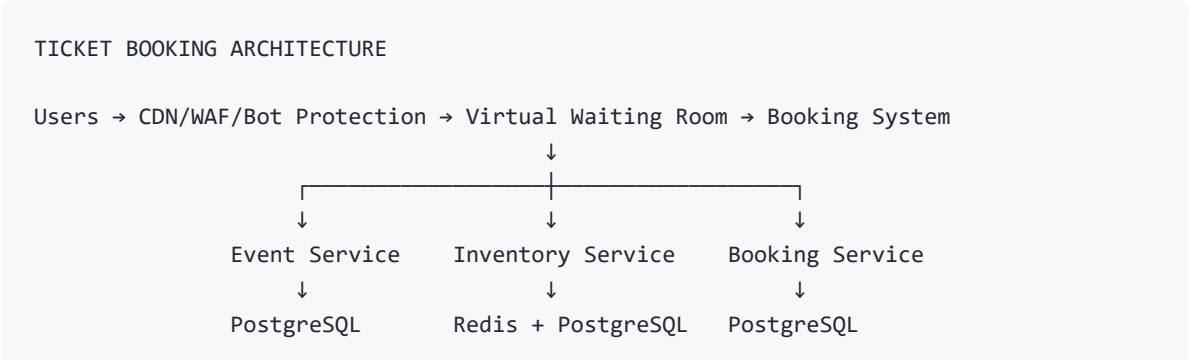
Design Reasoning

> **Core philosophy**: Absorb massive traffic spikes with a virtual waiting room that meters admission, then use Redis atomic operations to guarantee no overselling even under extreme concurrency.

Decision	Choice	Why	Alternative Considered
Traffic management	Virtual waiting room (Redis sorted set)	Meters admission at system capacity; prevents thundering herd from overwhelming booking services	Direct access — millions of concurrent users would crash the booking backend
Inventory tracking	Redis atomic Lua scripts	Single-threaded execution guarantees no race conditions; `DECRBY` is O(1) and handles thousands of concurrent holds	Database locks (SELECT FOR UPDATE) — much slower under high contention, causes lock escalation

Persistent storage	PostgreSQL	ACID transactions for bookings and payment records; well-suited for relational event/booking/user data	NoSQL — eventual consistency is unacceptable for financial booking records
Hold expiration	Redis key TTL (10 min)	Automatic cleanup without cron jobs; expired holds trigger inventory release via keyspace notifications	Application-level timers — fragile across restarts; background sweepers add latency
Booking orchestration	Saga pattern	Multi-step flow (hold → pay → confirm) with compensating actions on failure (release inventory, refund)	Two-phase commit — blocks resources across services; too slow for user-facing checkout
Fairness	FCFS via sorted set (timestamp score)	Transparent, auditable ordering; users see their position and progress	Lottery — fairer for high-demand events but frustrates users who arrived early
Bot prevention	Layered (rate limit + CAPTCHA + fingerprint)	No single defense is sufficient; layered approach catches scripts, headless browsers, and distributed bots	CAPTCHA only — sophisticated bots solve CAPTCHAs; rate limiting alone blocks shared IPs

High-Level Design



API Design

Event APIs

Method	Endpoint	Description
`GET`	`/api/v1/events`	List events
`GET`	`/api/v1/events/{eventId}`	Get event details
`GET`	`/api/v1/events/{eventId}/sections`	Get seating sections and availability

Booking APIs

Method	Endpoint	Description
`POST`	`/api/v1/bookings/hold`	Hold seats (starts 10-min timer)
`POST`	`/api/v1/bookings/confirm`	Confirm booking and process payment
`DELETE`	`/api/v1/bookings/{bookingId}`	Cancel booking
`GET`	`/api/v1/bookings/{bookingId}`	Get booking details

Hold Seats Request

```
POST /api/v1/bookings/hold
{
  "event_id": "evt_123",
  "seats": ["A-12", "A-13"],
  "user_id": "user_456",
  "session_token": "queue_token_789"  // from waiting room
}

Response: 200 OK
{
  "hold_id": "hold_001",
  "seats": ["A-12", "A-13"],
  "expires_at": "2025-03-15T10:10:00Z",
  "total_price": 150.00
}
```

Waiting Room APIs

Method	Endpoint	Description
`POST`	`/api/v1/queue/join`	Join virtual waiting room
`GET`	`/api/v1/queue/position`	Get queue position

Virtual Waiting Room

```
REDIS SORTED SET QUEUE:
┌──────────────────────────────────────────────────────────────────────────┐
│  Key: queue:event:{event_id}                                             │
│  Members: user tokens                                                    │
│  Scores: join timestamp                                                  │
│                                                                          │
│  # User joins queue                                                      │
│  ZADD queue:event:123 1705312200 "token_abc123"                          │
│                                                                          │
│  # Get position                                                          │
│  ZRANK queue:event:123 "token_abc123" → 45230                            │
│                                                                          │
│  # Admit next 100 users                                                  │
│  ZPOPMIN queue:event:123 100                                             │
│                                                                          │
│  Admission rate = System capacity / Avg checkout time                    │
│  Example: 1000 concurrent / 3 min = 333 users/minute                     │
└──────────────────────────────────────────────────────────────────────────┘
```

Inventory Management

```
ATOMIC RESERVATION (Lua script):
┌──────────────────────────────────────────────────────────────────────────┐
│  local available = redis.call('GET', KEYS[1])                            │
│  local requested = tonumber(ARGV[1])                                     │
│                                                                          │
│  if tonumber(available) >= requested then                                │
│      redis.call('DECRBY', KEYS[1], requested)                            │
│      redis.call('INCRBY', KEYS[2], requested)  -- reserved               │
│      return 1  -- Success                                                │
│  else                                                                    │
│      return 0  -- Not enough                                             │
│  end                                                                     │
└──────────────────────────────────────────────────────────────────────────┘

HOLD PATTERN:
  1. User selects tickets → Reserve in Redis (atomic)
  2. Create hold record with 10-min expiry
  3. On payment success → Convert to booking, update sold count
  4. On expiry → Release back to available
```

Booking Flow

```
BOOKING STATE MACHINE:

  BROWSING → HELD (10 min timer) → CONFIRMED
```

```
                    ↓
              EXPIRED / FAILED
              (Release inventory)

HOLD IMPLEMENTATION:
  1. INSERT hold record (PostgreSQL)
  2. DECRBY available, INCRBY reserved (Redis atomic)
  3. SET hold:id "" EX 600 (10 min expiry trigger)
  4. On expire → release inventory, delete hold
```

Database Schema

```
  CREATE TABLE events (
      id BIGINT PRIMARY KEY,
      name VARCHAR(255),
      venue_id BIGINT,
      date TIMESTAMP,
      status VARCHAR(20),  -- on_sale, sold_out
      total_tickets INT
  );

  CREATE TABLE bookings (
      id BIGINT PRIMARY KEY,
      event_id BIGINT,
      user_id BIGINT,
      quantity INT,
      total_amount DECIMAL(10,2),
      status VARCHAR(20),  -- confirmed, cancelled
      created_at TIMESTAMP
  );

  CREATE TABLE holds (
      id BIGINT PRIMARY KEY,
      event_id BIGINT,
      user_id BIGINT,
      quantity INT,
      expires_at TIMESTAMP
  );
```

Bot Prevention

```
ANTI-BOT LAYERS:
  1. Rate limiting: 60 req/min per IP
  2. CAPTCHA: reCAPTCHA v3 (invisible) + v2 challenge if suspicious
  3. Device fingerprinting: Block known bot signatures
  4. Behavior analysis: Detect inhuman speed/patterns
  5. Purchase limits: Max 4 tickets per user per event
```

Key Components

Component	Purpose	Technology
Queue Service	Virtual waiting room	Redis Sorted Set
Inventory Service	Track availability	Redis + PostgreSQL
Booking Service	Purchase orchestration	Saga pattern
Payment Service	Process payments	Stripe
Bot Protection	Prevent abuse	Cloudflare + reCAPTCHA

Scalability Considerations

- **Queue handles spikes**: 100K users wait, 1K process at a time
- **Redis for hot inventory**: Atomic operations prevent overselling
- **Horizontal scaling**: Stateless services behind load balancer
- **CDN caching**: Event pages cached at edge
- **Async notifications**: Emails via queue

Trade-offs

Decision	Option A	Option B	Recommendation
Queue	In-memory	Redis	**Redis** - persistent, distributed
Inventory	Database	Redis	**Redis** - atomic, fast
Hold duration	5 min	15 min	**10 min** - balance UX vs lock
Fairness	FCFS	Lottery	**FCFS + queue** - transparent

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Virtual waiting room queue, atomic inventory counters, hold expiration	Tech Guide

PostgreSQL	Persistent storage for events, bookings, and holds	Tech Guide
Saga Pattern	Orchestrating booking flow with compensations for failures	Tech Guide

How to Explain It Well

> **Opening**: "Ticket booking handles extreme traffic spikes with limited inventory. I'll use a virtual queue to smooth demand and Redis atomic operations to prevent overselling."

> **Key talking points**:
>
> - "Virtual queue: Redis sorted set, admit users at controlled rate"
> - "Redis atomic decrement prevents overselling under high concurrency"
> - "Hold pattern: reserve tickets for 10 minutes during checkout"
> - "Expired holds auto-release inventory back to pool"
> - "Bot protection: rate limits, CAPTCHA, fingerprinting"
> - "Max 4 tickets per user to prevent scalping"

Common Follow-up Questions

1. **How do you handle abandoned checkouts?**
    - Hold expires after 10 minutes
    - Background job releases inventory
    - Reminder email at 5 minutes
2. **How do you prevent scalpers?**
    - Purchase limits (4 per user)
    - Verified identity required
    - Non-transferable tickets option
3. **What if Redis goes down?**
    - Redis Cluster with replicas
    - Fallback to PostgreSQL
    - Queue pauses until recovery

---

43. Design Proximity Service

Key Terms:

Term	Full Name	What it does
Geohash	Geographic Hash	Encodes lat/lng to string; nearby points share prefix
QuadTree	Quadrant Tree	Recursive spatial subdivision (adapts to density)
GEORADIUS	Redis Geo Command	Finds all points within radius
PostGIS	PostgreSQL Geographic	Spatial extension for PostgreSQL
Haversine	Distance formula	Calculates distance on sphere (great-circle)
LBS	Location-Based Service	Service using geographic position

Problem Statement

Design a service that finds nearby places based on user location, like Yelp, Google Maps, or Uber's restaurant finder.

Clarifying Questions to Ask

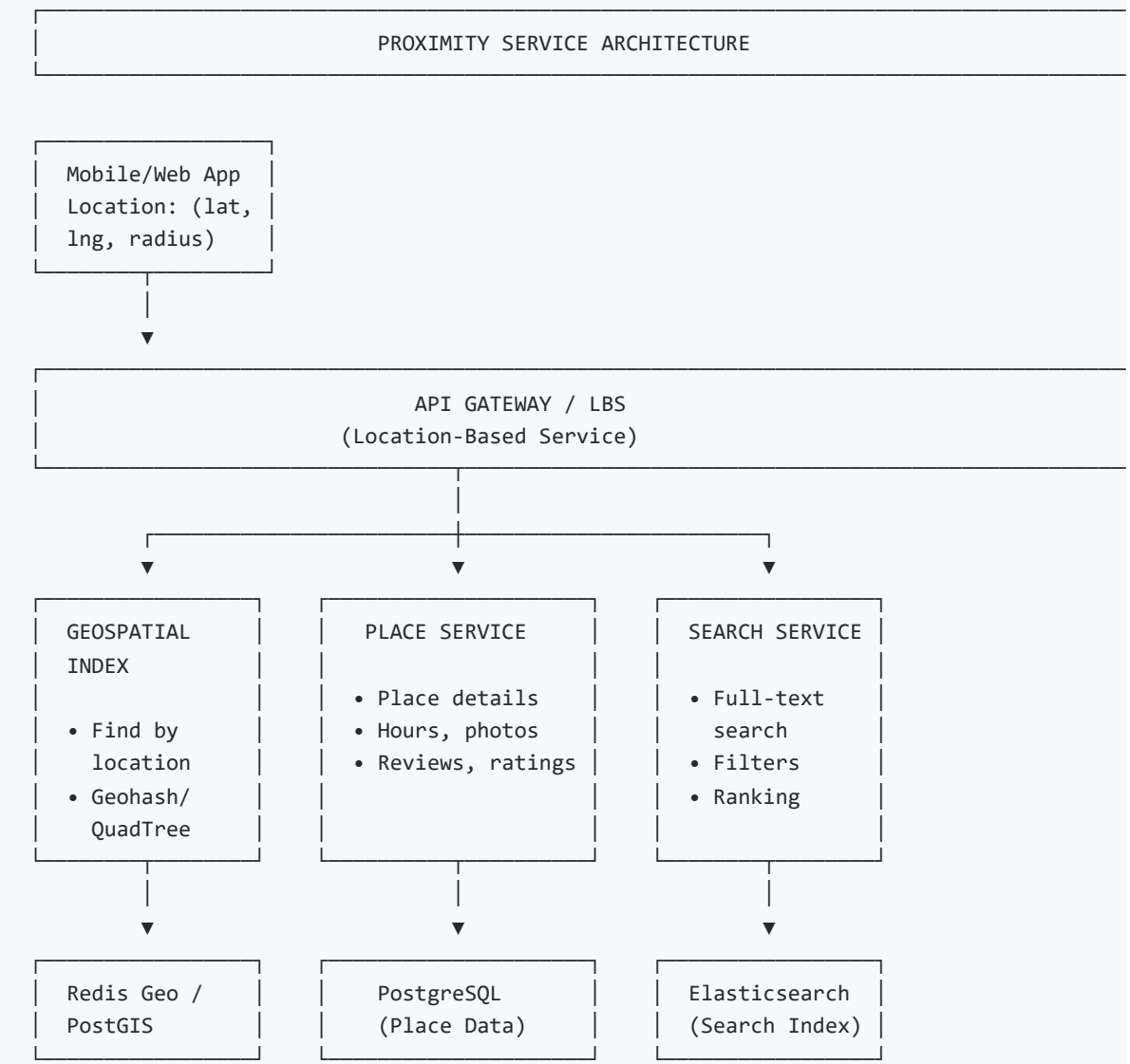
1. **Scale**: How many places in the system? (200M businesses globally?)
2. **Query volume**: How many searches per second? (10K QPS?)
3. **Radius**: Fixed radius or user-specified? (1km to 50km?)
4. **Results**: How many results to return? (20-50?)
5. **Ranking**: By distance only, or also ratings/relevance?
6. **Filters**: Category, price range, open now, ratings?
7. **Updates**: How often do places change? (locations rarely, hours frequently)
8. **Real-time**: Need real-time availability (restaurant seats, parking spots)?

Design Reasoning

> **Core philosophy**: Convert 2D geographic coordinates into 1D indexable strings using geohashing, enabling standard database prefix queries for spatial lookups, with Redis Geo for sub-millisecond proximity searches.

Decision	Choice	Why	Alternative Considered
Spatial index	Geohash	Simple prefix-based queries on standard databases; nearby points share prefixes enabling efficient range lookups	QuadTree — adapts to density but harder to distribute across nodes and more complex to implement
Fast proximity queries	Redis Geo (GEORADIUS)	Built-in sorted-set-backed geospatial index; O(log N) radius search with distance sorting in-memory	PostGIS only — powerful but adds disk I/O latency for the hot-path proximity lookup
Place data store	PostgreSQL + PostGIS	Mature spatial extension; supports complex geo queries, joins with business attributes, and GIST indexing	MongoDB geospatial — capable but PostGIS has deeper spatial function library and stronger ACID guarantees
Text search	Elasticsearch	Superior relevance ranking, fuzzy matching, and faceted filtering for "coffee shops with wifi" style queries	PostgreSQL FTS — adequate for simple search but lacks relevance tuning and faceted filtering
Boundary handling	Query center cell + 8 neighbors	Solves the geohash boundary problem where two nearby points fall in different cells	Single cell query — misses nearby places across cell boundaries, producing incorrect results
Caching	CDN + Redis for popular areas	Downtown and airport queries are repeated thousands of times; caching at edge eliminates redundant computation	No caching — every query hits the database, wasting resources on identical popular-area lookups
Data updates	Batch index rebuilds	Place locations rarely change; batching updates avoids write amplification on the geospatial index	Real-time index updates — unnecessary overhead since physical locations move infrequently

High-Level Design



API Design

Search APIs

Method	Endpoint	Description
`GET`	`/api/v1/places/nearby?lat={lat}&lng={lng}&radius={m}&type={type}`	Search nearby places
`GET`	`/api/v1/places/{placeId}`	Get place details
`GET`	`/api/v1/places/search?q={query}&lat={lat}&lng={lng}`	Text search with location bias

Nearby Search Response

```
GET /api/v1/places/nearby?lat=47.606&lng=-122.332&radius=1000&type=restaurant

Response: 200 OK
{
  "places": [
    {
      "id": "place_123",
      "name": "Sushi Bar",
      "location": { "lat": 47.607, "lng": -122.333 },
      "distance_m": 142,
      "rating": 4.5,
      "type": "restaurant"
    }
  ],
  "total": 25,
  "next_cursor": "cursor_abc"
}
```

Business APIs (for place owners)

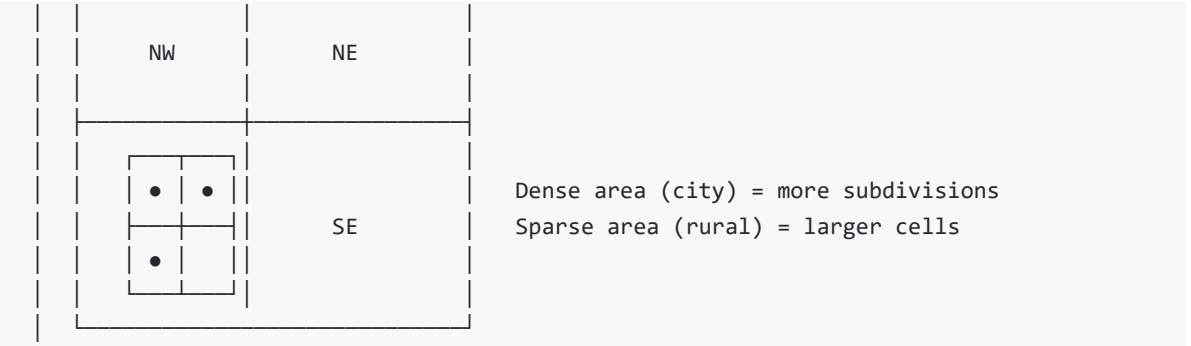
Method	Endpoint	Description
`POST`	`/api/v1/places`	Register a new place
`PUT`	`/api/v1/places/{placeId}`	Update place info
`PUT`	`/api/v1/places/{placeId}/location`	Update place location
`DELETE`	`/api/v1/places/{placeId}`	Remove a place

Geospatial Indexing Approaches

Option 1: Geohash

```
GEOHASH ENCODING:
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  Latitude/Longitude → Base32 String                                          │
│                                                                              │
│  (47.6062, -122.3321) → "c23nb62w"                                           │
│                                                                              │
│  Precision levels:                                                           │
│    Length 4: ~20km  × 20km   (city level)                                    │
│    Length 5: ~5km   × 5km    (neighborhood)                                  │
│    Length 6: ~1.2km × 0.6km  (streets)                                       │
│    Length 7: ~150m  × 150m   (block level)                                   │
```

```
│    Length 8: ~40m   × 20m    (building level)
│
│  KEY PROPERTY: Nearby locations share common prefix
│    "c23nb62w" and "c23nb62x" are neighbors
│
└──────────────
```



Option 3: Redis Geo (Recommended for simplicity)

```
REDIS GEO COMMANDS:
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                      │
│  # Add places                                                                                        │
│  GEOADD places -122.3321 47.6062 "starbucks_123"                                                     │
│  GEOADD places -122.3400 47.6100 "mcdonalds_456"                                                     │
│                                                                                                      │
│  # Find within radius                                                                                │
│  GEORADIUS places -122.3321 47.6062 5 km WITHDIST WITHCOORD COUNT 50 ASC                             │
│                                                                                                      │
│  Returns:                                                                                            │
│    1) "starbucks_123", 0.0, [-122.3321, 47.6062]                                                     │
│    2) "mcdonalds_456", 0.8, [-122.3400, 47.6100]                                                     │
│                                                                                                      │
│  # Under the hood: Uses geohash + sorted set (ZSET)                                                  │
│  # Time complexity: O(N+log(M)) where N = results, M = total items                                   │
│                                                                                                      │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Database Schema

```sql
-- Places table (PostgreSQL + PostGIS)
CREATE TABLE places (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100),           -- restaurant, cafe, gym
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    geohash VARCHAR(12),             -- precomputed for indexing
    location GEOGRAPHY(POINT, 4326), -- PostGIS type
    address TEXT,
```

```
    city VARCHAR(100),
    country VARCHAR(2),
    rating DECIMAL(2, 1),
    review_count INT,
    price_level INT,                -- 1-4 ($-$$$$)
    hours JSONB,                    -- {"mon": "9:00-17:00", ...}
    attributes JSONB,               -- {"wifi": true, "parking": true}
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

-- Indexes
CREATE INDEX idx_places_geohash ON places (geohash);
CREATE INDEX idx_places_location ON places USING GIST (location);
CREATE INDEX idx_places_category ON places (category);

-- Query example: Find restaurants within 5km
SELECT id, name, rating,
       ST_Distance(location, ST_MakePoint(-122.33, 47.60)::geography) AS distance
FROM places
WHERE ST_DWithin(location, ST_MakePoint(-122.33, 47.60)::geography, 5000)
  AND category = 'restaurant'
ORDER BY distance
LIMIT 50;
```

Key Components

Component	Purpose	Technology
Geospatial Index	Fast location queries	Redis Geo / PostGIS
Place Database	Store place details	PostgreSQL
Search Service	Full-text + filters	Elasticsearch
CDN	Cache popular areas	Azure CDN
Place Ingestion	Add/update places	Kafka → Workers
Photo Storage	Place images	Blob Storage + CDN

Query Flow

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                              SEARCH QUERY FLOW                              │
└─────────────────────────────────────────────────────────────────────────────┘

User: "Coffee shops near me" (lat: 47.6, lng: -122.3, radius: 2km)
```

```
1. PARSE REQUEST
   ├── Location: (47.6, -122.3)
   ├── Radius: 2km
   ├── Category: coffee (from NLP/search)
   └── Filters: open_now=true

2. GEOSPATIAL QUERY (Redis Geo or PostGIS)
   ├── GEORADIUS places 47.6 -122.3 2 km
   └── Returns: [place_id_1, place_id_2, ...] with distances

3. ENRICH + FILTER (Place Service)
   ├── Fetch details for each place_id
   ├── Apply filters: category=coffee, open_now=true
   └── Calculate "open now" from hours + timezone

4. RANK RESULTS
   ├── Primary: distance (closest first)
   ├── Secondary: rating × log(review_count)
   ├── Boost: sponsored/promoted places
   └── Personalization: user preferences, history

5. RETURN RESPONSE
   └── Top 20 places with details
```

Scalability Considerations

- **Sharding by region**: Partition data by country/city for locality
- **Read replicas**: Most queries are reads; use replicas per region
- **Caching**: Cache popular areas (downtown, airports) at edge
- **Pre-aggregation**: Pre-compute "top restaurants in Seattle" daily
- **Batch updates**: Places don't change often; batch index updates
- **Tiered storage**: Hot data (active places) in Redis, all data in PostgreSQL

Trade-offs

Decision	Option A	Option B	Recommendation
Index structure	Geohash	QuadTree	**Geohash** - simpler, works with Redis
Primary store	PostgreSQL+PostGIS	MongoDB	**PostGIS** - mature geo support
Cache layer	Redis Geo	Custom	**Redis Geo** - built-in, fast
Search	Elasticsearch	PostgreSQL FTS	**Elasticsearch** - better relevance

Ranking	Distance only	Distance + rating	**Combined** - better UX

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Geohash	Converting 2D coordinates to 1D strings for efficient range queries	Tech Guide
QuadTree	Adaptive spatial indexing that subdivides based on density	Tech Guide
Redis	GEORADIUS commands for fast proximity queries with sorted results	Tech Guide

How to Explain It Well

> **Opening**: "A proximity service needs to find places near a location efficiently. I'll use geohashing for O(1) lookups with Redis Geo, backed by PostgreSQL for full place data."

> **Key talking points**:
>
> - "Geohash converts 2D coordinates to 1D strings - nearby places share prefixes"
> - "Always query center cell + 8 neighbors to handle boundary cases"
> - "Redis GEORADIUS gives us sorted results by distance in O(log N)"
> - "Combine with Elasticsearch for full-text search and filtering"
> - "Cache popular areas (downtown, airports) - they're queried frequently"
> - "Shard by region for data locality - Seattle queries hit Seattle servers"

Common Follow-up Questions

1. **How do you handle the boundary problem in geohash?**
    - Query 8 neighboring cells, not just the center
    - Filter results by exact Haversine distance
2. **How do you rank results?**
    - Distance is primary, but also factor in ratings
    - Score = distance_score × quality_score

- Allow sponsored results with clear labeling

3. **How do you keep data fresh?**
   - Places rarely move; batch update locations weekly
   - Hours/availability update more frequently (daily sync)
   - User reports trigger immediate review

4. **How do you handle "open now" queries?**
   - Store hours in structured format with timezone
   - Compute at query time using current time + place timezone

---

44. Design Web Analytics System

Key Terms:

Term	Full Name	What it does
HLL	HyperLogLog	Probabilistic unique count (~2% error, 12KB memory)
OLAP	Online Analytical Processing	Database optimized for aggregations (vs OLTP)
Columnar	Column-oriented storage	Stores data by column for faster analytics
sendBeacon	Browser API	Reliable data sending even on page unload
Lambda Architecture	Batch + Speed	Combines real-time stream with batch processing
TTL	Time To Live	Auto-delete data after retention period
Roll-up	Pre-aggregation	Summarize raw events into hourly/daily totals

Problem Statement

Design a real-time analytics system like Google Analytics that tracks user behavior (page views, clicks, conversions) and provides dashboards and reports.

Clarifying Questions to Ask

1. **Scale**: How many events per second? (100K-1M events/sec?)
2. **Latency**: Real-time dashboards or delayed OK? (seconds vs hours)
3. **Retention**: How long to keep raw data? (30 days? 1 year?)
4. **Queries**: What queries are most common? (page views, funnels, cohorts)
5. **Cardinality**: How many unique users/pages? (affects storage)
6. **Custom events**: Support custom event tracking?
7. **Attribution**: Need multi-touch attribution for marketing?

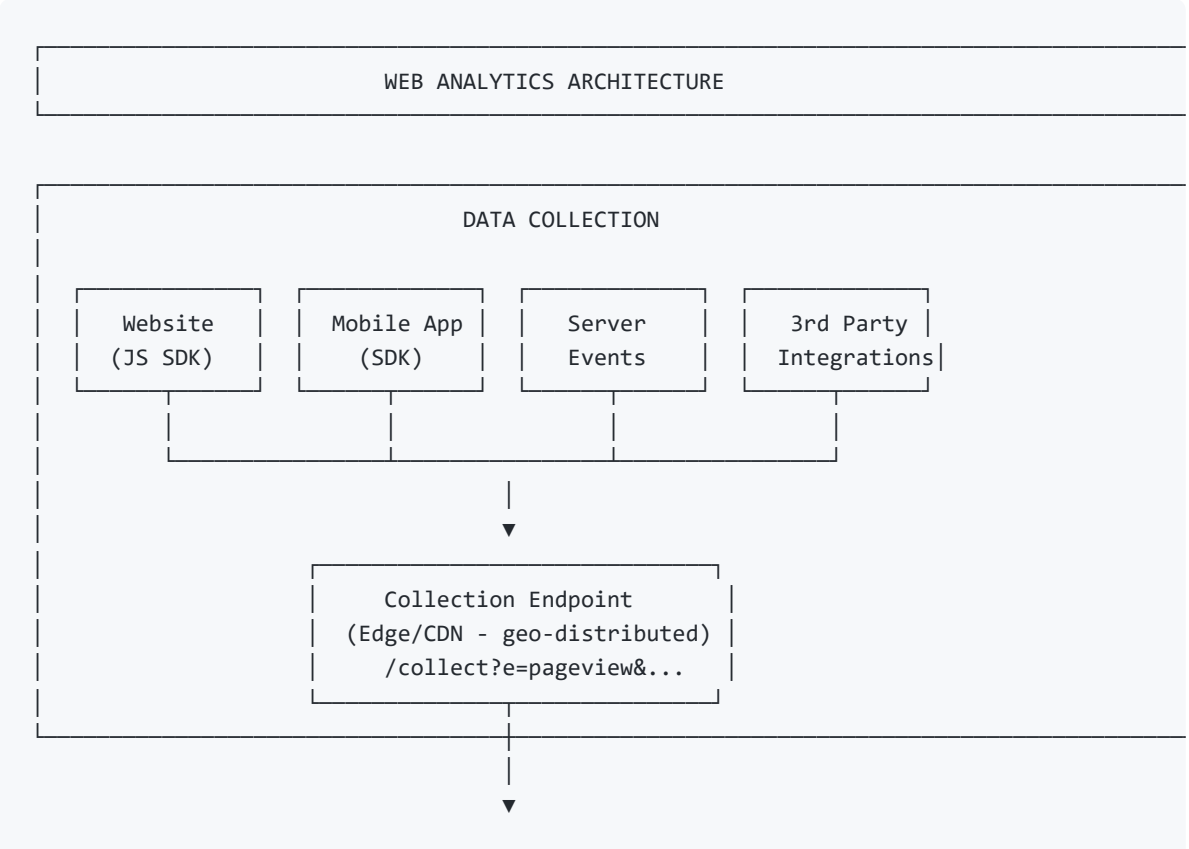
Design Reasoning

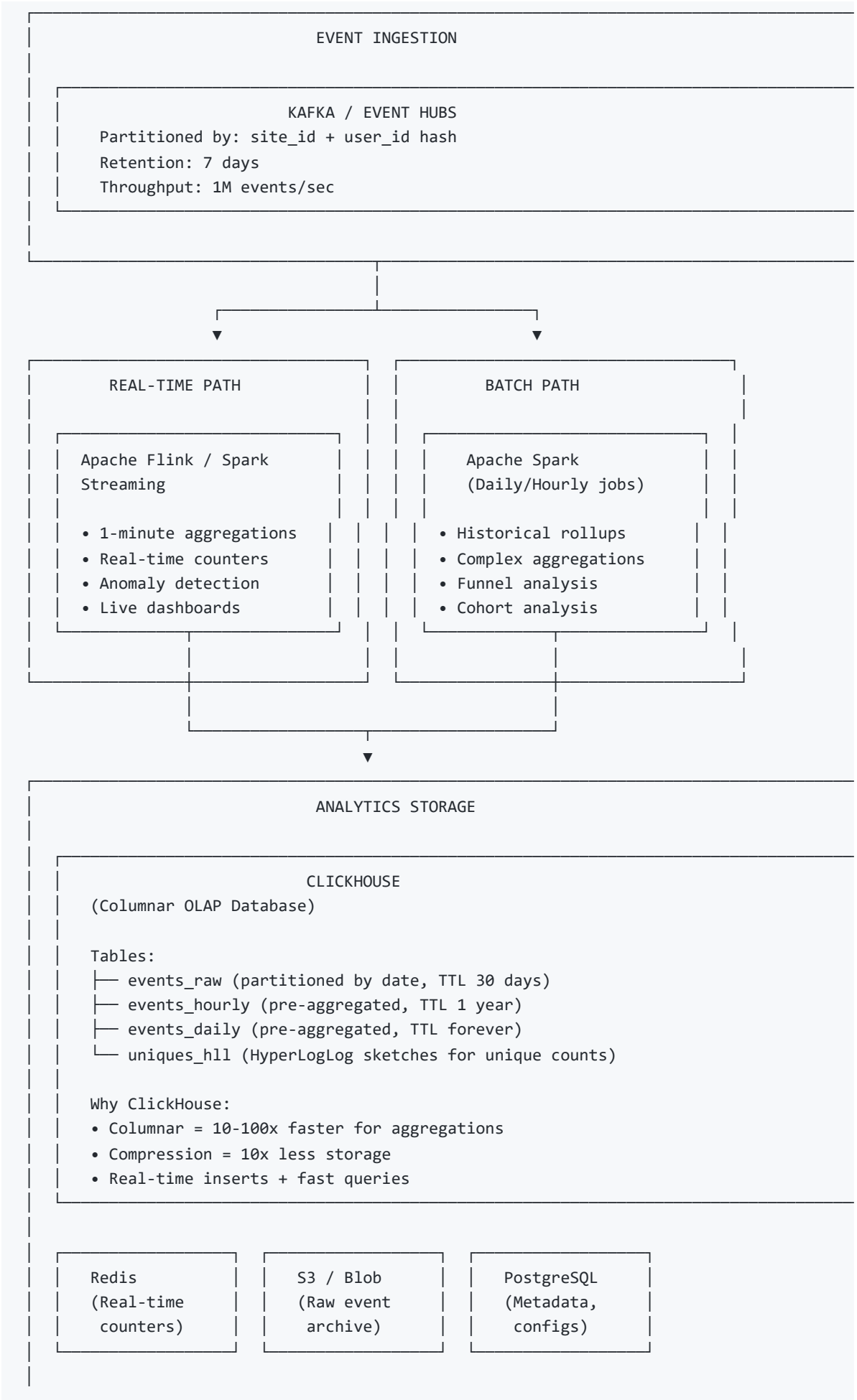
> **Core philosophy**: Ingest millions of events per second through a Lambda architecture — real-time streaming for live dashboards, batch processing for historical accuracy — stored in a columnar OLAP database optimized for aggregation queries.

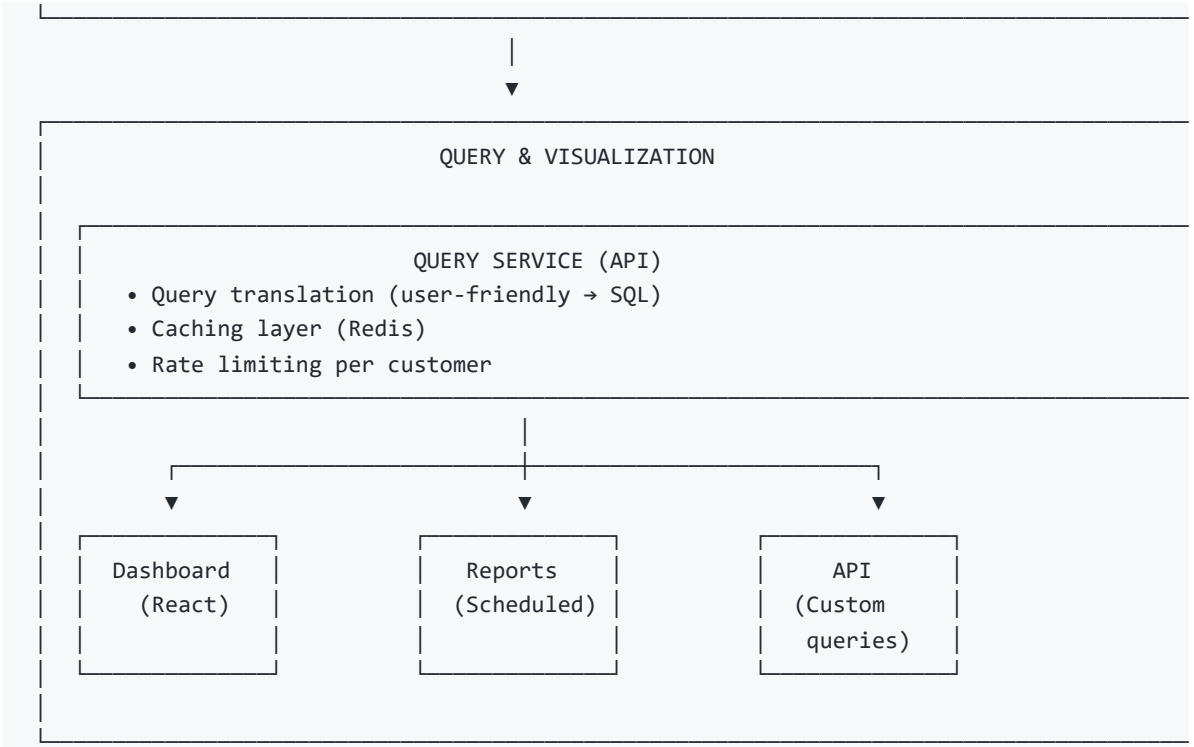
Decision	Choice	Why	Alternative Considered
Analytics database	ClickHouse (columnar)	10-100x faster aggregations than row stores; columnar compression reduces storage 10x; handles real-time inserts	PostgreSQL — row-oriented storage is too slow for scanning billions of analytics events
Event ingestion	Kafka / Event Hubs	Handles 1M+ events/sec with partitioned durability; decouples collection from processing; 7-day retention for replay	Direct-to-database — no back-pressure; a traffic spike would overwhelm the storage layer
Processing architecture	Lambda (real-time + batch)	Flink delivers second-level dashboards; Spark batch jobs correct late-arriving data and compute complex funnels	Real-time only — stream processing can lose events or produce approximate results without batch correction
Unique visitor counting	HyperLogLog (~2% error)	12KB memory vs 400MB for exact counts; mergeable	Exact COUNT(DISTINCT) — requires storing all user

		across time windows; ~2% error is acceptable for dashboards	IDs; too expensive at billions of events
Data collection	Client-side JS SDK + sendBeacon	Captures rich client context (viewport, referrer); sendBeacon survives page unload events	Server-side only — misses client-side context like scroll depth, viewport, and client timings
Data retention	TTL tiers (raw 30d → hourly 1y → daily forever)	Balances query speed and storage cost; pre-aggregated rollups make dashboard queries 1000x faster	Keep all raw data — storage costs grow unbounded; queries over months of raw data are too slow
Collection endpoint	Edge/CDN-distributed	Reduces collection latency globally; absorbs traffic spikes at the edge before they reach origin	Centralized endpoint — high latency for global users; single region becomes a bottleneck

High-Level Design




EVENT INGESTION
KAFKA / EVENT HUBS
Partitioned by: site_id + user_id hash
Retention: 7 days
Throughput: 1M events/sec
REAL-TIME PATH
Apache Flink / Spark Streaming
• 1-minute aggregations
• Real-time counters
• Anomaly detection
• Live dashboards
BATCH PATH
Apache Spark (Daily/Hourly jobs)
• Historical rollups
• Complex aggregations
• Funnel analysis
• Cohort analysis
ANALYTICS STORAGE
CLICKHOUSE
(Columnar OLAP Database)
Tables:
├── events_raw (partitioned by date, TTL 30 days)
├── events_hourly (pre-aggregated, TTL 1 year)
├── events_daily (pre-aggregated, TTL forever)
└── uniques_hll (HyperLogLog sketches for unique counts)
Why ClickHouse:
• Columnar = 10-100x faster for aggregations
• Compression = 10x less storage
• Real-time inserts + fast queries
Redis (Real-time counters)
S3 / Blob (Raw event archive)
PostgreSQL (Metadata, configs)



API Design

Event Collection APIs

Method	Endpoint	Description
`POST`	`/api/v1/events`	Collect a single event
`POST`	`/api/v1/events/batch`	Collect batch of events
`GET`	`/api/v1/events/pixel.gif?...`	Tracking pixel (for email/no-JS)

Event Payload

```
POST /api/v1/events
{
  "site_id": "site_123",
  "event": "page_view",
  "url": "https://example.com/pricing",
  "referrer": "https://google.com",
  "user_id": "anon_456",
  "session_id": "sess_789",
  "timestamp": 1672531200000,
  "properties": {
    "viewport": "1920x1080",
    "browser": "Chrome 120"
```

```
    }
  }
```

Query/Reporting APIs

Method	Endpoint	Description
`GET`	`/api/v1/analytics/pageviews?siteId={id}&from={date}&to={date}`	Get page view metrics
`GET`	`/api/v1/analytics/sessions?siteId={id}&groupBy=source`	Get session analytics
`GET`	`/api/v1/analytics/funnel?siteId={id}&steps={steps}`	Funnel analysis
`GET`	`/api/v1/analytics/realtime?siteId={id}`	Real-time active users
`POST`	`/api/v1/analytics/query`	Custom analytics query

Dashboard APIs

Method	Endpoint	Description
`POST`	`/api/v1/dashboards`	Create dashboard
`GET`	`/api/v1/dashboards/{dashId}`	Get dashboard with widgets

JavaScript SDK (Collection)

```
// analytics.js - Lightweight SDK (~2KB gzipped)
(function() {
  const ENDPOINT = 'https://collect.analytics.com/e';
  const queue = [];

  // Generate or retrieve anonymous ID
  const getAnonymousId = () => {
    let id = localStorage.getItem('_aid');
    if (!id) {
      id = 'a_' + Math.random().toString(36).substr(2, 16);
      localStorage.setItem('_aid', id);
    }
    return id;
  };

  // Core tracking function
```

```
  window.analytics = {
    track: function(event, properties = {}) {
      const payload = {
        e: event,                          // event name
        aid: getAnonymousId(),             // anonymous ID
        uid: this._userId,                 // user ID (if logged in)
        url: window.location.href,
        ref: document.referrer,
        ts: Date.now(),
        ua: navigator.userAgent,
        sr: `${screen.width}x${screen.height}`,
        ...properties
      };

      // Use sendBeacon for reliability (survives page unload)
      if (navigator.sendBeacon) {
        navigator.sendBeacon(ENDPOINT, JSON.stringify(payload));
      } else {
        // Fallback to image pixel
        new Image().src = `${ENDPOINT}?d=${encodeURIComponent(JSON.stringify(payload
      }
    },

    page: function(name, properties) {
      this.track('pageview', { page: name, ...properties });
    },

    identify: function(userId, traits) {
      this._userId = userId;
      this.track('identify', { traits });
    }
  };

  // Auto-track page views
  analytics.page();
})();
```

Event Schema

```
{
  "event_id": "evt_abc123",
  "site_id": "site_456",
  "event_type": "pageview",
  "anonymous_id": "a_xyz789",
  "user_id": "user_123",           // null if not logged in
  "session_id": "sess_def456",
  "timestamp": "2024-01-15T10:30:00Z",

  "page": {
    "url": "https://example.com/products/123",
    "path": "/products/123",
```

```
    "title": "Product Details",
    "referrer": "https://google.com"
  },

  "context": {
    "ip": "203.0.113.50",
    "user_agent": "Mozilla/5.0...",
    "locale": "en-US",
    "timezone": "America/Los_Angeles",
    "screen": "1920x1080",
    "viewport": "1200x800"
  },

  "properties": {
    "product_id": "123",
    "category": "electronics",
    "price": 299.99
  },

  "geo": {                          // Enriched from IP
    "country": "US",
    "region": "CA",
    "city": "San Francisco"
  }
}
```

ClickHouse Schema

```
-- Raw events table (hot data, 30-day TTL)
CREATE TABLE events_raw (
    event_id String,
    site_id String,
    event_type LowCardinality(String),
    anonymous_id String,
    user_id Nullable(String),
    session_id String,
    timestamp DateTime64(3),

    -- Page data
    url String,
    path String,
    referrer String,

    -- Context
    country LowCardinality(String),
    device_type LowCardinality(String),
    browser LowCardinality(String),
    os LowCardinality(String),

    -- Custom properties (flexible schema)
    properties String,  -- JSON string
```

```
    -- Partitioning
    date Date DEFAULT toDate(timestamp)
)
ENGINE = MergeTree()
PARTITION BY toYYYYMM(date)
ORDER BY (site_id, date, event_type, anonymous_id)
TTL date + INTERVAL 30 DAY;

-- Hourly aggregations (pre-computed for fast queries)
CREATE TABLE events_hourly (
    site_id String,
    hour DateTime,
    event_type LowCardinality(String),

    -- Counts
    event_count UInt64,
    unique_users AggregateFunction(uniq, String),
    unique_sessions AggregateFunction(uniq, String),

    -- Dimensions
    country LowCardinality(String),
    device_type LowCardinality(String),
    path String
)
ENGINE = AggregatingMergeTree()
PARTITION BY toYYYYMM(hour)
ORDER BY (site_id, hour, event_type, country, device_type, path);

-- Materialized view for automatic aggregation
CREATE MATERIALIZED VIEW events_hourly_mv TO events_hourly AS
SELECT
    site_id,
    toStartOfHour(timestamp) AS hour,
    event_type,
    count() AS event_count,
    uniqState(anonymous_id) AS unique_users,
    uniqState(session_id) AS unique_sessions,
    country,
    device_type,
    path
FROM events_raw
GROUP BY site_id, hour, event_type, country, device_type, path;
```

Key Patterns

1. Pre-Aggregation (Roll-ups)

```
RAW EVENTS → MINUTE → HOUR → DAY → MONTH

┌──────────────────────────────────────────────────────────────────────┐
```

```
│  Query: "Page views last 7 days"                                    │
│                                                                     │
│  Without pre-agg: Scan 100M raw events (~10 seconds)                │
│  With pre-agg:    Scan 168 hourly rows (~10 milliseconds)           │
│                                                                     │
│  1000x faster!                                                      │
└─────────────────────────────────────────────────────────────────────┘
```

2. HyperLogLog for Unique Counts

```
┌─────────────────────────────────────────────────────────────────────┐
│  PROBLEM: Count unique visitors across 1 billion events             │
│                                                                     │
│  Exact count: Store all 50M user IDs = 400MB memory                 │
│  HyperLogLog: 12KB memory, ~2% error                                │
│                                                                     │
│  ClickHouse: uniq() function uses HLL automatically                 │
│                                                                     │
│  SELECT uniq(anonymous_id) FROM events                              │
│  WHERE site_id = 'abc' AND date >= today() - 30                     │
│                                                                     │
│  Result: 1,234,567 (±2% = ±24,691)                                  │
│  For dashboards, this accuracy is acceptable!                       │
└─────────────────────────────────────────────────────────────────────┘
```

3. Session Stitching

```
┌─────────────────────────────────────────────────────────────────────┐
│  Events from same user need to be grouped into sessions             │
│                                                                     │
│  SESSION RULES:                                                     │
│    • Same anonymous_id                                              │
│    • Gap between events < 30 minutes                                │
│    • Same day (midnight = new session)                              │
│                                                                     │
│  IMPLEMENTATION:                                                    │
│    Flink: Session windows with 30-min gap                           │
│    Or: Compute session_id client-side                               │
└─────────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Purpose	Technology
JS SDK	Client-side tracking	JavaScript (2KB)
Collection API	Receive events	Edge functions (Cloudflare/CDN)

Event Queue	Buffer and distribute	Kafka / Event Hubs
Stream Processor	Real-time aggregation	Apache Flink
Batch Processor	Historical analysis	Apache Spark
Analytics DB	Query storage	ClickHouse
Cache	Fast dashboard queries	Redis
Archive	Raw event backup	S3 / Blob Storage

Scalability Considerations

- **Collection at edge**: CDN endpoints reduce latency, handle spikes
- **Kafka partitioning**: Partition by site_id for parallelism
- **ClickHouse sharding**: Shard by site_id for multi-tenant isolation
- **Pre-aggregation**: 1000x query speedup for dashboards
- **TTL policies**: Raw data 30 days, aggregates forever
- **Sampling**: For very high volume, sample raw events (keep 10%)

Trade-offs

Decision	Option A	Option B	Recommendation
Storage	Row-based (PostgreSQL)	Columnar (ClickHouse)	**ClickHouse** - 10-100x faster
Unique counts	Exact	HyperLogLog (~2% error)	**HLL** - 1000x less memory
Processing	Real-time only	Lambda (real-time + batch)	**Lambda** - accuracy + speed
Collection	Server-side	Client-side (JS)	**Client-side** - richer context
Retention	Keep everything	TTL + sampling	**TTL** - cost effective

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

Kafka	High-throughput event ingestion buffer partitioned by site_id	Tech Guide
ClickHouse	Columnar OLAP database for 10-100x faster aggregations	Tech Guide
HyperLogLog	Approximate unique visitor counts using 12KB instead of 400MB	Tech Guide
Flink/Spark	Real-time stream processing for minute aggregations and batch rollups	Tech Guide

How to Explain It Well

> **Opening**: "Web analytics needs to handle millions of events per second with real-time dashboards. I'll use a Lambda architecture with ClickHouse for fast OLAP queries."

> **Key talking points**:
>
> - "Lightweight JS SDK (~2KB) with sendBeacon for reliable collection"
> - "Kafka buffers events, Flink aggregates in real-time"
> - "ClickHouse columnar storage gives 10-100x faster aggregations than PostgreSQL"
> - "Pre-aggregate into hourly/daily rollups for O(1) dashboard queries"
> - "HyperLogLog for unique visitor counts - 12KB memory instead of 400MB, 2% error"
> - "TTL policies: raw events 30 days, aggregates forever"

Common Follow-up Questions

1. **How do you handle ad blockers?**
    - First-party domain for collection (analytics.yoursite.com)
    - Server-side tracking as backup
    - Accept some data loss (~10-20%)
2. **How do you ensure data accuracy?**
    - Deduplication by event_id
    - Late-arriving events handled by batch reprocessing
    - Checksums for data integrity
3. **How do you handle GDPR/privacy?**
    - Anonymize IPs (truncate last octet)
    - Cookie consent required for tracking

- Data deletion API for right-to-be-forgotten
- Configurable data retention per site

---

45. Design Ad Bidding System

Key Terms:

Term	Full Name	What it does
RTB	Real-Time Bidding	Auction for ad impressions in milliseconds
DSP	Demand-Side Platform	Advertiser's bidding system
SSP	Supply-Side Platform	Publisher's ad inventory system
pCTR	Predicted Click-Through Rate	ML model predicting click probability
pCVR	Predicted Conversion Rate	ML model predicting conversion probability
CPM	Cost Per Mille	Price per 1000 impressions
CPC	Cost Per Click	Price per click
Bid Shading	Bid reduction	Bidding below value in first-price auctions
Pacing	Budget spread	Spending budget evenly throughout day
Frequency Cap	Impression limit	Max times user sees same ad

Problem Statement

Design a real-time bidding (RTB) system that auctions ad impressions in milliseconds, like Google Ads, Microsoft Ads, or Meta Ads.

Clarifying Questions to Ask

1. **Latency**: What's the bid response deadline? (50-100ms typical)
2. **Scale**: How many bid requests per second? (1M+ QPS?)

3. **Auction type**: First-price or second-price auction?
4. **Budget**: Real-time budget enforcement or eventual?
5. **Targeting**: What targeting dimensions? (geo, demographics, interests)
6. **ML**: Use ML for bid optimization?
7. **Fraud**: How to detect invalid traffic?

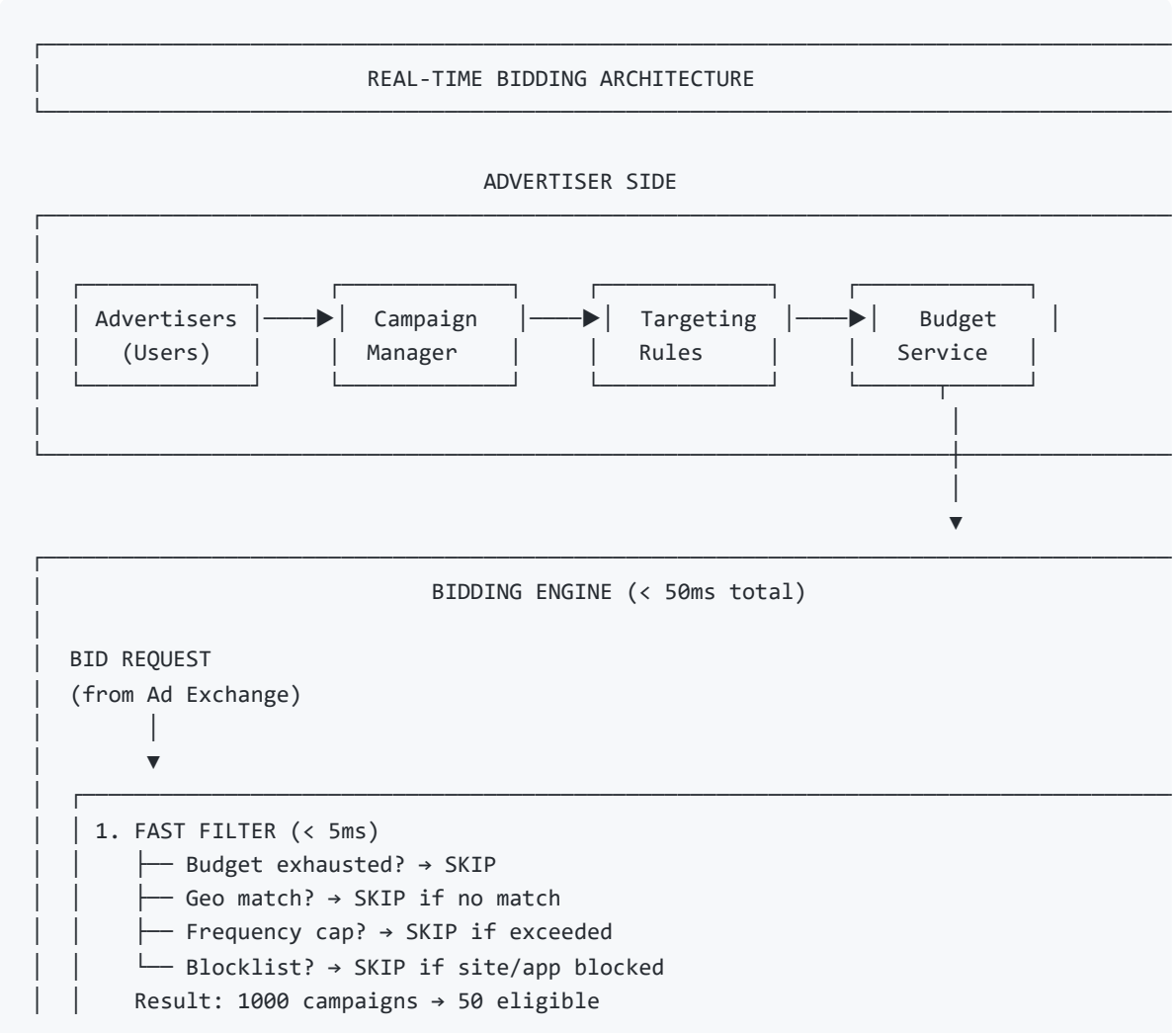
Design Reasoning

> **Core philosophy**: Respond to bid requests in under 50ms by aggressively filtering 90% of campaigns on cheap rules first, then using ML scoring only for viable candidates, with Redis atomic counters enforcing real-time budget constraints.

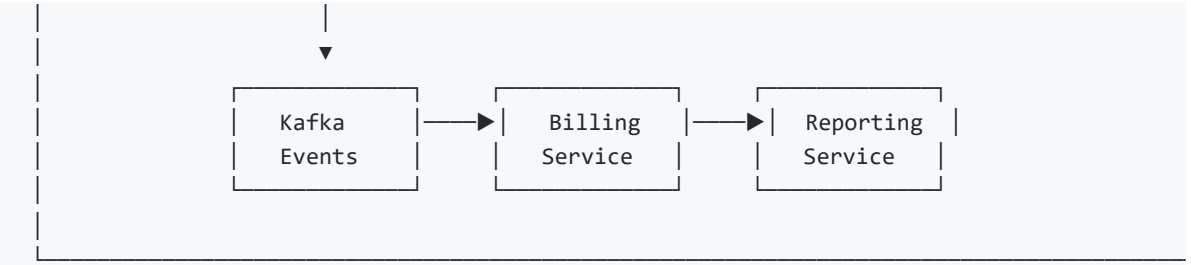
Decision	Choice	Why	Alternative Considered
Bidding language	Go/Rust	Low GC pause and predictable latency under 50ms deadline; high throughput for 1M+ QPS	Java — GC pauses can blow the 50ms deadline; not acceptable for latency-critical bid paths
Filtering strategy	Two-phase (rules → ML)	Fast rule filters (budget, geo, frequency cap) eliminate 90% of campaigns in <5ms; ML runs only on 50 viable candidates	ML-only scoring — running inference on 1000 campaigns per request would exceed the latency budget
Budget tracking	Redis atomic counters	DECRBY is O(1) and guarantees no overspend under concurrent bidding; rollback on auction loss	Database-based — too slow for per-request budget checks at 1M QPS; eventual consistency risks overspend
ML model serving	TensorFlow Serving with batching	Batched inference gives 10x throughput; GPU utilization improves with batch sizes >16	Per-request inference — wastes GPU cycles on single samples; cannot meet latency at scale
Feature store	Redis	Sub-millisecond feature lookup for user profiles and ad features; pre-computed features avoid runtime joins	Database queries — 5-10ms per lookup is too slow inside a 50ms total budget

Event pipeline	Kafka → ClickHouse	Kafka durably captures impressions, clicks, and conversions; ClickHouse enables fast billing aggregation and reporting	Synchronous logging — blocks the bid response path; risks data loss under load
Pacing algorithm	PID-controller-style adjustment	Dynamically adjusts bid multiplier to spread budget evenly through the day; corrects for traffic fluctuations	Uniform sampling — doesn't adapt to traffic patterns; wastes budget during low-value hours
Auction strategy	First-price with bid shading	Industry-standard; bid shading model learns optimal discount to avoid overpaying while maintaining win rate	Second-price — simpler bidding logic but exchanges are shifting to first-price; must adapt

High-Level Design




2. ML SCORING (< 20ms)
For each eligible campaign:
├── pCTR = P(click | user, ad, context)
├── pCVR = P(conversion | click)
└── Expected Value = pCTR × pCVR × conversion_value
3. BID CALCULATION (< 5ms)
bid_amount = expected_value × bid_multiplier × pacing_factor
Constraints:
├── bid ≥ floor_price (from exchange)
├── bid ≤ max_cpc (from advertiser)
└── bid ≤ remaining_budget
BID RESPONSE
{campaign_id, bid_amount, ad_creative_url}
AD EXCHANGE
AUCTION
DSP 1: $2.50
DSP 2: $3.00
DSP 3: $1.80
SECOND-PRICE AUCTION
Winner: DSP 2
Pays: $2.51 (2nd price + $0.01)
AD SERVING
Return winning creative to publisher → User sees ad
EVENT TRACKING
Impression
Click
Conversion



API Design

Bid Request/Response APIs (Real-Time Bidding)

Method	Endpoint	Description
`POST`	`/api/v1/bid`	Submit bid request (from ad exchange)
`POST`	`/api/v1/win`	Win notification
`GET`	`/api/v1/ad/{adId}/render`	Render ad creative

Bid Request (OpenRTB-compatible)

```
POST /api/v1/bid
{
  "id": "req_123",
  "imp": [{
    "id": "imp_1",
    "banner": { "w": 300, "h": 250 },
    "bidfloor": 0.50
  }],
  "site": { "domain": "example.com", "cat": ["IAB1"] },
  "user": { "id": "user_anon_456" },
  "device": { "ua": "...", "geo": { "country": "US" } }
}

Response: 200 OK (< 100ms)
{
  "seatbid": [{
    "bid": [{
      "id": "bid_789",
      "impid": "imp_1",
      "price": 1.25,
      "adm": "<creative_markup>"
    }]
  }]
}
```

Campaign Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/campaigns`	Create an ad campaign
`PUT`	`/api/v1/campaigns/{campaignId}`	Update campaign settings
`PUT`	`/api/v1/campaigns/{campaignId}/budget`	Update budget/pacing
`GET`	`/api/v1/campaigns/{campaignId}/stats`	Get campaign performance

Bid Request/Response Format

```
// BID REQUEST (from Ad Exchange)
{
  "id": "req_abc123",
  "imp": [{
    "id": "imp_1",
    "banner": {"w": 300, "h": 250},
    "bidfloor": 0.50
  }],
  "site": {
    "domain": "news.example.com",
    "page": "https://news.example.com/article/123",
    "cat": ["IAB12"]  // News category
  },
  "device": {
    "ua": "Mozilla/5.0...",
    "ip": "203.0.113.50",
    "geo": {"country": "USA", "region": "CA", "city": "SF"}
  },
  "user": {
    "id": "user_xyz789",
    "buyeruid": "dsp_user_456"  // DSP's user ID
  }
}

// BID RESPONSE
{
  "id": "req_abc123",
  "seatbid": [{
    "bid": [{
      "id": "bid_001",
      "impid": "imp_1",
      "price": 2.50,
      "adm": "<script>...</script>",  // Ad creative
      "nurl": "https://dsp.com/win?price=${AUCTION_PRICE}",
      "adomain": ["advertiser.com"]
    }]
```

```
    }]
  }
```

Budget & Pacing

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                            BUDGET MANAGEMENT                                │
└─────────────────────────────────────────────────────────────────────────────┘

REAL-TIME BUDGET TRACKING:
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│   Redis Atomic Operations:                                                  │
│                                                                             │
│   # Check and reserve budget atomically                                     │
│   DECRBY campaign:123:budget 250  # Deduct bid amount (cents)               │
│   → Returns new balance                                                     │
│   → If negative, bid was invalid (rollback)                                 │
│                                                                             │
│   # On auction win                                                          │
│   DECRBY campaign:123:budget -250  # Refund reserved amount                 │
│   DECRBY campaign:123:budget 150   # Charge actual price (2nd price)        │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘

PACING (Spread budget evenly throughout day):
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│   Daily budget: $1000                                                       │
│   Current time: 6:00 PM (75% of day elapsed)                                │
│   Spent so far: $600 (60% of budget)                                        │
│                                                                             │
│   Expected spend: $750 (75% of budget)                                      │
│   Actual spend:   $600 (60% of budget)                                      │
│   Status: UNDERPACING (behind schedule)                                     │
│                                                                             │
│   Pacing factor = expected / actual = 750 / 600 = 1.25                      │
│   → Increase bids by 25% to catch up                                        │
│                                                                             │
│   If OVERPACING: Reduce bids or throttle participation                      │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

ML Models for Bidding

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                            PREDICTION MODELS                                │
└─────────────────────────────────────────────────────────────────────────────┘
```

```
1. CLICK-THROUGH RATE (pCTR)
    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
    │  Features:                                                                                                   │
    │    • User: demographics, interests, past behavior                                                            │
    │    • Ad: creative type, advertiser category                                                                  │
    │    • Context: site category, time of day, device                                                             │
    │    • Historical: ad CTR, user's avg CTR                                                                      │
    │                                                                                                              │
    │  Model: Gradient Boosted Trees or Deep Learning                                                              │
    │  Latency: < 10ms (use feature store for fast lookup)                                                         │
    │  Output: P(click) ∈ [0, 1]                                                                                   │
    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

2. CONVERSION RATE (pCVR)
    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
    │  P(conversion | click)                                                                                       │
    │  Often combined with pCTR: pCTCVR = pCTR × pCVR                                                              │
    │                                                                                                              │
    │  Challenge: Sparse data (few conversions)                                                                    │
    │  Solution: Multi-task learning, transfer learning                                                            │
    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

3. BID LANDSCAPE
    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
    │  Predict: P(win | bid_price)                                                                                 │
    │  Use: Optimize bid to maximize ROI                                                                           │
    │                                                                                                              │
    │  optimal_bid = argmax(P(win|bid) × value - bid × P(win|bid))                                                 │
    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Purpose	Technology
Bid Service	Process bid requests	Go/Rust (low latency)
Filter Engine	Fast campaign matching	Bloom filters + rules
ML Scoring	Predict CTR/CVR	TensorFlow Serving
Budget Tracker	Real-time spend	Redis Cluster
Feature Store	Fast feature lookup	Redis / Aerospike
Campaign DB	Campaign configs	PostgreSQL + cache
Event Pipeline	Track impressions/clicks	Kafka → ClickHouse
Reporting	Analytics dashboards	ClickHouse + Grafana

Auction Types

```
SECOND-PRICE AUCTION (Traditional):
┌─────────────────────────────────────────────────────────────────────────────┐
│  Bids: A=$3.00, B=$2.50, C=$2.00
│  Winner: A
│  Pays: $2.51 (second price + $0.01)
│
│  Pros: Truthful bidding (bid your true value)
│  Cons: Revenue not optimal for exchange
└─────────────────────────────────────────────────────────────────────────────┘

FIRST-PRICE AUCTION (Modern trend):
┌─────────────────────────────────────────────────────────────────────────────┐
│  Bids: A=$3.00, B=$2.50, C=$2.00
│  Winner: A
│  Pays: $3.00 (what they bid)
│
│  Pros: Simpler, more revenue
│  Cons: Requires "bid shading" (bid below true value)
│
│  BID SHADING MODEL:
│    shaded_bid = true_value × (1 - shade_factor)
│    shade_factor = f(market_competitiveness, historical_win_rate)
└─────────────────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Stateless bidders**: Any server can handle any request
- **Geo-distributed**: Deploy close to exchanges (reduce latency)
- **Feature caching**: Pre-compute user features, cache in Redis
- **Model serving**: Use TensorFlow Serving with batching
- **Circuit breakers**: Skip slow campaigns, maintain SLA
- **Sampling**: For very high QPS, sample requests (bid on 10%)

Trade-offs

Decision	Option A	Option B	Recommendation
Auction	Second-price	First-price	**First-price** - industry trend
Budget	Eventual	Real-time (Redis)	**Real-time** - avoid overspend
ML inference	Per-request	Batched	**Batched** - 10x throughput
Language	Java	Go/Rust	**Go** - fast, simple, GC-friendly
Feature store	Redis	Aerospike	**Redis** - simpler, good enough

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event pipeline for impressions, clicks, and conversions to billing/reporting	Tech Guide
Redis	Real-time budget tracking with atomic counters, feature store for ML	Tech Guide
Flink/Spark	Model training and feature engineering for CTR/CVR prediction	Tech Guide

How to Explain It Well

> **Opening**: "RTB systems must respond in <100ms. I'll design a bidding engine that filters 90% of campaigns on rules, then uses ML for the rest."

> **Key talking points**:
>
> - "Fast filtering first: budget, geo, frequency cap - eliminates 90% of campaigns in <5ms"
> - "ML scoring for remaining campaigns: pCTR × pCVR × value = expected value"
> - "Redis atomic counters for real-time budget - can't afford eventual consistency"
> - "Pacing algorithm spreads budget evenly throughout the day"
> - "Second-price auction encourages truthful bidding; first-price needs bid shading"
> - "Feature store (Redis) for <1ms feature lookup - can't afford DB calls"

Common Follow-up Questions

1. **How do you handle latency spikes?**
    - Set hard timeout (50ms) - return no-bid if exceeded
    - Circuit breaker for slow models/services
    - Pre-compute as much as possible
2. **How do you prevent overspending?**
    - Redis atomic decrements with rollback on loss
    - Conservative budget reservation (reserve full bid, refund on 2nd price)
    - Async reconciliation every hour

3. **How do you detect ad fraud?**
    - IP/device fingerprint analysis
    - Click pattern anomalies (too fast, same user)
    - Third-party verification (IAS, DoubleVerify)
    - Conversion verification with advertiser

---

46. Design E-commerce Platform

Key Terms:

Term	Full Name	What it does
SKU	Stock Keeping Unit	Unique product variant identifier
OMS	Order Management System	Handles order lifecycle
WMS	Warehouse Management System	Manages inventory and fulfillment
PDP	Product Detail Page	Individual product page
PLP	Product Listing Page	Category/search results page
Cart Abandonment	Incomplete checkout	User adds items but doesn't purchase
Flash Sale	Time-limited sale	High traffic spike (10-100x normal)
Faceted Search	Filter by attributes	Search with category/price/brand filters

Problem Statement

Design the backend for a large-scale e-commerce platform like Amazon, handling product catalog, shopping cart, checkout, orders, and inventory management.

Clarifying Questions to Ask

1. **Scale**: How many products? Daily orders? Peak traffic? (1M products, 100K orders/day?)
2. **Geography**: Single region or global? Multi-currency?
3. **Inventory**: Single warehouse or distributed? Real-time stock?
4. **Checkout**: Payment processing in-house or third-party?

5. **Search**: Full-text search with filters/facets?
6. **Personalization**: Product recommendations?
7. **Flash sales**: Handle traffic spikes (10x-100x normal)?

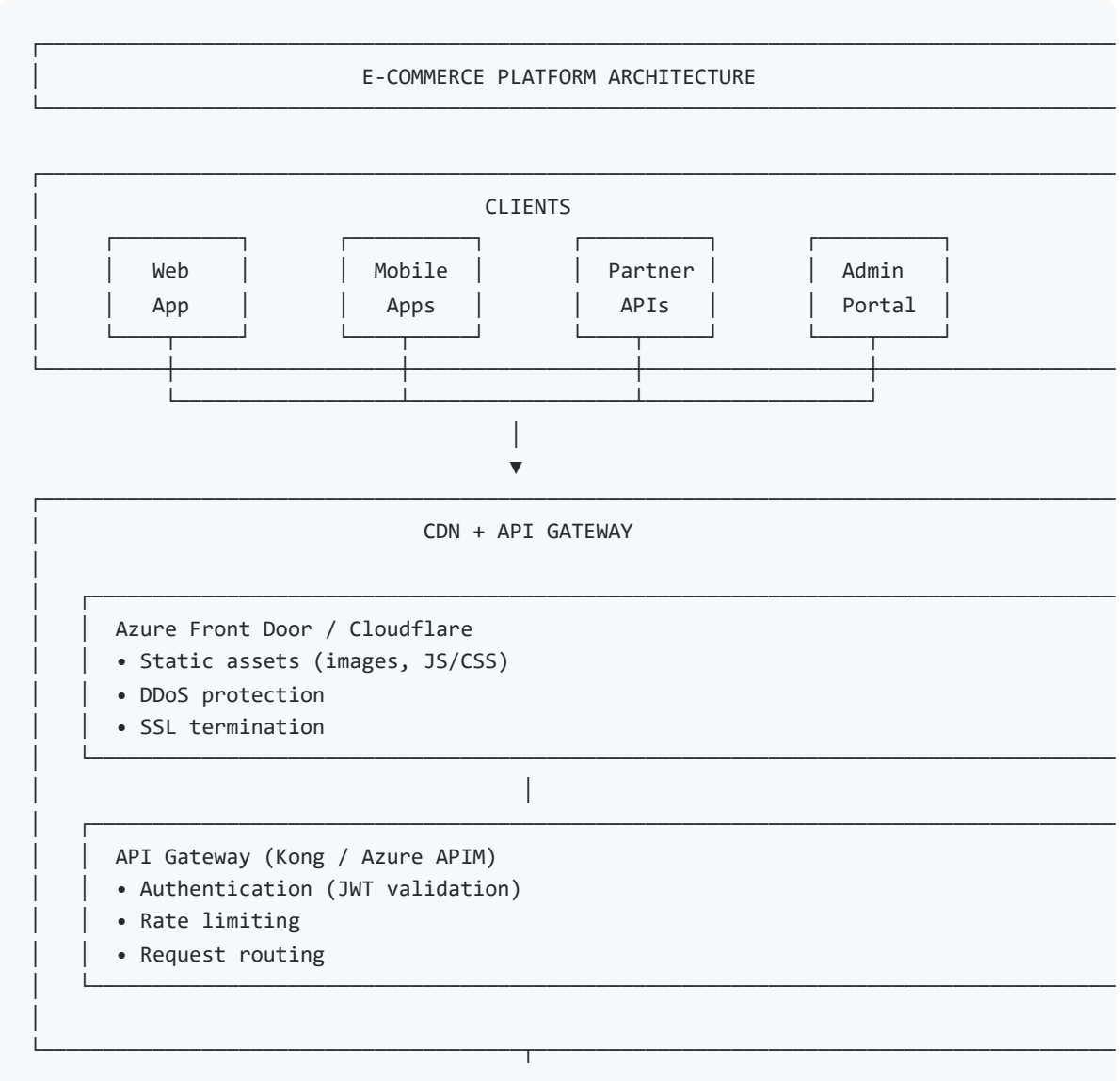
Design Reasoning

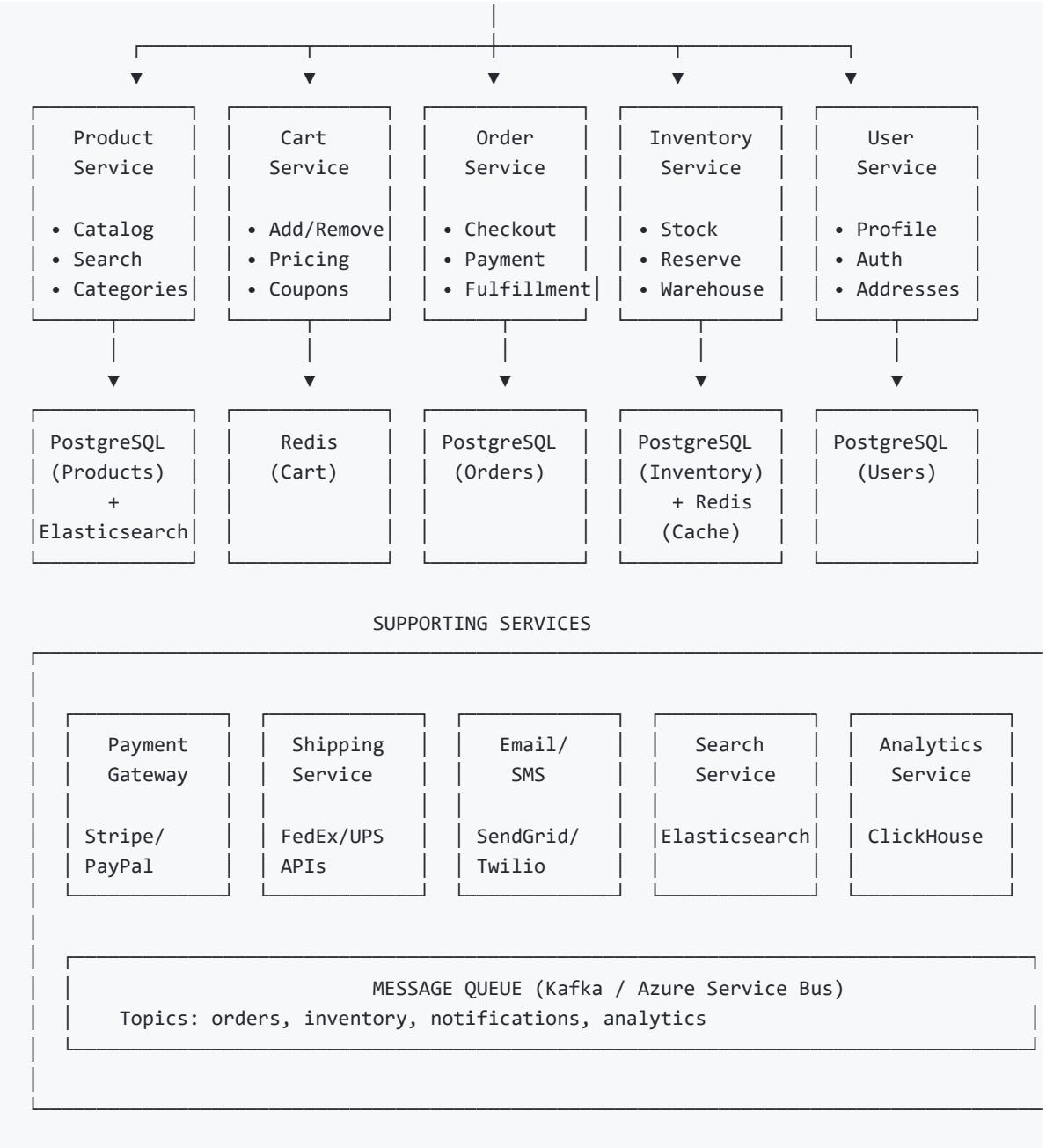
> **Core philosophy**: Guarantee zero overselling through atomic inventory operations and saga-based distributed transactions, while absorbing massive traffic spikes via cache layers and queue-based throttling.

Decision	Choice	Why	Alternative Considered
Cart storage	Redis Hash with 7-day TTL	Carts are ephemeral — sub-ms access, automatic expiration, no ACID overhead needed	PostgreSQL — overkill durability for temporary state, harder to expire old carts
Checkout orchestration	Saga pattern with compensations	Handles distributed tx across Inventory → Order → Payment without long-held locks; graceful rollback on failure	2-Phase Commit — blocks inventory during slow payment calls, poor scalability
Inventory consistency	PostgreSQL atomic UPDATE + reservation pattern	`WHERE available_qty >= @qty` prevents overselling; 10-min reservation hold protects stock during checkout	Eventual consistency — causes overselling during flash sales, erodes customer trust
Product search	Elasticsearch with nested mappings	Faceted filtering (category, price, brand), autocomplete, typo-tolerance; scales independently from transactional DB	PostgreSQL Full-Text Search — limited faceting, competes with transactional queries
Flash sale throttling	Redis atomic DECR + queue-based workers	Pre-generated tokens prevent thundering herd; atomic counters guarantee no overselling; workers	Direct DB updates — would crash under 100× traffic spike, no fairness guarantee

		process at sustainable rate	
Data architecture	Database-per-service (microservices)	Independent scaling, failure isolation, optimized indexes per domain; enables independent deployments	Shared monolithic DB — single point of failure, schema changes require cross-team coordination
Caching strategy	Multi-layer: CDN + Redis + read replicas	CDN for static assets, Redis for hot inventory, read replicas for catalog; different TTLs per data type	Single cache layer — stale data or DB overload; different data needs different caching strategies

High-Level Design





API Design

Product APIs

Method	Endpoint	Description
`GET`	`/api/v1/products?category={cat}&page={n}`	List products
`GET`	`/api/v1/products/{productId}`	Get product details
`GET`	`/api/v1/products/search?q={query}`	Search products

Cart APIs

Method	Endpoint	Description
`GET`	`/api/v1/cart`	Get current cart
`POST`	`/api/v1/cart/items`	Add item to cart
`PUT`	`/api/v1/cart/items/{itemId}`	Update item quantity
`DELETE`	`/api/v1/cart/items/{itemId}`	Remove item from cart

Order APIs

Method	Endpoint	Description
`POST`	`/api/v1/orders`	Place an order (from cart)
`GET`	`/api/v1/orders/{orderId}`	Get order details
`GET`	`/api/v1/orders?status={status}`	List user orders
`POST`	`/api/v1/orders/{orderId}/cancel`	Cancel order
`GET`	`/api/v1/orders/{orderId}/tracking`	Get shipping tracking

Payment APIs

Method	Endpoint	Description
`POST`	`/api/v1/payments`	Process payment
`POST`	`/api/v1/payments/{paymentId}/refund`	Initiate refund

Inventory APIs (Internal)

Method	Endpoint	Description
`GET`	`/api/v1/inventory/{productId}`	Check stock availability
`PUT`	`/api/v1/inventory/{productId}/reserve`	Reserve inventory
`PUT`	`/api/v1/inventory/{productId}/release`	Release reservation

Product Service

```
-- Product catalog schema
CREATE TABLE products (
```

```
    id BIGINT PRIMARY KEY,
    sku VARCHAR(50) UNIQUE,
    name VARCHAR(255),
    description TEXT,
    category_id BIGINT,
    brand_id BIGINT,
    price DECIMAL(10, 2),
    compare_at_price DECIMAL(10, 2),  -- Original price for discounts
    currency VARCHAR(3) DEFAULT 'USD',
    status VARCHAR(20),  -- active, draft, archived
    attributes JSONB,    -- {"color": "red", "size": "M"}
    images JSONB,        -- ["url1", "url2"]
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

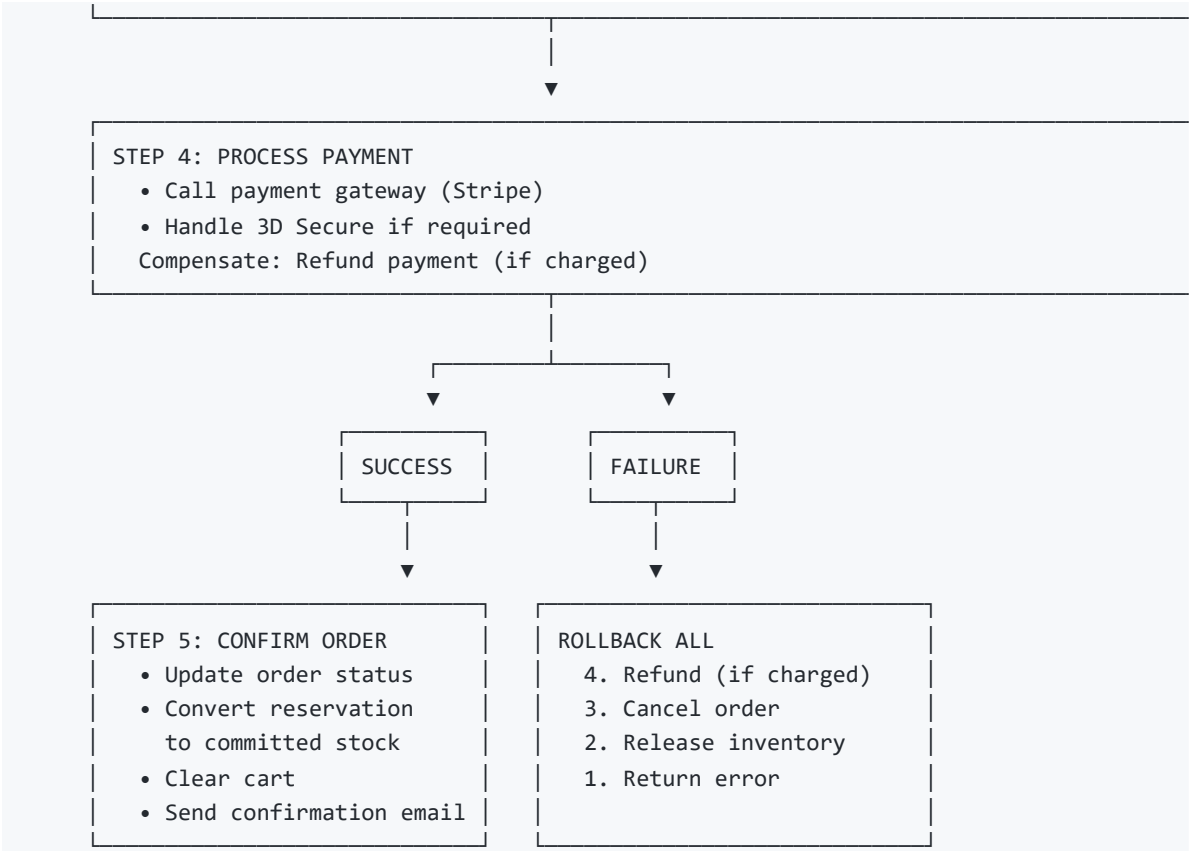
CREATE TABLE categories (
    id BIGINT PRIMARY KEY,
    parent_id BIGINT REFERENCES categories(id),
    name VARCHAR(100),
    slug VARCHAR(100),
    path LTREE  -- PostgreSQL extension for hierarchical data
);

-- Elasticsearch index for search
{
  "mappings": {
    "properties": {
      "name": {"type": "text", "analyzer": "standard"},
      "description": {"type": "text"},
      "category": {"type": "keyword"},
      "brand": {"type": "keyword"},
      "price": {"type": "float"},
      "attributes": {"type": "nested"},
      "in_stock": {"type": "boolean"}
    }
  }
}
```

Shopping Cart (Redis)

```
CART DATA MODEL (Redis Hash):
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│   KEY: cart:{user_id}                                                       │
│                                                                             │
│   HASH FIELDS:                                                              │
│     item:{product_id} → {"qty": 2, "price": 29.99, "added_at": "..."}       │
│     item:{product_id} → {"qty": 1, "price": 49.99, "added_at": "..."}       │
│     coupon → "SAVE10"                                                       │
│     updated_at → "2024-01-15T10:30:00Z"                                     │
│                                                                             │
```

```
│   EXPIRATION: 7 days (TTL)
│
│   Commands:
│     HSET cart:123 item:456 '{"qty": 2, "price": 29.99}'
│     HGETALL cart:123
│     HDEL cart:123 item:456
│     EXPIRE cart:123 604800  # 7 days
│
└──────────────
```



Inventory Management

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          INVENTORY TRACKING                                 │
└─────────────────────────────────────────────────────────────────────────────┘

DATABASE SCHEMA:
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│  CREATE TABLE inventory (                                                   │
│      product_id BIGINT,                                                     │
│      warehouse_id BIGINT,                                                   │
│      available_qty INT,      -- Can sell                                    │
│      reserved_qty INT,       -- Reserved for pending orders                 │
│      on_hand_qty INT,        -- Physical stock = available + reserved       │
│      PRIMARY KEY (product_id, warehouse_id)                                 │
│  );                                                                         │
│                                                                             │
│  CONSTRAINT: available_qty >= 0                                             │
│  CONSTRAINT: on_hand_qty = available_qty + reserved_qty                     │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘

RESERVATION PATTERN:
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│  -- Reserve inventory (atomic)
```

```
  UPDATE inventory
  SET available_qty = available_qty - @qty,
      reserved_qty = reserved_qty + @qty
  WHERE product_id = @pid
    AND warehouse_id = @wid
    AND available_qty >= @qty;  -- Prevent negative

  -- On order completion: Commit reservation
  UPDATE inventory
  SET reserved_qty = reserved_qty - @qty,
      on_hand_qty = on_hand_qty - @qty
  WHERE product_id = @pid AND warehouse_id = @wid;

  -- On order cancellation: Release reservation
  UPDATE inventory
  SET available_qty = available_qty + @qty,
      reserved_qty = reserved_qty - @qty
  WHERE product_id = @pid AND warehouse_id = @wid;
```

```
REDIS CACHE (Hot products):
```

```
  # Fast availability check
  GET inventory:{product_id}:{warehouse_id}
  → "45"  (available quantity)

  # Atomic decrement for high-traffic products
  DECRBY inventory:{product_id}:{warehouse_id} 1
  → If result < 0, rollback and return "out of stock"

  # Sync DB → Redis every few seconds
```

Flash Sale Handling

```
                          FLASH SALE ARCHITECTURE
```

```
PROBLEM: 100x traffic spike for limited inventory (1000 items, 100K users)

SOLUTION:
```

```
  1. QUEUE-BASED THROTTLING
     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
     │   Users      │────▶│   Queue      │────▶│  Workers     │
     │  (100K)      │     │  (Redis)     │     │  (Process    │
     │              │     │              │     │   1K/sec)    │
     └──────────────┘     └──────────────┘     └──────────────┘
```

```
│
│  2. PRE-GENERATE TOKENS
│     • Generate 1000 purchase tokens before sale
│     • First 1000 users get tokens, others get "sold out"
│     • Token holders have 10 minutes to complete purchase
│
│  3. REDIS STOCK COUNTER
│     SETNX flash:sale:123:stock 1000  # Set only if not exists
│     DECR flash:sale:123:stock        # Atomic decrement
│     → If result < 0, return "sold out"
│
│  4. RATE LIMITING
│     • 1 request per user per second
│     • CAPTCHA for suspicious traffic
│
└──────────────────────────────────────────────────────────────────────────────────────────────
```

Key Components

Service	Purpose	Database	Key Features
Product	Catalog management	PostgreSQL + Elasticsearch	Full-text search, facets
Cart	Shopping cart	Redis	TTL, merge on login
Order	Order lifecycle	PostgreSQL	Saga, idempotency
Inventory	Stock management	PostgreSQL + Redis	Reservation pattern
Payment	Payment processing	- (External: Stripe)	Idempotent, retry
Search	Product search	Elasticsearch	Facets, autocomplete
User	Auth, profiles	PostgreSQL	OAuth, JWT

Scalability Considerations

- **Read replicas**: Product catalog is read-heavy; use replicas
- **Caching**: CDN for images, Redis for product details
- **Database sharding**: Shard orders by user_id
- **Async processing**: Order confirmation emails via queue
- **Search scaling**: Elasticsearch cluster with shards/replicas
- **Flash sales**: Queue-based throttling, pre-generated tokens

Trade-offs

Decision	Option A	Option B	Recommendation
Cart storage	Database	Redis	**Redis** - ephemeral, fast
Checkout	2PC	Saga	**Saga** - better failure handling
Search	PostgreSQL FTS	Elasticsearch	**ES** - facets, relevance
Inventory	Eventual	Real-time	**Real-time** - avoid overselling
Payment	Synchronous	Async	**Sync** - user expects immediate feedback

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Shopping cart storage, inventory caching, flash sale counters	Tech Guide
PostgreSQL	Product catalog, orders, inventory, and user data persistence	Tech Guide
Kafka	Async messaging for orders, inventory updates, and notifications	Tech Guide
Elasticsearch	Full-text product search with faceted filtering	Tech Guide
Saga Pattern	Checkout orchestration: reserve → order → charge → confirm with compensations	Tech Guide
CDN	Static assets, product images, and edge caching	Tech Guide

How to Explain It Well

> **Opening**: "E-commerce needs to handle catalog browsing, cart management, and checkout atomically. I'll use microservices with a Saga pattern for distributed transactions."

> **Key talking points**:
>
> - "Redis for cart - ephemeral data, fast operations, automatic expiry"

> - "Saga pattern for checkout: reserve → create order → charge → confirm, with compensations"
> - "Inventory uses reservation pattern: separate available vs reserved quantities"
> - "Elasticsearch for search with faceted filtering (category, price, brand)"
> - "Flash sales: Redis atomic counters + queue-based throttling"
> - "Idempotency keys for payment retries - charge exactly once"

Common Follow-up Questions

1. **How do you handle overselling?**
    - Atomic inventory reservation (UPDATE with WHERE qty >= needed)
    - Redis counters for flash sales
    - Compensation: refund + notify customer
2. **How do you handle abandoned carts?**
    - Redis TTL (7 days)
    - Send reminder email after 1 hour, 24 hours
    - Release inventory reservation after 10 minutes
3. **How do you handle price changes during checkout?**
    - Lock price at cart addition
    - Re-validate at checkout
    - Show user if price changed, require confirmation

---

47. Design Payment Gateway

Key Terms:

Term	Full Name	What it does
PCI-DSS	Payment Card Industry Data Security Standard	Security requirements for card processing
TPS	Transactions Per Second	Throughput measure for payment systems
3DS	3D Secure	Additional authentication (Verified by Visa, etc.)

Tokenization	Card data replacement	Replace card number with non-sensitive token
Idempotency Key	Duplicate prevention	Ensures same request isn't processed twice
Chargeback	Transaction dispute	Customer disputes charge with bank
ACH	Automated Clearing House	US bank-to-bank transfer system
Settlement	Funds transfer	Moving money from buyer to seller account
Acquirer	Merchant's bank	Bank that receives payment for merchant
Issuer	Cardholder's bank	Bank that issued the credit card

Problem Statement

Design a payment gateway like Stripe or PayPal that securely processes credit card transactions with fraud detection, PCI compliance, and high availability.

Clarifying Questions to Ask

1. **Scale**: Transactions per second? (1K-100K TPS?)
2. **Payment types**: Credit cards only, or also ACH, wallets, crypto?
3. **Regions**: Global or specific markets? Multi-currency?
4. **Compliance**: PCI-DSS level? (Level 1 for >6M transactions/year)
5. **Settlement**: Real-time or batch settlement?
6. **Retry policy**: How to handle network failures?
7. **Chargebacks**: Dispute handling workflow?

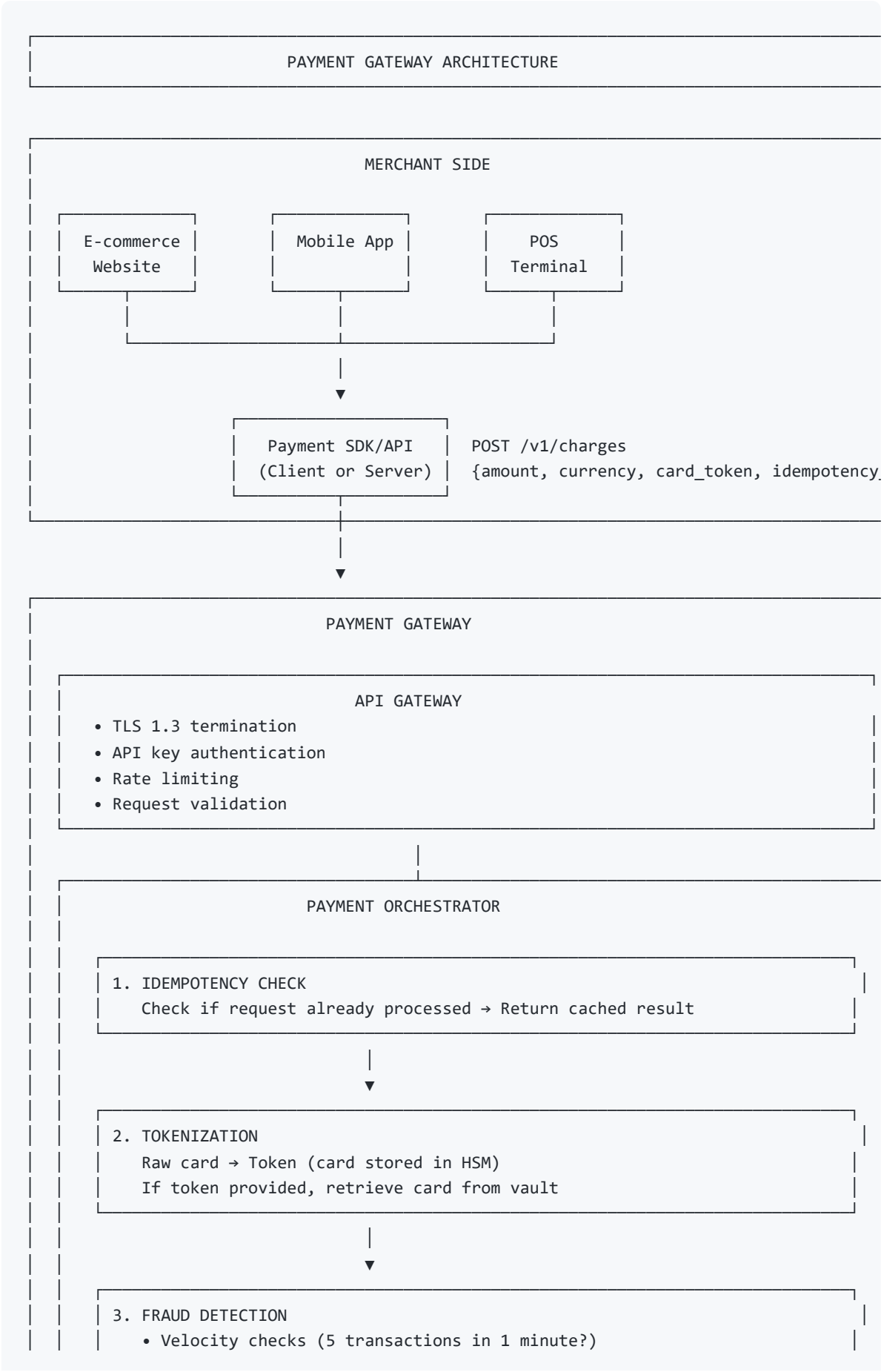
Design Reasoning

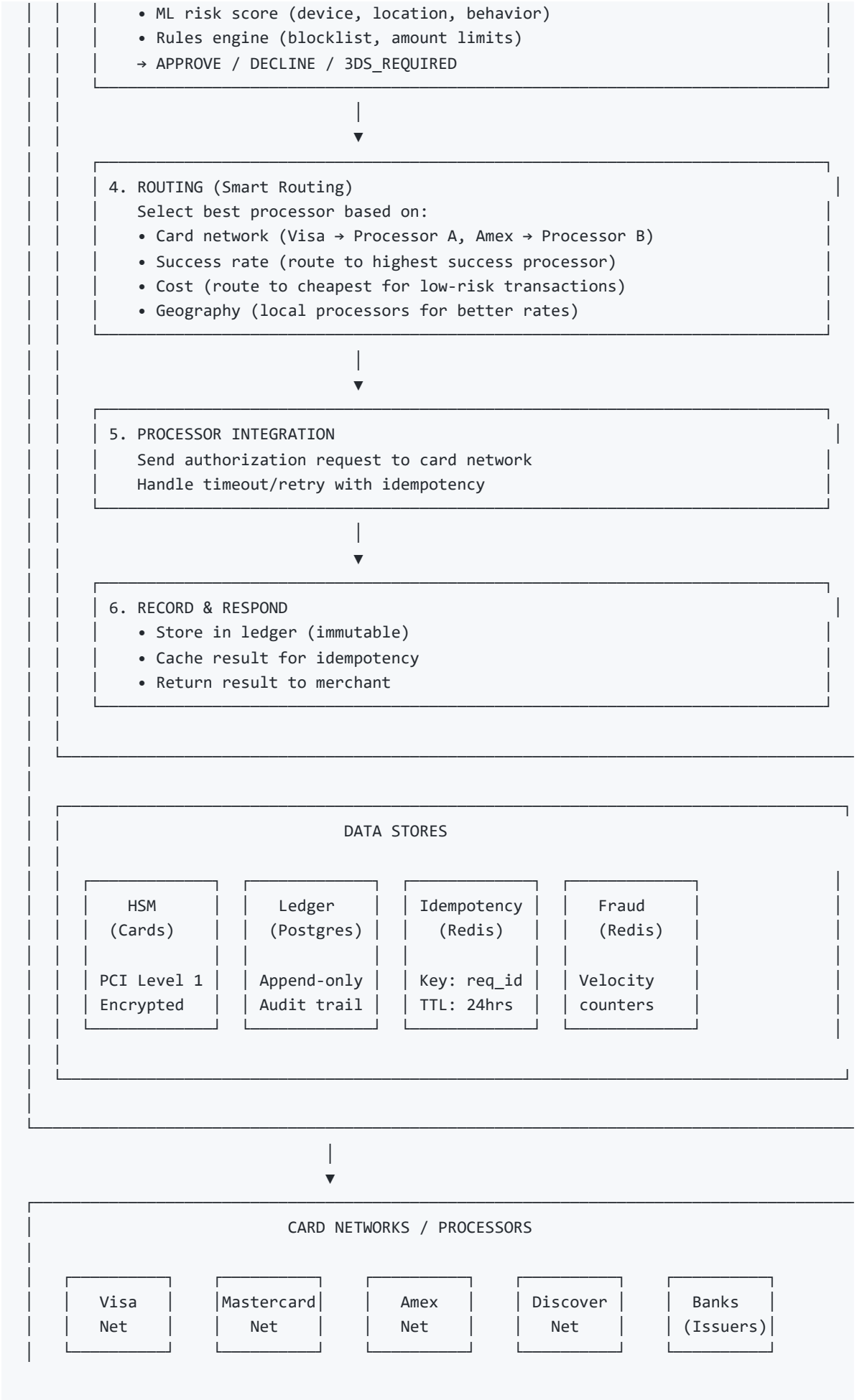
> Core philosophy: Financial accuracy and PCI compliance above all — immutable ledgers prevent double-charging, HSM tokenization eliminates card data exposure, and layered fraud detection adapts from millisecond rules to ML models, ensuring exactly-once transaction processing with complete auditability.

Decision	Choice	Why	Alternative Considered

Idempotency store	Redis with NX atomic SET + 24h TTL	O(1) lookup, atomic compare-and-set prevents race conditions on retries, TTL auto-expires old keys	PostgreSQL — slower B-tree lookups, requires explicit lock management, no built-in TTL
Card tokenization	Hardware Security Module (HSM)	PCI-DSS Level 1 requires tamper-proof hardware for >6M tx/year; keys never leave the module	Software-based encryption — doesn't meet highest compliance tier, greater liability in audits
Transaction ledger	PostgreSQL append-only double-entry	ACID guarantees for financial accuracy, immutable audit trail, supports complex reconciliation queries	Event sourcing — good for event tracking but double-entry ledger is simpler for accounting
Fraud detection	Three-layer: Rules (<5ms) + Velocity (Redis) + ML scoring	Rules catch obvious fraud instantly; velocity detects automated attacks; ML handles sophisticated patterns	Rules-only — misses complex patterns; ML-only — can't meet <5ms latency requirement
Processor routing	Multi-processor smart routing (card type, geography, success rate)	Optimizes success rates per card network, reduces costs, provides automatic failover	Single processor — single point of failure, no geographic optimization
Webhook delivery	Kafka + async worker pool	Decouples payment processing from notification; at-least-once semantics, retry with backoff, replay on failure	Synchronous HTTP callbacks — blocks critical payment path, vulnerable to slow endpoints
Network protocol	ISO 8583 adapters	Industry standard mandated by Visa/Mastercard; well-defined message format for auth, capture, reversal	REST APIs — card networks don't expose REST; ISO 8583 is a regulatory requirement

High-Level Design




• ML risk score (device, location, behavior)
• Rules engine (blocklist, amount limits)
→ APPROVE / DECLINE / 3DS_REQUIRED
4. ROUTING (Smart Routing)
Select best processor based on:
• Card network (Visa → Processor A, Amex → Processor B)
• Success rate (route to highest success processor)
• Cost (route to cheapest for low-risk transactions)
• Geography (local processors for better rates)
5. PROCESSOR INTEGRATION
Send authorization request to card network
Handle timeout/retry with idempotency
6. RECORD & RESPOND
• Store in ledger (immutable)
• Cache result for idempotency
• Return result to merchant
DATA STORES
HSM (Cards)
PCI Level 1 Encrypted
Ledger (Postgres)
Append-only Audit trail
Idempotency (Redis)
Key: req_id
TTL: 24hrs
Fraud (Redis)
Velocity counters
CARD NETWORKS / PROCESSORS
Visa Net
Mastercard Net
Amex Net
Discover Net
Banks (Issuers)

```
|
└──────────────
```

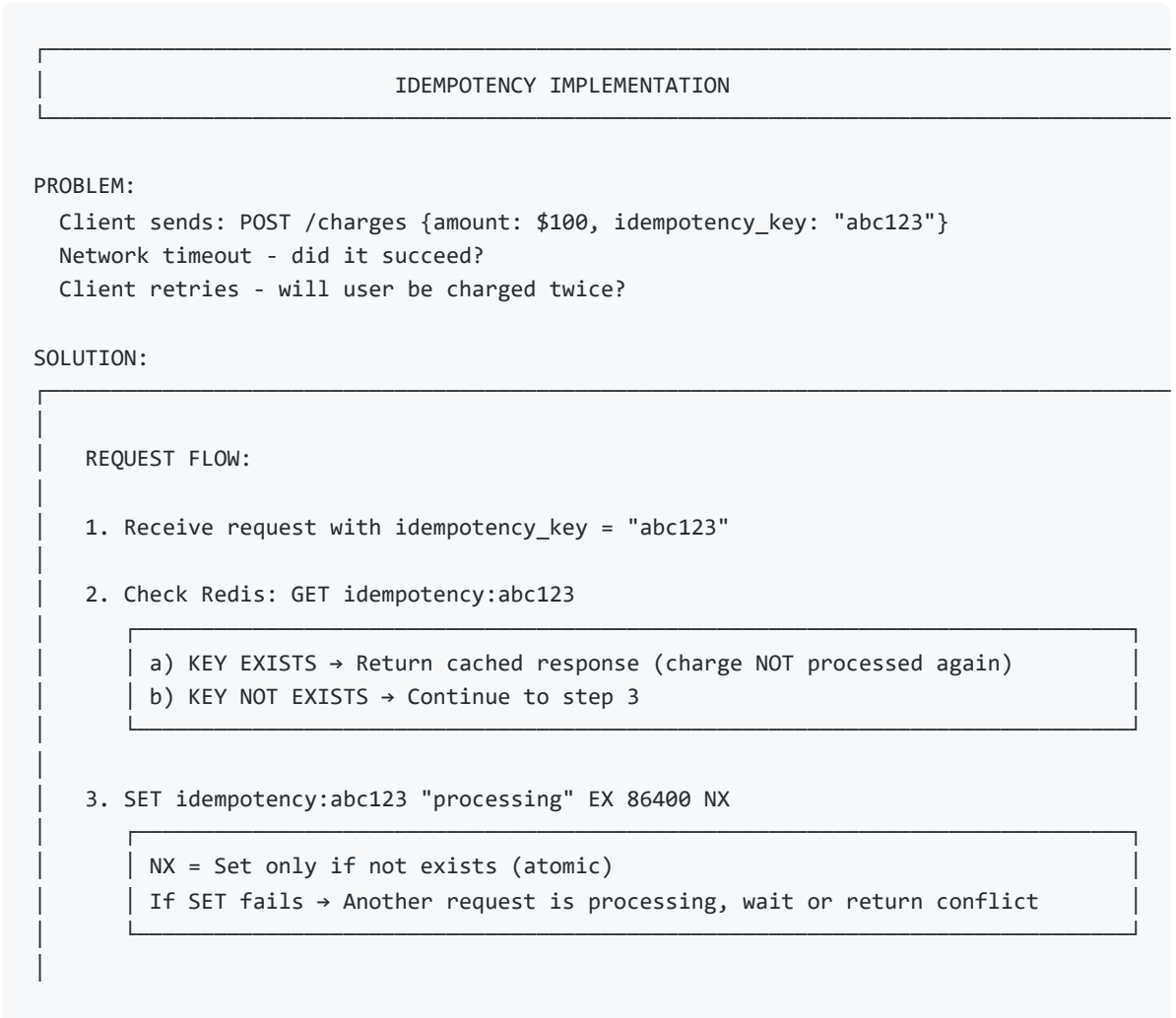
Webhook APIs

Method	Endpoint	Description
`POST`	`/api/v1/webhooks`	Register webhook endpoint
`GET`	`/api/v1/webhooks/{webhookId}/deliveries`	Get delivery history

Webhook Events

Event	Payload
`payment.authorized`	`{ paymentId, amount, status }`
`payment.captured`	`{ paymentId, capturedAmount }`
`payment.failed`	`{ paymentId, error, failureCode }`
`refund.completed`	`{ refundId, paymentId, amount }`

Idempotency (Critical for Payments)



```
│   4. Process payment
│
│   5. Store result: SET idempotency:abc123 {result_json} EX 86400
│
│   6. Return result
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────

IDEMPOTENCY KEY STORAGE:
{
  "key": "abc123",
  "status": "completed",
  "result": {
    "charge_id": "ch_xyz789",
    "amount": 10000,
    "currency": "usd",
    "status": "succeeded"
  },
  "created_at": "2024-01-15T10:30:00Z",
  "expires_at": "2024-01-16T10:30:00Z"
}
```

Tokenization & PCI Compliance

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                    TOKENIZATION FLOW                                                 │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘

INITIAL CARD SUBMISSION (Client-side):
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                      │
│   Browser/App                          Tokenization Service (PCI Scope)                              │
│        │                                          │                                                 │
│        │  POST /tokens                            │                                                 │
│        │  {card: "4242424242424242",              │                                                 │
│        │   exp: "12/25", cvc: "123"}              │                                                 │
│        │──────────────────────────────────────────▶│                                                 │
│        │                                          │                                                 │
│        │                                  ┌───────┴──────────┐                                      │
│        │                                  │ Encrypt card     │                                      │
│        │                                  │ Store in HSM     │                                      │
│        │                                  │ Generate token   │                                      │
│        │                                  └───────┬──────────┘                                      │
│        │                                          │                                                 │
│        │  {token: "tok_abc123"}                   │                                                 │
│        │◀─────────────────────────────────────────│                                                 │
│        │                                          │                                                 │
│                                                                                                      │
│   TOKEN USAGE (Server-side - OUT of PCI scope):                                                      │
│   POST /charges                                                                                      │
│   {amount: 10000, source: "tok_abc123"}                                                              │
│                                                                                                      │
│   Merchant server NEVER sees actual card number!
```

```
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘

PCI-DSS REQUIREMENTS:
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   DO:
│     ✓ Encrypt card data at rest (AES-256)
│     ✓ Encrypt in transit (TLS 1.3)
│     ✓ Store encryption keys in HSM
│     ✓ Tokenize before storing
│     ✓ Mask card numbers in logs (****4242)
│     ✓ Quarterly vulnerability scans
│     ✓ Annual penetration testing
│
│   DON'T:
│     ✗ Log full card numbers
│     ✗ Store CVV/CVC (ever!)
│     ✗ Store unencrypted card data
│     ✗ Email card numbers
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Fraud Detection

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                              FRAUD DETECTION LAYERS
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘

LAYER 1: RULES ENGINE (Fast, <5ms)
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   Rule Examples:
│     • Amount > $10,000 → FLAG
│     • Country mismatch (card country ≠ IP country) → FLAG
│     • Card on blocklist → DECLINE
│     • New account + high value → FLAG
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘

LAYER 2: VELOCITY CHECKS (Redis counters)
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   Redis Keys (with TTL):
│     velocity:card:{hash}:1h   → Count transactions in last hour
│     velocity:ip:{ip}:1h       → Count from same IP
│     velocity:device:{id}:1d   → Count from same device
│
│   Thresholds:
│     > 5 transactions per card per hour → FLAG
│     > 10 transactions per IP per hour → FLAG
│     > 3 declines in a row → BLOCK
│
```

```
└──────────────────────────────────────────────────────────────────────────────┘

LAYER 3: ML MODEL (Risk scoring)
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│   Features:                                                                  │
│     • Transaction: amount, currency, merchant category                       │
│     • Card: issuer, country, age, historical fraud rate                      │
│     • Device: fingerprint, known device?, proxy/VPN?                         │
│     • Behavior: time since last transaction, typical spending pattern        │
│     • Context: time of day, merchant reputation                              │
│                                                                              │
│   Output: Risk score 0-100                                                   │
│     0-30:  APPROVE                                                           │
│     30-70: 3D SECURE (step-up authentication)                                │
│     70-100: DECLINE                                                          │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

Ledger (Double-Entry Bookkeeping)

```sql
-- Immutable transaction ledger (append-only)
CREATE TABLE ledger_entries (
    id BIGINT PRIMARY KEY,
    transaction_id VARCHAR(50),
    account_id VARCHAR(50),      -- "merchant:123", "fees", "reserves"
    entry_type VARCHAR(10),      -- 'debit' or 'credit'
    amount DECIMAL(15, 2),
    currency VARCHAR(3),
    created_at TIMESTAMP,
    metadata JSONB
);

-- Every transaction creates balanced entries (debits = credits)
-- Example: $100 charge with 2.9% + $0.30 fee

-- 1. Charge customer card
INSERT INTO ledger_entries VALUES (1, 'txn_123', 'customer_card', 'debit', 100.00, '
INSERT INTO ledger_entries VALUES (2, 'txn_123', 'merchant:456', 'credit', 97.00, 'U
INSERT INTO ledger_entries VALUES (3, 'txn_123', 'stripe_fees', 'credit', 2.90, 'USD
INSERT INTO ledger_entries VALUES (4, 'txn_123', 'stripe_fixed_fee', 'credit', 0.10,

-- Verify: SUM(debits) = SUM(credits)
-- $100 = $97 + $2.90 + $0.10 ✓
```

Key Components

Component	Purpose	Technology

API Gateway	Auth, rate limiting	Kong / Azure APIM
Payment Orchestrator	Workflow management	Go / Java
Tokenization Vault	Secure card storage	HSM (Thales/AWS CloudHSM)
Fraud Service	Risk scoring	Python ML + Rules engine
Processor Gateway	Network integration	ISO 8583 adapters
Ledger	Audit trail	PostgreSQL (append-only)
Idempotency Store	Deduplication	Redis (24hr TTL)
Notification Service	Webhooks	Kafka + Workers

Scalability Considerations

- **Stateless services**: Any server can handle any request
- **Database sharding**: Shard ledger by merchant_id
- **Read replicas**: Reporting queries on replicas
- **Async processing**: Webhooks, settlement via queues
- **Circuit breakers**: Fail fast if processor is down
- **Multi-region**: Active-passive for DR, active-active for global

Trade-offs

Decision	Option A	Option B	Recommendation
Card storage	Own vault	Third-party (Stripe)	**Third-party** unless at scale
Idempotency	Database	Redis	**Redis** - faster, TTL built-in
Ledger	Single DB	Event sourcing	**Event sourcing** for audit
Fraud	Rules only	Rules + ML	**Both** - rules fast, ML accurate
3DS	Always	Risk-based	**Risk-based** - better UX

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

PostgreSQL	Double-entry ledger for immutable transaction audit trail	Tech Guide
HSM	Hardware Security Module for secure card tokenization and encryption	Tech Guide
Saga Pattern	Payment orchestration with compensations for failed/refunded transactions	Tech Guide
Redis	Idempotency store (24hr TTL), velocity counters for fraud detection	Tech Guide

How to Explain It Well

> **Opening**: "Payment gateways need to ensure exactly-once processing, PCI compliance, and fraud prevention. I'll design with idempotency keys, tokenization, and layered fraud detection."

> **Key talking points**:
>
> - "Idempotency keys ensure charge-once: store result, return cached on retry"
> - "Tokenization: raw card numbers never touch merchant servers - PCI scope reduced"
> - "Cards encrypted in HSM, only tokenization service can decrypt"
> - "Fraud: rules for fast checks, velocity counters, ML for complex patterns"
> - "Double-entry ledger: every transaction balances, immutable audit trail"
> - "Smart routing: select processor by card network, success rate, cost"

Common Follow-up Questions

1. **What if the processor times out?**
   - Don't retry blindly (might double-charge)
   - Query processor for transaction status
   - Use idempotency key for safe retry
2. **How do you handle chargebacks?**
   - Webhook from processor triggers dispute workflow
   - Gather evidence (receipt, delivery confirmation)
   - Submit to processor within deadline
   - Reserve funds during dispute
3. **How do you ensure high availability?**

- Multi-region deployment
- Multiple processor integrations (failover)
- Circuit breakers to fail fast
- 99.99% SLA = <52 minutes downtime/year

---

52. Design Twitter / X

Problem Statement

Design a social media platform like Twitter/X that allows users to post short messages (tweets), follow other users, view a personalized home timeline (news feed), search content, and engage with tweets via likes, retweets, and replies. The system must handle hundreds of millions of users, celebrity accounts with 100M+ followers, and viral content that can spike traffic 10-100x in seconds.

Clarifying Questions to Ask

1. **Core features**: Tweet, follow, timeline, search, notifications — which to prioritize?
2. **Tweet length**: 280 characters? Or also support long-form posts, images, video?
3. **Timeline**: Chronological, algorithmic (ranked), or both?
4. **Celebrity problem**: How to handle users with 100M+ followers? (fan-out challenge)
5. **Real-time**: How fresh should the timeline be? (seconds vs minutes)
6. **Search**: Full-text search over all tweets? Trending topics?
7. **Scale**: How many DAU? (300M+ DAU, 500M+ tweets/day)
8. **Media**: Support images, video, polls, or text-only for now?
9. **Notifications**: Real-time push for mentions, likes, retweets?
10. **Ads**: Inject promoted tweets into timeline?

Design Reasoning

> **Core philosophy**: Trade write complexity for read performance — pre-compute and cache timelines for sub-10ms reads, accept eventual consistency for engagement counters, and use a hybrid fan-out model that adapts distribution strategy based on follower count to handle celebrity-scale asymmetry.

Decision	Choice	Why	Alternative Considered

Timeline distribution	Hybrid fan-out: push for normal users (<500K followers), pull for celebrities	Pure push causes 100M writes per celebrity tweet; pure pull requires 200 parallel queries per read; hybrid solves both	Fixed threshold — celebrity status is dynamic; pure push/pull — each breaks at different scale extremes
Tweet ID generation	Snowflake ID (64-bit: 41 time + 10 machine + 12 sequence)	Time-sortable enables cursor pagination without coordination; generates ~4M IDs/sec per machine; distributed	UUIDs — not time-sortable, requiring additional sort overhead for chronological feeds
Timeline cache	Redis Sorted Sets (tweet IDs scored by timestamp, max 800/user)	O(log N) insertion, O(1) range reads; pre-computed timelines avoid 200-query merge at read time; <10ms p99	Fan-out on read for all — merge-sort on 200+ queries yields 200-500ms latency
Fan-out processing	Kafka (partitioned by author_id) + 100+ workers with Redis pipelining	Kafka preserves per-author order, provides replay on failure; pipelining achieves ~100K ZADD/s per connection	Direct fan-out from tweet service — single-threaded bottleneck, couples write to fan-out latency
Tweet search	Elasticsearch with 7-30 day rolling window (~100 nodes)	Full-text search with real-time indexing from Kafka; rolling window keeps index manageable (~100 TB)	Index all tweets forever — petabytes of hot storage, slower queries; 7-30 day window covers >95% of searches
Engagement counters	Separate `tweet_counts` table, eventually	Avoids row-lock contention on immutable tweets	Strong consistency on main row — 1M likes/min on viral

	consistent, counter sharding for viral tweets	table; viral tweets shard across 100 sub-counters	tweets creates extreme single-row contention
Feed ranking	Multi-stage ML: candidate retrieval → feature extraction → LightGBM scoring → business rules	LightGBM inference <20ms for 800 candidates; multi-stage reduces ML load; diversity rules prevent filter bubbles	Pure chronological — significantly lower engagement; pure engagement scoring — creates filter bubbles
Persistent storage	Vitess (sharded MySQL) for tweets and social graph	ACID for critical operations; Snowflake-based sharding puts recent tweets on fewer, faster shards	NoSQL (Cassandra/DynamoDB) — Twitter requires ACID for some operations; Vitess provides both

---

Requirements & Sizing

Functional

- **Tweet**: Post a tweet (text, media, polls), delete own tweets
- **Follow/Unfollow**: Follow other users, see follower/following lists
- **Home Timeline**: Personalized feed of tweets from followed users
- **User Timeline**: All tweets by a specific user (profile page)
- **Like / Retweet / Reply**: Engage with tweets
- **Search**: Full-text search across tweets, users, hashtags
- **Trending**: Real-time trending topics by region
- **Notifications**: Mentions, likes, retweets, new followers

Non-functional

Metric	Target
Timeline latency	< **200ms** p99 for home timeline load
Tweet publish	< **500ms** until visible to author

Tweet fan-out	< **5s** for tweet to reach followers' timelines
Availability	**99.99%** (four 9s)
Scale	**300M DAU**, **500M tweets/day**
Search latency	< **300ms** for keyword search
Storage	Petabytes of tweets + media

Capacity Estimates

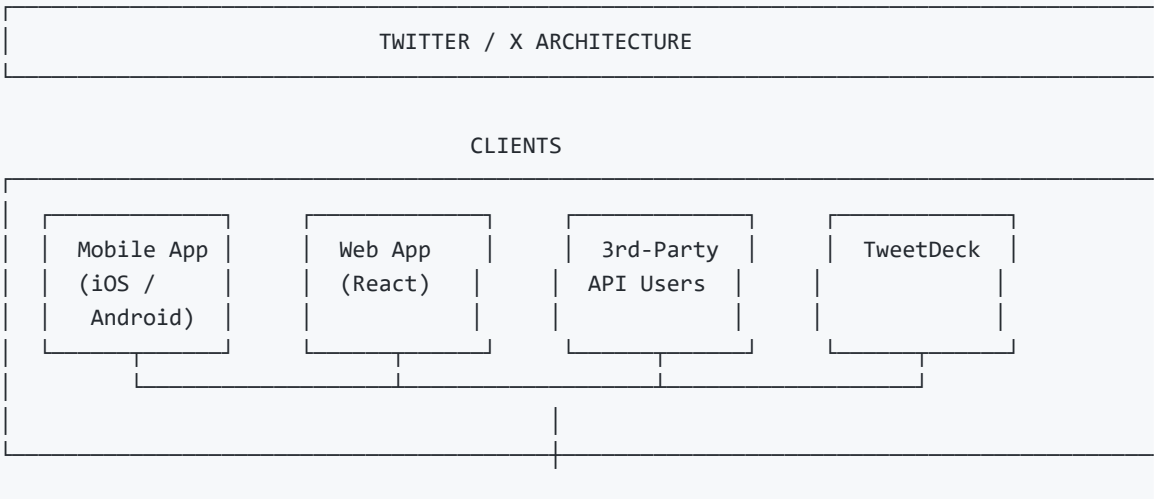
```
Users:
- 1B total accounts, 300M DAU
- Average user follows ~200 accounts
- Average user reads timeline ~10 times/day

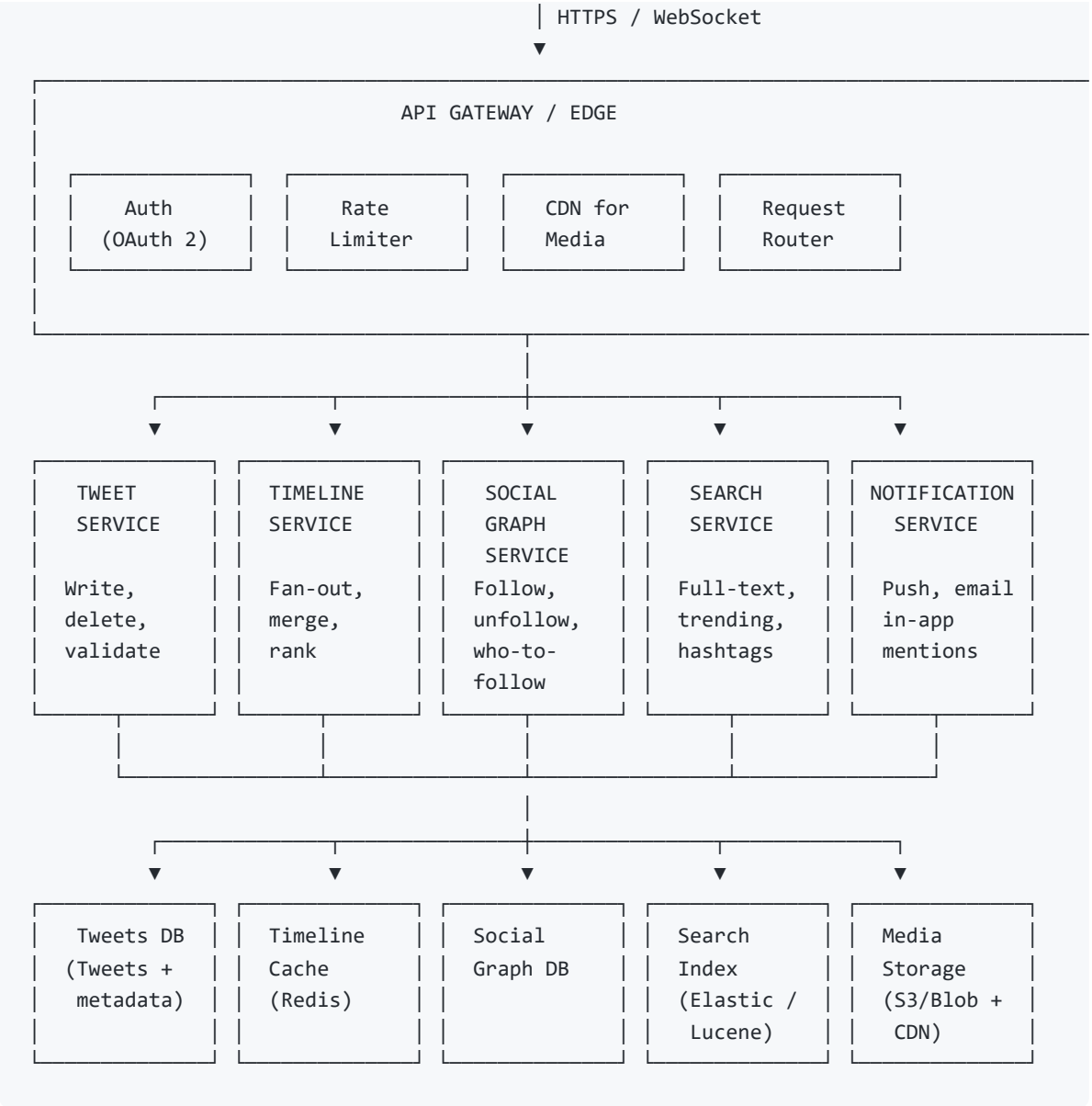
Tweets:
- 500M tweets/day ≈ 6,000 tweets/second (average)
- Peak: 30,000 tweets/second (during major events)
- Average tweet: 200 bytes text + 500 bytes metadata = ~700 bytes
- Daily tweet storage: 500M × 700B ≈ 350 GB/day ≈ 125 TB/year

Timeline reads:
- 300M DAU × 10 reads/day = 3B timeline reads/day
- ≈ 35,000 reads/second (average), 150K+ at peak

Fan-out:
- Average user: 200 followers → 500M × 200 = 100B fan-out ops/day
- Celebrity (100M followers): 1 tweet = 100M writes
- This is THE hard problem
```

High-Level Design





API Design

Tweet APIs

Method	Endpoint	Description
`POST`	`/v2/tweets`	Create a tweet
`DELETE`	`/v2/tweets/{id}`	Delete a tweet
`GET`	`/v2/tweets/{id}`	Get a single tweet
`POST`	`/v2/tweets/{id}/retweet`	Retweet
`POST`	`/v2/tweets/{id}/like`	Like a tweet

`DELETE`	`/v2/tweets/{id}/like`	Unlike a tweet

Timeline APIs

Method	Endpoint	Description
`GET`	`/v2/timeline/home`	Get home timeline (paginated)
`GET`	`/v2/timeline/user/{userId}`	Get user's tweets (profile)
`GET`	`/v2/timeline/home?cursor={cursor}`	Pagination via cursor

Social Graph APIs

Method	Endpoint	Description
`POST`	`/v2/users/{id}/follow`	Follow a user
`DELETE`	`/v2/users/{id}/follow`	Unfollow a user
`GET`	`/v2/users/{id}/followers`	List followers (paginated)
`GET`	`/v2/users/{id}/following`	List following (paginated)

Search APIs

Method	Endpoint	Description
`GET`	`/v2/search?q={query}`	Search tweets
`GET`	`/v2/trends/{region}`	Get trending topics

Create Tweet Request

```
POST /v2/tweets
{
  "text": "System design is fun! #coding",
  "media_ids": ["media_abc123"],
  "reply_to": null,
  "quote_tweet_id": null,
  "poll": null
}
```

Create Tweet Response

```
{
  "id": "1234567890",
  "text": "System design is fun! #coding",
  "author_id": "user_42",
  "created_at": "2026-02-22T19:00:00Z",
  "metrics": {
    "likes": 0,
    "retweets": 0,
    "replies": 0,
    "views": 0
  },
  "entities": {
    "hashtags": [{ "tag": "coding", "start": 26, "end": 33 }]
  }
}
```

Home Timeline Response

```
GET /v2/timeline/home?cursor=cursor_xyz&count=20

{
  "tweets": [
    {
      "id": "1234567890",
      "text": "System design is fun! #coding",
      "author": { "id": "user_42", "name": "Jane", "handle": "@jane" },
      "created_at": "2026-02-22T19:00:00Z",
      "metrics": { "likes": 142, "retweets": 23, "replies": 5, "views": 8420 },
      "ranked_score": 0.87
    }
  ],
  "next_cursor": "cursor_abc",
  "has_more": true
}
```

Tweet Storage

Tweet Table Schema

```
┌──────────────────────────────────────────────────────────────┐
│                    TWEET STORAGE                             │
│                                                              │
│  PRIMARY STORE: Distributed SQL (Vitess / CockroachDB)       │
│                                                              │
```

```
┌──────────────────────────────────────────────────────────────────┐
│  tweets table:                                                   │
│  ├── tweet_id       BIGINT (Snowflake ID, time-sortable)         │
│  ├── author_id      BIGINT                                       │
│  ├── text           VARCHAR(280)                                 │
│  ├── media_urls     JSON                                         │
│  ├── reply_to_id    BIGINT (nullable)                            │
│  ├── quote_tweet_id BIGINT (nullable)                            │
│  ├── created_at     TIMESTAMP                                    │
│  ├── is_deleted     BOOLEAN                                      │
│  └── hashtags       TEXT[] (extracted at write time)             │
│                                                                  │
│  Sharding key: tweet_id (Snowflake = time-ordered)               │
│  Why: Most queries are by tweet_id or time range                 │
│                                                                  │
│  tweet_counts table (separate for high-write):                   │
│  ├── tweet_id       BIGINT                                       │
│  ├── like_count     INT                                          │
│  ├── retweet_count  INT                                          │
│  ├── reply_count    INT                                          │
│  └── view_count     BIGINT                                       │
│                                                                  │
│  WHY SEPARATE COUNTS?                                            │
│  - Likes/views update millions of times per second               │
│  - Core tweet data is write-once, read-many                      │
│  - Separate table avoids row-lock contention                     │
│  - Counts can be eventually consistent (it's OK to show          │
│    "142 likes" when the true count is 143)                       │
│                                                                  │
│  STORAGE ESTIMATE:                                               │
│  - 500M tweets/day × 700 bytes ≈ 350 GB/day                      │
│  - 5 years ≈ 640 TB of tweet data                                │
│  - Sharded across 100+ database nodes                            │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Snowflake ID (Tweet ID Generation)

```
┌──────────────────────────────────────────────────────────────────┐
│                    SNOWFLAKE ID                                  │
│                                                                  │
│  64-bit unique ID, time-sortable:                                │
│                                                                  │
│  ┌──────────┬──────────┬──────────┬──────────┐                   │
│  │  1 bit   │ 41 bits  │  10 bits │  12 bits │                   │
│  │ (unused) │ (millis  │ (machine │ (sequence│                   │
│  │          │  since   │   ID)    │  number) │                   │
│  │          │  epoch)  │          │          │                   │
│  └──────────┴──────────┴──────────┴──────────┘                   │
│                                                                  │
│  Properties:                                                     │
│  ├── Globally unique without coordination                        │
│  ├── Time-sortable (newer tweets = higher IDs)                   │
│  ├── 41 bits = ~69 years of milliseconds                         │
│  ├── 10 bits = 1,024 machines                                    │
```

```
│  ├── 12 bits = 4,096 IDs per machine per millisecond            │
│  └── Total: ~4M IDs/second/machine                              │
│                                                                  │
│  Used as: tweet_id, cursor for pagination ("tweets after X")    │
└──────────────────────────────────────────────────────────────────┘
```

The Fan-Out Problem (Core Challenge)

This is the **defining problem** of Twitter's architecture.

Fan-Out on Write (Push Model)

```
┌──────────────────────────────────────────────────────────────────┐
│                    FAN-OUT ON WRITE                              │
│                                                                  │
│  When user A posts a tweet:                                      │
│  1. Write tweet to tweets table                                  │
│  2. Look up A's followers: [B, C, D, ... N]                      │
│  3. For EACH follower, append tweet_id to their timeline cache   │
│                                                                  │
│  User A (1,000 followers) tweets "Hello world"                   │
│       │                                                          │
│       ▼                                                          │
│  Tweet Service → Write to tweets DB                              │
│       │                                                          │
│       ▼                                                          │
│  Fan-out Service → Get followers of A = [B, C, D, ...]           │
│       │                                                          │
│       ▼                                                          │
│  For each follower, append to their timeline cache:              │
│  ┌──────────────────────────────────────────┐                    │
│  │  Redis timeline cache (sorted set):      │                    │
│  │  user:B:timeline → [(tweet_id, score=timestamp), ...]        │
│  │  user:C:timeline → [(tweet_id, score=timestamp), ...]        │
│  │  user:D:timeline → [(tweet_id, score=timestamp), ...]        │
│  └──────────────────────────────────────────┘                    │
│                                                                  │
│  Timeline read = just fetch from Redis (O(1), < 10ms)            │
│                                                                  │
│  ✅ PROS:                                                        │
│  ├── Timeline reads are extremely fast (pre-computed in cache)   │
│  ├── Simple read path: GET user:X:timeline from Redis            │
│  └── Scales reads infinitely (most operations are reads)         │
│                                                                  │
│  ❌ CONS:                                                        │
│  ├── Write amplification: 1 tweet × N followers = N writes       │
│  ├── Celebrity problem: 1 tweet × 100M followers = 100M writes!  │
│  ├── High Redis memory: 300M users × 800 tweet IDs × 8 bytes     │
│  │   ≈ ~2 TB of just timeline cache                              │
```

```
│   └── Fan-out delay: 100M writes takes minutes even at 1M/s      │
│                                                                   │
└─────────────────────────────────────────────────────────────────┘
```

Fan-Out on Read (Pull Model)

```
┌─────────────────────────────────────────────────────────────────┐
│                    FAN-OUT ON READ                              │
│                                                                 │
│  When user B reads their timeline:                              │
│  1. Look up who B follows: [A, X, Y, Z, ...]                    │
│  2. Fetch recent tweets from each followed user                 │
│  3. Merge and sort by time (or ranking score)                   │
│  4. Return top N                                                │
│                                                                 │
│  User B opens app → GET /timeline/home                          │
│       │                                                         │
│       ▼                                                         │
│  Timeline Service → Get B's following list = [A, X, Y, Z, ...]  │
│       │                                                         │
│       ▼                                                         │
│  Fetch recent tweets from each (parallel):                      │
│  ├── tweets WHERE author_id = A ORDER BY created_at DESC LIMIT 20│
│  ├── tweets WHERE author_id = X ORDER BY created_at DESC LIMIT 20│
│  ├── tweets WHERE author_id = Y ORDER BY created_at DESC LIMIT 20│
│  └── ... (for all 200 followed users)                           │
│       │                                                         │
│       ▼                                                         │
│  Merge sort → Rank → Return top 20                              │
│                                                                 │
│  ✅ PROS:                                                       │
│  ├── No write amplification (tweet write = just 1 DB write)     │
│  ├── No celebrity problem (no fan-out at write time)            │
│  └── Less memory (no pre-computed timeline caches)              │
│                                                                 │
│  ❌ CONS:                                                       │
│  ├── Slow reads: 200 DB queries merged in real-time             │
│  ├── High read latency: N parallel queries + merge = slow       │
│  └── Doesn't scale: 35K timeline reads/sec × 200 queries each   │
│      = 7M DB queries/sec just for timelines                     │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Hybrid Approach (Twitter's Actual Solution) ⭐

```
┌─────────────────────────────────────────────────────────────────┐
│                    HYBRID FAN-OUT                               │
│                                                                 │
│  RULE: Use fan-out on write for normal users,                   │
│        fan-out on read for celebrities (>500K followers)        │
│                                                                 │
```

```
When NORMAL user tweets (< 500K followers):
└── Fan-out on write → push to all followers' timeline caches

When CELEBRITY tweets (> 500K followers):
└── DON'T fan out → just write to tweets DB

When user reads timeline:
┌──────────────────────────────────────────────────────┐
│  1. Fetch pre-computed timeline from Redis           │
│     (contains tweets from normal followed users)     │
│                                                      │
│  2. For each celebrity the user follows:             │
│     Fetch their recent tweets from DB (parallel)     │
│     (typically user follows < 20 celebrities)        │
│                                                      │
│  3. MERGE both sets, sort/rank, return top N         │
└──────────────────────────────────────────────────────┘

WHY THIS WORKS:
├── Normal users (99.9%): Fan-out on write is fine (< 500K)
├── Celebrities (0.1%): Skip fan-out, query at read time
├── Read merge: User follows maybe 5-20 celebrities
│   → only 5-20 extra DB queries per timeline read
├── Total read cost: Redis GET + 5-20 small DB queries
│   → still < 50ms
└── Eliminates the 100M-write-per-celebrity-tweet problem

CELEBRITY THRESHOLD:
├── Configurable (Twitter used ~30K-500K depending on era)
├── Can also be dynamic based on engagement rate
└── Some accounts are "super-celebrities" (>50M) — always pull
```

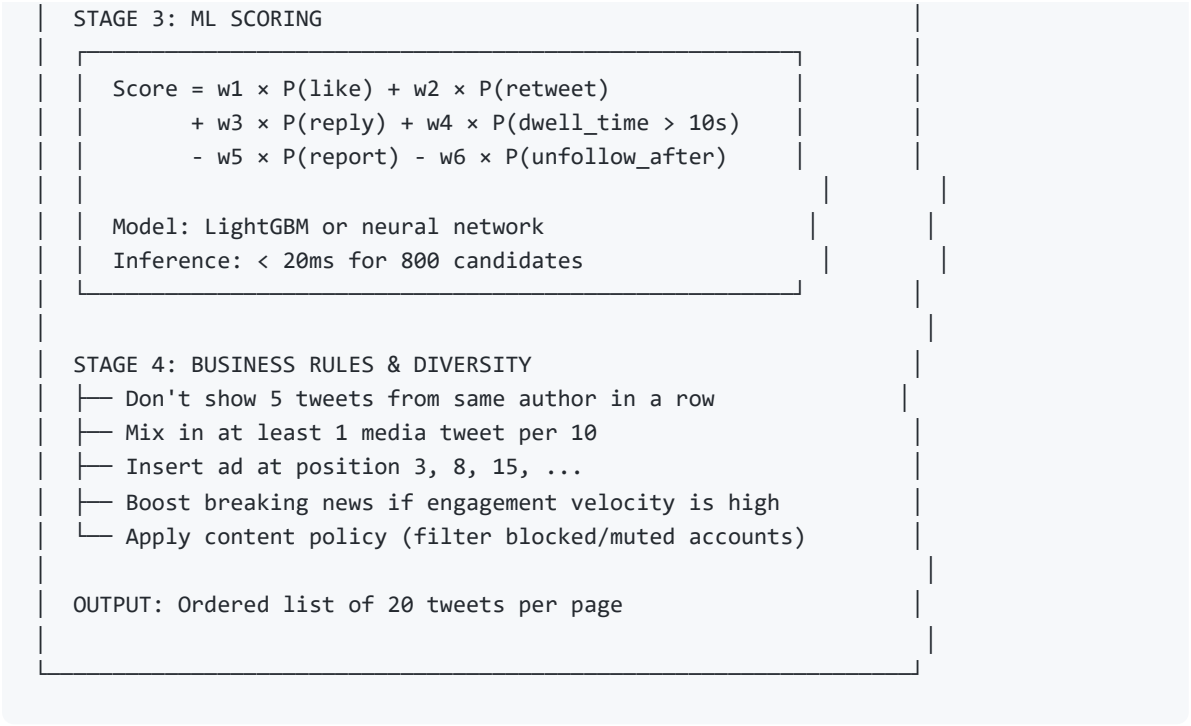
Fan-Out Service Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    FAN-OUT SERVICE                           │
│                                                              │
│  Tweet Published                                             │
│       │                                                      │
│       ▼                                                      │
│  ┌──────────┐                                                │
│  │  Kafka   │  Topic: tweet-published                        │
│  │  Queue   │  Partitioned by author_id                      │
│  └────┬─────┘                                                │
│       │                                                      │
│       ▼                                                      │
│  ┌──────────────────────────────────────────────────┐        │
│  │  Fan-out Worker Pool (100+ workers)              │        │
│  │                                                  │        │
│  │  For each tweet:                                 │        │
│  │  1. Check author follower count                  │        │
│  │     ├── > threshold → Skip fan-out (celebrity)   │        │
```

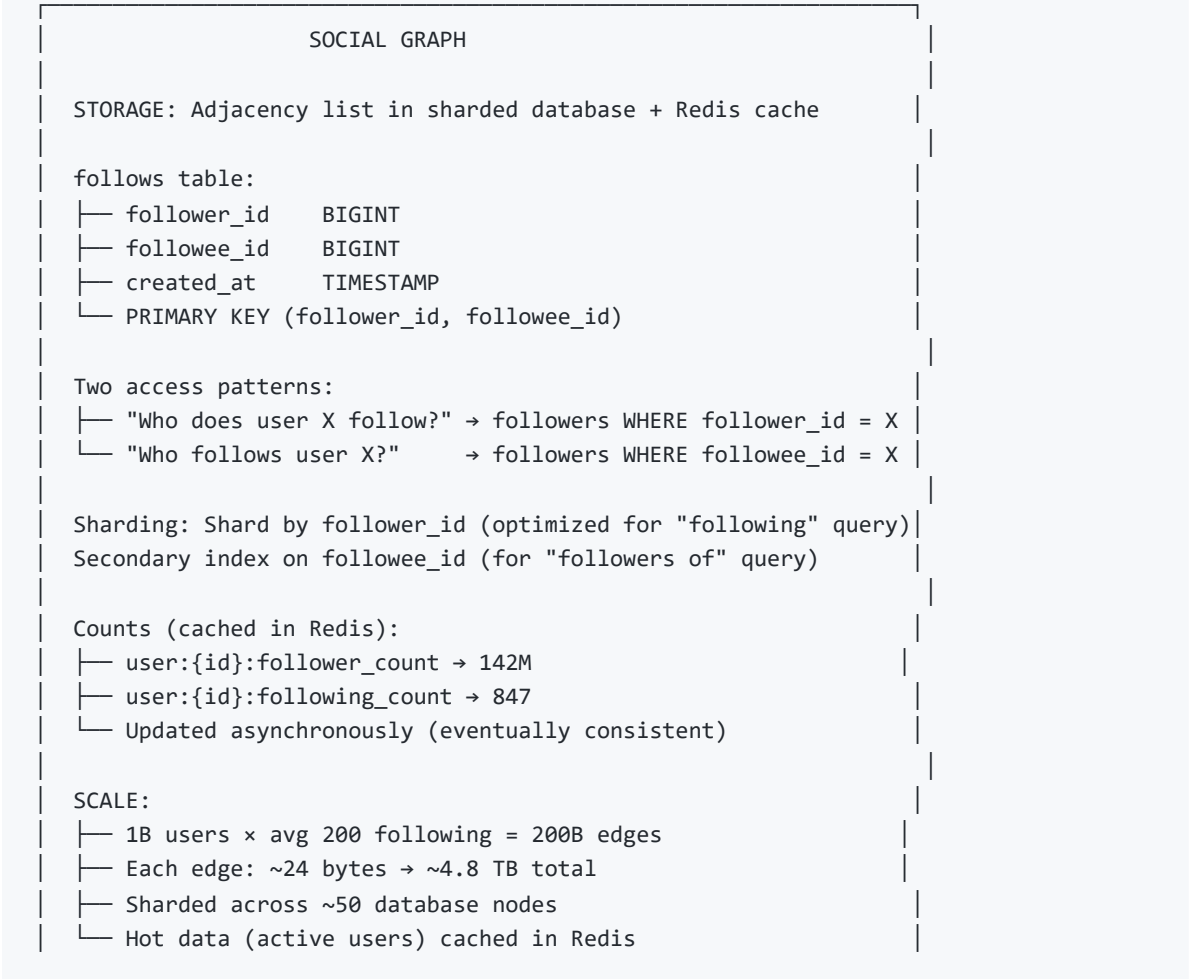
```
│  │     └── ≤ threshold → Proceed with fan-out                │         │
│  │                                                           │         │
│  │  2. Page through follower list (1,000 at a time)          │         │
│  │                                                           │         │
│  │  3. For each batch of followers:                          │         │
│  │     Pipeline Redis commands:                              │         │
│  │     ZADD user:{followerId}:timeline {timestamp}           │         │
│  │           {tweet_id}                                      │         │
│  │     ZREMRANGEBYRANK user:{followerId}:timeline            │         │
│  │           0 -(MAX_TIMELINE_SIZE+1)   // trim to 800       │         │
│  │                                                           │         │
│  │  Redis pipelining: ~100K ZADD/s per connection            │         │
│  │  With 100 workers: ~10M fan-out writes/second             │         │
│  │                                                           │         │
│  └───────────────────────────────────────────────────────────┘         │
│                                                                        │
│  TIMELINE CACHE (Redis Cluster):                                       │
│  ├── Data structure: Sorted Set per user                               │
│  ├── Score: tweet timestamp (for chronological ordering)               │
│  ├── Member: tweet_id (8 bytes)                                        │
│  ├── Max size per user: 800 entries (trim older)                       │
│  ├── Memory per user: ~800 × 16 bytes ≈ 12.5 KB                        │
│  ├── Total: 300M active users × 12.5 KB ≈ 3.5 TB                       │
│  └── Redis cluster: ~50 nodes (64 GB each)                             │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Timeline Ranking (Algorithmic Feed)

```
┌────────────────────────────────────────────────────────────────────────┐
│                    TIMELINE RANKING                                    │
│                                                                        │
│  Chronological feed is simple but low-engagement.                      │
│  Ranked feed shows "best" tweets first.                                │
│                                                                        │
│  INPUT: ~800 candidate tweet IDs from timeline cache + celeb DB        │
│                                                                        │
│  STAGE 1: CANDIDATE RETRIEVAL                                          │
│  ├── From Redis timeline cache (pre-computed)                          │
│  ├── From celebrity DB queries (fan-out on read)                       │
│  ├── Inject: Promoted tweets (ads)                                     │
│  └── Inject: "You might like" (recommendation model)                   │
│                                                                        │
│  STAGE 2: FEATURE EXTRACTION                                           │
│  For each candidate tweet, compute features:                           │
│  ├── Tweet features: age, media type, length, hashtags                 │
│  ├── Author features: follower count, engagement rate, verified        │
│  ├── User-author affinity: past interactions, mutual followers         │
│  ├── Engagement signals: like count, retweet count, reply count        │
│  └── Content signals: topic match with user interests                  │
│                                                                        │
```



Social Graph Service



```
│                                                                  │
│  CELEBRITY DETECTION:                                            │
│  └── Maintain a celebrity list (updated hourly):                 │
│      SELECT followee_id FROM follows                             │
│      GROUP BY followee_id                                        │
│      HAVING COUNT(*) > 500000                                    │
│      → Materialized view, used by fan-out service                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Search & Trending

```
┌──────────────────────────────────────────────────────────────────┐
│                    SEARCH SERVICE                                │
│                                                                  │
│  INDEXING PIPELINE:                                              │
│  Tweet published → Kafka → Search Indexer → Elasticsearch        │
│                                                                  │
│  INDEX SCHEMA:                                                   │
│  ├── tweet_id (keyword)                                          │
│  ├── text (full-text, analyzed with language-specific tokenizer) │
│  ├── author_id, author_handle (keyword)                          │
│  ├── hashtags (keyword array)                                    │
│  ├── created_at (date)                                           │
│  ├── lang (keyword)                                              │
│  ├── engagement_score (float, for ranking)                       │
│  └── geo (geo_point, if geotagged)                               │
│                                                                  │
│  SEARCH RANKING:                                                 │
│  score = text_relevance × recency_boost × engagement_score       │
│          × author_authority                                      │
│                                                                  │
│  SCALE:                                                          │
│  ├── 500M tweets/day indexed within seconds of posting           │
│  ├── Index size: ~100 TB (searchable window = 7-30 days)         │
│  ├── Elasticsearch cluster: ~100+ nodes                          │
│  └── Older tweets archived, searchable via cold storage query    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│                    TRENDING TOPICS                               │
│                                                                  │
│  PIPELINE:                                                       │
│  Tweets → Extract hashtags/keywords → Sliding window counter     │
│                                                                  │
│  ALGORITHM:                                                      │
│  ├── Count hashtag frequency in 5-min tumbling windows           │
│  ├── Compare current window to historical baseline               │
│  ├── Trending = significant spike over baseline                  │
│  │   (not just high volume — spike matters more)                 │
```

```
│  ├── Filter: Remove always-popular terms (#news, #love)        │
│  └── Regional: Compute per country/city using geo data         │
│                                                                 │
│  IMPLEMENTATION:                                                │
│  ├── Stream processor (Kafka Streams / Flink)                  │
│  ├── Count-min sketch for approximate frequency counting       │
│  ├── Redis sorted sets for top-K per region                    │
│  └── Update trending list every 1-5 minutes                    │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Notification Service

```
┌─────────────────────────────────────────────────────────────────┐
│                    NOTIFICATION SERVICE                         │
│                                                                 │
│  TRIGGERS:                                                      │
│  ├── Someone likes your tweet                                  │
│  ├── Someone retweets your tweet                               │
│  ├── Someone replies to your tweet                             │
│  ├── Someone mentions you (@handle)                            │
│  ├── New follower                                              │
│  └── DM received                                               │
│                                                                 │
│  PIPELINE:                                                      │
│  Event (like, retweet, ...) → Kafka → Notification Worker      │
│       │                                                         │
│       ▼                                                         │
│  ┌──────────────────────────────────────────────────┐          │
│  │  Notification Worker:                            │          │
│  │  1. Check user preferences (push enabled? muted?)│          │
│  │  2. Aggregate if burst (100 likes → "100 people  │          │
│  │     liked your tweet" not 100 separate notifs)   │          │
│  │  3. Determine delivery channel:                  │          │
│  │     ├── In-app (always, stored in notification DB)│          │
│  │     ├── Push (APNs / FCM if user has push enabled)│          │
│  │     └── Email (daily digest for inactive users)  │          │
│  │  4. Write to notification store + send push      │          │
│  └──────────────────────────────────────────────────┘          │
│                                                                 │
│  AGGREGATION:                                                   │
│  ├── Window: 5 minutes                                         │
│  ├── "Jane and 42 others liked your tweet"                     │
│  └── Prevents notification bombing on viral tweets             │
│                                                                 │
│  CELEBRITY NOTIFICATIONS:                                       │
│  ├── If tweet gets 1M likes, DON'T send 1M push notifs         │
│  ├── Aggregate: "Your tweet reached 1M likes!"                 │
│  └── Mentions: Only notify for mentions by followed users      │
│     (to prevent @spam)                                         │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Technology	Purpose
Tweet Store	Vitess (sharded MySQL) / CockroachDB	Persistent tweet storage, sharded by tweet_id
Timeline Cache	Redis Cluster (sorted sets)	Pre-computed timelines, O(1) reads
Social Graph	Sharded MySQL + Redis	Follow relationships, follower/following queries
Fan-out Service	Kafka + worker pool	Distribute tweets to followers' timeline caches
Search Index	Elasticsearch / Lucene	Full-text tweet search, hashtag search
Trending Engine	Kafka Streams / Flink + Count-min sketch	Real-time trending topic detection
Ranking Service	LightGBM / neural network	Algorithmic feed scoring
Media Storage	S3/Blob Storage + CDN	Image/video storage and global delivery
Notification Service	Kafka + APNs/FCM	Push notifications, in-app notifications
ID Generator	Snowflake	Globally unique, time-sortable tweet IDs
Rate Limiter	Redis (token bucket)	Per-user API rate limiting
CDN	CloudFront / Akamai	Static assets, media, JS/CSS bundles
Metrics	Prometheus + Grafana	System monitoring, SLO tracking

Scalability Considerations

1. Timeline Cache Scaling

```
Challenge: 300M users × 12.5 KB per timeline = 3.5 TB in Redis

Solutions:
├── Only cache ACTIVE users (logged in within 7 days)
│   → reduces to ~100M users → ~1.2 TB
├── Evict inactive timelines (rebuild on next login from DB)
├── Redis Cluster: 50 nodes, 64 GB each, hash-slot sharding
├── Hot/cold: Popular users cached longer, LRU eviction
└── Trim timelines to 800 entries (older = query from DB)
```

2. Fan-Out Throughput

```
Challenge: 500M tweets/day with avg 200 followers = 100B cache writes/day

Solutions:
├── Kafka partitioning: Partition tweet events by author_id
├── Worker pool: 100+ fan-out workers, each handles 1M writes/s
├── Redis pipelining: Batch 100 ZADD commands per round-trip
├── Skip celebrities: Eliminate 100M+ write spikes entirely
└── Priority queue: Fan-out high-engagement tweets first
```

3. Database Sharding

```
┌──────────────────────────────────────────────────────────────┐
│                    SHARDING STRATEGY                         │
│                                                              │
│  Tweets: Shard by tweet_id (Snowflake ID, time-based)        │
│  ├── Recent tweets on fewer, faster shards                   │
│  ├── Old tweets on many, cheaper shards                      │
│  └── Hot shard mitigation: Snowflake has machine_id bits     │
│                                                              │
│  Social graph: Shard by follower_id                          │
│  ├── "Who do I follow?" = single-shard query                 │
│  ├── "Who follows me?" = scatter-gather (cross-shard)        │
│  └── Cache follower lists in Redis to avoid scatter-gather   │
│                                                              │
│  User profiles: Shard by user_id                             │
│  └── Simple key-value, mostly cached                         │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

4. Handling Viral Tweets

```
Problem: A viral tweet gets 1M likes/min → extreme write load on tweet_counts row

Solutions:
├── Counter sharding: Split like_count across 100 sub-counters
│   like_count = SUM(like_count_shard_0 ... like_count_shard_99)
```

```
│   Each LIKE increments a random shard → no single hot row
├── Async aggregation: Aggregate sharded counters every 5 seconds
├── Redis for real-time display: Atomic INCR in Redis, flush to DB async
└── View counts: Sample (count every 10th view, multiply by 10)
    → 10x less write load, acceptable accuracy
```

Trade-offs

Decision	Option A	Option B	Twitter's Choice
Fan-out strategy	Write (push to cache)	Read (query at read time)	**Hybrid** — push for normal users, pull for celebrities
Timeline storage	Database	In-memory cache (Redis)	**Redis** — pre-computed timelines for sub-10ms reads
Feed ordering	Chronological	Algorithmic (ranked)	**Both** — user can toggle; default is ranked
Tweet ID	UUID	Snowflake (time-sortable)	**Snowflake** — time-sortable enables cursor-based pagination
Engagement counts	Strong consistency	Eventual consistency	**Eventual** — "142 likes" vs "143 likes" doesn't matter
Search index	Index all tweets forever	Rolling window (7-30 days)	**Rolling window** — index recent tweets, archive old ones
Celebrity threshold	Fixed (500K followers)	Dynamic (based on engagement)	**Dynamic** — adjust based on fan-out capacity and tweet frequency
Media serving	Direct from storage	Via CDN	**CDN** — essential for global low-latency media delivery

---

Technologies Deep Dive

Data Layer

Technology	Purpose	Why
Vitess	Sharded MySQL for tweets + social graph	Horizontal sharding, connection pooling, used by Twitter-scale companies
Redis Cluster	Timeline cache, counters, rate limiting	Sub-millisecond reads, sorted sets are perfect for timelines
Elasticsearch	Tweet search + trending	Full-text search, real-time indexing, distributed
Kafka	Event streaming (fan-out, search indexing, notifications)	High throughput, partitioned, replay capability

Compute

Technology	Purpose	Why
Kafka Streams / Flink	Trending topics, real-time analytics	Windowed aggregations, exactly-once semantics
gRPC	Inter-service communication	Low-latency, protobuf serialization, streaming
LightGBM	Timeline ranking model	Fast inference (<20ms for 800 candidates)

Infrastructure

Technology	Purpose	Why
CDN (CloudFront/Akamai)	Media + static content delivery	Global edge caching, essential for images/video
Snowflake ID generator	Unique tweet IDs	Time-sortable, no coordination, ~4M IDs/sec/machine
Kubernetes	Service orchestration	Auto-scaling, rolling deploys, service mesh
APNs + FCM	Push notifications	iOS + Android native push channels

---

How to Explain It Well

30-Second Pitch

> "Twitter's core challenge is the fan-out problem: when a user tweets, how do you distribute it to all their followers' timelines efficiently? The solution is a hybrid approach — fan-out on write for normal users (push tweet IDs into followers' Redis timeline caches at write time) and fan-out on read for celebrities with millions of followers (query their tweets at read time and merge). Timeline reads are sub-10ms from Redis sorted sets. Tweet IDs use Snowflake for time-sortable, globally unique IDs. Search is handled by Elasticsearch with a 7-30 day rolling index window. The key numbers: 500M tweets/day, 300M DAU, 35K timeline reads/second."

Key Talking Points

1. **Fan-out problem is THE question**: Be ready to discuss push vs pull vs hybrid in depth
2. **Snowflake ID**: Time-sortable, enables cursor-based pagination, no coordination needed
3. **Redis sorted sets**: Perfect data structure for pre-computed timelines (ZADD, ZRANGE, ZREMRANGEBYRANK)
4. **Celebrity threshold**: Dynamic — too low and you lose cache benefits; too high and you get write storms
5. **Engagement counters**: Eventually consistent, sharded counters to avoid hot rows on viral tweets

Whiteboard Sequence

1. Start with tweet write → fan-out → timeline cache (the core flow)
2. Show the celebrity problem: 100M writes from 1 tweet
3. Introduce hybrid: push for normal, pull for celebrities, merge at read
4. Add search pipeline: Kafka → Elasticsearch
5. Discuss Snowflake ID and Redis sorted sets as enabling data structures

---

Common Follow-up Questions

"Why not just use fan-out on read for everyone?"

- A user follows ~200 accounts → 200 parallel DB queries per timeline read
- At 35K timeline reads/sec → 7M DB queries/sec just for timelines
- Even with caching, the merge-sort step adds latency
- Pre-computed timelines in Redis give us **< 10ms reads** vs **200-500ms** with fan-out on read
- But for celebrities, the **write cost** of fan-out on write is worse than the **read cost** of fan-out on read

"How do you handle a user with 100M followers posting 50 tweets/day?"

- **Don't fan out**: Celebrity tweets stay in the tweets DB only
- At **read time**, fetch the celebrity's recent tweets (one indexed query by author_id)
- Most users follow < 20 celebrities → 20 small DB queries merged with Redis timeline
- The queries are fast because tweets are indexed by `(author_id, created_at DESC)`
- This is the entire point of the hybrid model

"How do you handle the thundering herd when a celebrity tweets?"

- **No thundering herd** — because celebrity tweets aren't fanned out
- Followers' next timeline reads naturally pick up the tweet via fan-out on read
- The celebrity's tweet row may get hot for reads → **cache it** in a tweet object cache (Redis/Memcached)
- Celebrity tweet cache TTL: 5 minutes (short enough to stay fresh)

"How does cursor-based pagination work with Snowflake IDs?"

- Snowflake IDs are **time-ordered**: higher ID = newer tweet
- First page: `ZREVRANGE user:B:timeline 0 19` → returns 20 newest tweet IDs
- Last tweet ID on page = `cursor` for next page
- Next page: `ZREVRANGEBYSCORE user:B:timeline (cursor -inf LIMIT 0 20`
- No offset-based pagination → consistent results even as new tweets arrive

"How do you implement 'For You' recommendations?"

- Beyond the follow graph: recommend tweets from accounts the user **doesn't** follow
- **Collaborative filtering**: "Users similar to you also engaged with these tweets"
- **Content-based**: Match tweet topics to user's interest profile
- **Social signals**: "Your friends liked this tweet" (second-degree engagement)
- These are injected as candidates into the ranking pipeline alongside the follow-based timeline
- Typically 20-40% of the ranked feed is from non-followed accounts

---

54. Design a Web Search Engine (Google / Bing)

Problem Statement

Design a web-scale search engine like Google or Bing that crawls billions of web pages, builds a searchable index, and returns highly relevant results in milliseconds. The system must handle tens of billions of indexed pages, 8.5 billion searches per day, support autocomplete, spell correction, knowledge panels, ads, and personalized ranking — all with sub-second latency worldwide.

Clarifying Questions to Ask

1. **Scale**: How many pages to index? (100B+ pages in the known web)
2. **Query volume**: How many searches per day? (8.5B+ for Google)
3. **Freshness**: How quickly should new/changed pages appear in results? (minutes for news, days for evergreen)
4. **Features**: Just web results, or also images, videos, news, maps, knowledge graph, ads?
5. **Autocomplete**: Provide query suggestions as the user types?
6. **Spell correction**: "Did you mean...?" for typos?
7. **Personalization**: Personalized results based on search history, location, language?
8. **Ads**: Auction-based sponsored results alongside organic results?
9. **Latency**: Target for end-to-end search result page? (< 200ms)
10. **International**: Multi-language support? Different ranking for different regions?

Design Reasoning

> **Core philosophy**: Decouple expensive offline computation (crawling, PageRank, ML training) from latency-critical online serving — achieve sub-200ms query latency across 100 billion pages through document-partitioned parallel fan-out, tiered indexing, and multi-stage ranking that dynamically trades freshness, relevance, and speed.

Decision	Choice	Why	Alternative Considered
Index sharding	Document-partitioned across 1,000+ shards	Multi-term queries resolved locally within each shard; embarrassingly parallel fan-out; each shard is a self-contained mini-search engine	Term-partitioned — multi-term queries need cross-shard joins, creating massive network latency at 100K+ QPS

Index implementation	Custom inverted index with delta-encoded posting lists (4-8× compression)	No off-the-shelf engine handles 100B docs at petabyte scale with sub-10ms latency; enables optimal SSD layout	Elasticsearch/Lucene — can't meet scale and latency requirements for 100B+ documents
Ranking pipeline	Three-stage: BM25 (recall) → LambdaMART (precision on top 1K) → Neural/BERT (semantics on top 50)	BM25 is microseconds/doc; LambdaMART is 100× faster than neural; neural provides deep semantic understanding on final candidates	BM25 alone — insufficient quality; all-neural — too slow for 100K QPS
Indexing architecture	Dual-path: batch (1-30 day recrawl) + real-time stream (RSS/social, 2-5 min freshness)	Batch is efficient for 10B pages/day; real-time ensures breaking news in results within minutes via Tier 0 segments	Batch-only — misses breaking news; all real-time — wastes resources crawling low-priority content
Index tiering	Tier 0 (in-memory, hours) → Tier 1 (SSD, top 10B pages) → Tier 2 (all 100B pages)	90%+ queries satisfied by Tier 1 alone; reduces working set and improves cache hits; Tier 2 for long-tail coverage	Single unified index — much slower; all queries would traverse 100B docs instead of high-quality 10B subset
Authority signal	PageRank via graph-parallel (Pregel/GraphX) on 4 trillion edges, weekly full + daily incremental	Foundational ranking signal; graph-parallel distributes computation across thousands of machines	Centralized computation — impossible at 4T edges; MapReduce alone — billions of shuffle operations
Web crawling	10K+ distributed crawlers with priority + per-host politeness	Two-level queues: global priority by freshness tier + per-host FIFO respecting	Aggressive crawling without rate limits — gets blocked by target websites

	queues (1 req/sec/domain)	robots.txt; achieves 10B pages/day	
Result caching	Redis for head queries (top 10K = ~30% traffic) + autocomplete trie + rate limiting	Sub-millisecond access for hot queries avoids entire index lookup; atomic ops for ad budget tracking	Memcached — lacks persistence for budget tracking; CDN caching — misses personalization

---

Requirements & Sizing

Functional

- **Web Search**: User types a query, gets ranked list of web page results (title, snippet, URL)
- **Crawling**: Continuously discover and fetch web pages across the internet
- **Indexing**: Build and maintain an inverted index over billions of documents
- **Ranking**: Return the most relevant results using hundreds of ranking signals
- **Autocomplete**: Query suggestions as the user types (typeahead)
- **Spell Correction**: "Did you mean..." for misspelled queries
- **Knowledge Panel**: Structured answers for entity queries (people, places, facts)
- **Ads**: Sponsored results via real-time ad auction
- **Snippets**: Generate highlighted text excerpts matching the query
- **Freshness**: Near-real-time indexing for breaking news

Non-functional

Metric	Target
Search latency	< **200ms** p99 end-to-end
Crawl coverage	**100B+ pages** indexed
Crawl freshness	**Minutes** for news, **days** for long-tail pages
Availability	**99.99%** (four 9s)
Query volume	**100K+ queries/second** (8.5B/day)
Index size	**100+ PB** (inverted index + document store)
Autocomplete latency	< **50ms**

Capacity Estimates

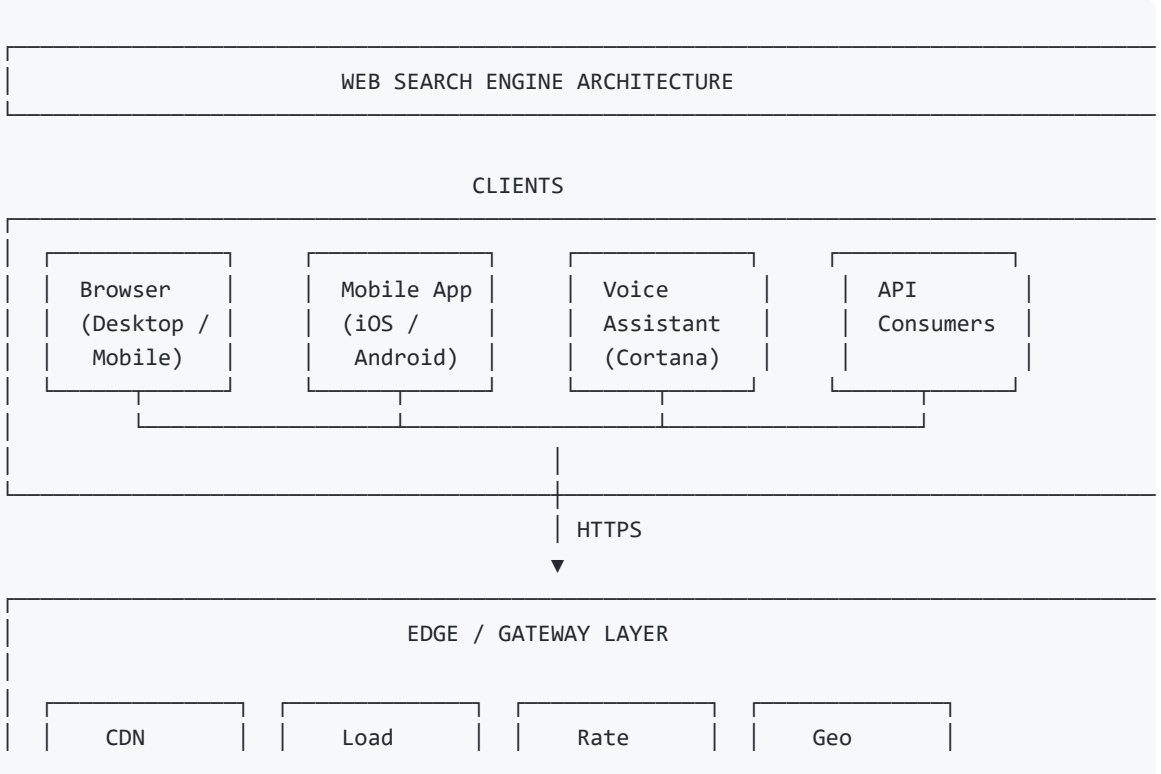
```
Queries:
- 8.5B queries/day ≈ 100K queries/second (average)
- Peak: 300K+ queries/second (during major events)
- Average query: 3-4 words, ~25 bytes
- Query log storage: 8.5B × 100 bytes ≈ 850 GB/day

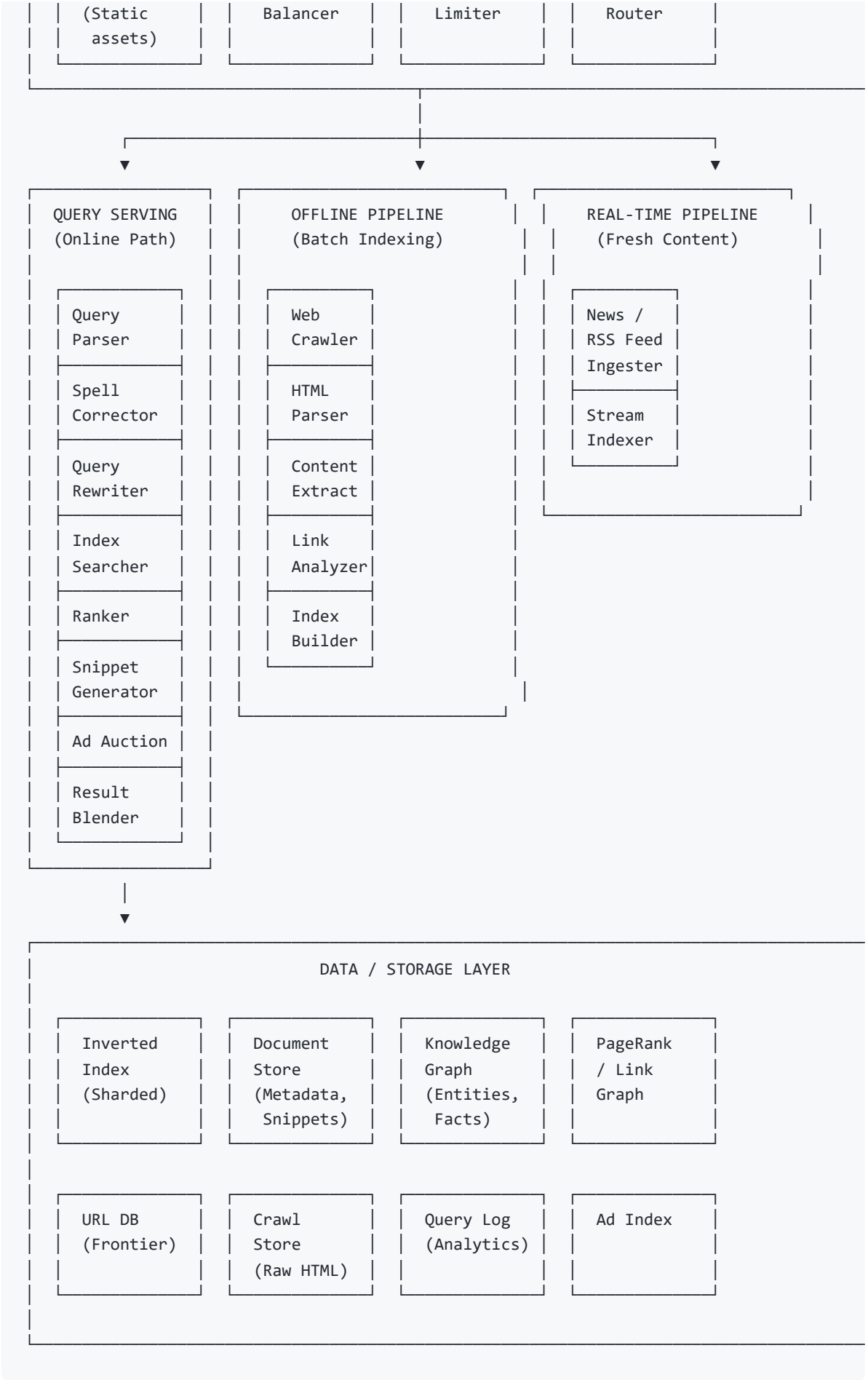
Index:
- 100B web pages indexed
- Average page: 50 KB raw HTML → 10 KB extracted text
- Raw content: 100B × 10 KB = 1 EB (exabyte)
- Inverted index: ~100-200 PB (compressed posting lists)
- Document metadata store: ~50 PB

Crawling:
- Recrawl cycle: Top 1B pages every day, long-tail every 30 days
- Crawl rate: ~10B pages/day ≈ 120K pages/second
- Bandwidth: 120K × 50 KB = 6 GB/second inbound crawl traffic
- robots.txt cache: ~500M domains × 2 KB ≈ 1 TB

Serving:
- Each query fans out to 1,000+ index shards in parallel
- Each shard returns top ~100 candidates in < 10ms
- Merger combines and re-ranks → final top 10 in < 50ms
- Total with network + rendering: < 200ms
```

High-Level Design




(Static assets)
Balancer
Limiter
Router
QUERY SERVING (Online Path)
Query Parser
Spell Corrector
Query Rewriter
Index Searcher
Ranker
Snippet Generator
Ad Auction
Result Blender
OFFLINE PIPELINE (Batch Indexing)
Web Crawler
HTML Parser
Content Extract
Link Analyzer
Index Builder
REAL-TIME PIPELINE (Fresh Content)
News / RSS Feed Ingester
Stream Indexer
DATA / STORAGE LAYER
Inverted Index (Sharded)
Document Store (Metadata, Snippets)
Knowledge Graph (Entities, Facts)
PageRank / Link Graph
URL DB (Frontier)
Crawl Store (Raw HTML)
Query Log (Analytics)
Ad Index

API Design

Search APIs

Method	Endpoint	Description
`GET`	`/v1/search?q={query}&start={offset}&count={n}`	Web search (paginated)
`GET`	`/v1/suggest?q={prefix}&lang={lang}`	Autocomplete suggestions
`GET`	`/v1/images?q={query}`	Image search
`GET`	`/v1/news?q={query}`	News search
`GET`	`/v1/knowledge?entity={id}`	Knowledge panel for entity
`GET`	`/v1/spellcheck?q={query}`	Spell correction suggestion

Search Request

```
GET /v1/search?q=distributed+systems+interview&start=0&count=10&lang=en&region=us

Headers:
  Authorization: Bearer <api_key>
  X-Request-ID: req_abc123
  Accept-Language: en-US
```

Search Response

```
{
  "query": "distributed systems interview",
  "corrected_query": null,
  "results": {
    "organic": [
      {
        "position": 1,
        "url": "https://example.com/distributed-systems-guide",
        "title": "Complete Guide to Distributed Systems Interviews",
        "snippet": "Master <b>distributed systems</b> concepts for your next <b>inte
        "domain": "example.com",
```

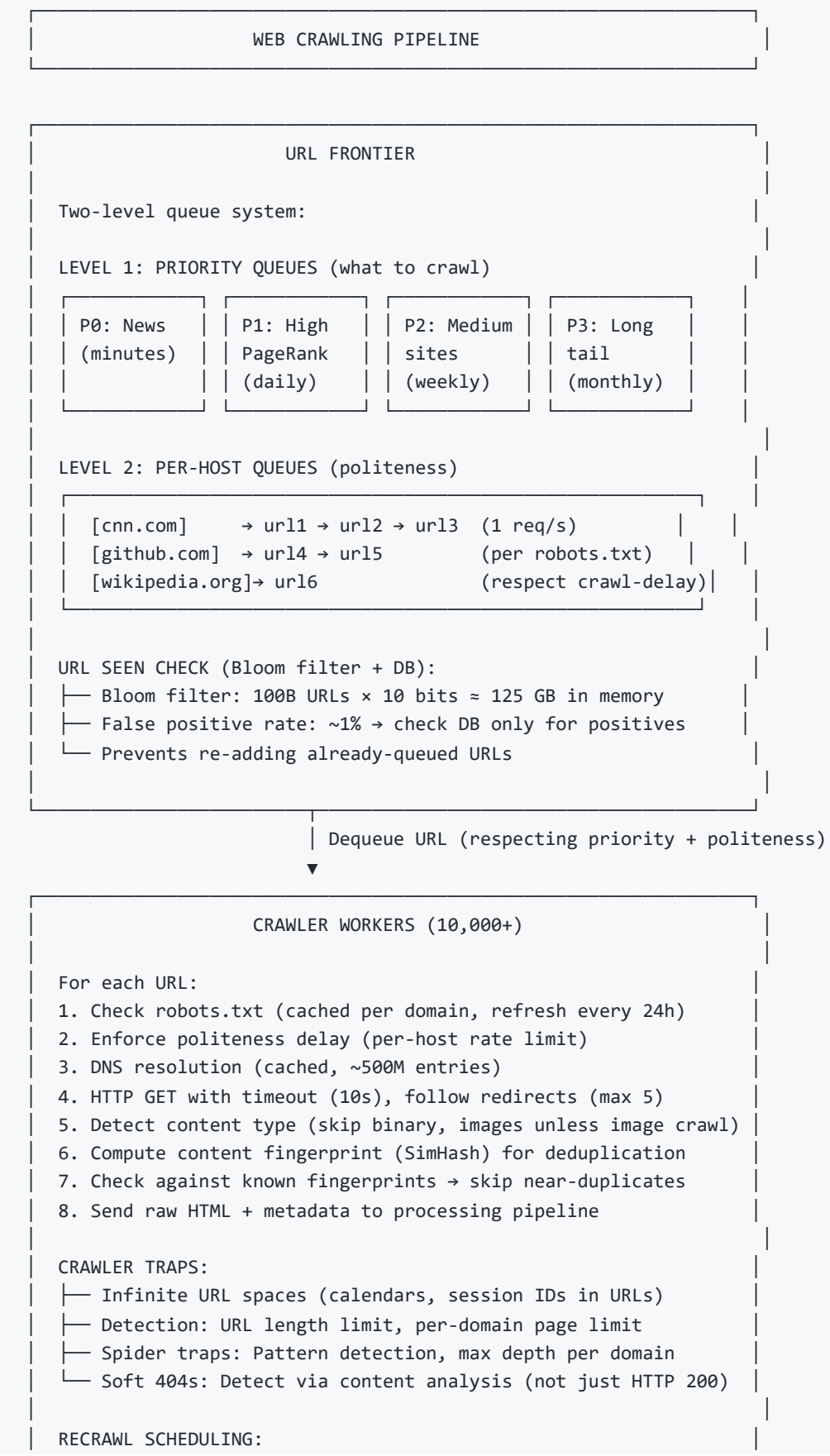
```
        "favicon": "https://example.com/favicon.ico",
        "last_crawled": "2026-03-01T12:00:00Z",
        "sitelinks": [
          { "title": "CAP Theorem", "url": "https://example.com/cap" },
          { "title": "Consistent Hashing", "url": "https://example.com/hashing" }
        ]
      }
    ],
    "knowledge_panel": {
      "title": "Distributed Systems",
      "description": "A distributed system is a system whose components are located
      "source": "Wikipedia",
      "related_topics": ["CAP Theorem", "Consensus Algorithms", "MapReduce"]
    },
    "ads": [
      {
        "position": 0,
        "url": "https://sponsor.com/course",
        "title": "Learn Distributed Systems — Online Course",
        "snippet": "Ace your system design interview. 50,000+ students enrolled.",
        "label": "Sponsored"
      }
    ],
    "related_searches": [
      "system design interview questions",
      "distributed systems book recommendations",
      "CAP theorem explained"
    ]
  },
  "metadata": {
    "total_results": 142000000,
    "search_time_ms": 187,
    "next_cursor": "start=10"
  }
}
```

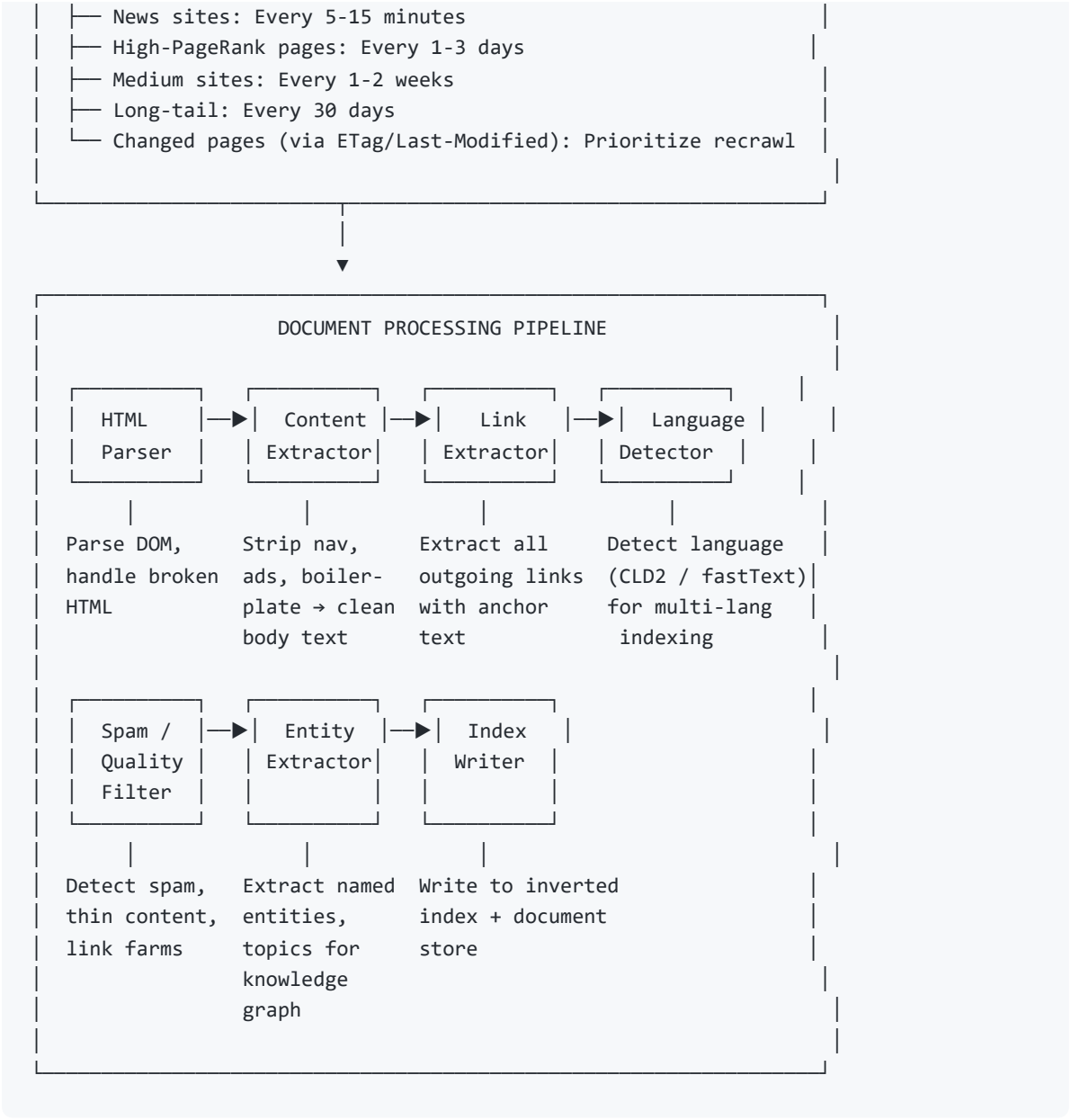

Autocomplete Response


```
GET /v1/suggest?q=distrib&lang=en

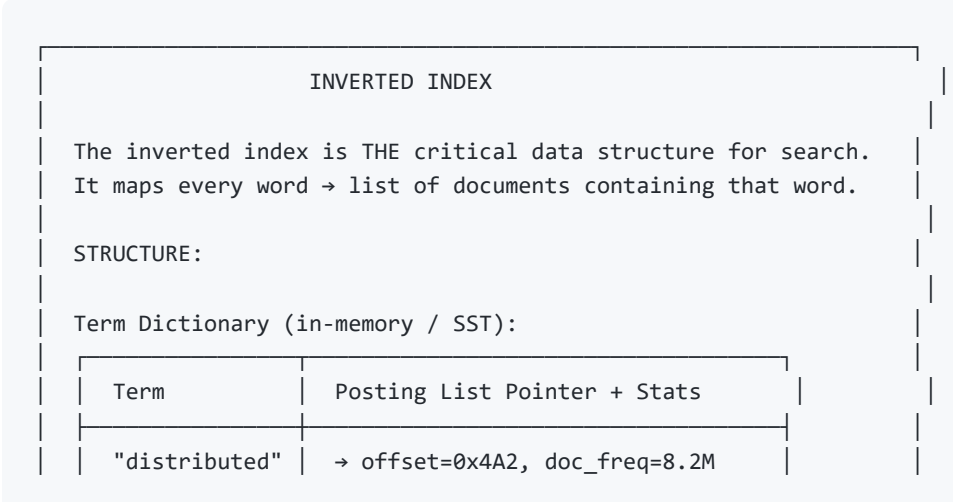
{
  "prefix": "distrib",
  "suggestions": [
    { "text": "distributed systems", "score": 0.95 },
    { "text": "distributed computing", "score": 0.82 },
    { "text": "distributed database", "score": 0.76 },
    { "text": "distributed systems interview questions", "score": 0.71 }
  ],
  "latency_ms": 12
}
```

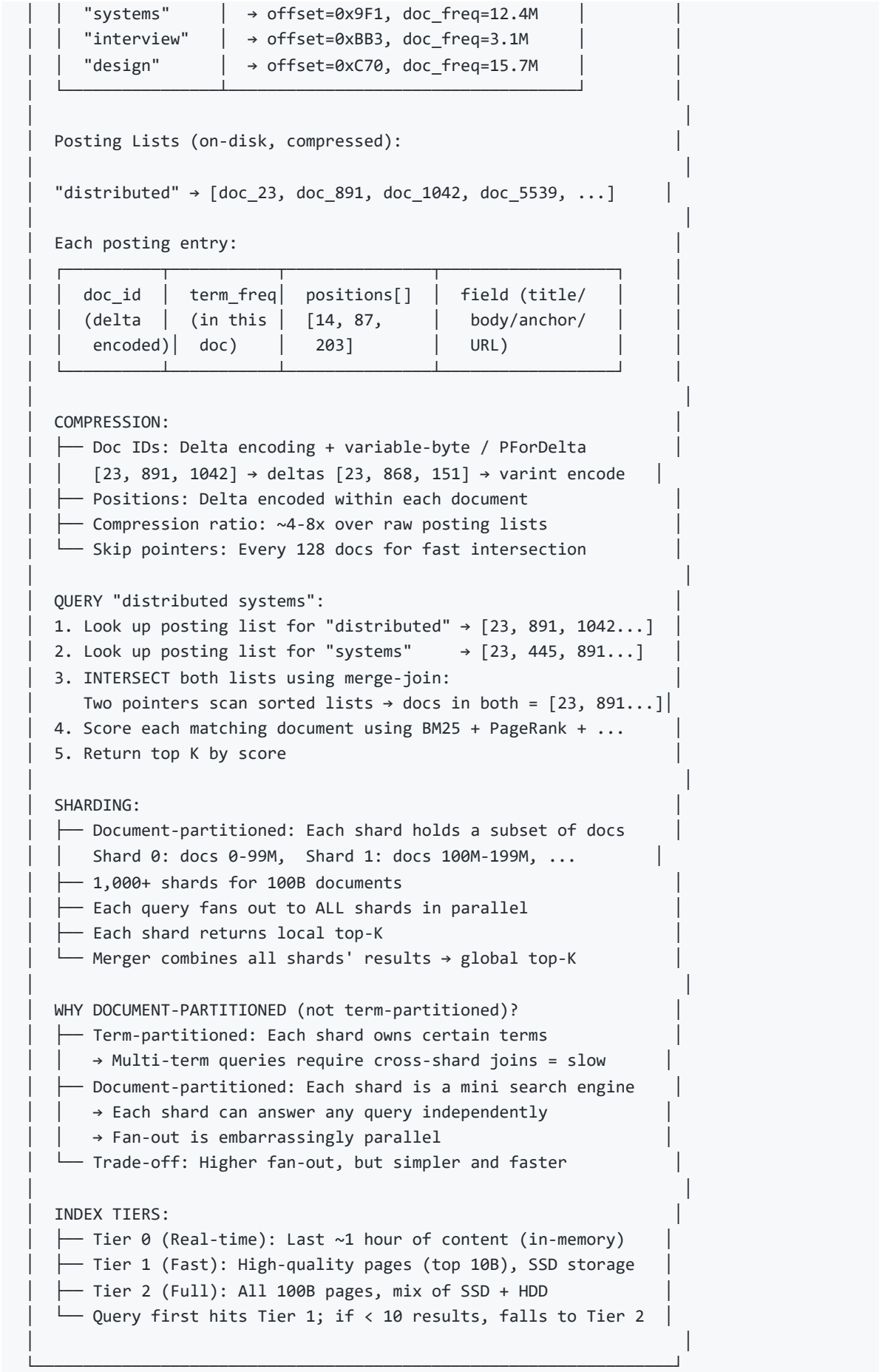
Web Crawling Pipeline





Inverted Index (Core Data Structure)



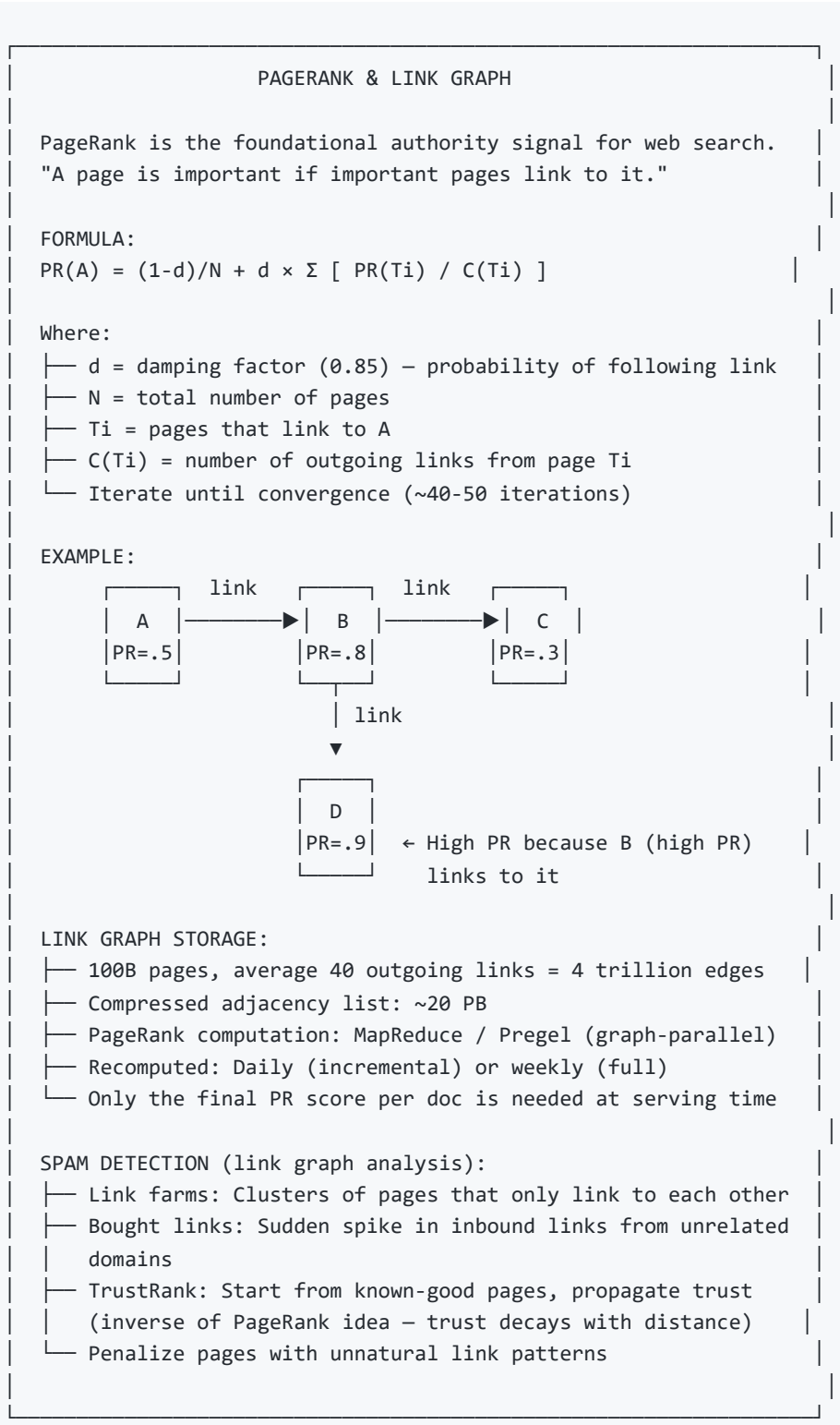

"systems" → offset=0x9F1, doc_freq=12.4M
"interview" → offset=0xBB3, doc_freq=3.1M
"design" → offset=0xC70, doc_freq=15.7M
Posting Lists (on-disk, compressed):
"distributed" → [doc_23, doc_891, doc_1042, doc_5539, ...]
Each posting entry:
doc_id (delta encoded)
term_freq (in this doc)
positions[] [14, 87, 203]
field (title/ body/anchor/ URL)
COMPRESSION:
├── Doc IDs: Delta encoding + variable-byte / PForDelta
[23, 891, 1042] → deltas [23, 868, 151] → varint encode
├── Positions: Delta encoded within each document
├── Compression ratio: ~4-8x over raw posting lists
└── Skip pointers: Every 128 docs for fast intersection
QUERY "distributed systems":
1. Look up posting list for "distributed" → [23, 891, 1042...]
2. Look up posting list for "systems" → [23, 445, 891...]
3. INTERSECT both lists using merge-join:
Two pointers scan sorted lists → docs in both = [23, 891...]
4. Score each matching document using BM25 + PageRank + ...
5. Return top K by score
SHARDING:
├── Document-partitioned: Each shard holds a subset of docs
Shard 0: docs 0-99M, Shard 1: docs 100M-199M, ...
├── 1,000+ shards for 100B documents
├── Each query fans out to ALL shards in parallel
├── Each shard returns local top-K
└── Merger combines all shards' results → global top-K
WHY DOCUMENT-PARTITIONED (not term-partitioned)?
├── Term-partitioned: Each shard owns certain terms
→ Multi-term queries require cross-shard joins = slow
├── Document-partitioned: Each shard is a mini search engine
→ Each shard can answer any query independently
→ Fan-out is embarrassingly parallel
└── Trade-off: Higher fan-out, but simpler and faster
INDEX TIERS:
├── Tier 0 (Real-time): Last ~1 hour of content (in-memory)
├── Tier 1 (Fast): High-quality pages (top 10B), SSD storage
├── Tier 2 (Full): All 100B pages, mix of SSD + HDD
└── Query first hits Tier 1; if < 10 results, falls to Tier 2

Ranking System (Core Challenge)

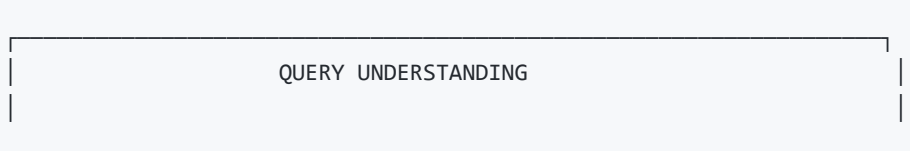
```
┌──────────────────────────────────────────────────────────────────────┐
│                    RANKING PIPELINE                                  │
│                                                                      │
│  Query: "distributed systems interview"                              │
│                                                                      │
│  STAGE 0: QUERY UNDERSTANDING                                        │
│  ┌──────────────────────────────────────────────────────┐            │
│  │  1. Tokenize: ["distributed", "systems", "interview"]│            │
│  │  2. Spell correction: None needed                    │            │
│  │  3. Query expansion / synonyms:                      │            │
│  │     "distributed systems" → also match "distributed  │            │
│  │      computing", "microservices"                     │            │
│  │  4. Intent classification:                           │            │
│  │     Informational (not navigational or transactional)│            │
│  │  5. Named entity recognition:                        │            │
│  │     "distributed systems" = CS topic                 │            │
│  └──────────────────────────────────────────────────────┘            │
│                                                                      │
│  STAGE 1: RETRIEVAL (Recall phase — find candidates)                 │
│  ├── Fan-out query to 1,000+ index shards in parallel                │
│  ├── Each shard: Intersect posting lists (AND / OR logic)            │
│  ├── Each shard scores with BM25 (lightweight):                      │
│  │                                                                   │
│  │   BM25(doc, query) = Σ IDF(term) ×                                │
│  │     [ TF(term,doc) × (k1+1) ]                                     │
│  │     ─────────────────────────────────                             │
│  │     [ TF(term,doc) + k1 × (1 - b + b × |doc|/avgDL) ]             │
│  │                                                                   │
│  │   Where: k1 = 1.2, b = 0.75                                       │
│  │   IDF = log((N - df + 0.5) / (df + 0.5))                          │
│  │                                                                   │
│  ├── Each shard returns top 100 candidates                           │
│  └── Merger: Combine 1,000 × 100 = 100K candidates → top 1,000       │
│                                                                      │
│  STAGE 2: FEATURE EXTRACTION                                         │
│  For each of the ~1,000 candidate documents:                         │
│  ┌──────────────────────────────────────────────────────┐            │
│  │  TEXT FEATURES:                                      │            │
│  │  ├── BM25 score (title, body, anchor text, URL)      │            │
│  │  ├── Term proximity (how close query terms appear)   │            │
│  │  ├── Exact phrase match in title?                    │            │
│  │  ├── Query coverage (% of query terms matched)       │            │
│  │  └── TF-IDF variants across fields                   │            │
│  │                                                      │            │
│  │  DOCUMENT FEATURES:                                  │            │
│  │  ├── PageRank (authority score from link graph)      │            │
│  │  ├── Domain authority (aggregate PageRank of domain) │            │
│  │  ├── Page freshness (last modified / crawl date)     │            │
│  │  ├── Content length and quality score                │            │
│  │  ├── HTTPS vs HTTP                                   │            │
│  │  ├── Mobile-friendliness                             │            │
```



PageRank & Link Graph



Query Understanding & Spell Correction



```
│  SPELL CORRECTION:                                                    │
│  ├── Input: "distribted systms intrvew"                               │
│  ├── Output: "distributed systems interview"                          │
│  │                                                                    │
│  ├── APPROACH 1: Edit Distance (Levenshtein)                          │
│  │   For each word, find dictionary words within edit distance 2      │
│  │   "distribted" → "distributed" (edit distance 1: insert 'u')       │
│  │                                                                    │
│  ├── APPROACH 2: Noisy Channel Model (used at scale)                  │
│  │   P(correction | misspelling) ∝ P(misspelling | correction)         │
│  │                                  × P(correction)                  │
│  │   P(correction) = language model (n-gram from query logs)         │
│  │   P(misspelling | correction) = error model (common typos)        │
│  │                                                                    │
│  ├── APPROACH 3: Sequence-to-Sequence Neural Model                    │
│  │   Encoder: misspelled query → hidden state                        │
│  │   Decoder: hidden state → corrected query                         │
│  │   Trained on (misspelled, corrected) pairs from query logs        │
│  │                                                                    │
│  └── FAST LOOKUP: SymSpell / BK-tree for candidate generation        │
│      Pre-compute all edits within distance 2 of dictionary words     │
│      Lookup: O(1) average case                                       │
│                                                                       │
│  QUERY EXPANSION:                                                     │
│  ├── Synonyms: "car" → also match "automobile", "vehicle"            │
│  ├── Stemming: "running" → "run" (match "runs", "runner")            │
│  ├── Acronyms: "ML" → "machine learning"                             │
│  └── Related terms from query co-occurrence logs                     │
│                                                                       │
│  INTENT CLASSIFICATION:                                               │
│  ├── Navigational: "facebook login" → user wants facebook.com        │
│  │   → Boost the exact domain, show site links                       │
│  ├── Informational: "how does DNS work" → user wants articles        │
│  │   → Boost comprehensive articles, show "People also ask"          │
│  ├── Transactional: "buy laptop online" → user wants to purchase     │
│  │   → Boost shopping results, show product listings                 │
│  └── Local: "pizza near me" → user wants local results               │
│      → Trigger maps/local pack, use geo location                     │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

Snippet Generation

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SNIPPET GENERATION                                │
│                                                                      │
│  Snippets are the 2-3 line text excerpts shown under each            │
│  search result. They must be relevant, readable, and fast.           │
│                                                                      │
│  APPROACH:                                                           │
│  1. At index time: Store compressed page text in document store      │
```

```
│  2. At query time: For each result (top 10 only):                │
│     a. Load compressed text from document store                  │
│     b. Find passages containing query terms                      │
│     c. Score passages by:                                        │
│        ├── Number of query terms present                         │
│        ├── Proximity of query terms to each other                │
│        ├── Position in document (early paragraphs preferred)     │
│        └── Presence of complete sentences                        │
│     d. Select best passage, truncate to ~160 characters          │
│     e. Bold the matching query terms with <b> tags               │
│                                                                  │
│  EXAMPLE:                                                        │
│  Query: "distributed systems interview"                          │
│  Page text: "...this guide covers everything you need to         │
│  know about distributed systems for your next technical          │
│  interview. We cover consensus, replication, and..."             │
│                                                                  │
│  Snippet: "This guide covers everything you need to know         │
│  about <b>distributed systems</b> for your next technical        │
│  <b>interview</b>. We cover consensus, replication..."           │
│                                                                  │
│  FEATURED SNIPPETS (Position 0):                                 │
│  ├── Detect "what is X", "how to X" queries                      │
│  ├── Extract best paragraph / list / table from top result       │
│  └── Display above organic results in highlighted box            │
│                                                                  │
│  LATENCY BUDGET: < 20ms for 10 snippets                          │
│  ├── Document store must support fast random reads               │
│  ├── Compressed text cached on SSD, hot docs in memory           │
│  └── Snippet extraction runs in parallel for all 10 results      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Autocomplete (Typeahead)

```
┌──────────────────────────────────────────────────────────────────┐
│                    AUTOCOMPLETE SERVICE                          │
│                                                                  │
│  As user types "distri", suggest "distributed systems" etc.      │
│                                                                  │
│  DATA SOURCE:                                                    │
│  ├── Query logs: Aggregate popular queries from search logs      │
│  ├── Frequency: Count queries per day, weighted by recency       │
│  ├── Freshness: Boost trending queries (spike detection)         │
│  └── Filter: Remove offensive / low-quality suggestions          │
│                                                                  │
│  DATA STRUCTURE: Trie + Top-K Cache                              │
│                                                                  │
│  ┌──────────────────────────────────────────────────┐            │
│  │                    TRIE                          │           │
│  │             (root)                               │           │
```

```
│  │            /     \                                  │        │
│  │           d       h                                 │        │
│  │          /         \                                │        │
│  │        i            o                               │        │
│  │       /              \                              │        │
│  │      s                w                             │        │
│  │     / \                                             │        │
│  │    t   c                                            │        │
│  │   /     \                                           │        │
│  │ "distributed   "discord"                            │        │
│  │  systems"       [freq=8M]                           │        │
│  │  [freq=2M]                                          │        │
│  │                                                     │        │
│  │  At each node, cache top-10 completions:            │        │
│  │  Node "dis" → ["discord", "disney", "distributed    │        │
│  │                systems", "discount code", ...]      │        │
│  └─────────────────────────────────────────────────────┘        │
│                                                                 │
│  SERVING:                                                       │
│  ├── Trie is replicated across edge data centers                │
│  ├── Rebuilt every ~15 minutes from updated query logs          │
│  ├── Size: ~10-50 GB per replica (top 100M queries)             │
│  ├── Lookup: O(prefix_length) ≈ sub-millisecond                 │
│  └── Client-side debounce: Only send request after 100ms pause  │
│                                                                 │
│  PERSONALIZATION:                                               │
│  ├── Blend global suggestions with user's recent searches       │
│  ├── Weight by user's language and region                       │
│  └── E.g., "py" → "python" for a developer vs "pyramid" for     │
│      a history student                                          │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Knowledge Graph

```
┌─────────────────────────────────────────────────────────────────┐
│                    KNOWLEDGE GRAPH                              │
│                                                                 │
│  Structured database of entities and their relationships.       │
│  Powers knowledge panels, instant answers, and entity search.   │
│                                                                 │
│  ENTITY EXAMPLE:                                                │
│  ┌─────────────────────────────────────────────────┐            │
│  │  Entity: "Alan Turing"                          │            │
│  │  Type: Person                                   │            │
│  │  ├── born: 1912-06-23                           │            │
│  │  ├── died: 1954-06-07                           │            │
│  │  ├── occupation: Mathematician, Computer Scientist          │
│  │  ├── known_for: ["Turing Machine", "Enigma",    │            │
│  │  │               "Turing Test"]                 │            │
│  │  ├── education: Princeton University            │            │
```

```
│  │  └── awards: ["OBE"]                                    │        │
│  │                                                         │        │
│  │  Relations:                                             │        │
│  │  ├── influenced_by: Alonzo Church                       │        │
│  │  ├── influenced: John von Neumann                       │        │
│  │  └── worked_at: Bletchley Park, University of           │        │
│  │                 Manchester                              │        │
│  └─────────────────────────────────────────────────────────┘        │
│                                                                     │
│  SOURCES:                                                           │
│  ├── Wikipedia / Wikidata (structured infoboxes)                    │
│  ├── Freebase (Google Knowledge Graph origin)                       │
│  ├── Web extraction (NER on crawled pages)                          │
│  └── Licensed data (medical, geographic, etc.)                      │
│                                                                     │
│  STORAGE:                                                           │
│  ├── Graph database or triple store (subject, predicate, object)    │
│  ├── ~5B entities, ~500B facts/triples                              │
│  ├── Entity linking: Map query terms → entity IDs                   │
│  └── Cached in memory for hot entities (top 100M)                   │
│                                                                     │
│  USE IN SEARCH:                                                     │
│  ├── Query "Alan Turing" → show knowledge panel on right side       │
│  ├── Query "who invented the Turing machine" → instant answer       │
│  ├── Query "Turing machine" → entity disambiguation                 │
│  └── Related entities: "People also search for..."                  │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

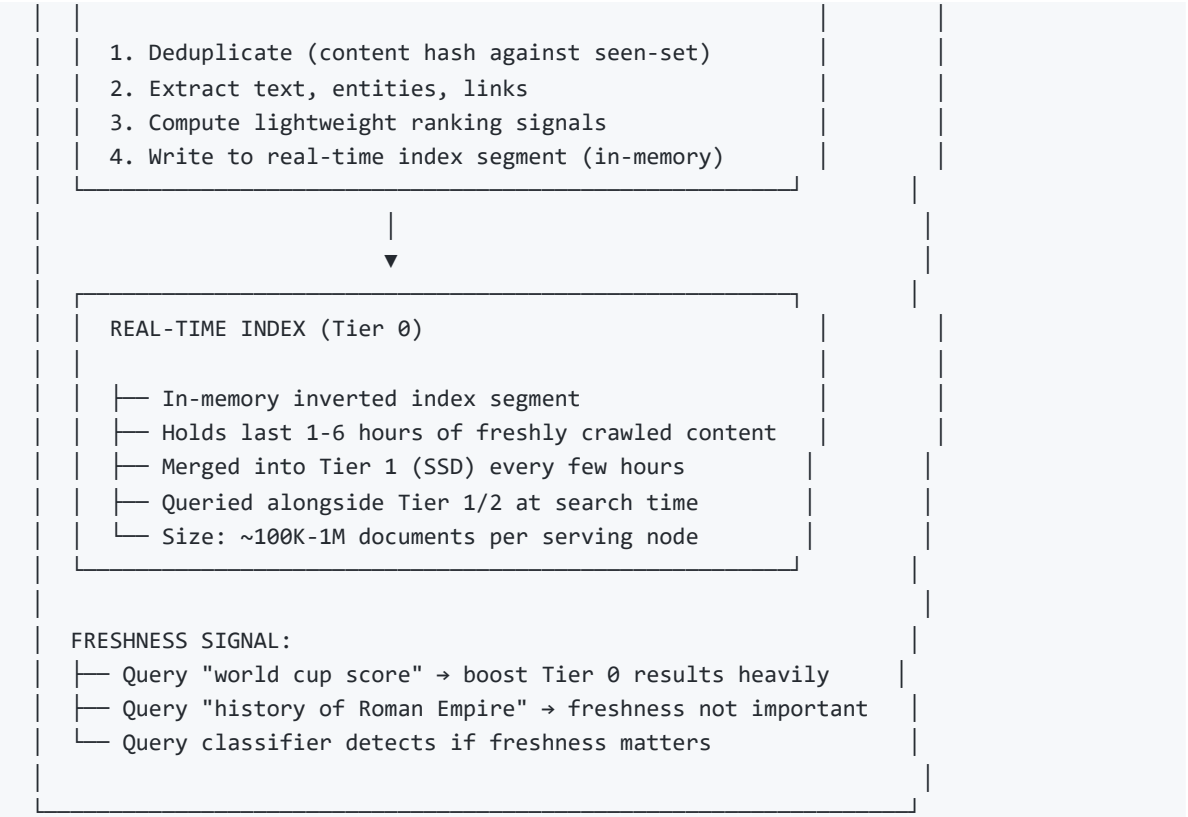
Ad Auction System

```
┌─────────────────────────────────────────────────────────────────────┐
│                    AD SERVING (REAL-TIME AUCTION)                   │
│                                                                     │
│  Ads are the primary revenue source for search engines.             │
│  Every search triggers a real-time ad auction in < 20ms.            │
│                                                                     │
│  FLOW:                                                              │
│  User searches "best laptop 2026"                                   │
│       │                                                             │
│       ▼                                                             │
│  1. AD RETRIEVAL                                                    │
│     ├── Match query to advertiser keyword bids                      │
│     ├── Filter by targeting (region, device, time, demographics)    │
│     └── Candidate ads: ~100-1,000 eligible ads                      │
│       │                                                             │
│       ▼                                                             │
│  2. AD SCORING (Generalized Second-Price Auction)                   │
│     ┌──────────────────────────────────────────────┐                │
│     │  Ad Rank = Bid × Quality Score               │                │
│     │                                              │                │
```

```
│     │  Quality Score:                                          │         │
│     │  ├── Expected CTR (predicted click-through rate)         │         │
│     │  ├── Ad relevance to query                               │         │
│     │  ├── Landing page quality                                │         │
│     │  └── Ad format (extensions, sitelinks boost)             │         │
│     │                                                          │         │
│     │  Winner pays: (Next Ad Rank / Winner's QS) + $0.01       │         │
│     │  (Generalized second-price → incentivizes bidding        │         │
│     │   true value)                                            │         │
│     └──────────────────────────────────────────────────────────┘         │
│       │                                                                  │
│       ▼                                                                  │
│  3. AD RENDERING                                                         │
│     ├── Top 1-3 ads shown above organic results                          │
│     ├── Bottom 1-3 ads shown below organic results                       │
│     └── Each labeled "Sponsored" / "Ad"                                  │
│                                                                          │
│  BUDGET MANAGEMENT:                                                      │
│  ├── Each advertiser sets daily budget                                   │
│  ├── Pacing: Spread budget evenly across the day                         │
│  ├── Real-time budget tracking (Redis counters)                          │
│  └── Stop showing ads when budget depleted                               │
│                                                                          │
│  CLICK FRAUD DETECTION:                                                  │
│  ├── Detect repeated clicks from same IP / device                        │
│  ├── Bot detection via behavioral analysis                               │
│  ├── Invalid click filtering: ~15-20% of clicks are invalid              │
│  └── Refund advertisers for detected fraud                               │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Real-Time Indexing Pipeline

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    REAL-TIME INDEXING                                    │
│                                                                          │
│  Breaking news must appear in search results within minutes,             │
│  not hours. This requires a separate real-time indexing path.            │
│                                                                          │
│  ┌────────────┐     ┌────────────┐     ┌────────────┐                    │
│  │  News      │     │  Social    │     │  High-     │                    │
│  │  RSS /     │────▶│  Media     │────▶│  Priority  │                    │
│  │  Atom      │     │  Firehose  │     │  Recrawl   │                    │
│  │  Feeds     │     │  (X API)   │     │  Queue     │                    │
│  └────────────┘     └────────────┘     └────────────┘                    │
│        │                 │                  │                            │
│        └─────────────────┴──────────────────┘                            │
│                          │                                               │
│                          ▼                                               │
│  ┌──────────────────────────────────────────────────────┐                │
│  │  STREAM PROCESSOR (Kafka Streams / Flink)            │                │
```



Key Components

Component	Technology	Purpose
URL Frontier	Priority queue + per-host queues (Kafka / custom)	Manage crawl order with priority and politeness
Crawler Workers	Distributed HTTP fetchers (10K+ machines)	Download web pages in parallel
Document Processor	MapReduce / Spark pipeline	Parse HTML, extract text, entities, links
Inverted Index	Custom (Google: Caffeine; Bing: custom) on SSDs	Core search data structure — term → doc mapping
Document Store	Columnar storage (Bigtable / HBase)	Store page metadata, compressed text for snippets
Link Graph	Graph-parallel system (Pregel / GraphX)	Store link structure, compute PageRank
Knowledge Graph	Graph DB / Triple store	Entity database for knowledge panels

Ranking Service	LambdaMART + neural re-ranker (BERT-based)	Multi-stage relevance scoring
Spell Corrector	SymSpell + neural seq2seq model	"Did you mean..." suggestions
Autocomplete	Distributed trie + top-K cache	Query suggestions as user types
Ad Auction	Real-time bidding engine (< 20ms)	Match and rank ads per query
Real-Time Indexer	Kafka Streams / Flink + in-memory index	Index breaking content in minutes
Snippet Generator	Document store + passage scorer	Generate text excerpts for results
Query Logger	Kafka → HDFS / data warehouse	Store query logs for analytics and ML training
CDN	CloudFront / Akamai / Google's edge network	Serve static assets, cache homepage globally

---

Scalability Considerations

1. Index Serving Scalability

```
Challenge: 100B pages × 1,000+ shards × 100K queries/second

Solutions:
├── Document-partitioned sharding: Each shard is independent
│   → Each query fans out to ALL shards in parallel
│   → Merger aggregates top-K from each shard
├── Replicas: Each shard has 3-5 replicas for throughput
│   → 1,000 shards × 5 replicas = 5,000 index serving nodes
├── Tiered index: Query Tier 1 first (top 10B pages)
│   → Only fall through to Tier 2 if insufficient results
│   → 90%+ of queries satisfied by Tier 1 alone
├── Caching: Cache results for popular queries
│   → Top 10K queries (head queries) = ~30% of all traffic
│   → Cache hit avoids index lookup entirely
└── Geographic distribution: Index replicas in each region
    → US, Europe, Asia each have full index copies
    → Route queries to nearest data center
```

2. Crawl Scalability

```
Challenge: Crawl 10B pages/day while being polite to websites

Solutions:
├── Distributed crawlers: 10,000+ machines across data centers
├── Per-host rate limiting: 1 request per second per domain
├── DNS caching: Pre-resolve and cache 500M+ domain→IP mappings
├── Prioritized crawling: High-PageRank pages crawled more often
├── Incremental recrawl: Use HTTP ETag / If-Modified-Since
│   → Only re-download changed pages (saves ~60% bandwidth)
├── Bloom filter for URL dedup: 125 GB in-memory filter
│   → O(1) check for "have I seen this URL?"
└── Sitemaps: Parse sitemap.xml for efficient URL discovery
```

3. Ranking Scalability

```
Challenge: Score 1,000+ candidates per query in < 50ms

Solutions:
├── Multi-stage ranking: Cheap filter → expensive re-rank
│   Stage 1 (BM25): Score 100K candidates in 5ms (per shard)
│   Stage 2 (LambdaMART): Re-rank top 1,000 in 10ms
│   Stage 3 (Neural): Re-rank top 50 in 20ms
├── Feature pre-computation: PageRank, domain authority stored
│   at index time (not computed per query)
├── Model distillation: Compress large neural ranker into
│   smaller model for faster inference
└── GPU inference: For neural re-ranker stage, batch scoring
    on GPUs for throughput
```

4. Freshness vs Throughput

```
Challenge: New content must be searchable in minutes, but batch
           indexing is more efficient

Solutions:
├── Dual-path architecture:
│   ├── Batch path: Recrawl + reindex billions of pages (hours)
│   └── Real-time path: Stream new content into in-memory index
├── Tier 0 merge: Real-time segments merged into Tier 1 hourly
├── Priority signals: RSS feeds, social media spikes trigger
│   immediate recrawl of breaking content
└── QPS allocation: Reserve 10% of crawl capacity for
    real-time/priority crawls
```

Trade-offs

Decision	Option A	Option B	Search Engine Choice
Index partitioning	Document-partitioned	Term-partitioned	**Document-partitioned** — each shard handles any query; simpler, faster
Ranking model	BM25 only (simple)	ML-based (LambdaMART + neural)	**Multi-stage ML** — BM25 for recall, ML for precision
Crawl strategy	Crawl everything equally	Prioritized by importance	**Prioritized** — high-PageRank pages crawled more often
Freshness	Batch-only index rebuild	Real-time + batch hybrid	**Hybrid** — batch for base index, streaming for breaking content
Snippet generation	Pre-compute at index time	Generate at query time	**Query-time** — snippets must match the specific query terms
Autocomplete data	Full query logs	Aggregated popular queries	**Aggregated** — privacy + efficiency; filter offensive content
PageRank freshness	Recompute daily	Recompute weekly	**Weekly full + daily incremental** — balance freshness vs compute cost
Spell correction	Dictionary-based	Neural (seq2seq)	**Hybrid** — dictionary for speed, neural for hard cases
Ad placement	Mixed with organic	Clearly separated	**Clearly separated** — labeled "Sponsored" above and below organic
Cache strategy	Cache full result pages	Cache index postings	**Both** — full pages for head queries, postings for warm queries

Technologies Deep Dive

Data Layer

Technology	Purpose	Why
Custom Inverted Index (Caffeine / custom)	Core search index	No off-the-shelf solution handles 100B+ docs at this scale
Bigtable / HBase	Document store (metadata, snippets)	Petabyte-scale, fast random reads by doc_id
HDFS / GFS	Raw crawl data, index build intermediates	Distributed file system for batch processing
Kafka	Event streaming (crawl events, query logs)	High throughput, partitioned, replay for reprocessing
Redis	Query result cache, rate limiting, ad budgets	Sub-millisecond reads for hot query cache
Graph DB / Pregel	Link graph, PageRank computation	Graph-parallel processing at web scale

Compute

Technology	Purpose	Why
MapReduce / Spark	Index building, PageRank, batch analytics	Batch processing of petabyte-scale data
Kafka Streams / Flink	Real-time indexing, trending queries	Stream processing for fresh content
LambdaMART	Learning-to-rank model	Fast inference (~µs per doc), interpretable features
BERT / Transformer	Neural re-ranker (top-50 candidates)	Deep semantic understanding of query-document relevance
gRPC	Inter-service communication	Low-latency, protobuf, streaming for index fan-out

Infrastructure

Technology	Purpose	Why
Custom hardware	Index serving (SSD-heavy), crawling	Optimized cost per query at massive scale

CDN (global edge)	Static assets, cached result pages	Sub-50ms page load worldwide
DNS-based load balancing	Route users to nearest data center	Geographic latency optimization
Kubernetes / Borg	Service orchestration	Auto-scaling, rolling deploys

---

How to Explain It Well

30-Second Pitch

> "A web search engine has two main paths: the offline pipeline crawls billions of pages, extracts text, builds an inverted index (term → document mapping), and computes PageRank for authority. The online serving path takes a user query, fans it out to 1,000+ index shards in parallel, each shard returns top candidates scored by BM25, then a multi-stage ML ranker re-ranks the top 1,000 using hundreds of features — text relevance, PageRank, freshness, click signals. The final top 10 results get snippets generated and ads injected, all in under 200ms. The key insight is document-partitioned sharding: each shard is a mini search engine that can answer any query independently."

Key Talking Points

1. **Inverted index is THE data structure**: Understand posting lists, intersection, compression, and sharding
2. **Document-partitioned sharding**: Each shard holds a subset of all documents and can answer any query
3. **Multi-stage ranking**: BM25 (fast, recall) → LambdaMART (features, precision) → Neural (top-50, deep relevance)
4. **PageRank**: "Important pages are linked to by important pages" — the foundational authority signal
5. **Freshness architecture**: Dual-path — batch indexing for base coverage, streaming for breaking news

Whiteboard Sequence

1. Start with the query serving flow: Query → fan-out to shards → merge → rank → results
2. Draw the inverted index structure: term dictionary → posting lists → intersection
3. Show the offline crawl pipeline: Frontier → Crawler → Parser → Index Builder
4. Add PageRank: Link graph → iterative computation → score per document
5. Discuss multi-stage ranking: BM25 → ML re-ranker → business rules → final top 10

Common Follow-up Questions

"Why document-partitioned instead of term-partitioned?"

- **Term-partitioned**: Each shard owns certain terms (e.g., shard A has "distributed", shard B has "systems")
- Multi-term queries like "distributed systems" require fetching posting lists from **different shards** and joining them across the network — very slow
- **Document-partitioned**: Each shard is a complete mini search engine with all terms for its subset of documents
- Multi-term queries are answered **locally** on each shard — just intersect posting lists in memory
- Trade-off: Higher fan-out (query all shards), but each shard's work is independent and fast
- At Google/Bing scale, the network overhead of term-partitioned cross-shard joins is prohibitive

"How does PageRank actually work at 100B page scale?"

- Represented as a **sparse matrix multiplication** problem: PR = d × M × PR + (1-d)/N
- Computed using **graph-parallel frameworks** (Google's Pregel, Apache Giraph, Spark GraphX)
- Each iteration: every page sends its current rank ÷ outgoing link count to all pages it links to
- Typically converges in **40-50 iterations**
- At 100B pages with 4T edges, this is a massive computation — runs on thousands of machines
- Full recomputation: weekly. Incremental updates for new/changed pages: daily
- The final output is just one float per document (the PageRank score), stored in the index

"How do you handle 100K queries per second?"

- **Fan-out + parallelism**: Each query hits 1,000+ shards in parallel, each shard has 3-5 replicas
- **Caching**: Top 10K queries (~30% of traffic) served from result cache — no index lookup needed
- **Tiered serving**: 90% of queries answered by Tier 1 (top 10B pages) — smaller, faster index
- **Geographic replication**: Full index copies in US, Europe, Asia — route to nearest
- **Lightweight first pass**: BM25 scoring costs microseconds per doc; heavy ML only on top candidates
- Total serving fleet: ~50,000+ machines for index serving alone

"How does real-time indexing work for breaking news?"

- News sources (RSS feeds, social media APIs) continuously monitored by ingestion workers
- New content enters a **Kafka topic**, processed by stream indexer (Flink / Kafka Streams)
- Stream indexer builds **in-memory index segments** (Tier 0) on each serving node
- These segments are queried alongside the persistent Tier 1/2 index
- A "freshness boost" in the ranker promotes Tier 0 content for time-sensitive queries
- Every few hours, Tier 0 segments are **merged into Tier 1** (compaction)
- End-to-end latency: Breaking news article → searchable in **2-5 minutes**

"What about spam and SEO manipulation?"

- **Link spam**: Detected via link graph analysis — link farms, bought links, unnatural patterns
- **TrustRank**: Propagate trust from known-good seed pages; pages far from trusted seeds are suspicious
- **Content spam**: Keyword stuffing, hidden text, doorway pages — detected by content classifiers
- **Cloaking**: Show different content to crawler vs user — detected by re-fetching as a user agent
- **Manual penalties**: Human reviewers flag spam rings; penalties applied at domain level
- **ML classifier**: Trained on labeled spam/not-spam pages; features include link patterns, content quality, user engagement signals
- Arms race: Spammers adapt, search engine updates models continuously

"How would you handle a query like 'weather' differently from 'history of quantum computing'?"

- **Intent classification** at query understanding stage detects query type
- "weather" → **Local + Instant Answer**: Trigger weather widget (structured data from weather API), boost local results, freshness matters
- "history of quantum computing" → **Informational**: Boost comprehensive long-form articles, show knowledge panel for "quantum computing" entity, freshness less important
- This classification affects: which index tiers to prioritize, freshness boosting weight, whether to show widgets/panels, ad relevance thresholds
- Classified using an ML model trained on query logs + human-labeled intent data

---